Contents

Proven C Library Complete Manual (v26.09.04a)

The manual of `proven`, a C23 systems library. Every chapter is one file under `manual/`; the Korean edition mirrors it under `manual-ko/`.

Contents

Copyright

- Author: rubidus — rubidus@gmail.com
- Repository: github.com/rubidus-api/proven_c_lib

- Edition: v26.09.04a

- Every code example in this manual is compiled, and the quoted ones are run, by the build.

Tutorial: the library in six short programs

Part I — Start here. No prerequisites beyond one introductory C book. **After this tutorial** you can read the greeting program in Chapter 0 line by line, and the reference chapters stop looking like a wall of new words.

Why this exists

Chapter 0 shows a working program on its first page. That program is thirty lines long and every one of them is a new idea: an allocator you pass in, a result you have to check, a view that carries its length, an append that refuses rather than truncates, a destroy paired with the allocator that created it. If you have finished one C book and no more, five new ideas arriving together is four too many.

So this tutorial takes them one at a time. Six programs, each a few lines longer than the last, each introducing exactly one thing. Every one of them is a real file under `manual/examples/` that the build compiles and runs, so what you read here is what actually ran.

Build any of them the way you build any C program — the library is source you compile with your own code, so there is nothing to install:

```
cc -std=c23 -Iinclude your_program.c src/proven/*.c platform/*.c -o your_program
```

If that line is unfamiliar, read Chapter 0 §4 first and come back.

Table of contents

Lesson 1 — printing, and a build that works

The one new thing: `proven_println`, and the fact that its arguments are checked.

`printf("%d", 3.5)` compiles. It is wrong, and on a bad day it prints garbage or crashes, because the format string is a *string* — nothing connects it to the arguments that follow. `proven_println` uses `{}` as the placeholder and wraps each argument in `PROVEN_ARG`, which records the argument's type. The two cannot disagree, because the type travels with the value.

```
/*
 * Lesson 1 - get one line to build and run.
 *
 * Nothing here is about the library's ideas yet. The only question this program
 * answers is "does my build command work?", and it is worth answering on its
 * own, because every later lesson assumes it.
 *
 * printf(fmt, ...) is not checked: the compiler cannot tell you that "%d" was
 * handed a double. proven_println checks, because every argument is wrapped by
 * PROVEN_ARG and carries its own type.
 */

int main(void) {
    proven_println("hello from proven");

    /* {} is the placeholder. The value goes through PROVEN_ARG, which records
```

```
     * what type it is, so the format string and the argument cannot disagree. */
    proven_println("one number: {}", PROVEN_ARG(42));
    proven_println("two of them: {} and {}", PROVEN_ARG(1), PROVEN_ARG(2));

    return EXAMPLE_OK();
}
```

Running it prints three lines. If it built and ran, your include path and your source list are right, and everything after this is about the library rather than about your build.

Common first stumble. `{}` is not `%s`. There is no letter to get wrong, and there is nothing to remember about `long` versus `int`: `PROVEN_ARG` knows.

Lesson 2 — text that knows how long it is

The one new thing: the *view* — a pointer and a size that travel together.

In the C you know, a string is a pointer, and its length is wherever the first zero byte happens to be. Every function that touches it has to walk the bytes to find out how much there is; if the zero is missing, it walks off the end. That single design decision is behind a large share of the security advisories in the language's history.

A view makes the missing half explicit. It does not own the bytes — it borrows them, and it never frees anything.

```
/*
 * Lesson 2 - text that knows how long it is.
 *
 * In the C you already know, a string is a pointer and the length is wherever
 * the first zero byte happens to be. Every function has to walk the bytes to
 * find out how much there is, and if the zero is missing it walks off the end.
 *
 * A view is the pair that C leaves implicit: a pointer AND a size, travelling
 * together. It borrows - it does not own the bytes and never frees them.
 */

int main(void) {
    /* PROVEN_LIT makes a view from a literal. The size is computed at compile
     * time, so no scan happens here - unlike strlen. */
    proven_u8str_view_t hello = PROVEN_LIT("hello");

    EXAMPLE_REQUIRE(hello.size == 5, "the view already knows its own length");
    EXAMPLE_REQUIRE(hello.ptr != NULL, "and it points at the literal's bytes");

    /* Because the length travels with the pointer, comparing is a size check
     * plus a memcmp. No walking, and no way to run off the end. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(hello, PROVEN_LIT("hello")), "same text");
    EXAMPLE_REQUIRE(!proven_u8str_view_eq(hello, PROVEN_LIT("hell")), "shorter text differs");

    /* A view can name PART of something without copying it. Here is the middle
     * of the literal - still borrowed, still no allocation. */
    proven_u8str_view_t ell = { .ptr = hello.ptr + 1, .size = 3 };
    EXAMPLE_REQUIRE(proven_u8str_view_eq(ell, PROVEN_LIT("ell")), "a window onto the same bytes");

    proven_println("whole: {} (size {})", PROVEN_ARG(hello), PROVEN_ARG(hello.size));
    proven_println("part : {} (size {})", PROVEN_ARG(ell), PROVEN_ARG(ell.size));

    return EXAMPLE_OK();
}
```

What to notice. `PROVEN_LIT` computes the size at compile time, so it is not `strlen`. And the third view in the program names the *middle* of the literal without copying it: a view can describe part of something, which is why splitting and parsing in this library allocate nothing.

Common first stumble. A view is borrowed. If the bytes it points at go away — a local buffer that leaves scope, a string you destroyed — the view is dangling, exactly like any other C pointer. The library never extends a lifetime behind your back.

Lesson 3 — a call that can fail says so

The one new thing: `proven_err_t`, and refusal instead of truncation.

C reports failure in three unrelated ways: a magic return value, a null pointer, or the global `errno` that the next call overwrites. All three are easy not to look at, and nothing complains when you do not. Here, a call that can fail returns an error *value* — and these functions are marked `[[nodiscard]]`, so throwing it away is a compiler warning rather than a habit.

```
/*
 * Lesson 3 - a call that can fail says so, in its return value.
 *
 * The C you know reports failure in three different ways: a magic return value
 * (-1), a null pointer, or a global called errno that the next call overwrites.
 * All three are easy to not look at, and nothing complains when you don't.
 *
 * Here a fallible call returns proven_err_t. It is an ordinary value: you can
 * store it, compare it, and pass it upward. And [[nodiscard]] on these
 * functions means ignoring it is a compiler warning, not a habit.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* Room for exactly 8 bytes. (Chapter 2 is about where that room comes
     * from; for now, notice only that we said how much.) */
    proven_result_u8str_t s = proven_u8str_create(alloc, 8);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "8 bytes should be available");

    /* This fits. */
    proven_err_t err = proven_u8str_append(&s.value, PROVEN_LIT("12345678"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "eight bytes into eight bytes fits exactly");

    /* This does not - and the library REFUSES it. It does not append the part
     * that would fit, because half a word is not a shorter word, it is a
     * different one. */
    proven_err_t too_much = proven_u8str_append(&s.value, PROVEN_LIT("9"));
    EXAMPLE_REQUIRE(!proven_is_ok(too_much), "one byte more than capacity must fail");
    EXAMPLE_REQUIRE(too_much == PROVEN_ERR_OUT_OF_BOUNDS, "and it says why: out of bounds");

    /* The refusal changed nothing. The string is exactly as it was, which is
     * what "failure-atomic" means: a failed call leaves no half-done state. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), PROVEN_LIT("12345678")),
                    "the refused append must not have written anything");

    proven_println("after the refusal, the string is still: {}",
                   PROVEN_ARG(proven_u8str_as_view(&s.value)));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

What to notice. The append that does not fit changes *nothing*. It does not store the part that would have fit. This is called failure atomicity, and the reason for it is that a truncated string is not a shorter message, it is a different one — a truncated path names a different file, and a truncated command is a different command.

Common first stumble. `proven_is_ok(err)` is the check; `err == PROVEN_OK` says the same thing. What you must not do is ignore it and read the value anyway.

Lesson 4 — the value and the error arrive together

The one new thing: result structs — `.err` and `.value` in one return.

When a call has nothing to give back, an error code is enough. When it does have something, the library returns both in one small struct, and the rule is one sentence: `value` **means nothing until you have looked at** `err`**.**

```
/*
 * Lesson 4 - when the call has a value to give back, the value and the error
 * arrive together.
 *
 * proven_err_t alone is enough when there is nothing to return. When there IS
 * something, you get a small struct with two fields: `err` and `value`. The
 * rule is one sentence: `value` means nothing until you have looked at `err`.
 *
 * This is the same discipline as checking malloc for NULL, except the checking
 * place is part of the type instead of a convention you have to remember.
 */

/* A function of your own can return one too. Nothing in the library is magic. */
static proven_result_size_t safe_div(proven_size_t a, proven_size_t b) {
    proven_result_size_t res = {0};
    if (b == 0) {
        res.err = PROVEN_ERR_INVALID_ARG;    /* value stays 0 - and means nothing */
        return res;
    }
    res.err = PROVEN_OK;
    res.value = a / b;
    return res;
}

int main(void) {
    proven_result_size_t ok = safe_div(10, 2);
    EXAMPLE_REQUIRE(proven_is_ok(ok.err), "dividing by 2 is fine");
    EXAMPLE_REQUIRE(ok.value == 5, "and only now may we read the value");

    proven_result_size_t bad = safe_div(10, 0);
    EXAMPLE_REQUIRE(!proven_is_ok(bad.err), "dividing by zero must fail");
    EXAMPLE_REQUIRE(bad.err == PROVEN_ERR_INVALID_ARG, "and say which rule was broken");
    /* bad.value is 0, but that is not an answer. It is the absence of one. */

    /* The library's own calls have the same shape. proven_u8str_create hands
     * back the string it made together with the error that guards it. */
    proven_allocator_t alloc = proven_heap_allocator();
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
    if (!proven_is_ok(s.err)) return 1;      /* nothing was created; nothing to destroy */

    proven_println("10 / 2 = {}", PROVEN_ARG(ok.value));

    proven_u8str_destroy(alloc, &s.value);
```

```
    return EXAMPLE_OK();
}
```

What to notice. `safe_div` is *your* function, not the library's. The pattern is a plain struct return; there is no machinery underneath, which is why you can use it in your own code the moment you have read this page.

Common first stumble. On failure the `value` field usually holds a zero. That zero is not an answer — it is the absence of one. Checking `err` is what tells them apart.

Lesson 5 — who gives out the memory is an argument

The one new thing: `proven_allocator_t`, passed in rather than assumed.

`malloc` is a decision made for you: one heap, one strategy, and nothing at the call site says so. In this library, anything that needs memory takes an allocator and uses only that one. Two things follow, and both are the point: you can always answer "who allocated this?" by reading the call, and you can hand the same code a different allocator without changing it.

```
/*
 * Lesson 5 - who gives out the memory is an argument, not a global.
 *
 * malloc is a global decision made for you: one heap, one strategy, invisible
 * at the call site. Here, anything that needs memory takes a
 * proven_allocator_t and uses only that. Two consequences follow, and both are
 * the point.
 *
 *   - You can always answer "who allocated this?" by looking at the call.
 *   - You can hand a different allocator to the same code without changing it.
 */

/* This function does not know or care where the memory comes from. */
static proven_result_u8str_t make_greeting(proven_allocator_t alloc,
                                           proven_u8str_view_t name) {
    proven_result_u8str_t out = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(out.err)) return out;

    proven_err_t err = proven_u8str_append(&out.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&out.value, name);
    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &out.value);   /* undo what we made */
        out.err = err;
        out.value = (proven_u8str_t){0};
    }
    return out;
}

int main(void) {
    /* (a) the ordinary heap - malloc and free underneath */
    proven_allocator_t heap = proven_heap_allocator();

    proven_result_u8str_t a = make_greeting(heap, PROVEN_LIT("world"));
    EXAMPLE_REQUIRE(proven_is_ok(a.err), "the heap should be able to give 64 bytes");
    proven_println("from the heap : {}", PROVEN_ARG(proven_u8str_as_view(&a.value)));
    /* Destroyed with the SAME allocator that created it. That pairing is the
     * whole ownership rule of this library. */
    proven_u8str_destroy(heap, &a.value);

    /* (b) an arena - one block of memory handed out in order, freed all at once.
     *     Note that make_greeting above did not change at all. */
```

```
    alignas(PROVEN_MAX_ALIGN) proven_byte_t backing[512];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = backing, .size = sizeof backing });
    proven_allocator_t from_arena = proven_arena_as_allocator(&arena);

    proven_result_u8str_t b = make_greeting(from_arena, PROVEN_LIT("arena"));
    EXAMPLE_REQUIRE(proven_is_ok(b.err), "the arena has room for this too");
    proven_println("from an arena : {}", PROVEN_ARG(proven_u8str_as_view(&b.value)));

    /* No destroy loop here: an arena frees everything by being reset. That is
     * lesson 7's subject; the point now is only that the caller chose. */
    proven_arena_reset(&arena);

    return EXAMPLE_OK();
}
```

What to notice. `make_greeting` is written once and runs against the heap and against an arena — a block of memory you own, handed out by bumping a pointer, and released all at once. The function did not change, and nothing global was configured.

The ownership rule, in one line: whatever created it destroys it, with the *same* allocator.

Common first stumble. With an arena, `destroy` reclaims nothing — an arena frees by being reset. Call it anyway: the pairing is what makes the code correct when the allocator later changes.

Lesson 6 — the Chapter 0 program, read line by line

The one new thing: nothing. That is the exercise.

```
/*
 * Lesson 6 - the program from Chapter 0, read line by line.
 *
 * Nothing new is introduced here. This is the same greeting program the manual
 * opens with, and the point of the lesson is that you can now name every part
 * of it: the allocator you pass in (lesson 5), the result you must check
 * (lesson 4), the error a refusal returns (lesson 3), the view that carries its
 * own length (lesson 2), and the printing that checks its arguments (lesson 1).
 *
 * If that reads as ordinary now, the tutorial has done its job and the
 * reference chapters are open to you.
 */

int main(void) {
    /* lesson 5 - the caller decides where memory comes from */
    proven_allocator_t alloc = proven_heap_allocator();

    /* lesson 4 - the string and the error that guards it, together */
    proven_result_u8str_t greeting = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(greeting.err)) return 1;

    /* lesson 2 - borrowed text that knows its own size */
    proven_u8str_view_t name = PROVEN_LIT("world");

    /* lesson 3 - each append either fits or refuses; none of them truncates */
    proven_err_t err = proven_u8str_append(&greeting.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, name);
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, PROVEN_LIT("!"));

    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &greeting.value);
```

```
        return 1;
    }

    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&greeting.value),
                                         PROVEN_LIT("hello, world!")),
                    "the three appends should have built the whole greeting");

    /* lesson 1 - the format string and the argument cannot disagree */
    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&greeting.value)));

    /* lesson 5 again - destroyed with the allocator that created it */
    proven_u8str_destroy(alloc, &greeting.value);
    return EXAMPLE_OK();
}
```

If those comments now read as labels for things you know rather than as new information, you are ready for the reference chapters.

Where to go next

If you want to	Read
the full version of everything above	Chapter 0
the types, errors and contracts in detail	Chapter 1
arenas, pools and buffers properly	Chapter 2
owned strings, splitting, encodings	Chapter 3
arrays, maps, lists, hashing	Chapter 4
files, streams, time, randomness	Chapter 5
a word you do not recognise	the glossary

Chapter 0: Start Here

Part I — Start here. No prerequisites beyond one introductory C book. **After this chapter** you can build a program against `proven`, read any other chapter, and look up a term you do not recognise.

Table of contents

---

1. Who this manual is for

This manual assumes you have finished one introductory C book. Concretely, it assumes you are comfortable with variables and control flow, functions, arrays, `struct`, pointers and `*`/`&`, `malloc` and `free`, `printf`, `char *` strings with `strlen`, and compiling a program with `gcc main.c -o main`. If all of that is familiar, you have enough.

It does **not** assume you have met ownership as a discipline, borrowed versus owned data, arenas or pools, function-pointer tables used as an interface, C23 attributes, undefined behaviour as something a compiler actively exploits, alignment beyond "it works", atomics, or the idea that a library might *refuse* an operation instead of doing its best. Every one of those is explained where it is first used, and every one is in the glossary in §13.

This is not a C tutorial. It will not explain what a pointer is. It will explain, at length, why this library hands you a `struct` containing an error instead of setting `errno`, because that is a design decision you are entitled to disagree with, and you cannot disagree with a decision nobody explained.

---

2. Why this library exists

C gives you almost nothing and trusts you completely. That is its great strength — there is no runtime, no hidden allocation, no cost you did not write — and it is why C is still the language of operating systems, embedded devices and everything that has to be small and predictable.

It is also why the same five bugs have been shipping for fifty years. This library is a set of answers to those five bugs. Each answer costs something, and this section says what.

The string functions do not know how big anything is

```
char buf[64];
strcpy(buf, name);            /* wrong: how long is name? strcpy never asks */
strcat(buf, ", welcome!");    /* wrong: and how much room is left now? */
```

`strcpy` receives a destination pointer and a source pointer. Nothing in that signature carries the size of the destination, so nothing can check it. The function will happily write the 200th byte into a 64-byte buffer, and what it corrupts is whatever the compiler happened to put next — often the return address. This is not a rare mistake by careless people; it is the single most exploited class of bug in the history of the language, and the API makes it the *default* behaviour.

`strncpy` is the traditional answer and it is a trap of its own: it does not always NUL-terminate, so the "safe" version silently produces a string that is not a string.

What this library does instead. A string carries its length. `proven_u8str_view_t` is a pointer *and* a size, together, always. An append checks the destination's capacity because it knows the capacity, and when the text does not fit it **returns an error and writes nothing** — it does not truncate, because a truncated path is a wrong path and a truncated command is a different command. See Chapter 3.

What it costs. Every string is two words instead of one, and you cannot pass a `proven` string straight to a `printf("%s")` without asking for a NUL-terminated form.

Nothing makes you check for failure

```
char *p = malloc(n);
p[0] = 'x';                   /* wrong: malloc returns NULL when it fails */
```

`malloc` reports failure by returning `NULL`, and C will not say a word if you never look. The same is true of `fopen`, of `realloc`, of every function that returns a sentinel. The error is *available*; noticing it is optional.

`errno` is worse, because it is a global that survives the call. You must check it at exactly the right moment, before any other library call has a chance to overwrite it, and you must remember that a successful call is allowed to set it to garbage.

What this library does instead. A function that can fail returns its error *as a value*, and when it also has a result, the two come back together in one `struct`:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
if (!proven_is_ok(s.err)) return 1;      /* s.value means nothing until you check */
```

Functions whose only job can fail are marked `[[nodiscard]]`, which means the compiler refuses to build code that throws the error away. You can still ignore it deliberately — `(void)` in front of the call — and having to type that is the point. See Chapter 1.

What it costs. More `if`s. There is no exception mechanism to jump you out of a deep failure, so error paths are visible in the shape of the code. That visibility is the feature.

`printf` believes whatever you tell it

```
printf("%d\n", 3.0);          /* wrong: %d with a double. This compiles. */
printf("%s\n", 42);           /* wrong: and this one crashes */
```

The format string is checked by nothing at runtime. Modern compilers warn about literal formats, which helps exactly until the format is a variable, and then you are back to a function that reads whatever bytes the varargs stack happens to hold, in whatever shape the string demanded.

What this library does instead. `{}` is a placeholder with no type in it, and the type comes from the argument, resolved at compile time by `_Generic`:

```
proven_println("{} is {}", PROVEN_ARG(name), PROVEN_ARG(count));
```

There is no `%d`-versus-`double` mismatch available, because you never wrote the type twice. See Chapter 3 §3 for the tutorial and Chapter 8 for the full grammar.

What it costs. `PROVEN_ARG` around each argument, and a format language that is not the one in your fingers.

Who frees this?

```
char *s = build_message();    /* do I free this? the type does not say */
```

A `char *` returned from a function might be freshly allocated, might point into a caller's buffer, might be a string literal in read-only memory, or might be a pointer into a static buffer that the next call will overwrite. The type is identical in all four cases. The answer lives in documentation, and documentation drifts.

What this library does instead. Ownership is in the type name and in the signature. A `proven_u8str_t` is **owned** — you got it from a `_create`, and you must `_destroy` it with the same allocator. A `proven_u8str_view_t` is **borrowed** — it points at bytes someone else owns, you never destroy it, and it stops being valid when the owner does. Any function that might allocate takes the allocator as a parameter, so a signature without an allocator cannot allocate.

What it costs. Two types where C has one, and the discipline of asking "who owns this?" at every boundary — which you were paying anyway, just later and in a debugger.

The comparison function nobody can typecheck

```
qsort(a, n, sizeof *a, cmp);  /* cmp takes const void*; get it wrong and it is UB */
```

`qsort` takes a comparator through a `void *` interface, so a comparator with the wrong parameter types still compiles. The classic version of this bug is comparing the pointers instead of what they point at, and it produces a program that runs, sorts nothing correctly, and never crashes.

What this library does instead. The same `void *` shape — this is C, there is no other way — but the library documents the contract precisely, gives you working comparators to copy, and `proven_array_sort` is an introsort with an *O(n log n)* guarantee rather than a quicksort that degrades to $O(n^2)$ on the input an attacker chooses. See Chapter 4.

The bytes have a type, even when you did not choose one

You think of memory as bytes. C's abstract machine does not: it treats memory as *typed*, and reading the same bytes through pointers of two different types is undefined behaviour — which the compiler is allowed to exploit, silently, only when optimisations are on. This is called **strict aliasing**, and it is the trap under every hand-written parser that reads a byte buffer through pointers of different widths:

```
void *buf = malloc(8);
uint32_t *w = buf;      /* the same memory, seen as 32-bit — no cast, no warning */
uint16_t *h = buf;      /* the same memory, seen as 16-bit */
*w = 0xAAAAAAAAu;
*h = 0x1234;            /* change the low half */
printf("%08x\n", *w);   /* wrong to expect aaaa1234: at -O2 this prints aaaaaaaa */
```

Compiled at `-O0` it prints `aaaa1234`; at `-O2` it prints `aaaaaaaa`, because the compiler assumed a `uint16_t` write and a `uint32_t` read cannot touch the same memory and dropped the write. No warning is given, and the program passed every test you ran in a debug build. This is the class of bug the Linux kernel avoids by compiling with `-fno-strict-aliasing` — a whole flag, for one rule.

What this library does instead. Raw memory is `proven_byte_t`, an alias of `unsigned char` — the one type the rule explicitly exempts, because the standard lets you inspect any object's bytes through it. The ordinary API never quietly reinterprets your bytes as a wider type, so the bug above cannot be written through it. (Strict aliasing has a subtler sibling, *provenance*, that the library is named after; Chapter 6 §3 and the project README cover it.)

What this library is not

It is not a framework, and it does not want to own your `main`. It has no global state to initialise, starts no threads, registers no `atexit` handler, and allocates nothing you did not hand it an allocator for. Every module is usable on its own. Most of it runs with no operating system at all — see freestanding mode.

The problem	What C gives you	What `proven` gives you	Cost
Buffer overruns	`strcpy`, `strcat` — no size anywhere	Views carry a length; writes refuse rather than truncate	Two words per string
Unchecked failure	`NULL` returns and `errno`	Errors returned as values, `[[nodiscard]]` on the ones you must not drop	More `if`s
Format mismatch	`printf` trusts the format string	`{}` with the type taken from the argument	`PROVEN_ARG` at each call
Unclear ownership	`char *` means four different things	Owned and borrowed are different types	Two types instead of one
Hidden allocation	Anything may call `malloc`	Only functions taking an allocator can allocate (one bounded exception: `proven_println` on an over-long line)	The parameter is on the signature
Bytes with a hidden type	Reinterpreting memory through a wider pointer is UB the optimiser exploits	Raw bytes go through `proven_byte_t`, the type the rule exempts	—

---

3. Your first program

This is the whole of it. Every line is one of the contracts in §5, and the build compiles and runs this exact file, so it cannot quietly stop being true.

```
/*
 * The first program. It is deliberately small, and every line of it is one of
 * the five contracts you will meet on every page of this manual.
 *
 * Compare it with the C you already know:
 *
 *     char buf[64];
 *     strcpy(buf, name);          <- how big is name? strcpy does not ask.
 *     strcat(buf, ", welcome!");  <- and now? strcat does not ask either.
 *     printf("%s\n", buf);
 *
 * That program is correct until the day `name` is longer than you assumed, and
 * then it is a security advisory. The version below cannot do that: every write
 * knows the size of its destination, and every operation that could fail hands
 * you back an error you are not allowed to ignore silently.
 */

int main(void) {
    /* (1) You pass the allocator in. The library never reaches for a global
```

```
     *     malloc behind your back, so you always know who allocated what. */
    proven_allocator_t alloc = proven_heap_allocator();

    /* (2) Anything that can fail returns its error WITH its value. There is no
     *     errno to remember to check, and `greeting.value` means nothing until
     *     you have looked at `greeting.err`. */
    proven_result_u8str_t greeting = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(greeting.err)) return 1;

    /* (3) A view is borrowed text that knows its own length. PROVEN_LIT builds
     *     one from a literal at compile time - no strlen scan happens here. */
    proven_u8str_view_t name = PROVEN_LIT("world");

    /* (4) The append refuses rather than truncates. If "hello, " and the name
     *     did not fit in the 64 bytes asked for above, this returns
     *     PROVEN_ERR_OUT_OF_BOUNDS and writes nothing - it never quietly stores
     *     half a word and lets you carry on. */
    proven_err_t err = proven_u8str_append(&greeting.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, name);
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, PROVEN_LIT("!"));

    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &greeting.value);
        return 1;
    }

    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&greeting.value),
                                         PROVEN_LIT("hello, world!")),
                    "the three appends should have built the whole greeting");

    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&greeting.value)));

    /* (5) You created it with `alloc`, so you destroy it with the SAME `alloc`.
     *     Owning things are destroyed exactly once; borrowed things - like
     *     `name` above - are never destroyed at all. */
    proven_u8str_destroy(alloc, &greeting.value);

    return EXAMPLE_OK();
}
```

`EXAMPLE_REQUIRE` and `EXAMPLE_OK` are not part of the library — they come from `manual/examples/example.h` and exist so the build can check that every example in this manual still does what the text says it does. Your own program would use neither.

Three details worth pausing on, because they recur everywhere:

- `proven_u8str_create(alloc, 64)` **asks for 64 bytes of capacity**, and that capacity includes the NUL terminator. It does not grow on its own. `proven_u8str_append` is the fixed-capacity form: it fails when the text does not fit. When you want growth, you call `proven_u8str_append_grow` and pass the allocator, and the signature tells you at a glance which one you are using.
- `PROVEN_LIT("hello, ")` **costs nothing at runtime.** It is a compile-time `sizeof` on a string literal. `proven_u8str_view_from_cstr(s)` exists for the case where the text comes from elsewhere, and that one really does scan for the NUL — the two are spelled differently because one is free and the other is *O(n)*.
- **The `destroy` takes the allocator again.** The string does not remember which allocator made it; you must pass the same one. That keeps the string small and makes the dependency visible, and it is a real hazard — see §5.

---

4. Building and including

There is one header for everything:

```
#include "proven.h"
```

It pulls in the whole public API. If you prefer to include only what you use, every module has its own header under `include/proven/` — `#include "proven/u8str.h"`, `#include "proven/fs.h"` and so on — and §14 below maps every header to the chapter that documents it.

Compiling by hand, which is all this library needs:

```
gcc -std=c2x -Iinclude -Iplatform your_program.c src/proven/*.c platform/*.c -o your_program
```

`src/proven/` is the portable library. `platform/` is the small layer that talks to the operating system — the **PAL**, or platform abstraction layer. Everything that makes a syscall lives there and nowhere else, which is why the rest of the library can be compiled for a target with no operating system at all.

The repository builds itself with a C program rather than a build system:

```
cc -std=c2x -o nob nob.c     # build the build driver, once
./nob build                  # compile everything and run the whole test suite
./nob release                # the same, optimised
```

`./nob` with no arguments lists the other modes — sanitizers, freestanding, cross-compilation and the benchmark. There is no `make`, no CMake, and nothing to install.

A C23 compiler is required. The library uses C23 features deliberately, `[[nodiscard]]` most visibly. GCC 13+, Clang 16+ and recent MSVC all work; the build driver probes for `-std=c23` and falls back to `-std=c2x` for slightly older compilers.

---

5. The five contracts you will meet on every page

These five rules explain most of the library's shape. Each is stated once here and assumed everywhere else. §8 below gives the formal versions; these are the plain-language ones.

#	Contract	In one line	Chapter
1	**Errors are values**	Fallible calls return `proven_err_t`, or a `{ err, value }` struct	1
2	**Views are borrowed**	A view is a pointer + a length into memory someone else owns	3
3	**Allocation is a parameter**	If a function can allocate, it takes an allocator; if it cannot, it does not	2
4	**Caller-owned state must not be copied**	Some structs point into themselves; copying them creates dangling pointers	5
5	**Refuse, never truncate**	An operation that does not fit fails and writes nothing	3

1. Errors are values

Every fallible function hands the error back. When there is nothing else to return it is a bare `proven_err_t`; when there is a value, the value and the error travel together and the value is meaningless until you have checked the error. `PROVEN_OK` is zero, and `proven_is_ok(err)` reads better than `err == 0`.

Wrong — reading the value before the error:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
proven_u8str_append(&s.value, text);   /* wrong: s.value is garbage if create failed */
```

The whole point of pairing them is that the value is only valid on the success path. Check first, every time.

2. Views are borrowed
`proven_u8str_view_t` is a pointer and a size. It owns nothing, allocates nothing, and is free to copy — but it stops being valid the moment the thing it points into is destroyed or moved.

Wrong — a view that outlives what it points at:

```
proven_u8str_view_t name;
{
    proven_result_u8str_t owned = proven_u8str_create(alloc, 16);
    (void)proven_u8str_append(&owned.value, PROVEN_LIT("temp"));
    name = proven_u8str_as_view(&owned.value);
    proven_u8str_destroy(alloc, &owned.value);   /* the bytes are gone */
}
use(name);                                       /* wrong: dangling view */
```

This is the same lifetime bug as a dangling pointer, and it is worth stating separately because a view *looks* like a value. It is not; it is a pointer wearing a struct.

3. Allocation is a parameter
Read the signature. `proven_u8str_append(str, data)` cannot allocate, so it fails when the text does not fit. `proven_u8str_append_grow(alloc, str, data)` takes an allocator, so it can grow. Nothing in this library calls `malloc` behind your back, which is what makes it usable in an arena, in a pool, or on a device with no heap.

The corollary is the hazard: **you must destroy with the allocator you created with.** The object does not remember.

Wrong — mismatched allocators:

```
proven_result_u8str_t s = proven_u8str_create(arena_alloc, 64);
proven_u8str_destroy(heap_alloc, &s.value);   /* wrong: heap free on arena memory */
```

Nothing checks this today. It is heap corruption that surfaces later, somewhere else.

4. Caller-owned state must not be copied
Some objects are structs you own and pass by pointer — buffered writers, line readers, directory iterators. Several of them contain a pointer to one of their own fields, so copying the struct copies a pointer that still points at the *original*.

Wrong — copying a state struct:

```
proven_writer_buf_t a = ...;
proven_writer_buf_t b = a;          /* wrong: b's internals still point into a */
```

§9.2 below lists all sixteen of these types. The rule is simple: create it where it lives, pass `&it`, and do not assign it.

5. Refuse, never truncate
When a result does not fit, this library fails the operation and leaves the destination unchanged. It does not write "as much as fits". A truncated path opens the wrong file, a truncated command runs the wrong command, and a truncated number is a different number — and every one of those is worse than an error you can see.

Where truncation genuinely is what you want, there is a separate, differently named function that tells you how much it wrote:

```
proven_result_size_t r = proven_u8str_append_partial(&s, huge);
/* r.value is how many bytes were actually appended. Reading it is the point. */
```

Wrong — assuming the two behave alike:

```
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the count WAS the result */
```

Everything up to here is the plain-language introduction. The rest of this chapter is the reference the other chapters lean on: the library's intent and build model stated formally, the global contracts, the ownership matrix, the behaviour classes, the reading order, platform support, and the appendices. It used to be a document of its own, the manual's front page; it lives here now so that the front page is nothing but the table of contents.

6. Intent and design philosophy

`proven` is a compact C23 systems foundation library. It is intended for C programs that want practical infrastructure without hiding memory ownership, error control flow, or platform access behind global state.

It is not a libc replacement. It provides a focused set of allocator-driven memory tools, byte views, containers, strings, formatting, scanning, hashing (FNV, SipHash, CRC-32, SHA-256), hex/Base64 encoding, OS-strength randomness, filesystem helpers, buffered streams, time helpers, memory mapping, stackless coroutine macros, and a bounded job system.

Core design principles:

- C23 first: the build driver uses `-std=c23`.
- Explicit errors: fallible functions return `proven_err_t` or `proven_result_*_t`.
- Explicit ownership: owned objects have clear destroy functions and allocator rules.
- Failure atomicity: grow/realloc-style APIs preserve the old object on allocation failure unless documented otherwise.
- Pointer provenance discipline: raw object access uses `proven_byte_t` and bounded views.
- PAL isolation: hosted OS services live under `platform/` and are called through public wrappers.
- Core containers do not add hidden locks. Shared mutation requires caller synchronization.
- The build system is a single checked-in `nob.c`; tests are plain C executables.

7. Build and include model

There is nothing to install

This library has no `configure`, no CMake, no package to fetch, and no shared object to place somewhere. It is C source: you compile it into your program alongside your own files.

That is a deliberate choice and it costs something. You do not get a system package, and updating means pulling new source rather than bumping a version constraint. What you get is that the library cannot be a different version than the one you are looking at, cannot pick up a build flag you did not choose, and cannot fail to link because a distribution built it with different options. For a library that is meant to run on hosted systems *and* on bare metal, "compile it with your program" is the only model that works in both places.

Two directories matter. `src/proven/` is the portable library — no OS calls anywhere. `platform/` is the small layer that makes syscalls, and it is the only part a new target has to replace. A freestanding build simply leaves the hosted files out; see the freestanding guide.

The build driver, `nob.c`, is a C program rather than a build system, and you compile it the same way you compile everything else here. It is checked into the repository, so there is no bootstrap step and no version of it to install either.

A C23 compiler is required — GCC 13+, Clang 16+, or recent MSVC. The driver probes `-std=c23` and falls back to `-std=c2x` for compilers that still use the transitional spelling.

Build and run the hosted test suite:

```
cc nob.c -o nob
./nob build
```

Common validation commands:

```
./nob release
./nob strict
./nob strict-error
./nob asan
./nob ubsan
./nob tsan
./nob regression
./nob regression-asan
./nob regression-ubsan
./nob freestanding
./nob cross -build-root build-out/proven_c_lib
```

Use the umbrella header when you want the full hosted API:

```
#include "proven.h"
```

Use smaller includes when you want a narrower translation unit:

```
#include "proven/heap.h"
#include "proven/u8str.h"
#include "proven/fmt.h"
```

A direct hosted application build can follow the same source layout as `nob.c`:

```
cc -std=c23 -D_DEFAULT_SOURCE -D_POSIX_C_SOURCE=200809L \
  -Iinclude -Iplatform \
  app.c src/proven/*.c platform/proven_sys_*.c \
  -pthread -o app
```

8. Global contracts

8.1 Result contract

A result value is usable only when `err == PROVEN_OK`.

Correct:

```
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 128, PROVEN_DEFAULT_ALIGNMENT);
if (proven_is_ok(r.err)) {
    /* only now does r.value mean anything */
    proven_mem_mut_t mem = r.value;
    (void)proven_mem_copy(mem.ptr, mem.size, proven_mem_view_from_u8(PROVEN_LIT("hi")));
    alloc.free_fn(alloc.ctx, mem.ptr);
}
```

Wrong (does not compile as shown - the value is read before the error is checked):

```
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 128, PROVEN_DEFAULT_ALIGNMENT);
use_bytes(r.value.ptr, r.value.size); /* wrong: r.err was not checked */
```

8.2 Public struct contract

Many public structs expose layout for C usability. This does not mean arbitrary field mutation is supported. Treat direct mutation of internal fields as caller misuse unless the header explicitly allows it.

Wrong:

```
arr.len = 999;      /* wrong: breaks array invariants */
str.internal.cap=0; /* wrong: breaks string invariants */
```

Use the functions and macros that maintain invariants.

8.3 Borrowed view contract

Views do not own memory. A `proven_u8str_view_t`, `proven_u16str_view_t`, `proven_mem_view_t`, or `proven_mem_mut_t` is valid only while the referenced storage remains alive and unmoved.

Wrong:

```
proven_u8str_view_t v = proven_u8str_as_view(&s);
proven_u8str_append_grow(alloc, &s, PROVEN_LIT("more"));
use_view(v); /* wrong: growth may reallocate s */
```

8.4 Allocator contract

A `proven_allocator_t` is valid only when all three function pointers are present. Reallocation must be failure-atomic: if it fails, the old allocation remains valid.

```
proven_allocator_t heap = proven_heap_allocator();
if (!proven_alloc_is_valid(heap)) {
    /* no usable allocator here: e.g. the freestanding heap stub */
    proven_panic("no heap allocator on this target");
}
```

8.5 PAL boundary contract

Code under `src/proven/` should not depend on OS headers directly. Hosted services should route through `platform/proven_sys_*.[ch]` and public wrappers such as `proven_fs_*`, `proven_sysio_*`, `proven_time_*`, and `proven_mmap_*`.

Application code should prefer public APIs. Direct PAL calls are for porting and platform integration.

9. Ownership and destruction matrix

There are **two kinds of object** in this library, and the difference decides what you must do with them.

Owning objects hold storage they allocated, and you must destroy them. They are the table immediately below.

Caller-owned state objects allocate nothing. They are scratch structs you declare — usually on the stack — and hand to a constructor, which returns a small by-value handle that points *into* them. They have **no destroy function**, because there is nothing to free. What they have instead is a rule that owning objects do not have, and it is the one that bites:

> **A caller-owned state object must not be copied or moved once a handle has been made from it.** The handle holds a pointer into the struct. Copy the struct and the handle still addresses the original; let the original go out of scope and the handle points at dead memory.

They are listed in §4.2.

9.1 Owning objects — you must destroy these

Object	Owns storage	Stores allocator	Destroy function	Notes
`proven_arena_t`	no — caller owns the backing slice	no	`proven_arena_destroy(&arena)` (no-op)	Bump pointer over caller memory: `alloc` advances an offset, `free` is a no-op, `reset` rewinds to

				empty. The caller owns/frees the backing block.
`proven_pool_t`	yes — items + recycle bin	yes (`base_alloc`)	`proven_pool_destroy(&pool)`	Fixed item size. You use it **through the allocator trait** (`proven_pool_as_allocator`): the trait's `free_fn` returns a slot to the recycle bin for reuse rather than freeing it. There is no `proven_pool_free` — freeing goes through the trait, like every other allocator.
`proven_buf_t`	yes	no	`proven_buf_destroy(alloc, &buf)`	Caller must pass the matching allocator.
`proven_u8str_t`	yes	no	`proven_u8str_destroy(alloc, &str)`	Always NUL-terminated when valid.
`proven_u16str_t`	yes	no	`proven_u16str_destroy(alloc, &str)`	Tracks byte length internally; API length is in `proven_u16` units.
`proven_array_t`	yes	yes	`proven_array_destroy(&arr)` or `PROVEN_ARRAY_DESTROY(&arr)`	Pointers into elements may be invalidated by growth.
`proven_ring_t`	yes	yes	`proven_ring_destroy(&ring)` or `PROVEN_RING_DESTROY(&ring)`	Fixed capacity; no growth.
`proven_map_t`	yes	yes	`proven_map_destroy(&map)` or `PROVEN_MAP_DESTROY(&map)`	Borrowed U8 keys are not copied.
`proven_array_t` from `proven_fs_list()`	yes	yes plus owned entry names	`proven_fs_list_destroy(alloc, &list)`	Do not use plain array destroy; entry names need cleanup.
`proven_mmap_t`	OS mapping	OS handle state	`proven_mmap_destroy(&map)`	Views into the mapping die with the mapping.

`proven_job_sys_t *`	yes	internal	`proven_job_system_close(sys)` then `proven_job_system_destroy(sys)`	Destroy must not race with producers.

9.2 Caller-owned state — no destroy, do not copy

These allocate nothing and free nothing. You declare one, pass its address to a constructor, and use the small handle you get back. The struct must **outlive the handle**, and must **not be copied or moved** while the handle is alive.

State object	Constructed by	The handle it backs	Notes
`proven_sha256_t`	`proven_sha256_init`	(used directly)	A hashing context. Safe to copy *before* you start, meaningless to copy mid-stream unless you intend to fork the hash.
`proven_xoshiro256ss_t`	`proven_xoshiro256ss_seed`	`proven_rng_t`	Copying it **clones the sequence** — deliberate and useful for a replay, a bug anywhere else.
`proven_chacha_rng_t`	`proven_chacha_rng_seed` / `_seed_from_entropy`	`proven_rng_t`	Copying it clones the keystream: two "independent" tokens become the same token.
`proven_writer_buf_t`	`proven_writer_from_buffer`	`proven_writer_t`	**Do not copy.**
`proven_writer_u8str_t`	`proven_writer_from_u8str`	`proven_writer_t`	**Do not copy.** The string and allocator must also outlive it.
`proven_writer_buffered_t`	`proven_writer_buffered`	`proven_writer_t`	**Do not copy.** You must `proven_writer_flush` before it or its buffer dies.
`proven_reader_view_t`	`proven_reader_from_view`	`proven_reader_t`	**Do not copy.**
`proven_reader_buffered_t`	`proven_reader_buffered`	`proven_reader_t`	**Do not copy.** Views returned by `proven_reader_read_line` point *into* its buffer.
`proven_sysio_std_t`	`proven_sysio_stdout_writer` / `_stderr_writer` / `_stdin_reader`	`proven_writer_t` / `proven_reader_t`	**Do not copy.** Holds the standard handle the writer points at.
`proven_sysio_out_t`	`proven_sysio_stdout_buffered` / `_file_buffered`	`proven_writer_t`	**Do not copy.** Must be flushed.
`proven_sysio_lines_t`	`proven_sysio_lines_open` / `_stdin_lines`	(used via `proven_sysio_read_line`)	The one exception: `proven_sysio_read_line` re-binds it on every call,

			so this one **may** be moved.
`proven_sysio_scanner_t`	`proven_sysio_scanner_init`	(used directly)	The exception in the other direction: this one **does** own a buffer, and you must call `proven_sysio_scanner_deinit`.

Wrong — the copy looks harmless and is a use-after-free:

```
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);

proven_sysio_out_t saved = out;   /* wrong: `w` still points into `out`, not `saved` */
use_elsewhere(&saved);            /* and if `out` goes out of scope, `w` is dangling */
```

Wrong — returning the state by value from a factory function does the same thing:

```
proven_sysio_out_t make_logger(void) {
    proven_sysio_out_t out;
    proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);
    (void)w;
    return out;   /* wrong: any writer made from `out` addresses this dead frame */
}
```

Correct — the state stays put, and the handle travels:

```
proven_byte_t buf[512];
proven_sysio_out_t out;                                  /* lives as long as `w` */
proven_writer_t w = proven_sysio_stdout_buffered(&out,
    (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });

(void)proven_fprintln(w, "one syscall, not {}", PROVEN_ARG(100));
(void)proven_writer_flush(w);                            /* or the bytes never happened */
```

10. Operation behavior classes

One operation, three honest answers

"Append this text to that string" has three defensible behaviours when the text does not fit, and most libraries pick one and hide the choice. This library exposes all three, gives them different names, and puts the difference in the signature — because which one you want depends on what the text *is*, and only the caller knows that.

Consider appending to a filesystem path:

- If the path does not fit, **truncating is catastrophic.** `documents/report.pdf` becomes `documents/rep`, which is a different, possibly existing, file. The operation must fail and change nothing.
- Now consider appending to a log line. If it does not fit, **truncating is fine** — you would rather have most of the message than none of it — as long as you are told how much was written.
- And appending to a buffer you are building up, where you would simply like it to **grow**.

The three classes below are those three answers. The one you get is decided by the function name and the presence of an allocator parameter, never by a flag or a global:

- `proven_u8str_append(str, data)` — no allocator, so it cannot grow: **refuses**.
- `proven_u8str_append_partial(str, data)` — returns a count: **truncates and tells you**.
- `proven_u8str_append_grow(alloc, str, data)` — takes an allocator: **grows**.

The default across the library is the first one, and Chapter 0 §5 explains why: a truncated path opens the wrong file, a truncated command runs the wrong command, and a truncated number is a different number.

Wrong — ignoring the count from the truncating form:

```
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the count WAS the answer */
```

Several APIs intentionally expose three behavior classes:

Class	Example	Behavior on insufficient capacity
Atomic fixed-capacity	`proven_u8str_append`, `proven_u16str_append`, `proven_u8str_append_fmt`	Return an error and leave the old object unchanged.
Best-effort truncating	`proven_u8str_append_partial`, `proven_u16str_append_partial`, `proven_u8str_append_fmt_trunc`	Write as much as fits, preserve a valid object, report how much was written.
Atomic growable	`proven_u8str_append_grow`, `proven_u16str_append_grow`, `proven_u8str_append_fmt_grow`	Grow with an allocator; on allocation failure, leave the old object unchanged.
Refuse, never truncate	`proven_hex_encode`, `proven_base64_encode`, `proven_base64_decode`, `proven_reader_read_line`	Write **nothing** and return `PROVEN_ERR_OUT_OF_BOUNDS`. A half-encoded string or a shortened line is a wrong answer that looks like a right one, so these do not produce one. Size the buffer with the module's own size function (`proven_base64_encoded_size`, ...), not by eye.

Choose the class deliberately. Do not treat a truncating function as an all-or-nothing function.

Wrong — assuming the truncating and the atomic form behave alike:

```
/* `_partial` wrote what fit and told you so; the error you did not read is the
   difference between "all of it" and "some of it". */
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the result was the point */
```

11. Manual chapters

Just finished an introductory C book? Start with the tutorial. It is six short programs, each introducing exactly one idea, ending with the greeting program from this chapter read line by line. This chapter shows that program on its first page and it carries five new ideas at once; the tutorial hands them over one at a time.

Otherwise start with this chapter. It is the only chapter that assumes nothing: why the library exists, argued from the C bugs it is answering; a hello-world program; how to build; the five contracts the rest of the manual takes for granted; and a glossary plus a libc-to-`proven` table. Every other chapter assumes it.

The reading order

The chapters are grouped into parts, and the parts are ordered so that each one only needs the ones before it. That order is not the same as the header dependency graph, and it is not arbitrary: strings need allocators, containers need allocators, and hosted services need strings, so the sequence below is what the material itself requires.

The chapter *numbers* are stable identifiers, not a reading sequence. Two of them are out of order on purpose: the alias index is an appendix you look things up in, and Chapter 8 is a reference you read after Chapter 3 has introduced the subject.

Part	Read	Prerequisites	You can then
I — Start here	tutorial → 0	One introductory C book	Build against the library and read anything below
II — The vocabulary every program uses	1 → 2 → 3	Chapter 0	Handle errors as values, own memory deliberately, hold text safely
III — Data structures	4	Part II	Arrays, maps, lists, rings, sorting, searching, hashing, encoding
IV — Text in and out	8	Chapter 3 §3–§4	Format and parse anything, and teach the formatter your own types
V — Talking to the operating system	5	Part II	Files, directories, streams, standard I/O, time, randomness, mapping
VI — Going further	6 → freestanding	Parts II–V	Coroutines, jobs, thread-safety, bare metal, cross builds
Appendices	A, B, C, D	—	Look things up

The chapters

- **Tutorial**: the library in six short programs, one new idea at a time — *Part I, optional on-ramp*

1. **Start here**: why this exists, hello world, the five contracts, glossary, libc map — *Part I*
2. **Foundation**: types, errors, memory views, alignment, version, panic — *Part II*
3. **Allocation**: heap, arena, pool, byte buffers, and the allocator trait — *Part II*
4. **Strings and text**: U8, U16, and an introduction to formatting and scanning — *Part II; the tutorial half of the text material*
5. **Containers and algorithms**: array, list, ring, map, sort/search, hashing, encoding — *Part III*
6. **Hosted services**: filesystem, tree walk, streams, sysio, environment, randomness, mmap, time — *Part V*
7. **Execution and platform**: coroutines, jobs, thread-safety, aliases, PAL, cross builds — *Part VI*
8. **Appendix A — Alias index**: every `alias_xcv.h` spelling — *reference only; not reading material*
9. **Formatting and scanning**: the full `fmt.h` and `scan.h` reference — *Part IV; the reference half of the text material*

Chapters 3 and 8 both cover the formatter and the scanner, and the division is deliberate. Chapter 3 introduces them alongside strings, with the everyday cases and enough to be productive. Chapter 8 is the complete reference: the full format grammar, every argument constructor, the scanner's error codes and recovery rules, and how to teach the formatter a type of

your own. Read Chapter 3 first; reach for Chapter 8 when you need the exact behaviour of a specifier or a failure.

12. Platform support and verification

Primary verified hosted target:

- Linux x86_64 with GCC or Clang in C23 mode.

Compile-only cross coverage exists when the corresponding toolchains are installed:

- Linux AArch64.
- Linux ARM hard-float.
- Linux i686 through `i686-linux-gnu-gcc` or `gcc -m32` multilib.
- Windows x86_64 and i686 through MinGW/WinAPI paths.
- ARM Cortex-M freestanding.
- RISC-V ELF freestanding.

The cross matrix checks compilation, public header visibility, and target ABI assumptions. It does not replace runtime validation on the target platform.

Freestanding mode builds a reduced subset with OS-backed services removed. See `manual-freestanding.md` and Chapter 6 for details.

13. Appendix B: glossary

Terms this manual uses as if they were ordinary words. They are not ordinary C words, and every one of them is load-bearing.

Two rules the chapters follow, so that nothing here has to be guessed at:

- **No private vocabulary.** Where a widely used word exists for an idea, the manual uses that word. Where the library has invented something, it is defined here.
- **Every abbreviation is written out the first time it appears in a chapter**, and every one of them is listed below as well.

The Korean edition adds one more: an English term appears with its original spelling in parentheses the first time each chapter uses it — 뷰(view) — so a reader who learned the idea in English can find it, and a reader who did not is never handed an untranslated word.

Term	Meaning here
owned	You are responsible for destroying it. Comes from a `_create`, goes to a `_destroy`, exactly once.
borrowed	Points at memory someone else owns. Never destroyed. Valid only while the owner is.
view	A borrowed pointer + length pair, e.g. `proven_u8str_view_t`. Copyable, non-owning, no allocation.
allocator	A value carrying four things: a context pointer and three function pointers (alloc, realloc, free). Passed by value into anything that may allocate.
arena	An allocator that hands out memory by bumping a pointer through one block. Individual frees do nothing; you reset or destroy the whole arena at once. Fast, and perfect for "many small things with the same lifetime".

pool	An allocator for many objects of one fixed size, with a free list, so freeing really does recycle a slot.
trait	A struct of function pointers used as an interface — C's answer to a virtual table. `proven_allocator_t`, `proven_writer_t` and `proven_rng_t` are traits. Not a C keyword; borrowed terminology.
PAL	Platform abstraction layer: the code under `platform/` that makes actual syscalls. The only OS-dependent part.
freestanding	A build with no operating system and no libc — bare metal. `PROVEN_FREESTANDING` selects it.
failure atomicity	If an operation fails, it changes nothing. A failed grow leaves your old data intact and valid.
provenance	Which allocation a pointer came from. C's optimizer assumes pointers from different allocations never overlap; violating that is undefined behaviour, not merely surprising. Chapter 6 covers it.
UB (undefined behaviour)	Not "unpredictable output" — the standard imposes no requirement at all, and optimizers are allowed to assume it never happens. This is why UB can delete your `if`.
`[[nodiscard]]`	A C23 attribute. The compiler errors if you throw the return value away. Used on every function whose error you must not drop.
fixed-capacity	Will not grow. Fails when full. Takes no allocator.
growable	Will reallocate when full. Takes an allocator. Always spelled `_grow` in the name.
CSPRNG	Cryptographically secure pseudo-random number generator: output an attacker cannot predict even after seeing earlier output.
intrusive	The list's links live *inside* your struct rather than in separately allocated nodes. No allocation per element.
code unit	One element of an encoding: a byte in UTF-8, a 16-bit value in UTF-16. Not a character — one character can take several.
code point	One character's number in Unicode. A code point takes one to four bytes in UTF-8, and one or two code units in UTF-16 — which is why counting either one is not counting characters.
API (application programming interface)	The set of functions and types a library exposes for other programs to call. In this library, everything declared in `include/proven/`.
hosted	A build that has an operating system and a C standard library under it — the opposite of *freestanding*.

heap	The general-purpose pool of memory `malloc` hands out from. `proven_heap_allocator()` is the allocator that uses it.
slice	A borrowed pointer + length pair you may **write** through, e.g. `proven_mem_mut_t`. A *view* is the read-only form of the same idea.
result	A small struct carrying an error code and a value together, e.g. `proven_result_u8str_t`. The value means nothing until the error beside it has been checked.
panic	The deliberate stop taken when a failure has no caller to return to. `proven_set_panic_handler()` chooses what happens; the default stops the program.
reserve	Raise a container's capacity **now**, so later growth does not reallocate. Saves copying on the heap, and dead storage in an arena.
dangling	A pointer to memory that has been freed or moved. Using one is undefined behaviour; the usual cause here is holding a pointer across a call that may reallocate.
use-after-free	Reading or writing through a dangling pointer. The sanitizers below detect it.
sanitizer	A compiler mode that adds run-time checks: **ASan** (AddressSanitizer) finds memory errors, **UBSan** (UndefinedBehaviorSanitizer) finds undefined behaviour, **TSan** (ThreadSanitizer) finds data races. `./nob asan`, `./nob ubsan`, `./nob tsan`.
partial write / **short read**	A single write or read that moved *fewer* bytes than asked for. Normal, not an error — and treating a short read as end of input is the classic way to lose the tail of a file.
EOF (end of file)	There is no more input. Reported as `PROVEN_ERR_EOF`, never as a zero-byte success, so it cannot be confused with "nothing arrived yet".
flush	Push a buffered writer's accumulated bytes onward. Nothing in this library flushes on your behalf at exit.
back-pressure	Slowing a producer down because the consumer cannot keep up. `proven_writer_write_partial()` is the call that lets you notice and react.
durability	The guarantee that data survives a power cut. Reaching the operating system is not enough: `proven_fs_sync()` puts a file's bytes on the storage device, and `proven_fs_sync_dir()` does the same for a rename.
atomic rename	Replacing a file by renaming a finished temporary over it. A reader sees the whole old file or the whole new one, never a half-written mixture.

advisory lock	A lock that excludes only the processes that also ask for it (`proven_fs_lock()`). It is a convention between cooperating programs, not access control.
hard link	A second **name** for the same file. There is no original; the data lives until the last name is removed. Same filesystem only.
symbolic link	A small file that holds a path. It may cross filesystems, and it may point at nothing — following it then fails.
memory mapping	Making a file's contents appear at an address, so the processor reads it as memory (`mmap.h`). **SHARED** mappings write back to the file; **PRIVATE** ones are *copy-on-write*: the writes exist in your process and nowhere else.
copy-on-write	Sharing memory until somebody writes, at which point that writer gets a private copy. What `PROVEN_MMAP_PRIVATE` does.
cursor	The scanner's position in the text it is reading (`proven_scan_t.cursor`). Scanning advances it; `proven_scan_skip_*` moves it deliberately.
locale	The system's idea of local conventions — including whether the decimal separator is `.` or `,`. This library's number parsing is **locale-free**: a comma is never a decimal point, on any machine.
entropy	Genuinely unpredictable bits, from the operating system or from hardware. A generator is *seeded* from entropy; a clock or a counter is not entropy, however random it looks.
seed	The starting value of a generator. The same seed replays the same sequence — essential for a reproducible test, and fatal for a key.
PRNG / CSPRNG	A pseudo-random number generator computes a sequence from a seed. A **C*ryptographically** S*ecure one (CSPRNG) is additionally unpredictable to an attacker who has seen earlier output.
HashDoS	An attack that feeds a hash table keys chosen to collide, turning *O(1)* lookups into $O(n^2)$. `proven_map_create()` defends against it with a keyed hash; `proven_map_create_trusted()` opts out for keys you choose yourself.
open addressing	The map's layout: entries live in one flat bucket array and a collision moves to the next slot, rather than following a chain of separately allocated nodes.
tombstone	The marker left where a map entry was removed, so lookups keep probing past it. Tombstones count towards the load factor, which is why heavy remove traffic still triggers a rehash.

rehash	Rebuilding the bucket array at a new size. It invalidates every pointer a previous `get_mut` returned, which is why you hold the key rather than the pointer.
checksum	A short value that detects accidental corruption — `proven_crc32()`. It detects accidents, never tampering.
digest / **hash**	A fixed-size value computed from data. **SHA-256** is a cryptographic digest (tamper-evident); **FNV-1a** and **SipHash-2-4** are table hashes, and only SipHash is keyed.
hex / **Base64** / **Base64URL**	Ways to write bytes as text. Hex is two characters per byte; Base64 packs three bytes into four characters with `+` `/` `=`; Base64URL uses `-` `_` and no padding, so it is safe in a URL or filename.
padding	The `=` characters Base64 adds so the output length is a multiple of four. Base64URL leaves them out.
BMP (Basic Multilingual Plane)	The first 65,536 Unicode code points. A character outside it — an emoji, many rarer CJK characters — needs two UTF-16 code units.
NUL terminator	The zero byte that marks the end of a C string. A *view* does not have one, which is exactly why it carries a length instead.
shortest round trip	Printing the fewest digits that read back as exactly the same floating-point value. `PROVEN_FLOAT_FORMAT_MODE_SHORTEST` asks for it; it is what a serialiser wants.
dispatch macro	A macro built on C11 `_Generic` that picks a function from an argument's type — `PROVEN_ARG(x)` and `PROVEN_SCAN_ARG(&x)`. It chooses among the named constructors; it is not itself one.
identity constructor	`proven_arg_identity()` / `proven_scan_arg_identity()`: they take an argument that has already been built and pass it through, so macro-driven code can accept one.
scratch allocator	An allocator passed for temporary working memory only, separate from the one that owns the result — e.g. the `scratch` parameter of `proven_map_set_with_scratch()`.
stackless coroutine	A function that can suspend and resume without a stack of its own: its state lives in a struct you hold. `coro.h` implements it with a switch, so it costs no thread and no allocation.
bounded queue	A queue with a fixed capacity that refuses when full rather than growing. The job system uses one, so a producer that outruns the workers is told so instead of exhausting memory.
monotonic clock	A clock that only moves forward, for measuring how long something took. Distinct from the wall clock,

	which can jump when the system time is corrected — measuring a duration with the wall clock is how a negative elapsed time happens.

14. Appendix C: public header map

How to find things

There are 35 public headers and one umbrella. `#include "proven.h"` pulls in everything and is what the examples in this manual do; including individual headers is for when you care about compile time or want the dependency to be visible in the file.

Two things this table tells you that the file names do not:

- **Which chapter documents it.** Every header has exactly one chapter that explains it, and the build enforces that every public function is named somewhere under `manual/` — so if a symbol is not in the chapter you expect, it is in the manual somewhere and this map says where.
- **Whether it survives a freestanding build.** The headers assigned to Chapter 5 are the hosted ones: they need a filesystem, standard streams, a clock, virtual memory or threads. Everything else compiles with no operating system. The freestanding guide has the authoritative per-module table.

Header	Main purpose	Chapter
`proven.h`	Umbrella include	This file
`types.h`	Fixed-width aliases, checked arithmetic, error enum	Chapter 1
`error.h`	Error predicate helpers	Chapter 1
`memory.h`	Byte views, slicing, range checks, memcmp	Chapter 1
`align.h`	Alignment constants and align-up helpers	Chapter 1
`version.h`	Version macros	Chapter 1
`panic.h`	Registerable panic handler	Chapter 1
`config.h`	Compile-time feature toggles (`PROVEN_FREESTANDING`, `PROVEN_FMT_NO_FLOAT`, `PROVEN_NO_U16STR`, …)	Chapters 1 and 6
`allocator.h`	Allocator trait	Chapter 2
`heap.h`	PAL-backed heap allocator	Chapter 2
`arena.h`	Bump allocator	Chapter 2
`pool.h`	Fixed-size recycler allocator	Chapter 2
`buffer.h`	Fixed-capacity byte buffer	Chapter 2
`u8str.h`	Owned U8 string and borrowed U8 views	Chapter 3
`u16str.h`	Owned U16 string and borrowed U16 views	Chapter 3
`fmt.h`	Structural formatter and format arguments	Chapter 3
`scan.h`	Structural scanner and typed scan destinations	Chapter 3
`float_parse.h`	Locale-free decimal → `double`/`float` parser (`proven_strtod`, `proven_parse_double_ascii`)	Chapter 8
`float_format.h`	`double`/`float` → decimal formatter (fixed `%f`/`%e`, shortest)	Chapter 8
`float_config.h`	Float-engine tuning (`PROVEN_FLOAT_BIGINT_LIMBS`, precision caps)	Chapters 6 and 8
`array.h`	Generic growable vector	Chapter 4
`list.h`	Intrusive doubly-linked list	Chapter 4

ring.h	Fixed-capacity FIFO ring	Chapter 4
map.h	Open-addressing map	Chapter 4
algorithm.h	Array sort and search helpers	Chapter 4
hash.h	FNV-1a, SipHash-2-4, CRC-32, SHA-256, by use case	Chapter 4
encode.h	Hex and Base64 (standard + URL-safe), bytes to text and back	Chapter 4
fs.h	Files, directories, metadata, links, locks, read-all, tree walk	Chapter 5
stream.h	Buffered writers, readers, and a line reader — and, through sysio.h, the standard streams (hosted-only)	Chapter 5
sysio.h	Standard streams as writers/readers, line input from stdin, buffered output, printing, scanning, environment access	Chapter 5
random.h	Randomness by use case: xoshiro256** (reproducible), ChaCha20 (cryptographic), the OS CSPRNG, and unbiased range/shuffle helpers. The generators work freestanding; only the OS source is hosted.	Chapter 5
mmap.h	Memory-mapped file regions	Chapter 5
time.h	Timestamp, datetime, sleep, datetime formatting	Chapter 5
coro.h	Stackless coroutine macros	Chapter 6
job.h	Bounded worker-thread job system	Chapter 6
alias_xcv.h	Optional short alias layer and generated spelling map	Chapters 6 and 7

15. Appendix D: the libc map

If you already write C, this is the fastest way in. Every row links to the chapter that explains the trade.

You would write	Use instead	Why it differs
malloc / free	proven_heap_allocator() + _create / _destroy	The allocator is a parameter, so the same code runs on an arena or a pool. Ch 2
strcpy, strcat	proven_u8str_append, _append_grow	Sizes are known, so overruns are refused instead of performed. Ch 3
strlen	view.size	The length is already there; nothing scans. Ch 3
strcmp for equality	proven_u8str_view_eq	Works on text with embedded NULs, and does not walk past the end. Ch 3
strstr	proven_u8str_view_find	Returns an index or PROVEN_INDEX_NOT_FOUND; the search is not naive. Ch 3
strtok	*(no equivalent yet)*	strtok mutates its input and cannot be nested. A view-based splitter is designed in docs/RFC-0002.
printf	proven_println("{}", PROVEN_ARG(x))	Types come from the arguments, not from the format string. Ch 8
sprintf	proven_u8str_append_fmt	Writes into a sized destination and refuses to overrun it. Ch 8

`sscanf`	`proven_scan_*`, `proven_scan_fmt`	Reports which field failed and where the cursor stopped. Ch 8
`strtod`	`proven_parse_f64_ascii`	Correctly rounded, locale-independent, no `errno`. Ch 8
`fopen` / `fread` / `fclose`	`proven_fs_open`, `_read`, `_close`, or `proven_fs_read_all_u8str`	Explicit errors, no hidden buffering, one call for whole files. Ch 5
`fgets`	`proven_sysio_read_line`, `proven_reader_read_line`	A line that exactly fills the buffer is returned, not lost. Ch 5
`qsort`	`proven_array_sort`	Introsort: *O(n log n)* guaranteed, not quicksort's worst case. Ch 4
`bsearch`	`proven_array_binary_search`	Same shape, same comparator contract. Ch 4
`rand`	`proven_xoshiro256ss_*` or `proven_random_bytes`	Reproducible and fast, or unpredictable and secure — you pick, deliberately. Ch 5
`time` / `clock`	`proven_time_now`, `proven_time_breakdown`	Nanoseconds, explicit about wall clock versus monotonic. Ch 5
`assert`	`proven_panic` + a panic hook	Works in freestanding builds and is overridable. Ch 1

---

16. Where to go next

The chapters are ordered so that each one only needs the ones before it.

Part	Read	For
I	This chapter	The contracts and the vocabulary
II	1 → 2 → 3	Errors, memory, and text: what every program uses
III	4	Arrays, maps, lists, rings, sorting, hashing, encoding
IV	8	Formatting and scanning in full, once Chapter 3 has introduced them
V	5	Files, streams, standard I/O, time, randomness, mapping
VI	6 → freestanding	Coroutines, jobs, thread-safety, bare metal, cross builds
Appendices	A: alias index, B and D above	Looking things up

If you are an experienced C programmer in a hurry, read §15 above, then Chapter 1, then whichever chapter covers the thing you need. If you are newer, read Parts I and II in order — they are short, and everything later assumes them.

Chapter 1: Foundation — Errors, Types, and Memory Views

Part II — The vocabulary every program uses. Prerequisite: Chapter 0. After this chapter you can handle every failure this library reports, describe a region of memory without losing track of its size, and do arithmetic on sizes that cannot silently wrap.

This chapter covers `types.h`, `error.h`, `memory.h`, `align.h`, `version.h`, and `panic.h` — the pieces every other chapter is built out of. Nothing here allocates, and nothing here talks to the operating system. It is the shortest chapter that is genuinely required reading.

Table of contents

1. Errors are values

Why this is first

Every other page of this manual assumes you know how failure is reported here, so this is the one section you cannot skip.

C has never agreed with itself about how a function should report failure. `malloc` returns `NULL`. `fopen` returns `NULL`. `read` returns `-1`. `strtol` returns `0` and *maybe* sets `errno`, but only if you cleared `errno` first, because a successful call is permitted to leave junk in it. `printf` returns a negative number that almost nobody checks. Four conventions, and the only thing they share is that **ignoring them is silent and legal**:

```
char *p = malloc(n);
p[0] = 'x';                    /* wrong: malloc returns NULL on failure */

FILE *f = fopen(path, "r");
fread(buf, 1, n, f);           /* wrong: f may be NULL */

long v = strtol(s, NULL, 10);  /* wrong: 0 could be the value or the failure */
```

The deeper problem is not that these are easy to get wrong. It is that **the type does not say anything is possible.** `char *` is the same type whether or not it can be `NULL`, so nothing — not the compiler, not a reviewer skimming, not you at 2 a.m. — is prompted to check.

`errno` makes it worse by making the error *global and temporary*. It must be read at exactly the right moment; any intervening library call may overwrite it; and it is per-thread, so the fix for one bug class opened another.

What this library does instead

The error is a return value, and it has a type of its own.

When a function's only outcome is success or failure, it returns `proven_err_t` — a plain enum where `PROVEN_OK` is `0`:

```
proven_err_t err = proven_u8str_append(&s, PROVEN_LIT("hello"));
if (!proven_is_ok(err)) { /* handle it */ }
```

When a function has something to give back, the value and the error travel together in one struct, and **the value is meaningless until the error has been checked**:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
if (!proven_is_ok(s.err)) return;      /* s.value is not a string; do not touch it */
```

This is the same idea as `Result` in Rust or `std::expected` in C++23, expressed in the only way C allows — a struct returned by value. There is no allocation, no indirection, and no hidden control flow. What you give up is the ability to skip error handling silently, which is the point.

Three properties are worth stating explicitly because the rest of the library relies on them:

- `PROVEN_OK` **is zero**, so a result struct zero-initialised with `{0}` starts out meaning "success, empty".
- **Fallible functions are** `[[nodiscard]]` — a C23 attribute that makes the compiler reject code that throws the return value away. You can still ignore an error, but you have to write `(void)` in front of the call and thereby say that you meant it.
- **Failure is failure-atomic** unless a function documents otherwise: if an operation fails, it changed nothing. A failed append leaves your string exactly as it was.

Reference

```
typedef enum {
    PROVEN_OK = 0,
    PROVEN_ERR_NOMEM,
    PROVEN_ERR_OUT_OF_BOUNDS,
    PROVEN_ERR_INVALID_ENCODING,
    PROVEN_ERR_INVALID_ARG,
    PROVEN_ERR_IO,
    PROVEN_ERR_NOT_FOUND,
    PROVEN_ERR_INVALID_STATE,
    PROVEN_ERR_OVERFLOW,
    PROVEN_ERR_UNSUPPORTED,
    PROVEN_ERR_AGAIN,
    PROVEN_ERR_EOF,
    PROVEN_ERR_BUSY,
    PROVEN_ERR_PERMISSION,
    PROVEN_ERR_INVALID_FORMAT
} proven_err_t;
```

Error	Typical meaning	You usually respond by
`PROVEN_OK`	Success.	Carrying on.
`PROVEN_ERR_NOMEM`	The allocator could not provide memory.	Giving up on this operation; the object is unchanged.
`PROVEN_ERR_OUT_OF_BOUNDS`	An index, size, capacity, or range is invalid — including "this does not fit".	Growing the destination, or refusing the input.
`PROVEN_ERR_INVALID_ENCODING`	Encoded text failed validation (hex, Base64).	Rejecting the input as malformed.
`PROVEN_ERR_INVALID_ARG`	A null pointer, an invalid allocator, an impossible mode, a wrong argument count.	Fixing the call — this one usually means a bug in your code, not bad data.
`PROVEN_ERR_IO`	The OS or device failed the operation.	Retrying, or reporting to the user.
`PROVEN_ERR_NOT_FOUND`	A file, key, substring, or resource does not exist.	Taking the "absent" branch; often not an error at all.
`PROVEN_ERR_INVALID_STATE`	The object's state does not allow this operation.	Fixing the sequence of calls.

PROVEN_ERR_OVERFLOW	Integer conversion or size arithmetic overflowed.	Refusing the size; see §4.
PROVEN_ERR_UNSUPPORTED	Unavailable on this platform or build profile.	Taking a different route (e.g. a pipe cannot seek).
PROVEN_ERR_AGAIN	Not now; retry later.	Retrying, usually after waiting.
PROVEN_ERR_EOF	End of input.	Stopping the read loop — expected, not exceptional.
PROVEN_ERR_BUSY	A queue, lock, or resource is busy.	Backing off.
PROVEN_ERR_PERMISSION	Access denied.	Reporting; retrying will not help.
PROVEN_ERR_INVALID_FORMAT	A format or scan template is itself malformed.	Fixing the format string — a bug, not bad data.

```
static inline int proven_is_ok(proven_err_t err);
#define PROVEN_IS_OK(err) proven_is_ok(err)
```

`proven_is_ok` returns non-zero when `err == PROVEN_OK`. `PROVEN_IS_OK` is the macro spelling for places where you want it to look like a macro; they do the same thing.

Correct:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
if (!proven_is_ok(s.err)) {
    return;   /* nothing was created, so there is nothing to destroy */
}
(void)proven_u8str_append(&s.value, PROVEN_LIT("ready"));
proven_u8str_destroy(alloc, &s.value);
```

Wrong — testing the error as a bare truth value:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
if (s.err) {
    /* wrong: works today only because PROVEN_OK is 0. It reads as "if there is
       an error" to someone who already knows that, and as nothing at all to
       everyone else. Say what you mean: !proven_is_ok(s.err). */
    return s.err;
}
```

Wrong — the mistake this whole design exists to prevent:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
(void)proven_u8str_append(&s.value, text);   /* wrong: s.value may be garbage */
proven_u8str_destroy(alloc, &s.value);       /* wrong: destroying what was never created */
```

Reading `.value` before checking `.err` is the one error this convention cannot catch for you. The compiler will not complain — the struct exists either way — so it is worth making a habit of writing the check on the line immediately after the call.

Worked example: errors as values

This program is compiled and run by the test suite, so it cannot fall out of date. It shows the two shapes a fallible call takes — a bare `proven_err_t` when there is nothing to hand back, and a `proven_result_*_t` when there is — and why the value inside a result means nothing until you have checked the error beside it.

```
/*
 * Errors are values in proven: a fallible call hands back either an error or a
 * result, and the compiler makes you look at it. There is nothing to unwind and
 * nothing global to consult.
 */
```

```c
/* A fallible operation returns proven_err_t when it has no value to give back. */
static proven_err_t write_greeting(proven_u8str_t *out) {
    return proven_u8str_append(out, PROVEN_LIT("hello"));
}

/* When there IS a value, it comes wrapped with the error that guards it. The
 * value is only meaningful once you have checked `err`. */
static proven_result_size_t half(proven_size_t n) {
    proven_result_size_t res = {0};
    if (n % 2 != 0) {
        res.err = PROVEN_ERR_INVALID_ARG;   /* leave res.value at 0: it means nothing */
        return res;
    }
    res.err = PROVEN_OK;
    res.value = n / 2;
    return res;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- checking a plain proven_err_t ------------------------------------ */
    proven_result_u8str_t s = proven_u8str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "creating a 32-byte string should succeed");

    proven_err_t err = write_greeting(&s.value);
    if (!proven_is_ok(err)) {
        /* Nothing was appended, and the string is still valid: proven's
         * grow-style operations are failure-atomic. */
        proven_u8str_destroy(alloc, &s.value);
        return 1;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), PROVEN_LIT("hello")),
                    "the greeting should have been appended");

    /* --- checking a result struct ----------------------------------------- */
    proven_result_size_t ok = half(10);
    EXAMPLE_REQUIRE(proven_is_ok(ok.err), "10 is even, so halving it must succeed");
    EXAMPLE_REQUIRE(ok.value == 5, "half of 10 is 5");

    proven_result_size_t bad = half(7);
    EXAMPLE_REQUIRE(bad.err == PROVEN_ERR_INVALID_ARG, "7 is odd, so halving it must fail");
    /* bad.value is NOT to be read. It is 0 here, but that is an implementation
     * detail of this function, not a promise of the result type. */

    /* --- the error is impossible to drop by accident ---------------------- */
    /* proven_u8str_append is [[nodiscard]], so this would be a compile error:
     *
     *     proven_u8str_append(&s.value, PROVEN_LIT("!"));
     *
     * If you really do want to ignore a failure, you have to say so: */
    (void)proven_u8str_append(&s.value, PROVEN_LIT("!"));

    printf("greeting: %s\n", proven_u8str_as_cstr(&s.value));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

2. The types, and why they are spelled differently

Why not just use `int` and `char`?
C's built-in types are famously vague. `int` is at least 16 bits, `long` is at least 32, and `char` may be signed or unsigned depending on the compiler and the target. Code that assumes otherwise works on your machine and breaks on someone else's, and the break is usually silent.

C99 fixed the width problem with `<stdint.h>` (`int32_t`, `uint64_t` and friends). This library uses those underneath and gives them its own names, for one reason: **a single spelling that means the same thing on every target the library supports**, including the freestanding ones where `<stdint.h>` may be the only header available.

There is a second, subtler reason, and it is the one that matters in practice: `proven_u8` and `proven_byte_t` are both 8 bits, and they mean different things.

- `proven_u8` is a **number** between 0 and 255. Add it, compare it, print it.
- `proven_byte_t` is a **byte of some object's representation**. It is an alias of `unsigned char`, which is the one type C allows you to use to inspect the raw bytes of anything without invoking undefined behaviour.

Using `proven_byte_t` for raw memory is not a style preference. C's strict-aliasing rules say that reading an object through a pointer of the wrong type is undefined, with `unsigned char` as the explicit exception. Every buffer, view, and hash input in this library is `proven_byte_t` for exactly that reason.

Reference

Name	Meaning	Notes
`proven_i8`, `proven_i16`, `proven_i32`, `proven_i64`	Signed fixed-width integers	Aliases of the standard fixed-width types.
`proven_u8`, `proven_u32`, `proven_u64`	Unsigned fixed-width integers	`proven_u8` is **numeric**; use `proven_byte_t` for raw object bytes.
`proven_u16`	16-bit code unit type	Uses `char16_t` when `<uchar.h>` is available, otherwise `uint_least16_t`. The U16 string APIs use it for UTF-16 code units.
`proven_byte_t`	Byte-level object representation	Alias of `unsigned char`; the type you may legally inspect any object's bytes through.
`proven_size_t`	Size and index type	Alias of `size_t`. Unsigned — see §4 before you subtract two of them.
`proven_ptrdiff_t`	Pointer difference, signed offset	Alias of `ptrdiff_t`.
`proven_intptr_t`, `proven_uintptr_t`	Pointer-sized integers	Only for explicit pointer-to-integer work, such as the arena's range checks.

```
typedef struct {
    proven_err_t  err;
    proven_size_t value;
} proven_result_size_t;
```

`proven_result_size_t` returns either a size or an error; `value` is valid only when `err == PROVEN_OK`. It is what you get back from partial appends, file reads and writes, and size queries — anywhere the answer is "how many", and the operation could fail.

Wrong — treating a byte as a small number's storage:

```
proven_u8 *raw = (proven_u8 *)&some_struct;   /* wrong type for raw inspection */
for (size_t i = 0; i < sizeof some_struct; i++) hash ^= raw[i];
```

It happens to work on every mainstream compiler, and it is still the wrong type to have written: `proven_byte_t` is the one with the aliasing guarantee behind it, and using it documents that you are looking at representation rather than at a number.

3. Memory views: a pointer and a length, together

The problem: pointers forget
A pointer says where something starts and nothing else. Everything C has ever done about the "where does it end" half has been a convention layered on top:

- **A NUL terminator**, for strings — which costs an *O(n)* scan every time you want the length, cannot represent text containing a zero byte, and produces a buffer overrun the moment the terminator is missing.
- **A separate length parameter**, for everything else — which works right up until someone passes the wrong one, because nothing ties the two arguments together.

```
void process(const unsigned char *data, size_t len);
process(buf, sizeof buf);        /* fine */
process(buf, sizeof(buf) - 1);   /* also compiles; now the last byte is invisible */
process(other, sizeof buf);      /* also compiles; now it reads past the end */
```

Every one of those is a legal call. The relationship between the pointer and the length exists only in the programmer's head.

What this library does instead
A **view** is a pointer and a size in one struct, passed by value. They cannot get separated, because they are one thing.

```
typedef struct { const proven_byte_t *ptr; proven_size_t size; } proven_mem_view_t;  /* read-only */
typedef struct {       proven_byte_t *ptr; proven_size_t size; } proven_mem_mut_t;   /* writable  */
typedef struct {       proven_byte_t *ptr; proven_size_t size; } proven_mem_t;       /* owned     */
```

They are two words. Copying one is free, and there is no allocation anywhere in sight. What a view does *not* do is own anything: it points at bytes that belong to someone else, and it becomes invalid the moment they do. That rule is contract 2 from Chapter 0.

The three spellings differ only in intent, and the intent is the point:

- `proven_mem_view_t` — **borrowed, read-only.** The `const` is on the pointed-to bytes, so the compiler enforces it.
- `proven_mem_mut_t` — **borrowed, writable.** You may write through it; you still do not own it.
- `proven_mem_t` — **owned.** Somebody must free this with the allocator that produced it. The type does not enforce that; it announces it.

And two result wrappers, for the operations that can fail:

```
typedef struct { proven_err_t err; proven_mem_mut_t  value; } proven_result_mem_mut_t;
typedef struct { proven_err_t err; proven_mem_view_t value; } proven_result_mem_view_t;
```

`proven_result_mem_mut_t` is what an allocator hands back. `proven_result_mem_view_t` is what a *checked* slice hands back.

Slicing, checked and unchecked

Taking a sub-range of a view is the most common thing you will do with one, and it comes in two forms on purpose:

API	Purpose	Return
`proven_mem_view_from_owned(mem)`	Read-only view over an owned block.	`proven_mem_view_t`.
`proven_mem_mut_from_owned(mem)`	Writable view over an owned block.	`proven_mem_mut_t`.
`proven_mem_view_slice_checked(view, offset, size)`	Sub-view, validated.	`proven_result_mem_view_t`.
`proven_mem_view_slice_unchecked(view, offset, size)`	Sub-view, **not** validated.	`proven_mem_view_t`.
`proven_mem_mut_slice_checked(mut, offset, size)`	Writable sub-slice, validated.	`proven_result_mem_mut_t`.
`proven_mem_mut_slice_unchecked(mut, offset, size)`	Writable sub-slice, **not** validated.	`proven_mem_mut_t`.
`proven_range_contains_ptr(base, cap, ptr, size, out_offset)`	Is this pointer range inside that allocation? Integer address comparison, no pointer arithmetic on unrelated pointers.	`_Bool`.
`proven_memcmp(s1, s2, size)`	Compare raw memory.	Zero if equal; sign by byte order (unsigned).
`proven_mem_copy(dst, dst_cap, src)`	Bounded copy of a view into `dst`.	`PROVEN_OK`; `PROVEN_ERR_OUT_OF_BOUNDS` if it would not fit — **and nothing is written**; `PROVEN_ERR_INVALID_ARG` on a null pointer with a non-zero size. Regions must not overlap.
`proven_mem_move(dst, dst_cap, src)`	As `proven_mem_copy`, but the regions may overlap.	As above.

The checked form's exact behaviour:

- `view.size > 0 && view.ptr == NULL` → `PROVEN_ERR_INVALID_ARG` (a size with no memory behind it is a bug, not an empty view).
- The requested range falls outside the view → `PROVEN_ERR_OUT_OF_BOUNDS`.
- `size == 0` → an empty view (`ptr == NULL`, `size == 0`) with `PROVEN_OK`.

Use the checked form by default. The unchecked form exists for the case where you have already proved the range in the lines above — inside a loop whose bounds you computed, for instance — and the second check would be pure cost. That is a real case, and it is narrower than it feels.

Wrong — unchecked slicing on numbers that came from outside:

```
proven_mem_view_t part = proven_mem_view_slice_unchecked(view, user_offset, user_size);
/* wrong: user_offset and user_size were never validated. This is the buffer
   overrun from the top of section 3, reintroduced through a safer-looking API. */
```

Wrong — rejecting the empty view:

```
if (!view.ptr) return PROVEN_ERR_INVALID_ARG;   /* wrong: {NULL, 0} is legal */
```

An empty view is a normal value: a zero-length slice, a file with no content, a string that was just reset. The test for a *malformed* view is a null pointer with a non-zero size:

```
proven_mem_view_t view = {0};   /* size 0, ptr NULL: a legal empty view */
proven_err_t err = PROVEN_OK;

if (view.size > 0 && !view.ptr) {
    err = PROVEN_ERR_INVALID_ARG;   /* only a null pointer with bytes behind it is a bug */
}
(void)err;
```

4. Size arithmetic that cannot wrap

Why a multiplication needs a function call

`proven_size_t` is unsigned, and unsigned arithmetic in C does not overflow — it *wraps*, silently and legally, modulo 2^64. That single rule is behind a large fraction of the industry's allocation bugs:

```
proven_size_t total = count * sizeof(item_t);   /* wrong: wraps for large count */
void *p = malloc(total);                        /* succeeds, and is far too small */
items[count - 1] = x;                           /* writes way past the end */
```

With `count = 2^61` and a 16-byte item, `total` is `0`. `malloc(0)` succeeds. Every subsequent write is out of bounds, and nothing anywhere reported an error. The same shape appears whenever a size is computed from data that came from outside the program — a file header, a network packet, a user-supplied count.

Subtraction has the mirror problem, and it is easier to hit:

```
proven_size_t remaining = view.size - offset;   /* wrong when offset > size: a huge number */
```

What this library does instead

Three macros that do the arithmetic and tell you whether it fit.

```
#define PROVEN_CKD_ADD(res, a, b)   /* true on overflow */
#define PROVEN_CKD_SUB(res, a, b)
#define PROVEN_CKD_MUL(res, a, b)
```

Macro	Computes	Returns	Writes `*res`
`PROVEN_CKD_ADD(res, a, b)`	`a + b`	**true on overflow**, false on success	Only when it fits
`PROVEN_CKD_SUB(res, a, b)`	`a - b`	**true on overflow** (including unsigned underflow)	Only when it fits
`PROVEN_CKD_MUL(res, a, b)`	`a * b`	**true on overflow**	Only when it fits
`PROVEN_SIZE_MAX`	—	The largest value a `proven_size_t` can hold	—

- `res` is a **pointer** to where the answer goes.
- They return **true on overflow**, false on success. That reads backwards the first time; the convention comes from C23's `<stdckdint.h>`, which these use when the compiler provides it, and from the GCC/Clang `__builtin_*_overflow` family otherwise. Read the call as *"did this overflow?"* and it comes out right.
- `PROVEN_SIZE_MAX` is there for the cases where you want to compare against the limit directly rather than compute and check.

Correct:

```
typedef struct { int id; double weight; } item_t;

proven_size_t count = 1024;
proven_size_t total = 0;
proven_err_t err = PROVEN_OK;
```

```c
if (PROVEN_CKD_MUL(&total, count, sizeof(item_t))) {
    err = PROVEN_ERR_OVERFLOW;   /* refuse to allocate a size that wrapped */
} else {
    proven_result_mem_mut_t block = alloc.alloc_fn(alloc.ctx, total, alignof(item_t));
    err = block.err;
    if (proven_is_ok(err)) {
        alloc.free_fn(alloc.ctx, block.value.ptr);
    }
}
(void)err;
```

Wrong:

```c
proven_size_t total = count * sizeof(item_t);   /* wrong: may wrap silently */
```

You do not need these for every arithmetic expression in your program. You need them wherever a size is **computed from a value you did not choose yourself** — and that is precisely where the bugs are.

5. Alignment

What alignment is, and why the library talks about it

Every type in C has an alignment: an address it must start on for the hardware to load it efficiently, or at all. A `double` typically wants an address divisible by 8. Some architectures fault on a misaligned load; x86 merely makes it slow; and either way, C says accessing an object through a misaligned pointer is undefined behaviour.

You have not had to think about this, because `malloc` returns memory suitably aligned for anything. That guarantee is exactly what disappears the moment you hand out memory yourself — and handing out memory yourself is what Chapter 2's arenas and pools do. An arena that bumps a pointer by 13 bytes and hands you the result has just given you a misaligned `double`.

So the helpers here exist for the allocators, and for you if you write one.

Reference

Macro	Meaning
`PROVEN_DEFAULT_ALIGNMENT`	The default the library uses when you do not say otherwise. Currently 8.
`PROVEN_MAX_ALIGN`	`alignof(max_align_t)` — enough for any built-in type.

Function	Purpose	Return
`proven_is_pow2(x)`	Is `x` a non-zero power of two? Alignments must be.	`bool`.
`proven_mem_align_up(addr, align)`	Round a size or address up to the next multiple of `align`.	The aligned value, or **0** if `align` is invalid or the rounding overflowed.
`proven_uintptr_align_up(addr, align)`	The same, for a `proven_uintptr_t` address.	As above.

Note the error convention: these return **0 on failure**, because they have no room for an error code. Zero is never a valid aligned address, so testing for it is unambiguous.

Correct:

```c
typedef struct { int id; double weight; } item_t;

proven_size_t size = 100;
```

```
proven_size_t aligned = proven_mem_align_up(size, alignof(item_t));
proven_err_t err = (aligned == 0) ? PROVEN_ERR_OVERFLOW : PROVEN_OK;
(void)err;
```

Wrong — the classic bit-twiddling version:

```
proven_size_t aligned = (size + align - 1) & ~(align - 1);   /* wrong: may overflow */
```

That line is in a great deal of production code and is correct for every input except the ones near the top of the range, where `size + align - 1` wraps to a small number and the result is an address *below* where you started.

Worked example: views, slicing, and alignment together

The three ideas in this chapter — a view that carries its own length, a slice that is checked before it is taken, and a boundary that has to be a power of two — only show their point when a program uses them at once. This one keeps a small table of fixed-size records in a single block it owns, and then does the four things any such program has to do: hand the table to a reader that must not modify it, hand one row to code that may modify only that row, delete a row by moving the rows after it down over it, and decide whether a pointer that arrived from somewhere else even points into the table.

Two of those are the places C is most often quietly wrong. Moving overlapping bytes is not what `proven_mem_copy` promises, so it has a separate call, `proven_mem_move`. And comparing two pointers with `<` when they may belong to different objects is undefined behaviour — the compiler is entitled to assume it never happens — so "is this pointer inside my buffer?" is asked with `proven_range_contains_ptr` rather than with `>=` and `<`.

This program is compiled and run by the test suite, so it cannot fall out of date.

```
#include <string.h>

/*
 * One owned buffer, three ways of talking about it.
 *
 * The program holds a small table of fixed-size records in a single block of
 * memory it owns, and then does the four things every such program has to do:
 *
 *   1. hand the table to a reader that must not modify it (a read-only view),
 *   2. hand one row to a writer that may modify only that row (a slice),
 *   3. delete a row by moving the rows after it down over it (an overlapping
 *      copy, which plain copying is not allowed to do),
 *   4. take a pointer somebody else returned and decide whether it even points
 *      into the table before using it as a row index.
 *
 * Every step is a place where a bare `char *` loses the length and the program
 * finds out later. The types here carry the length with the pointer, so the
 * bounds question is answered where the mistake would be made.
 */

#define ROW_SIZE  8u
#define ROW_COUNT 4u

/* A reader gets a view: it can read every byte and write none of them. The
 * const in the type is not advice - assigning through it does not compile. */
static proven_size_t count_nonzero(proven_mem_view_t table) {
    proven_size_t n = 0;
    for (proven_size_t i = 0; i < table.size; ++i) {
        if (table.ptr[i] != 0) {
            ++n;
        }
```

```
    }
    return n;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* Allocate the table. The allocator returns a read-write slice, which is
     * what an owner holds: a pointer and the size that goes with it. */
    proven_result_mem_mut_t got = alloc.alloc_fn(alloc.ctx, ROW_SIZE * ROW_COUNT, alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(got.err), "allocating the record table must succeed");
    if (!proven_is_ok(got.err)) {
        return 1;
    }

    /* proven_mem_t is the "I own this" type. Keeping the owned block in one
     * variable, and handing out views and slices derived from it, is the whole
     * ownership discipline this library asks for. */
    proven_mem_t owned = { .ptr = got.value.ptr, .size = got.value.size };
    proven_mem_mut_t table = proven_mem_mut_from_owned(owned);

    /* Fill row i with the byte value i + 1, so a moved row is recognisable. */
    for (proven_u32 row = 0; row < ROW_COUNT; ++row) {
        proven_result_mem_mut_t slot = proven_mem_mut_slice_checked(table, row * ROW_SIZE, ROW_SIZE);
        EXAMPLE_REQUIRE(proven_is_ok(slot.err), "every row of the table must be in bounds");
        if (!proven_is_ok(slot.err)) {
            alloc.free_fn(alloc.ctx, owned.ptr);
            return 1;
        }
        memset(slot.value.ptr, (int)(row + 1), slot.value.size);
    }

    /* 1. Read-only use. The reader cannot modify the table and cannot run off
     *    the end of it, because it was handed both facts at once. */
    proven_mem_view_t read_only = proven_mem_view_from_owned(owned);
    EXAMPLE_REQUIRE(count_nonzero(read_only) == ROW_SIZE * ROW_COUNT,
                    "every byte of every row was written");

    /* 2. Asking for a row that does not exist is an error you can handle, not a
     *    crash you have to debug. Row 4 of a 4-row table starts one row past
     *    the end. */
    proven_result_mem_mut_t past_end = proven_mem_mut_slice_checked(table, ROW_COUNT * ROW_SIZE, ROW_SIZE);
    EXAMPLE_REQUIRE(past_end.err == PROVEN_ERR_OUT_OF_BOUNDS,
                    "slicing past the end must report OUT_OF_BOUNDS, not return a bad slice");

    /* 3. Delete row 1 by moving rows 2 and 3 down one row. Source and
     *    destination overlap, so this is the one case plain copying is not
     *    allowed to handle: proven_mem_copy documents non-overlapping regions,
     *    proven_mem_move is the one that accepts them. */
    proven_mem_view_t tail = {
        .ptr  = table.ptr + 2 * ROW_SIZE,
        .size = 2 * ROW_SIZE
    };
    proven_err_t moved = proven_mem_move(table.ptr + 1 * ROW_SIZE, table.size - 1 * ROW_SIZE, tail);
    EXAMPLE_REQUIRE(proven_is_ok(moved), "moving the tail of the table down must succeed");
    EXAMPLE_REQUIRE(table.ptr[1 * ROW_SIZE] == 3, "row 2 moved into row 1's place");
    EXAMPLE_REQUIRE(table.ptr[2 * ROW_SIZE] == 4, "row 3 moved into row 2's place");

    /* The move is bounded like everything else: a destination too small to hold
     * the source is refused, and nothing is written. */
```

```
    proven_err_t refused = proven_mem_move(table.ptr, ROW_SIZE, tail);
    EXAMPLE_REQUIRE(refused == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a move that would not fit must be refused before it writes");

    /* 4. A pointer from elsewhere. Comparing unrelated pointers with < is
     *    undefined behaviour in C - the compiler is allowed to assume it never
     *    happens - so the question "does this pointer point into my buffer?"
     *    has to be asked with a function that does the comparison correctly. */
    proven_size_t offset = 0;
    const proven_byte_t *from_elsewhere = table.ptr + 2 * ROW_SIZE;
    bool inside = proven_range_contains_ptr(table.ptr, table.size, from_elsewhere, ROW_SIZE, &offset);
    EXAMPLE_REQUIRE(inside, "a pointer to row 2 is inside the table");
    EXAMPLE_REQUIRE(offset / ROW_SIZE == 2, "and its offset recovers the row index");

    proven_u8 unrelated[ROW_SIZE] = { 0 };
    EXAMPLE_REQUIRE(!proven_range_contains_ptr(table.ptr, table.size, unrelated, ROW_SIZE, NULL),
                    "a pointer into a different object is not inside the table");

    /* A row that starts inside but ends outside is also outside: the check is
     * about the whole range, not just where it begins. */
    EXAMPLE_REQUIRE(!proven_range_contains_ptr(table.ptr, table.size,
                                               table.ptr + table.size - 1, ROW_SIZE, NULL),
                    "a range that starts inside but runs past the end is refused");

    /* 5. Once the range check has proved the row is in bounds, the unchecked
     *    slice is the right call: it does no work, and the proof is the line
     *    above it rather than a comment. Never write it without that proof. */
    proven_mem_mut_t row2 = proven_mem_mut_slice_unchecked(table, offset, ROW_SIZE);
    EXAMPLE_REQUIRE(row2.size == ROW_SIZE && row2.ptr[0] == 4,
                    "the unchecked slice names the row the checked test just proved");

    /* 6. Alignment, in the one place a caller meets it: laying two differently
     *    aligned things out inside one block. The address of the second one has
     *    to be rounded up, and rounding up is only defined for a power-of-two
     *    boundary - so the boundary is checked first. */
    proven_size_t want_align = alignof(proven_u32);
    EXAMPLE_REQUIRE(proven_is_pow2(want_align), "an alignment boundary must be a power of two");
    EXAMPLE_REQUIRE(!proven_is_pow2(24u), "24 is not a power of two, so it is not a valid boundary");

    proven_uintptr_t raw     = (proven_uintptr_t)(table.ptr + 1);   /* deliberately odd */
    proven_uintptr_t aligned = proven_uintptr_align_up(raw, want_align);
    EXAMPLE_REQUIRE(aligned >= raw, "aligning up never moves backwards");
    EXAMPLE_REQUIRE(aligned % want_align == 0, "and the result is on the boundary asked for");
    EXAMPLE_REQUIRE(aligned - raw < want_align, "the padding is never more than one boundary");

    /* A boundary that is not a power of two returns 0 rather than a plausible
     * wrong address, so the mistake stops here instead of downstream. */
    EXAMPLE_REQUIRE(proven_uintptr_align_up(raw, 24u) == 0,
                    "aligning to a non-power-of-two returns 0, not a wrong address");

    alloc.free_fn(alloc.ctx, owned.ptr);
    return EXAMPLE_OK();
}
```

Wrong — the same deletion written with the copy that forbids overlap:

```
/* rows 2..3 moved down onto row 1: source and destination overlap */
proven_mem_copy(table.ptr + ROW_SIZE, table.size - ROW_SIZE, tail);   /* wrong */
```

`proven_mem_copy` documents non-overlapping regions. It will not report this; it will simply be free to copy in whatever order is fastest, and on the day that order changes the table quietly ends up holding two copies of one row.

Wrong — the bounds question asked with plain pointer comparison:

```
if (ptr >= table.ptr && ptr + size <= table.ptr + table.size) { /* wrong */ }
```

If `ptr` came from a different allocation, this comparison is undefined behaviour, not merely a comparison that returns the wrong answer.

6. Panic: when there is no one left to return an error to

Why a library that returns errors has a panic at all
Everything in this chapter has argued that failure should be a value you can inspect. A panic is the admission that this is not always possible.

Some APIs have no error channel by construction. `proven_arena_alloc_or_panic()` returns a memory block, not a result, because it exists for the setup phase of a program where an allocation failure means the program cannot run at all and there is nothing sensible to return. In a freestanding build there may be no `abort()`, no `stderr`, and nothing to unwind to.

So the library raises a panic by calling `proven_panic()`, which dispatches to a handler you can replace. It does not call `exit`, print to `stderr`, or assume an operating system.

```
typedef void (*proven_panic_handler_t)(const char *msg);

void proven_panic(const char *msg);
void proven_set_panic_handler(proven_panic_handler_t handler);
```

API	Purpose	Notes
`proven_panic(msg)`	Raise a panic. Dispatches to the installed handler.	Called by the library's `_or_panic` APIs; you can call it too.
`proven_set_panic_handler(handler)`	Install a handler.	Pass `NULL` to restore the default. Not thread-safe — install it during start-up, before other threads exist.
`proven_panic_handler_t`	`void (*)(const char *msg)`	The handler must not return.

The default handler traps: `__builtin_trap()` on GCC and Clang, and an infinite loop on any other compiler. The dispatch goes through a function pointer rather than a weak symbol, so it links the same way on ELF and on PE/COFF (Windows, mingw-w64) toolchains.

Installing your own (a file-scope function, so this is a listing rather than something you paste inside another function):

```
static void my_panic(const char *msg) {
    log_critical(msg);            /* your logger */
    for (;;) {
        /* reset, halt, or wait for a debugger */
    }
}

proven_set_panic_handler(my_panic);   /* pass NULL to restore the default */
```

A panic handler must not return. If it does, the `_or_panic` family has nothing to give back and the validity of its result is not guaranteed.

Wrong — a handler that returns:

```
static void my_panic(const char *msg) {
    fprintf(stderr, "%s\n", msg);   /* wrong: this returns, and then the caller
                                       proceeds with a block that was never allocated */
}
```

Wrong — reaching for `_or_panic` in ordinary code:

```
proven_mem_mut_t block = proven_arena_alloc_or_panic(&arena, n);
/* wrong for anything that handles input: an arena that is merely full has just
   killed the process. Use proven_arena_alloc and read the error. */
```

7. Version macros

Compile-time identification, for diagnostics and for code that must adapt to the library version.

```
#define PROVEN_VERSION_STRING "proven_c_lib-v26.09.04a"
#define PROVEN_VERSION_NUM    260723
#define PROVEN_VERSION_SUFFIX "d"
```

`PROVEN_VERSION_STRING` is what you print in a build report or a `--version` flag. `PROVEN_VERSION_NUM` is `YYMMDD` as an integer, for numeric comparison — `#if PROVEN_VERSION_NUM

= 260720. PROVEN_VERSION_SUFFIX` distinguishes releases made on the same day.

These three are checked against `include/proven/version.h` by the build, so the manual cannot drift from the header. (It did, once, and this sentence is why it will not again: the string was gated and the other two were not.)

8. Examples and misuse cases

Safe result handling

```
proven_byte_t record[16] = {0};
proven_mem_view_t view = { .ptr = record, .size = sizeof record };

proven_result_mem_view_t part = proven_mem_view_slice_checked(view, 4, 8);
if (!proven_is_ok(part.err)) {
    return;   /* the range was not inside the view */
}

proven_byte_t payload[8];
proven_err_t err = proven_mem_copy(payload, sizeof payload, part.value);
(void)err;
```

Unchecked slicing only after proof

Correct when the caller already proved the range:

```
proven_byte_t record[16] = {0};
proven_mem_view_t view = { .ptr = record, .size = sizeof record };
proven_size_t offset = 4;
proven_size_t size = 8;

if (offset <= view.size && size <= view.size - offset) {
    proven_mem_view_t part = proven_mem_view_slice_unchecked(view, offset, size);
    (void)part;   /* proved in range: safe to read */
}
```

Note the shape of that test. `size <= view.size - offset` is written that way, rather than the more natural `offset + size <= view.size`, precisely because of §4: the sum can wrap and the difference cannot, given the `offset <= view.size` check that precedes it.

Wrong:

```
proven_mem_view_t part = proven_mem_view_slice_unchecked(view, user_offset, user_size);
/* wrong: user_offset and user_size were not validated */
```

Empty views are allowed
A zero-size view may have a null pointer. Do not reject it blindly.

Wrong:

```
if (!view.ptr) return PROVEN_ERR_INVALID_ARG; /* wrong for empty views */
```

Better:

```
proven_mem_view_t view = {0};   /* size 0, ptr NULL: a legal empty view */
proven_err_t err = PROVEN_OK;

if (view.size > 0 && !view.ptr) {
    err = PROVEN_ERR_INVALID_ARG;   /* only a null pointer with bytes behind it is a bug */
}
(void)err;
```

Where to go next
Chapter 2 is where these types start doing work: allocators produce `proven_mem_mut_t`, arenas use the alignment helpers from §5, and every growable container in the library takes the allocator as a parameter for the reasons §1 gave for taking the error as a return value — the cost should be visible in the signature.

Chapter 2: Allocation — Heap, Arenas, Pools, and Buffers

Part II — The vocabulary every program uses. Prerequisite: Chapter 1. After this chapter you can choose an allocation strategy deliberately instead of reaching for `malloc` by reflex, and you will know which of the three costs you are paying.

This chapter covers `heap.h`, `arena.h`, `pool.h`, `allocator.h`, and `buffer.h`. The thread-safety and pointer-provenance material that used to be section 7 here now lives in Chapter 6, with the rest of the concurrency subject — it was the hardest material in the book sitting in one of the first chapters.

Table of contents

1. Why allocation is a parameter, and the heap allocator

The problem with `malloc`

`malloc` is a global. Any function may call it, nothing in a signature says whether a function does, and every call goes to the same general-purpose allocator no matter what the memory is for.

That has four consequences you have probably met:

- **You cannot tell from a signature whether a function allocates.** `char *build_message(int n)` might allocate, might return a pointer into a static buffer, might return a literal. The type is identical in all three cases, so the caller cannot know whether to free, and the answer lives in a comment that may be wrong.
- **You cannot change the strategy for one part of a program.** If a parser makes ten thousand small allocations that all die at the end of the parse, `malloc` and `free` will do ten thousand general-purpose allocations and ten thousand frees. A bump allocator would do one. There is no way to say so without rewriting every call.
- **You cannot test the failure path.** Making `malloc` fail on demand means intercepting it globally — `LD_PRELOAD`, a linker trick, a `#define malloc my_malloc` that also catches the library's internal calls. Meanwhile the branch you most want to test is the one that runs when memory runs out.
- **You cannot use it at all where there is no heap.** Firmware, a kernel, a bootloader: no `malloc`, and every library that assumes one is unusable.

`malloc` is not badly designed. It is a fixed policy, hardcoded into every call site by the fact that it takes no parameter saying otherwise.

What this library does instead

An allocator is a value, and it is a parameter. A function that can allocate takes a `proven_allocator_t`; a function that cannot, does not. That is the whole idea, and every consequence follows from it:

```
proven_err_t proven_u8str_append(proven_u8str_t *str, proven_u8str_view_t data);
proven_err_t proven_u8str_append_grow(proven_allocator_t alloc, proven_u8str_t *str, proven_u8str_view_t
data);
```

Read the signatures. The first cannot grow the string, so it fails when the text does not fit. The second can, and says so by taking the allocator. You never have to wonder which one you called.

The same code now works with three different strategies, chosen by the caller:

Allocator	Get one from	Frees individually?	Use it when
Heap	`proven_heap_allocator()`	Yes	The general case. Objects with unrelated lifetimes.
Arena	`proven_arena_create(backing)`, then `proven_arena_as_allocator(&a)`	**No** — free is a no-op; you reset or destroy the whole thing	Many allocations that all die at the same moment: one request, one frame, one parse.
Pool	`proven_pool_init(&p, base, size, align, bin_cap)`, then `proven_pool_as_allocator(&p)`	Yes, into a free list	Many objects of **one fixed size**, allocated and freed repeatedly.

Start with the heap. Reach for the other two when you have a reason, and the reason is usually a measurement.

The heap allocator

```
proven_allocator_t proven_heap_allocator(void);
```

This is `malloc`, `realloc` and `free` wearing the library's interface. It is what you should use unless you have a specific reason not to, and there is nothing clever about it — which is the point. Every example in this manual that does not have a reason to do otherwise uses it:

```
proven_allocator_t heap = proven_heap_allocator();

proven_result_u8str_t s = proven_u8str_create(heap, 64);
if (!proven_is_ok(s.err)) {
    return;   /* nothing was created, so there is nothing to destroy */
}
(void)proven_u8str_append(&s.value, PROVEN_LIT("ready"));
proven_u8str_destroy(heap, &s.value);   /* the SAME allocator */
```

The value is four words — a context pointer and three function pointers — and it is passed by value. Copying it is free, storing it in your own struct is fine, and it does not need to be destroyed.

In freestanding builds `proven_heap_allocator` does not exist. There is no `malloc` to wrap. That is not a limitation to work around; it is the reason the whole library takes allocators as parameters, and it is why an arena over a static array makes the same code run on a microcontroller. See freestanding mode.

Wrong — destroying with a different allocator than you created with:

```
proven_result_u8str_t s = proven_u8str_create(arena_alloc, 64);
proven_u8str_destroy(heap_alloc, &s.value);   /* wrong: heap free on arena memory */
```

The object does not remember which allocator produced it — that is what keeps it small. Nothing checks this today, and the failure is heap corruption that surfaces somewhere else, later. Pair them by construction: keep the allocator next to the object, or pass both together.

2. Arena: many things, one lifetime

The problem: many small allocations with the same death date

Consider parsing a configuration file. You allocate a string for each key, a string for each value, a node for each section — a few thousand small allocations. Then the parse finishes, you build your result, and every one of those allocations becomes garbage at the same instant.

With `malloc` you pay for that twice. Each allocation searches a free list, updates bookkeeping, and returns memory with a header attached; each `free` returns it and possibly coalesces neighbours. And you must not miss one, because a leak is one forgotten `free` away.

The observation an arena makes is that **all of those objects have the same lifetime**, so individual frees are pointless work. If they die together, they can be freed together.

How an arena works
An arena is a block of memory and an offset. Allocating means rounding the offset up to the alignment you asked for, returning the address, and moving the offset along. That is the entire algorithm:

```
[####used####¦                  free                  ]
             ^ offset
```

- **Allocation is a few instructions.** No search, no free list, no per-allocation header.
- `free` **does nothing at all.** It is a no-op, deliberately.
- **You reclaim by resetting or destroying the whole arena**, which sets the offset back to zero. One operation frees ten thousand objects.

The memory comes from you. `proven_arena_create` takes a `proven_mem_mut_t` — a block you got from the heap, or a `static` array, or a region on the stack. The arena never allocates on its own, which is what lets it work with no heap underneath.

What you give up
An arena is not a general-purpose allocator, and the trade is real:

- **You cannot free one object.** If the lifetimes are not actually shared, an arena leaks by design — memory is only reclaimed at reset.
- **Running out is a hard limit.** The backing block is fixed. `PROVEN_ERR_NOMEM` here does not mean the machine is out of memory, it means this arena is.
- **Every pointer into it dies at reset**, all at once, with nothing to warn you. A view that outlives the reset is a dangling view; see contract 2 in Chapter 0.

Use an arena when the shared lifetime is a fact about your program, not a hope.

`proven_arena_t`

```
typedef struct {
    proven_mem_mut_t backing;
    proven_size_t offset;
} proven_arena_t;
```

Fields:

- `backing`: mutable memory range owned by the caller.
- `offset`: next allocation position within `backing`.

Arena functions and helpers

API	Intent	Parameters	Return
`proven_arena_create(backing)`	Initialize an arena over a backing slice.	`backing`: caller-owned memory.	`proven_arena_t`.
`proven_arena_reset(arena)`	Discard all arena allocations.	`arena`.	void.
`proven_arena_destroy(arena)`	Formal cleanup.	`arena`.	void; no-op for caller-backed arenas.
`proven_arena_alloc_aligned(arena, size, align)`	Allocate with explicit alignment.	`arena`, byte `size`, power-of-two `align`.	`proven_result_mem_mut_t`.
`proven_arena_realloc_aligned(arena, old_ptr, old_size, new_size, align)`	Reallocate; can extend/shrink tail allocation in place.	old allocation details and new size.	`proven_result_mem_mut_t`.

`proven_arena_alloc(arena, size)`	Allocate with `PROVEN_DEFAULT_ALIGNMENT`.	`arena`, byte `size`.	`proven_result_mem_mut_t`.
`proven_arena_alloc_aligned(arena, size, align)`	Allocate or call panic hook.	`arena`, `size`, `align`.	`proven_mem_mut_t`.
`proven_arena_alloc_or_panic(arena, size)`	Default-aligned panic allocation.	`arena`, `size`.	`proven_mem_mut_t`.
`proven_arena_as_allocator(arena)`	Expose arena as `proven_allocator_t`.	`arena`.	allocator trait, or zero allocator for null arena.

Trait adapter helpers:

- `proven_arena_alloc_trait`
- `proven_arena_realloc_trait`
- `proven_arena_free_trait`

These are exposed because the allocator trait needs function pointers. Application code normally calls `proven_arena_as_allocator()` instead.

Example:

```
alignas(max_align_t) proven_byte_t storage[4096];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});

proven_result_mem_mut_t r = proven_arena_alloc(&arena, 64);
if (!proven_is_ok(r.err)) {
    return;   /* the arena cannot grow: it reports NOMEM instead */
}

proven_arena_reset(&arena);   /* reclaims r and everything else at once */
proven_arena_destroy(&arena);
```

Arena with growable containers

Growable containers can use an arena allocator, but growth may abandon earlier blocks because arena `free` is a no-op. Reserve capacity up front when possible.

Correct:

```
alignas(max_align_t) proven_byte_t storage[4096];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t a = proven_arena_as_allocator(&arena);

/* Ask for the capacity up front, so growth never has to abandon a block. */
proven_result_array_t ar = PROVEN_ARRAY_INIT(a, int, 128);
if (!proven_is_ok(ar.err)) {
    return;
}
(void)PROVEN_ARRAY_PUSH(&ar.value, int, 10);
PROVEN_ARRAY_DESTROY(&ar.value);   /* correct, but arena free reclaims nothing */
proven_arena_reset(&arena);        /* this is what gives the bytes back */
```

Wrong — growing into an arena from a tiny initial capacity:

```
proven_result_array_t ar = PROVEN_ARRAY_INIT(a, int, 1);
for (int i = 0; i < 10000; ++i) {
    PROVEN_ARRAY_PUSH(&ar.value, int, i); /* wrong: every regrow abandons the old block */
}
```

Each regrow asks the arena for a bigger block and then "frees" the old one — which, in an arena, does nothing. The array ends up correct and the arena ends up holding every intermediate size it ever allocated: 1, 2, 4, 8 … 8192 elements' worth of abandoned space, on top of the one buffer you wanted. Reserve the capacity up front, as the correct version above does.

Wrong — a view that outlives the reset:

```
proven_u8str_view_t name = /* ... built in the arena ... */;
proven_arena_reset(&arena);
use(name);                 /* wrong: those bytes are now free space */
```

This is the arena's sharpest edge. Reset does not touch the memory it reclaims, so the bytes are usually still there and the bug usually does not show up in testing — right up until the next allocation writes over them.

3. Pool: many things, one size

The problem an arena does not solve
An arena assumes shared lifetimes. Plenty of workloads have the opposite shape: objects of one type, created and destroyed continuously, in no particular order. A linked list of events. Nodes in a tree that grows and shrinks. Connection records that come and go.

An arena cannot do this — it never reclaims a single object, so a long-running program would grow without bound. The heap can, and that is exactly what it costs: every allocation searches, every free updates bookkeeping, and the general-purpose allocator does that general-purpose work for a request it already made a thousand times.

The observation a pool makes is that **all these objects are the same size**. If they are the same size, a freed one fits a future request exactly, so it can be handed straight back out with no search at all.

How a pool works
A pool keeps a small stack of freed blocks — the *bin* — and does the obvious thing:

- **Allocate**: if the bin has anything in it, pop one and return it. That is a pointer read and a decrement. Only when the bin is empty does it go to the underlying allocator.
- **Free**: if the bin has room, push the block onto it for reuse. If the bin is full, hand the memory back to the underlying allocator so the pool does not park memory forever.

`bin_cap` is therefore a dial: how much memory this pool is allowed to keep in reserve.

What you give up
- **One pool serves exactly one size and alignment.** A request for anything else is refused with `PROVEN_ERR_INVALID_ARG` — not served from somewhere else. A stricter alignment than the pool was built with is refused too; a looser one is fine, because the block already satisfies it.
- **The pool does not track live objects.** `proven_pool_destroy` frees what is in the bin, not what you are still holding. Free everything you allocated first, or you have leaked it *and* it is now dangling.

Arena versus pool, in one line each: an arena is for *allocate many, free all at once*; a pool is for *allocate and free the same size, over and over, cheaply*.

proven_pool_t
```
typedef struct {
    proven_allocator_t base_alloc;
    proven_size_t item_size;
    proven_size_t item_align;
```

```
    void **bin;
    proven_size_t bin_cap;
    proven_size_t bin_len;
} proven_pool_t;
```

Fields:

- `base_alloc`: allocator used for new blocks and the recycle bin.
- `item_size`: exact object size accepted by the pool. Any other size is rejected with `PROVEN_ERR_INVALID_ARG`.
- `item_align`: the alignment every block is allocated with. A request for a *stricter* alignment is rejected with `PROVEN_ERR_INVALID_ARG`; a looser one is served, since the block already meets it.
- `bin`: cached free-list array.
- `bin_cap`: maximum cached block count.
- `bin_len`: current cached block count.

How recycling works

The pool does not own one big slab; it allocates each item individually from `base_alloc` and keeps a small **stack of freed blocks** (the `bin`, an array of up to `bin_cap` pointers):

- **Allocate.** If `bin_len > 0`, pop the top pointer (O(1), no `base_alloc` call) — this is the whole point of the pool. Otherwise fall through to `base_alloc`.
- **Free.** If `bin_len < bin_cap`, push the pointer onto the bin for reuse; otherwise (bin full) free it straight back to `base_alloc`. So the bin caps how much memory the pool keeps parked for reuse.
- **Destroy.** `proven_pool_destroy` frees every pointer still in the bin and the bin array itself — but it does **not** track live (handed-out) items, so you must free everything you allocated before destroying the pool.

This makes the pool a churn optimizer for short-lived same-type objects (nodes, events), not a region allocator. Use an `arena` when you want "allocate many, free all at once"; use a `pool` when you want "allocate/free the same size over and over cheaply".

Counter-examples

```
/* WRONG: the pool only handles its configured size/alignment. */
proven_allocator_t a = proven_pool_as_allocator(&pool);   /* item_size == sizeof(Node) */
a.alloc_fn(a.ctx, sizeof(BigThing), alignof(BigThing));   /* not the pool's item -> rejected */

/* WRONG: destroying while items are still live leaks (and dangles) them. */
void *p = a.alloc_fn(a.ctx, sizeof(Node), alignof(Node)).value.ptr;
proven_pool_destroy(&pool);   /* `p` is NOT freed and is now dangling */
/* RIGHT: free every handed-out item first, then destroy. */
```

Pool functions

API	Intent	Return
`proven_pool_init(pool, base_alloc, item_size, item_align, bin_cap)`	Initialize a fixed-size pool.	`PROVEN_OK` or error.
`proven_pool_as_allocator(pool)`	Return allocator trait backed by the pool.	`proven_allocator_t`.
`proven_pool_destroy(pool)`	Free cached blocks and bin storage.	void.

Example:

```
typedef struct Node { int value; } Node;

proven_pool_t pool = {0};
proven_err_t e = proven_pool_init(&pool, alloc, sizeof(Node), alignof(Node), 64);
if (!proven_is_ok(e)) {
```

```
    return;
}

proven_allocator_t node_alloc = proven_pool_as_allocator(&pool);
proven_result_mem_mut_t n = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node), alignof(Node));
if (proven_is_ok(n.err)) {
    /* Hand it back before destroy: the pool does not track live items. */
    node_alloc.free_fn(node_alloc.ctx, n.value.ptr);
}

proven_pool_destroy(&pool);
```

Wrong:

```
node_alloc.alloc_fn(node_alloc.ctx, sizeof(LargerObject), alignof(LargerObject));
/* wrong: one pool is for one fixed object size and alignment */
```

4. The allocator trait

Why this section is fourth and not first

You have now used three allocators without seeing the interface they share, and that was deliberate. The interface is the least interesting part of the idea and the hardest thing to read cold: three function-pointer typedefs and an alignment contract mean very little until you have seen a heap, an arena and a pool behind them.

You need this section for two things: writing an allocator of your own, and understanding the rules every allocator in this library promises to follow. If you are only ever going to *use* the three above, `proven_heap_allocator()`, `proven_arena_as_allocator()` and `proven_pool_as_allocator()` are the whole API and you can skip ahead.

A trait, here, is a struct of function pointers used as an interface — C's version of a virtual table, written out by hand. `proven_allocator_t` is the library's most important one; `stream.h` and `random.h` use the same shape.

Function pointer types

```
typedef proven_result_mem_mut_t (*proven_alloc_fn_t)(
    void *ctx,
    proven_size_t size,
    proven_size_t align
);

typedef proven_result_mem_mut_t (*proven_realloc_fn_t)(
    void *ctx,
    void *old_ptr,
    proven_size_t old_size,
    proven_size_t new_size,
    proven_size_t align
);

typedef void (*proven_free_fn_t)(void *ctx, void *ptr);
```

Intent:

- `alloc_fn` allocates a new byte slice.
- `realloc_fn` changes allocation size and must be failure-atomic: on failure the old block is still valid and unmodified, so the caller has lost nothing.
- A block must be reallocated and freed with the **same** `align` it was allocated with. An allocator may pick a different underlying mechanism for over-aligned requests than for ordinary ones, and the heap allocator does: `align <= alignof(max_align_t)` — which is every string, buffer and byte array in this library — goes through `malloc`/`realloc` so that growth can happen in place, and

anything more strictly aligned goes through an aligned allocator. Handing a block back under a different alignment class is undefined.
- `free_fn` releases memory. If an allocator needs size metadata, it must track that internally.

proven_allocator_t

```
typedef struct {
    void *ctx;
    proven_alloc_fn_t alloc_fn;
    proven_realloc_fn_t realloc_fn;
    proven_free_fn_t free_fn;
} proven_allocator_t;
```

Fields:

- `ctx`: allocator-specific state.
- `alloc_fn`: allocation function.
- `realloc_fn`: reallocation function.
- `free_fn`: deallocation function.

Reference

Member / API	Shape	Contract
`ctx`	`void *`	The allocator's own state. Opaque to callers; passed back as the first argument.
`alloc_fn`	`proven_alloc_fn_t`	Allocate `size` bytes at `align`. Returns `proven_result_mem_mut_t`.
`realloc_fn`	`proven_realloc_fn_t`	Resize. **Failure-atomic**: on failure the old block is untouched and still valid.
`free_fn`	`proven_free_fn_t`	Release. Must be given the same `align` class the block was allocated with.
`proven_alloc_is_valid(alloc)`	`static inline bool`	True when every function pointer is non-null — i.e. the trait can be called.

The three rules that every allocator in this library obeys, and that yours must:

1. **Same allocator, whole lifetime.** Allocate, reallocate and free a block with one allocator.
2. **Same alignment class.** A block allocated at one alignment must be reallocated and freed at that alignment; allocators may use a different mechanism for over-aligned requests.
3. **Failure changes nothing.** A failed `realloc_fn` leaves the old block valid and unmodified.

```
static inline bool proven_alloc_is_valid(proven_allocator_t alloc);
```

Purpose: check that all function pointers are non-null.

Return: true if the trait can be called.

Correct:

```
proven_allocator_t heap = proven_heap_allocator();
proven_err_t err = proven_alloc_is_valid(heap) ? PROVEN_OK : PROVEN_ERR_UNSUPPORTED;
(void)err;   /* a freestanding build hands back a zero allocator, not a valid one */
```

Wrong:

```
proven_allocator_t alloc = {0};
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 64, 8); /* wrong: null call */
```

5. Raw byte buffer

What it is for

`proven_buf_t` is the plainest thing in this chapter: a pointer, a length and a capacity. It owns its bytes, it never grows, and it is what `proven_u8str_t` and `proven_u16str_t` are built out of.

Reach for it directly when you want *bytes* rather than *text* — a record you are assembling, a frame you are about to write to a socket — and you know the maximum size in advance. When the content is text, use the string types in Chapter 3 instead: they give you the same storage plus NUL-termination, views, searching and formatting.

Like everything else here it does **not** store its allocator, so `proven_buf_destroy` takes the same one `proven_buf_create` was given.

proven_buf_t

```
typedef struct {
    proven_byte_t *ptr;
    proven_size_t len;
    proven_size_t cap;
} proven_buf_t;
```

Fields:

- `ptr`: storage pointer.
- `len`: bytes currently used.
- `cap`: bytes allocated.

proven_result_buf_t

```
typedef struct {
    proven_err_t err;
    proven_buf_t value;
} proven_result_buf_t;
```

Buffer functions

API	Intent	Return
`proven_buf_create(alloc, cap)`	Allocate a non-zero-capacity buffer.	`proven_result_buf_t`; invalid allocator or zero cap returns `PROVEN_ERR_INVALID_ARG`.
`proven_buf_append(buf, data)`	Append raw bytes if they fit.	`PROVEN_OK` or error.
`proven_buf_destroy(alloc, buf)`	Free buffer storage with the matching allocator.	void.

Example:

```
proven_result_buf_t r = proven_buf_create(alloc, 64);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_t buf = r.value;

proven_err_t e = proven_buf_append(&buf, (proven_mem_view_t){
    .ptr = (const proven_byte_t *)"abc",
    .size = 3,
});
(void)e;   /* PROVEN_ERR_OUT_OF_BOUNDS if it would not fit: the buffer never grows */

proven_buf_destroy(alloc, &buf);
```

6. Examples and misuse cases

Match allocator on destroy

Correct:

```
proven_allocator_t heap = proven_heap_allocator();

proven_result_buf_t r = proven_buf_create(heap, 128);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_destroy(heap, &r.value);   /* the same allocator that created it */
```

Wrong:

```
proven_result_buf_t r = proven_buf_create(heap, 128);
proven_buf_destroy(other_alloc, &r.value); /* wrong: allocator mismatch */
```

Arena free is a no-op

```
alignas(max_align_t) proven_byte_t storage[1024];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t a = proven_arena_as_allocator(&arena);

proven_result_mem_mut_t r = proven_arena_alloc(&arena, 32);
if (proven_is_ok(r.err)) {
    /* Legal, and correct to write - but it intentionally reclaims nothing. */
    a.free_fn(a.ctx, r.value.ptr);
}
proven_arena_reset(&arena);   /* only this gives the 32 bytes back */
```

Pool size contract

Correct:

```
typedef struct Node { int value; } Node;

proven_pool_t pool = {0};
if (!proven_is_ok(proven_pool_init(&pool, alloc, sizeof(Node), alignof(Node), 8))) {
    return;
}
proven_allocator_t node_alloc = proven_pool_as_allocator(&pool);

/* The pool serves exactly the size and alignment it was configured with. */
proven_result_mem_mut_t n = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node), alignof(Node));
if (proven_is_ok(n.err)) {
    node_alloc.free_fn(node_alloc.ctx, n.value.ptr);
}

/* Anything else is refused with PROVEN_ERR_INVALID_ARG rather than served. */
proven_result_mem_mut_t wrong = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node) * 2, alignof(Node));
(void)wrong;

proven_pool_destroy(&pool);
```

Wrong:

```
node_alloc.alloc_fn(node_alloc.ctx, sizeof(Other), alignof(Other));
/* wrong: PROVEN_ERR_INVALID_ARG - a pool is for one size and one alignment */
```

Buffer append does not grow

Wrong:

```
proven_buf_append(&buf, huge_data); /* returns out-of-bounds; no automatic growth */
```

Use `proven_u8str_append_grow()` or an array if you need growth.

Buffer append tolerates overlap

Correct:

```
proven_result_buf_t r = proven_buf_create(alloc, 64);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_t buf = r.value;

proven_err_t e = proven_buf_append(&buf, (proven_mem_view_t){
    .ptr = (const proven_byte_t *)"abcdefgh",
    .size = 8,
});
if (proven_is_ok(e)) {
    /* The source view points back into the buffer's own bytes. That is allowed:
     * append moves rather than copies, so the overlap is well defined. */
    e = proven_buf_append(&buf, (proven_mem_view_t){
        .ptr = buf.ptr + 2,
        .size = 4,
    });
}
(void)e;

proven_buf_destroy(alloc, &buf);
```

Wrong:

```
proven_sys_mem_copy(buf.ptr + buf.len, buf.ptr + 2, 4); /* copy is not the public contract here */
```

Worked example: an arena over caller-supplied memory

Compiled and run by the test suite. It shows the bump-and-drop lifetime that makes an arena worth reaching for: allocations are nearly free, nothing is individually freed, and `proven_arena_reset` reclaims the whole region at once.

```
/*
 * An arena does not own memory: it bumps a pointer through memory YOU own. That
 * is the whole trade. Allocation is an add, individual frees do not exist, and
 * you get everything back at once with a reset.
 *
 * The shape that makes it worth using is bump-then-drop: a phase allocates
 * freely, the phase ends, one reset reclaims the lot. No per-object bookkeeping
 * to get wrong, and nothing to leak - the backing storage below is a plain
 * array with automatic storage duration.
 */

int main(void) {
    /* The backing store is the caller's. Over-align it so the arena can satisfy
     * any alignment a caller asks for out of the first byte. */
    alignas(max_align_t) static proven_byte_t storage[4096];

    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = storage,
        .size = sizeof storage,
    });

    /* --- bump ------------------------------------------------------------ */
    proven_result_mem_mut_t a = proven_arena_alloc(&arena, 64);
    EXAMPLE_REQUIRE(proven_is_ok(a.err), "64 bytes must fit in a 4 KiB arena");
```

```
    EXAMPLE_REQUIRE(a.value.ptr == storage, "the first allocation starts at the backing store");

    /* Explicit alignment when the type demands more than PROVEN_DEFAULT_ALIGNMENT.
     * The arena pads to reach it, so the bytes it skips are simply gone until reset. */
    proven_result_mem_mut_t b = proven_arena_alloc_aligned(&arena, 32, 64);
    EXAMPLE_REQUIRE(proven_is_ok(b.err), "an over-aligned block must still fit");
    EXAMPLE_REQUIRE(((uintptr_t)b.value.ptr % 64) == 0, "the block must honour the requested alignment");

    /* --- the arena as an allocator for another API ------------------------- */
    /* Anything in proven that takes a proven_allocator_t can be driven by the
     * arena. The string below therefore lives inside `storage`. */
    proven_allocator_t arena_alloc = proven_arena_as_allocator(&arena);
    EXAMPLE_REQUIRE(proven_alloc_is_valid(arena_alloc), "the arena must expose a usable allocator");

    proven_result_u8str_t s = proven_u8str_create(arena_alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "the arena should be able to back a 32-byte string");

    proven_err_t err = proven_u8str_append_grow(arena_alloc, &s.value, PROVEN_LIT("scratch line"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending into an arena-backed string must succeed");

    /* Destroying it is still correct and still required by the ownership rules -
     * but arena free is a no-op, so it reclaims nothing. That is not a leak: the
     * bytes belong to `storage`, and the reset below is what returns them. */
    proven_u8str_destroy(arena_alloc, &s.value);

    proven_size_t used = arena.offset;
    EXAMPLE_REQUIRE(used > 64, "every allocation above came out of the same backing store");

    /* --- drop -------------------------------------------------------------- */
    /* One statement frees the 64-byte block, the aligned block and the string.
     * Reset costs the same whether ten objects were allocated or ten thousand. */
    proven_arena_reset(&arena);
    EXAMPLE_REQUIRE(arena.offset == 0, "reset must reclaim every allocation at once");

    /* Proof that the storage really is reusable: the next allocation lands back
     * at the start. Every pointer handed out before the reset is dangling now -
     * that is the price of the reset being free. */
    proven_result_mem_mut_t c = proven_arena_alloc(&arena, 64);
    EXAMPLE_REQUIRE(proven_is_ok(c.err), "allocation after reset must succeed");
    EXAMPLE_REQUIRE(c.value.ptr == storage, "after reset the arena bumps from the beginning again");

    /* --- exhaustion is an error, not a crash ------------------------------- */
    proven_result_mem_mut_t too_big = proven_arena_alloc(&arena, sizeof storage);
    EXAMPLE_REQUIRE(too_big.err == PROVEN_ERR_NOMEM, "an arena cannot grow: it reports NOMEM instead");

    printf("arena: %zu bytes used before reset, %zu in use now\n",
           (size_t)used, (size_t)arena.offset);

    /* Formal cleanup. A no-op for a caller-backed arena, but writing it keeps
     * the lifetime obvious if the backing store later becomes heap memory. */
    proven_arena_destroy(&arena);
    return EXAMPLE_OK();
}
```

Worked example: a pool that recycles fixed-size blocks

Compiled and run by the test suite. It shows a freed block coming straight back out of the recycle bin, and what the pool refuses.

```
/*
 * A pool is a churn optimizer, not a region. It is for one type: allocate and
```

```
 * free the same fixed-size block over and over - list nodes, events, particles -
 * without paying malloc every time.
 *
 * It keeps a small stack of freed blocks (the "bin"). Freeing pushes a block
 * onto the bin instead of returning it to the base allocator; allocating pops
 * one back off. Both are O(1) and neither touches the heap. That recycling is
 * the entire point, and the check below proves it happens.
 *
 * Ownership: the pool caches freed blocks, but it does NOT track the blocks it
 * has handed out. Every block you take, you must give back before destroy - the
 * pool cannot free what it does not know about.
 */

typedef struct {
    int id;
    int score;
} node_t;

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    EXAMPLE_REQUIRE(proven_alloc_is_valid(heap), "hosted builds have a heap allocator");

    /* The pool takes a base allocator for the blocks it cannot serve from the
     * bin, plus the exact size and alignment of the one type it manages. The
     * last argument caps how many freed blocks are parked for reuse. */
    proven_pool_t pool = {0};
    proven_err_t err = proven_pool_init(&pool, heap, sizeof(node_t), alignof(node_t), 4);
    EXAMPLE_REQUIRE(proven_is_ok(err), "initializing a pool of node_t must succeed");
    if (!proven_is_ok(err)) {
        return 1;
    }

    proven_allocator_t nodes = proven_pool_as_allocator(&pool);

    /* --- first block: nothing in the bin, so it comes from the heap --------- */
    proven_result_mem_mut_t first = nodes.alloc_fn(nodes.ctx, sizeof(node_t), alignof(node_t));
    EXAMPLE_REQUIRE(proven_is_ok(first.err), "the pool must be able to serve its own item type");
    if (!proven_is_ok(first.err)) {
        proven_pool_destroy(&pool);
        return 1;
    }

    node_t *n = (node_t *)first.value.ptr;
    *n = (node_t){ .id = 1, .score = 100 };
    void *first_addr = n;

    /* --- hand it back: it lands in the bin, not back on the heap ------------ */
    nodes.free_fn(nodes.ctx, n);
    EXAMPLE_REQUIRE(pool.bin_len == 1, "a freed block is cached for reuse, not returned to the heap");
    /* `n` is dangling from here on. The pool owns those bytes again. */

    /* --- second block: the freed one is handed straight back ---------------- */
    proven_result_mem_mut_t second = nodes.alloc_fn(nodes.ctx, sizeof(node_t), alignof(node_t));
    EXAMPLE_REQUIRE(proven_is_ok(second.err), "allocating from a non-empty bin must succeed");
    EXAMPLE_REQUIRE(second.value.ptr == first_addr, "the recycled block is the one that was freed");
    EXAMPLE_REQUIRE(pool.bin_len == 0, "taking it back out empties the bin");

    /* Recycled memory is NOT zeroed for you - it is whatever the pool left there.
     * Initialize every field, exactly as you would for a fresh malloc. */
    node_t *m = (node_t *)second.value.ptr;
```

```
    *m = (node_t){ .id = 2, .score = 50 };
    EXAMPLE_REQUIRE(m->id == 2, "the recycled block is ours to overwrite");

    /* --- one pool serves one size and one alignment ------------------------- */
    /* A request for anything else is refused: this is not a general allocator, and it will
     * not silently hand you a block of the wrong size. The code is PROVEN_ERR_UNSUPPORTED -
     * "not my job" - and not INVALID_ARG, which would read as "you passed me garbage" and
     * send you hunting for a bug in your own code. */
    proven_result_mem_mut_t wrong = nodes.alloc_fn(nodes.ctx, sizeof(node_t) * 2, alignof(node_t));
    EXAMPLE_REQUIRE(wrong.err == PROVEN_ERR_UNSUPPORTED, "the pool only serves its configured item size");

    /* --- return every live block before destroying -------------------------- */
    /* proven_pool_destroy frees what is in the bin and the bin itself. `m` is
     * still handed out, so if we skipped this free it would leak. */
    nodes.free_fn(nodes.ctx, m);

    printf("pool: %zu block(s) cached for reuse at teardown\n", (size_t)pool.bin_len);

    proven_pool_destroy(&pool);
    return EXAMPLE_OK();
}
```

Worked example: start-up allocation, growing in place, and wrapping an allocator
The two examples above use an arena and a pool the way most code does. This one covers the three things you reach for once the program is real:

1. **Allocation during start-up.** Some allocations have no sensible failure path: if the program cannot get its own configuration table, there is nothing left for it to do. `proven_arena_alloc_or_panic()` and `proven_arena_alloc_aligned_or_panic()` return the block itself instead of a result, and hand the failure to the panic handler (Chapter 1 section 6). `proven_set_panic_handler()` is how you choose what happens then — the example installs one that records the message so the failure can be shown here, and puts the default back afterwards.
2. **Growing the block you took last.** `proven_arena_realloc_aligned()` extends it where it stands, because the most recent block is the one at the end of the used region. Any earlier block is copied to the end instead — still correct, no longer free.
3. **Wrapping an allocator.** An allocator is three function pointers and a context pointer, so a wrapper that counts, logs, or fails the tenth call on purpose is three forwarding functions. The arena's own three — `proven_arena_alloc_trait()`, `proven_arena_realloc_trait()` and `proven_arena_free_trait()` — are public so your wrapper can call them instead of re-implementing the arena. Everything in the library that takes a `proven_allocator_t` then runs through your wrapper without knowing it.

```
/*
 * Three things a program that owns its own memory eventually has to do, and the
 * calls that do them:
 *
 *   - Allocate during start-up, where running out of memory is not a condition
 *     the program can carry on from. That is what the `_or_panic` calls are
 *     for, and installing a panic handler is how you decide what "cannot carry
 *     on" means for your program - a log line and an exit, rather than a trap.
 *
 *   - Grow the block you allocated last, without copying it. An arena can do
 *     that in place, because the block that was allocated last is the one
 *     sitting at the end of the used region.
 *
 *   - Instrument the allocator - count allocations, or fail the tenth one on
 *     purpose in a test - without changing the code being measured. An
 *     allocator here is three function pointers and a context pointer, so
```

```
 *     wrapping one is writing three forwarding functions. The arena's own three
 *     are public for exactly this reason: your wrapper forwards to them.
 */

/* --- what a panic handler is for ---------------------------------------- */

static int g_panics = 0;
static char g_last_panic[128];

/* A panic handler receives the message and decides the program's fate. The
 * default one traps immediately, which is right in production and useless in a
 * test - so this one records the message and returns. Returning is allowed ONLY
 * when you are deliberately testing the panic path, and the memory block the
 * panicking call returns must then not be used. */
static void record_panic(const char *msg) {
    ++g_panics;
    snprintf(g_last_panic, sizeof g_last_panic, "%s", msg);
}

/* --- an allocator that counts what passes through it ---------------------- */

typedef struct {
    proven_arena_t *arena;
    proven_size_t   live_bytes;
    proven_size_t   alloc_calls;
    proven_size_t   free_calls;
} counting_ctx_t;

/* Each of the three matches one field of proven_allocator_t. Each does its own
 * bookkeeping and then forwards to the arena's public trait function, so the
 * behaviour being measured is exactly the arena's behaviour and not a
 * re-implementation of it. */
static proven_result_mem_mut_t counting_alloc(void *ctx, proven_size_t size, proven_size_t align) {
    counting_ctx_t *c = (counting_ctx_t *)ctx;
    proven_result_mem_mut_t r = proven_arena_alloc_trait(c->arena, size, align);
    if (proven_is_ok(r.err)) {
        c->live_bytes += size;
        ++c->alloc_calls;
    }
    return r;
}

static proven_result_mem_mut_t counting_realloc(void *ctx, void *old_ptr, proven_size_t old_size,
                                                proven_size_t new_size, proven_size_t align) {
    counting_ctx_t *c = (counting_ctx_t *)ctx;
    proven_result_mem_mut_t r = proven_arena_realloc_trait(c->arena, old_ptr, old_size, new_size, align);
    if (proven_is_ok(r.err)) {
        c->live_bytes = c->live_bytes - old_size + new_size;
    }
    return r;
}

static void counting_free(void *ctx, void *ptr) {
    counting_ctx_t *c = (counting_ctx_t *)ctx;
    ++c->free_calls;
    proven_arena_free_trait(c->arena, ptr);   /* an arena free is a no-op; the count is the point */
}

int main(void) {
    alignas(max_align_t) static proven_byte_t storage[1024];
```

```
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){ .ptr = storage, .size = sizeof storage });

    /* --- 1. start-up allocation that must not fail ------------------------ */

    proven_set_panic_handler(record_panic);

    /* No result to unwrap: these return the block directly, because there is no
     * error the caller could act on. That is the entire difference. */
    proven_mem_mut_t table = proven_arena_alloc_or_panic(&arena, 256);
    EXAMPLE_REQUIRE(table.ptr != NULL, "a 256-byte start-up allocation must succeed");
    EXAMPLE_REQUIRE(g_panics == 0, "a successful allocation must not panic");

    /* The aligned form, for a type that needs more than the default boundary -
     * a 64-byte cache line here, the usual reason. */
    proven_mem_mut_t cache_line = proven_arena_alloc_aligned_or_panic(&arena, 64, 64);
    EXAMPLE_REQUIRE(((proven_uintptr_t)cache_line.ptr % 64) == 0,
                    "the block must start on the boundary that was asked for");
    EXAMPLE_REQUIRE(g_panics == 0, "an over-aligned allocation that fits must not panic either");

    /* Now the failing case, on purpose: more than the arena could ever hold.
     * With the recording handler installed we can observe it; with the default
     * handler the program would stop here, which is what it is for. */
    proven_mem_mut_t impossible = proven_arena_alloc_or_panic(&arena, sizeof storage * 2);
    (void)impossible;   /* after a handler returns, this block means nothing */
    EXAMPLE_REQUIRE(g_panics == 1, "exhausting the arena through _or_panic must panic");
    EXAMPLE_REQUIRE(g_last_panic[0] != '\0', "the handler receives a message naming the call");
    printf("panic handler saw: %s\n", g_last_panic);

    /* Passing NULL puts the default trapping handler back. Leaving a test
     * handler installed turns a real failure into silent corruption. */
    proven_set_panic_handler(NULL);

    /* --- 2. growing the most recent block in place ------------------------ */

    /* A parser that reads a header, then discovers the body is longer than it
     * guessed, wants to extend the buffer it just took - not copy it. */
    proven_result_mem_mut_t buf = proven_arena_alloc_aligned(&arena, 32, alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(buf.err), "the initial 32-byte buffer must fit");

    proven_size_t before = arena.offset;
    proven_result_mem_mut_t grown = proven_arena_realloc_aligned(&arena, buf.value.ptr, 32, 96,
alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(grown.err), "growing the most recent block must succeed");
    EXAMPLE_REQUIRE(grown.value.ptr == buf.value.ptr,
                    "the most recent block grows in place: same address, no copy");
    EXAMPLE_REQUIRE(arena.offset == before + 64, "only the extra 64 bytes were taken");

    /* The in-place path is available only for the block allocated LAST. Take
     * another block, and the earlier one can no longer be extended where it
     * stands - the arena copies it to the end instead, and the old bytes are
     * dead until the next reset. Correct either way; just not free. */
    proven_result_mem_mut_t other = proven_arena_alloc(&arena, 16);
    EXAMPLE_REQUIRE(proven_is_ok(other.err), "a second block must fit");
    proven_result_mem_mut_t moved = proven_arena_realloc_aligned(&arena, grown.value.ptr, 96, 128,
alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(moved.err), "growing an older block must still succeed");
    EXAMPLE_REQUIRE(moved.value.ptr != grown.value.ptr, "but it is relocated, not extended");

    /* --- 3. the arena behind a counting wrapper --------------------------- */
```

```
    proven_arena_reset(&arena);

    counting_ctx_t counted = { .arena = &arena };
    proven_allocator_t alloc = {
        .ctx        = &counted,
        .alloc_fn   = counting_alloc,
        .realloc_fn = counting_realloc,
        .free_fn    = counting_free,
    };
    EXAMPLE_REQUIRE(proven_alloc_is_valid(alloc), "all three function pointers must be present");

    /* Any part of the library that takes an allocator now runs through the
     * wrapper, unchanged and unaware. */
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "creating a string through the wrapper must succeed");

    proven_err_t err = proven_u8str_append_grow(alloc, &s.value, PROVEN_LIT("a line long enough to need more
room"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending past the initial capacity must succeed");

    proven_u8str_destroy(alloc, &s.value);

    EXAMPLE_REQUIRE(counted.alloc_calls >= 1, "the wrapper saw the string being created");
    EXAMPLE_REQUIRE(counted.free_calls >= 1, "and saw it being destroyed");
    printf("counting allocator: %zu alloc call(s), %zu free call(s), %zu bytes handed out\n",
           (size_t)counted.alloc_calls, (size_t)counted.free_calls, (size_t)counted.live_bytes);

    proven_arena_destroy(&arena);
    return EXAMPLE_OK();
}
```

Wrong — leaving a test panic handler installed:

```
proven_set_panic_handler(record_panic);   /* returns instead of stopping */
/* ... the rest of the program ... */     /* wrong: no restore */
```

A handler that returns turns every later out-of-memory panic into a call that hands back a null block and carries on. Install one only around the code you are testing, and restore the default with `proven_set_panic_handler(NULL)`.

Wrong — using the block after a handler that returned:

```
proven_mem_mut_t m = proven_arena_alloc_or_panic(&arena, huge);
memset(m.ptr, 0, huge);   /* wrong: m.ptr is null if the handler returned */
```

Wrong — a wrapper that forgets one of the three:

```
proven_allocator_t alloc = { .ctx = &counted, .alloc_fn = counting_alloc };  /* wrong */
```

`proven_alloc_is_valid()` returns false for this, and the first `realloc` or `destroy` through it calls a null pointer. Fill in all three, even when two of them only forward.

Chapter 3: Strings, Formatting, and Scanning

Part II — The vocabulary every program uses. Prerequisites: Chapter 1 and Chapter 2. After this chapter you can hold text without a NUL terminator deciding your program's fate, build strings that refuse to overflow, and format and parse the everyday cases.

This chapter covers `u8str.h`, `u16str.h`, `fmt.h`, and `scan.h`. It is the **tutorial half** of the text material: it introduces the formatter and the scanner with the cases you meet daily. Chapter 8 is the reference half — the complete grammar, every argument constructor, and the scanner's error and recovery rules. Read this one first.

Table of contents

1. U8 strings and views

The problem: a C string does not know how long it is

A C string is a pointer, and where it ends is decided by a zero byte somewhere in memory. That one decision — made in 1972 to save a byte per string — is behind a remarkable amount of damage:

- **Length is a search.** `strlen` walks the string. A loop that checks `strlen(s)` on each iteration is quadratic, and it looks like ordinary code.
- **Text cannot contain a zero byte.** So a C string cannot hold a UTF-16 buffer, a protocol frame, a slice of a file, or any binary data — the string type and the byte type are different things and the language pretends otherwise.
- **A missing terminator is not detectable.** `strcpy` into a buffer with no room writes until it finds a zero somewhere in your stack frame. Nothing reports it. This is the single most exploited bug class in the language's history.
- **You cannot cheaply refer to part of a string.** "The third field of this line" means either copying those bytes out or writing a zero over the original, destroying it. `strtok` chose the second, which is why it mutates its input and cannot be nested.

`strncpy`, the traditional patch, does not always NUL-terminate — so the "safe" function can produce a string that is not a string.

What this library does instead

Two types, and the difference between them is ownership:

- `proven_u8str_view_t` — **borrowed.** A pointer and a size, together, pointing at bytes someone else owns. Copying it is free. It allocates nothing, destroys nothing, and stops being valid when its owner does. This is what you pass to functions.
- `proven_u8str_t` — **owned.** It has its own storage, a capacity, and a NUL terminator kept for you so `proven_u8str_as_cstr` can hand the bytes to a libc function that still wants one. You created it with an allocator; you destroy it with the same one.

Both count **bytes**, not characters. `"한"` is three bytes in UTF-8 and one character, and this library will tell you three, because that is what it knows. Text is a sequence of bytes here; interpreting those bytes as characters is a job for a Unicode layer this library does not have.

Because a view carries its length, everything the NUL terminator made hard becomes ordinary: length is a field, text may contain zero bytes, a sub-range is a view into the same memory with no copy, and a write that would not fit is refused rather than performed.

Wrong — treating a view as a C string:

```
proven_u8str_view_t v = proven_u8str_view_slice(line, 4, 8);   /* a field inside a line */
printf("%s\n", (const char *)v.ptr);   /* wrong: no terminator, prints until it finds a zero */
```

A view is deliberately *not* NUL-terminated: it usually points into the middle of someone else's buffer, where writing a terminator would corrupt the next field. Use `proven_println("{}", PROVEN_ARG(v))`, which takes the size, or copy it into an owned string first.

Structures

```
typedef struct {
    const proven_byte_t *ptr;
    proven_size_t size;
} proven_u8str_view_t;

typedef struct {
    proven_byte_t *ptr;
    proven_size_t size;
} proven_u8str_mut_t;

typedef struct {
    proven_buf_t internal;
    bool         borrowed;
} proven_u8str_t;

typedef struct {
    proven_err_t err;
    proven_u8str_t value;
} proven_result_u8str_t;

typedef struct {
    proven_err_t err;
    const char *value;
} proven_result_cstr_t;
```

Intent:

- `proven_u8str_view_t`: borrowed read-only byte string view.
- `proven_u8str_mut_t`: borrowed mutable byte string view.
- `proven_u8str_t`: owned string with NUL termination.
- `proven_result_u8str_t`: result wrapper for owned strings.
- `proven_result_cstr_t`: result wrapper for an allocated NUL-terminated C string.

Internal layout

The views are exactly what they look like — a pointer and a byte count:

```
typedef struct { const proven_byte_t *ptr; proven_size_t size; } proven_u8str_view_t;
typedef struct { proven_byte_t *ptr;       proven_size_t size; } proven_u8str_mut_t;
```

The owned string wraps a `proven_buf_t` (the fixed-capacity byte buffer from Chapter 2) plus one flag:

```
typedef struct {
    proven_buf_t internal;   /* the bytes + length + capacity; always keeps room for a NUL */
    bool         borrowed;   /* false = allocator-owned (default); true = wraps caller memory */
} proven_u8str_t;
```

- `internal.len` is the byte length (`proven_buf_t` is `ptr` / `len` / `cap` - there is no `size` member). The byte at `internal.len` is always the NUL terminator while the string is valid, so `proven_u8str_as_cstr()` is O(1).
- `borrowed` is `false` for a zero-initialized handle, so an allocator-owned string is the safe default. It is set `true` only by `proven_u8str_borrow`, which wraps `[buf, buf+cap)` you own: growing operations then refuse to reallocate (`PROVEN_ERR_OUT_OF_BOUNDS`) and `proven_u8str_destroy` is a no-op.

- **Do not** read or write these fields directly to change the string; use the functions below. Reading `internal.len` for length is fine, but prefer `proven_u8str_as_view()`.

Counter-example — treating a borrowed string like an owned one:

```
proven_byte_t stack[8];
proven_u8str_t s = proven_u8str_borrow(stack, sizeof stack);   /* borrowed */

/* This would have to reallocate caller memory, so it refuses: the call returns
   PROVEN_ERR_OUT_OF_BOUNDS and `stack` is left exactly as it was. A borrowed
   string never silently escapes to the heap. */
proven_err_t e = proven_u8str_append_grow(alloc, &s, PROVEN_LIT("far too long for eight bytes"));
(void)e;   /* == PROVEN_ERR_OUT_OF_BOUNDS */

/* destroy is a no-op here: `stack` is yours, and the library will not free it.
   Writing it anyway is correct, and it keeps teardown code uniform. */
proven_u8str_destroy(alloc, &s);
```

Macros

Macro	Intent	Notes
`PROVEN_LIT(s)`	Make a `proven_u8str_view_t` from a string literal.	Use only with string literals.
`PROVEN_LIT_INIT(s)`	Initializer form for literal views.	Use in aggregate initialization.
`PROVEN_INDEX_NOT_FOUND`	Sentinel returned by find functions.	Equal to `(proven_size_t)-1`.

U8 functions

API	Intent	Return
`proven_u8str_create(alloc, limit)`	Create an empty owned string with capacity for `limit` bytes plus NUL.	`proven_result_u8str_t`.
`proven_u8str_create_from_view(alloc, view)`	Copy a view into a new owned string.	`proven_result_u8str_t`.
`proven_u8str_borrow(buf, cap)`	Wrap caller-owned `[buf, buf+cap)` as a fixed-capacity string (no allocation). Fixed-capacity ops and `append_fmt` work; growing ops refuse to reallocate caller memory; destroy is a no-op. `cap` includes the NUL.	`proven_u8str_t`.
`proven_u8str_reset(str)`	Truncate to empty, keeping the buffer/capacity for reuse (owned or borrowed).	`proven_err_t`.
`proven_u8str_is_valid(str)`	Validate public string invariants.	`bool`.
`proven_u8str_reserve(alloc, str, new_cap)`	Ensure at least `new_cap` bytes of internal capacity. (Borrowed strings cannot grow.)	`proven_err_t`.
`proven_u8str_append(str, data)`	Atomic fixed-capacity append.	`PROVEN_OK` or `PROVEN_ERR_OUT_OF_BOUNDS`.

`proven_u8str_append_partial(str, data)`	Truncating append.	`proven_result_size_t`; `value` is bytes written.
`proven_u8str_append_grow(alloc, str, data)`	Atomic growable append.	`proven_err_t`.
`proven_u8str_append_byte(alloc, str, b)`	Append one byte, growing if needed.	`proven_err_t`.
`proven_u8str_replace_at(str, index, old_len, data)`	Replace an exact byte range with fixed-capacity semantics.	`proven_err_t`.
`proven_u8str_replace_at_grow(alloc, str, index, old_len, data)`	Like `replace_at`, but grows the buffer (doubling) when the edit does not fit instead of failing.	`proven_err_t`; string unchanged on alloc failure.
`proven_u8str_insert(str, index, data)`	Insert bytes at `index` with fixed-capacity semantics.	`proven_err_t`.
`proven_u8str_insert_grow(alloc, str, index, data)`	Like `insert`, but grows the buffer when needed. No manual `reserve` required.	`proven_err_t`; string unchanged on alloc failure.
`proven_u8str_remove(str, index, len)`	Remove a byte range.	`proven_err_t`.
`proven_u8str_replace_first(str, start_offset, target, replacement)`	Replace first matching target at or after start.	`PROVEN_OK` even when not found.
`proven_u8str_view_find(haystack, start_offset, needle)`	Find the first occurrence of a byte substring (correctly handles any byte values; not NUL-terminated).	index or `PROVEN_INDEX_NOT_FOUND`.
`proven_u8str_view_starts_with(str, prefix)`	Prefix test.	int truth value.
`proven_u8str_view_ends_with(str, suffix)`	Suffix test.	int truth value.
`proven_u8str_view_slice(str, index, len)`	Return a clamped subview.	`proven_u8str_view_t`.
`proven_u8str_as_cstr(str)`	Return internal NUL-terminated pointer.	`const char *`; invalidated by growth/destroy.
`proven_mem_view_from_u8(view)`	Convert U8 view to byte view.	`proven_mem_view_t`.
`proven_u8str_view_to_cstr(view, alloc)`	Allocate a NUL-terminated C string from any view.	`proven_result_cstr_t`; caller frees with allocator.
`proven_cstr_len(s)`	Count bytes until NUL.	`proven_size_t`.
`proven_u8str_view_from_cstr(s)`	Make a view from a trusted NUL-terminated string.	Empty view for null pointer.
`proven_u8str_view_eq(a, b)`	Byte equality test.	int truth value.
`proven_u8str_destroy(alloc, str)`	Free owned storage with matching allocator.	void.
`proven_u8str_as_view(str)`	Borrow current contents.	`proven_u8str_view_t`.

Basic U8 example

```
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("log"));
if (!proven_is_ok(r.err)) {
    return;
```

```
}
proven_u8str_t s = r.value;

proven_err_t e = proven_u8str_append_grow(alloc, &s, PROVEN_LIT(": ready"));
if (!proven_is_ok(e)) {
    proven_u8str_destroy(alloc, &s);
    return;
}

/* Valid until the next growing call: as_cstr points into the string's storage. */
const char *cstr = proven_u8str_as_cstr(&s);
(void)cstr;

proven_u8str_destroy(alloc, &s);
```

2. U16 strings and views

Why a second string type exists at all
UTF-8 is the right default and this library commits to it. `proven_u16str_t` exists for one reason: **the Windows API is UTF-16.** Every "wide" entry point — `CreateFileW`, `GetEnvironmentVariableW`, the whole `W` family — takes `wchar_t *`, which on Windows is 16 bits. A library that talks to those APIs needs a type that holds their code units without pretending they are bytes.

So this is a boundary type. You use it where you touch a UTF-16 API, and you use `proven_u8str_t` everywhere else. It is deliberately small — create, destroy, append, and length — because it is not meant to be the type your program thinks in.

Two things to hold on to, because they are the source of every UTF-16 bug:

- **A code unit is not a character.** UTF-16 encodes anything outside the Basic Multilingual Plane as a *surrogate pair* — two `proven_u16` values that mean one character. An emoji is two code units. Slicing between them produces an unpaired surrogate, which is not valid UTF-16.
- `size` **here counts code units, not bytes.** `proven_u16str_view_t.size` is a count of `proven_u16` values; the `proven_buf_t` underneath tracks bytes, which is why `proven_u16str_len` divides. Mixing the two units is the most common mistake with this type.

There is no conversion between UTF-8 and UTF-16 in this library. That is a real gap, and it is deliberate rather than forgotten: correct conversion means deciding what to do with invalid input, unpaired surrogates and overlong encodings, and that is a Unicode layer's job. Today you build a `proven_u16str_t` from a `u"..."` literal or from code units you already have.

U16 APIs are excluded when `PROVEN_NO_U16STR` is defined, which is the default in freestanding builds — a bare-metal target has no Windows API to talk to.

Wrong — assuming one code unit is one character:

```
proven_u16str_view_t v = ...;                /* text containing an emoji */
proven_size_t chars = proven_u16str_len(&s);  /* wrong: that is code units */
```

Structures

```
typedef struct {
    const proven_u16 *ptr;
    proven_size_t size;
} proven_u16str_view_t;

typedef struct {
    proven_buf_t internal;
} proven_u16str_t;

typedef struct {
```

```
    proven_err_t err;
    proven_u16str_t value;
} proven_result_u16str_t;
```

Macro

Macro	Intent
`PROVEN_U16_LIT(s)`	Make a `proven_u16str_view_t` from a UTF-16 literal expression `u"..."`.

U16 functions

API	Intent	Return
`proven_u16str_create(alloc, unit_limit)`	Create empty owned U16 string.	`proven_result_u16str_t`.
`proven_u16str_create_from_view(alloc, view)`	Copy U16 view into owned string.	`proven_result_u16str_t`.
`proven_u16str_destroy(alloc, str)`	Free owned U16 storage.	void.
`proven_u16str_append(str, data)`	Atomic fixed-capacity append.	`proven_err_t`.
`proven_u16str_append_partial(str, data)`	Truncating append.	`proven_result_size_t`.
`proven_u16str_append_grow(alloc, str, data)`	Atomic growable append.	`proven_err_t`.
`proven_u16str_as_ptr(str)`	Return internal `proven_u16` pointer.	`const proven_u16 *`.
`proven_u16str_len(str)`	Return length in code units.	`proven_size_t`.

Example:

```
#ifndef PROVEN_NO_U16STR
proven_result_u16str_t r =
    proven_u16str_create_from_view(alloc, PROVEN_U16_LIT("hello"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u16str_t s = r.value;

(void)proven_u16str_append_grow(alloc, &s, PROVEN_U16_LIT(" world"));

/* Length is in code units, not characters: "hello world" is 11 units. */
proven_size_t units = proven_u16str_len(&s);
(void)units;

proven_u16str_destroy(alloc, &s);
#endif
```

Note: `proven_u16` is a code unit, not necessarily one full Unicode character. UTF-16 surrogate pairs use two code units.

3. Formatting

The problem: `printf` is told the types twice

`printf("%d", x)` states the type of `x` twice — once in the format string and once by passing `x` — and nothing checks that the two agree. Varargs erases the type, so the function reads whatever bytes the calling convention left, in whatever shape the format demanded:

```
printf("%d\n", 3.0);      /* wrong: reads a double's bytes as an int */
printf("%s\n", 42);       /* wrong: dereferences 42 as a pointer */
printf("%d %d\n", 1);     /* wrong: reads an argument that was never passed */
```

All three compile. Modern compilers warn when the format is a literal, which helps until the format is a variable — and then you have a function that will read arbitrary stack memory on demand, which is a class of vulnerability with its own name.

The second problem is where the output goes. `sprintf` writes to a buffer whose size it does not know. `snprintf` takes a size and then **truncates**, returning the length it *would* have written — so the caller who forgets to compare gets a silently shortened path, command, or identifier.

What this library does instead

The placeholder has no type in it. `{}` says "a value goes here"; the type comes from the argument, resolved at compile time:

```
proven_println("{} scored {}", PROVEN_ARG(name), PROVEN_ARG(score));
```

`PROVEN_ARG` is a `_Generic` dispatch — the compiler picks the right constructor for the argument's static type. A mismatch between the format and the argument is not possible, because the format never states a type. What the spec after `:` controls is *presentation* — width, fill, alignment, precision, base — never interpretation.

The destination is a sized object, and the fixed-capacity form refuses rather than truncates: `proven_u8str_append_fmt` fails with `PROVEN_ERR_OUT_OF_BOUNDS` and writes nothing, while `proven_u8str_append_fmt_grow` takes an allocator and grows. Which one you called is visible in the call itself.

Your own types can join in. `PROVEN_ARG_OF(&obj, render_fn)` lets a type you defined print with `{}` like everything else — the extension point is compile-time and typed, not a registry of names. Chapter 8 §5.1 shows how.

The cost, stated plainly: `PROVEN_ARG` around each argument is more typing than `%d`, and the format language is not the one in your fingers. What you buy is that the class of bug at the top of this section cannot be written.

The formatter writes into `proven_u8str_t` or PAL-backed streams. It uses a small structural format language with `{}` placeholders, explicit indexes like `{1}`, escaped braces `{{` and `}}`, and width/alignment specs such as `{:0>5}`, `{:*^10}`, and `{:.<10}`.

Wrong — assuming the spec chooses the type:

```
proven_println("{:d}", PROVEN_ARG(3.5));   /* wrong: the spec formats, it does not convert */
```

Structures and enums

```
typedef struct {
    proven_err_t err;
    proven_size_t written;
    proven_size_t required;
} proven_fmt_result_t;
```

Fields:

- `err`: result code.
- `written`: bytes actually written.
- `required`: bytes required for full output.

`proven_arg_type_t` variants:

- `PROVEN_ARG_NONE`
- `PROVEN_ARG_I32`
- `PROVEN_ARG_U32`
- `PROVEN_ARG_I64`
- `PROVEN_ARG_U64`
- `PROVEN_ARG_F64` unless `PROVEN_FMT_NO_FLOAT` is defined
- `PROVEN_ARG_CSTR`
- `PROVEN_ARG_STR_VIEW`
- `PROVEN_ARG_DATETIME`

- `PROVEN_ARG_PTR`
- `PROVEN_ARG_FN`

```
typedef struct {
    proven_arg_type_t type;
    union {
        proven_i32 i32;
        proven_u32 u32;
        proven_i64 i64;
        proven_u64 u64;
        double f64;
        const char *cstr;
        proven_u8str_view_t str_view;
        proven_datetime_t datetime;
        const void *ptr;
        void (*fn)(void);
    } value;
} proven_arg_t;
```

Format argument constructors

API	Intent
`proven_arg_none()`	Internal sentinel value.
`proven_arg_i32(v)`, `proven_arg_u32(v)`	Integer arguments.
`proven_arg_i64(v)`, `proven_arg_u64(v)`	Wide integer arguments.
`proven_arg_f64(v)`	Floating-point argument unless float formatting is disabled.
`proven_arg_cstr(v)`	Trusted live NUL-terminated C string.
`proven_arg_cstr_n(v, max_len)`	Bounded C-string argument; scans for NUL only up to `max_len`.
`proven_arg_str_view(v)`	Borrowed string view argument.
`proven_arg_datetime(v)`	Datetime argument.
`proven_arg_ptr(v)`	Object pointer argument.
`proven_arg_fn(v)`	Function pointer argument.
`proven_arg_ucstr(v)`	Unsigned-char C string helper.
`proven_arg_identity(v)`	Pass-through for existing `proven_arg_t`.

Format macros

Macro	Intent
`PROVEN_ARG(x)`	`_Generic` selector for supported argument types.
`PROVEN_ARG_FN(f)`	Function pointer formatting helper.
`PROVEN_ARG_CSTR_N(v, max_len)`	Bounded C-string helper.
`proven_u8str_append_fmt(str, fmt, ...)`	Atomic fixed-capacity formatting.
`proven_u8str_append_fmt_trunc(str, fmt, ...)`	Best-effort truncating formatting.
`proven_u8str_append_fmt_grow(alloc, str, fmt, ...)`	Atomic growable formatting.
`proven_u8str_append_fmt_with_scratch(alloc, str, fmt, scratch, ...)`	Growable formatting with scratch allocator for temporary patch allocations.
`PROVEN_FMT_IS_OK(res)`	Check `proven_fmt_result_t`.

Formatting engine

```
proven_fmt_result_t proven_u8str_fmt_internal(
    proven_allocator_t alloc,
    proven_u8str_t *str,
```

```
    bool trunc,
    const char *fmt,
    proven_allocator_t scratch,
    const proven_arg_t *args,
    proven_size_t args_count
);
```

Normal user code should call the macros instead of this internal engine. The engine expects a leading `proven_arg_none()` sentinel at index 0.

Extra unused format arguments return `PROVEN_ERR_INVALID_ARG`.

Example:

```
proven_result_u8str_t r = proven_u8str_create(alloc, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

/* The target is only 8 bytes; the _grow form reallocates rather than truncate. */
proven_fmt_result_t fr = proven_u8str_append_fmt_grow(
    alloc,
    &s,
    "name={} score={:0>4}",
    PROVEN_ARG(PROVEN_LIT("ada")),
    PROVEN_ARG(42)
);
if (!PROVEN_FMT_IS_OK(fr)) {
    proven_u8str_destroy(alloc, &s);
    return;
}
/* s == "name=ada score=0042" */

proven_u8str_destroy(alloc, &s);
```

4. Scanning

The problem: `scanf` will not tell you where it stopped
Parsing input is the mirror of formatting, and libc's answer is worse. `sscanf` returns *how many fields it filled* and nothing else — not which one failed, not how far it got, not why:

```
int n = sscanf(line, "%d %d %d", &a, &b, &c);
if (n != 3) { /* wrong: which field? at what offset? was it malformed or missing? */ }
```

If the third field is malformed, you know only that you got two. You cannot report the position to the user, cannot skip the bad record and resume, and cannot tell "ran out of input" from "found something that is not a number". And `%s` into a `char *` has the same unbounded-write problem as `strcpy`, with the input now coming from outside your program.

Then there is `strtol`, whose contract requires you to clear `errno` first, check it after, and compare `endptr` against the input to detect "no digits at all" — three separate things to get right for one conversion.

What this library does instead
The cursor is yours. A `proven_scan_t` holds the view being parsed and an offset into it. Each scan reads from the cursor and moves it forward on success. Because the cursor is a field you can read, you always know exactly where parsing stopped — which is the position you show the user.

Each scan returns a result. `proven_scan_i64` hands back `{err, val}`, so "not a number", "out of range" and "end of input" are different errors rather than one missing field.

Failure restores the cursor for the primitive scanners: a failed `proven_scan_i64` leaves the cursor where it was, so you can try something else at the same position. That is what makes recovery possible instead of guesswork.

One thing to know before you rely on it: the *structural* scan (`proven_scan_fmt`, the `{}` form) is **not** transactional across fields. If the third placeholder fails, the first two destinations have already been written. Chapter 8 §11.1 covers the error codes and the recovery patterns in full.

The scanner parses from a borrowed `proven_u8str_view_t`. A cursor tracks progress.

Wrong — treating a partial structural scan as if nothing happened:

```
proven_err_t e = proven_scan_fmt(&sc, "{} {} {}", ...);
if (!proven_is_ok(e)) {
    /* wrong: destinations for the fields that DID parse have already been written */
}
```

Result structs

```
typedef struct { proven_err_t err; proven_i64 val; } proven_result_i64_t;
typedef struct { proven_err_t err; proven_u64 val; } proven_result_u64_t;
typedef struct { proven_err_t err; double val; } proven_result_f64_t;
typedef struct { proven_err_t err; proven_u8str_view_t val; } proven_result_u8str_view_t;
```

proven_scan_t

```
typedef struct {
    proven_u8str_view_t view;
    proven_size_t cursor;
} proven_scan_t;
```

Purpose: hold input and current parse position. `proven_scan_init()` normalizes invalid non-empty null views to empty views.

Scanner functions

API	Intent	Return
`proven_scan_init(view)`	Create scanner from view.	`proven_scan_t`.
`proven_scan_skip_whitespace(scan)`	Advance past whitespace.	void.
`proven_scan_i64(scan)`	Parse signed 64-bit integer.	`proven_result_i64_t`.
`proven_scan_u64(scan)`	Parse unsigned 64-bit integer.	`proven_result_u64_t`.
`proven_scan_f64(scan)`	Parse floating-point value.	`proven_result_f64_t`.
`proven_parse_double_ascii(view)`	Parse one locale-free ASCII float token without skipping whitespace.	`proven_parse_double_result_t`.
`proven_parse_f64_ascii(view)`	Compatibility alias for the same locale-free binary64 parser.	`proven_parse_f64_result_t`.
`proven_strtod(nptr, endptr)`	Parse one `strtod`-style token with whitespace skipping and `endptr` reporting.	`double`.
`proven_scan_str(scan)`	Parse a whitespace-delimited token as view into input.	`proven_result_u8str_view_t`.
`proven_scan_skip_until(scan, target)`	Move cursor to target if found.	`proven_err_t`.
`proven_scan_skip_until_number(scan)`	Move cursor to next number-looking position.	void.

Scan argument types

`proven_scan_arg_type_t` identifies the destination kind stored in a `proven_scan_arg_t`. `PROVEN_SCAN_ARG(&x)` supports pointers to:

- `short`, `unsigned short`
- `int`, `unsigned int`
- `long`, `unsigned long`
- `long long`, `unsigned long long`
- `double`
- `proven_u8str_view_t`

Native destination constructors:

- `proven_scan_arg_short`, `proven_scan_arg_ushort`
- `proven_scan_arg_int`, `proven_scan_arg_uint`
- `proven_scan_arg_long`, `proven_scan_arg_ulong`
- `proven_scan_arg_llong`, `proven_scan_arg_ullong`

Explicit fixed-width and utility helpers:

- `proven_scan_arg_i32`, `proven_scan_arg_u32`
- `proven_scan_arg_i64`, `proven_scan_arg_u64`
- `proven_scan_arg_f64`
- `proven_scan_arg_str_view`
- `proven_scan_arg_none`
- `proven_scan_arg_identity`

Long aliases:

```
#define PROVEN_SCAN_ARG_LONG(ptr)  proven_scan_arg_long(ptr)
#define PROVEN_SCAN_ARG_ULONG(ptr) proven_scan_arg_ulong(ptr)
```

Format scanning macros

Macro	Intent
`PROVEN_SCAN_ARG(x)`	`_Generic` destination selector.
`proven_scan_fmt_cursor(scan_ptr, fmt, ...)`	Scan from an existing cursor.
`proven_scan_fmt(view, fmt, ...)`	Scan from a view with a temporary cursor.

Scan engine

```
proven_err_t proven_scan_fmt_internal(
    proven_scan_t *scan,
    const char *fmt,
    const proven_scan_arg_t *args,
    proven_size_t args_count
);
```

API	Intent	Return
`proven_scan_fmt_internal(scan, fmt, args, args_count)`	Structural scanner engine over an existing cursor.	`proven_err_t`.
`proven_scan_fmt_internal_view(view, fmt, args, count)`	Convenience wrapper that creates a temporary scanner over a view.	`proven_err_t`.

Normal user code should call the macros. The engine expects a leading sentinel argument.

Important behavior: `proven_scan_fmt_internal()` may advance the cursor and write earlier destinations before returning an error if a later literal mismatch occurs. Save the cursor and destination values first if you need transaction-like parsing.

Example:

```
proven_scan_t scan = proven_scan_init(PROVEN_LIT("ID: 402 SCORE: 99.5 ada"));
int id = 0;
double score = 0.0;
proven_u8str_view_t user = {0};

proven_err_t e = proven_scan_fmt_cursor(
    &scan,
    "ID: {} SCORE: {} {}",
    PROVEN_SCAN_ARG(&id),
    PROVEN_SCAN_ARG(&score),
    PROVEN_SCAN_ARG(&user)
);
if (!proven_is_ok(e)) {
    return;
}
/* id == 402, score == 99.5, user borrows "ada" out of the input - it is not a
 * copy, so it is only valid while the scanned bytes are. */
```

Float parsing notes

- `proven_scan_f64()` and `proven_parse_double_ascii()` route through the shared decimal-to-binary64 backend.
- Finite decimal inputs are rounded to IEEE-754 binary64 with round-to-nearest, ties-to-even behavior.
- The current conversion stack is `Clinger fast path -> staged Eisel-Lemire layer -> exact bigint fallback`, with internal counters used by tests to confirm which path took a representative input.
- The staged Eisel-Lemire layer currently accepts generated-`5^q` positive exponent cases and exact negative-exponent cases where the decimal significand cleanly cancels the required `5^q`.
- On compilers with `__uint128_t`, the same layer also accepts a conservative rounded negative-exponent ratio subset in normal-range cases, including wider left-shift normalization than the original narrow prototype allowed.
- The widened cached-power product path now also stages some subnormal cases, including `5e-324`, while values below the half-threshold to true-min still defer to the exact bigint fallback.
- The checked-in cached `5^q` constants are generated locally by `scripts/generate_float_decimal_tables.py`.
- `proven_parse_double_ascii()` does not skip leading whitespace.
- `proven_strtod()` skips leading ASCII whitespace, updates `endptr`, returns signed infinity on overflow, and preserves signed zero on underflow.

5. Examples and misuse cases

Views are not C strings

Wrong:

```
proven_u8str_view_t view = get_view();
printf("%s\n", (const char *)view.ptr); /* wrong: view may not be NUL-terminated */
```

Correct:

```
proven_u8str_view_t view = proven_u8str_view_slice(PROVEN_LIT("/etc/hosts"), 5, 5);

/* A view is a pointer and a length into somebody else's bytes. To hand it to a
 * C API that wants a NUL, allocate a real C string from it. */
proven_result_cstr_t c = proven_u8str_view_to_cstr(view, alloc);
if (!proven_is_ok(c.err)) {
    return;
}
```

```
/* c.value is "hosts", NUL-terminated. It is yours, so free it. */
alloc.free_fn(alloc.ctx, (void *)c.value);
```

`PROVEN_LIT` is for literals

Correct:

```
proven_u8str_view_t a = PROVEN_LIT("abc");
(void)a;
```

Wrong:

```
const char *runtime = getenv("NAME");
proven_u8str_view_t a = PROVEN_LIT(runtime); /* wrong: macro requires literal syntax */
```

Use:

```
const char *runtime = "NAME=value";   /* any trusted NUL-terminated string */
proven_u8str_view_t a = proven_u8str_view_from_cstr(runtime);
(void)a;
```

Distinguish not found from replaced

`proven_u8str_replace_first()` returns `PROVEN_OK` when the target is not found. Search first if that matters.

```
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("the old way"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

proven_size_t at = proven_u8str_view_find(proven_u8str_as_view(&s), 0, PROVEN_LIT("old"));
if (at != PROVEN_INDEX_NOT_FOUND) {
    (void)proven_u8str_replace_first(&s, 0, PROVEN_LIT("old"), PROVEN_LIT("new"));
}

proven_u8str_destroy(alloc, &s);
```

Bounded format input

Wrong:

```
char *untrusted = get_untrusted_pointer();
proven_println("{}", PROVEN_ARG(untrusted));
/* wrong: C-string formatting scans until NUL */
```

Correct:

```
char untrusted[16] = { 'n', 'o', ' ', 'n', 'u', 'l', ' ', 'h', 'e', 'r', 'e', '!', '!', '!', '!', '!' };
proven_size_t max_len = sizeof untrusted;

/* The bounded form stops looking for a NUL after max_len bytes. */
proven_println("{}", PROVEN_ARG_CSTR_N(untrusted, max_len));
```

Borrowed fixed-capacity strings

Use `proven_u8str_borrow` to format into a stack or static buffer without any allocation — useful in allocator-free code and on hot paths. Use the fixed-capacity operations (and `proven_u8str_append_fmt`), reuse with `proven_u8str_reset`, and do not call growing operations or `proven_u8str_destroy` on it (the caller owns the memory).

```
int cur = 3;
int total = 10;

proven_byte_t line[64];
proven_u8str_t s = proven_u8str_borrow(line, sizeof line);   /* cap includes NUL */

proven_fmt_result_t fr = proven_u8str_append_fmt(&s, "L{}/{}", PROVEN_ARG(cur), PROVEN_ARG(total));
```

```c
if (PROVEN_FMT_IS_OK(fr)) {
    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
}

(void)proven_u8str_reset(&s);   /* reuse next frame, no allocation */
```

Misuse: a growing call that would exceed the borrowed capacity returns `PROVEN_ERR_OUT_OF_BOUNDS` rather than reallocating caller memory.

```c
proven_byte_t small[4];
proven_u8str_t t = proven_u8str_borrow(small, sizeof small);
proven_err_t e = proven_u8str_append_grow(alloc, &t, PROVEN_LIT("toolong"));
(void)e;   /* e == PROVEN_ERR_OUT_OF_BOUNDS; small[] is untouched */
```

Self-referential formatting

Wrong:

```c
const char *inside = proven_u8str_as_cstr(&s);
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(inside));
/* wrong: self-aliasing C-string arguments are rejected */
```

Correct:

```c
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("ab"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

/* A view carries a length, so the formatter knows exactly which bytes to snapshot. */
proven_u8str_view_t before = proven_u8str_as_view(&s);
proven_fmt_result_t fr = proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(before));
(void)fr;   /* s == "abab" */

proven_u8str_destroy(alloc, &s);
```

Transactional scanning

Wrong assumption:

```c
proven_err_t e = proven_scan_fmt_cursor(&scan, "{} suffix", PROVEN_SCAN_ARG(&x));
/* if suffix mismatches, x and scan.cursor may already have changed */
```

Correct pattern:

```c
proven_scan_t scan = proven_scan_init(PROVEN_LIT("42 prefix"));
int x = -1;

proven_size_t old_cursor = scan.cursor;
int old_x = x;

proven_err_t e = proven_scan_fmt_cursor(&scan, "{} suffix", PROVEN_SCAN_ARG(&x));
if (!proven_is_ok(e)) {
    /* The literal "suffix" did not match, but 42 was already written into x and
     * the cursor already moved. Put both back yourself. */
    scan.cursor = old_cursor;
    x = old_x;
}
```

Worked example: owned strings and borrowed strings

Compiled and run by the test suite. The distinction that matters: an owned string may reallocate and must be destroyed; a borrowed one wraps caller memory, never reallocates, and refuses to grow past the buffer you gave it rather than quietly moving it.

```c
/*
 * There are two string handles here and the difference is ownership, not size:
 *
 *   proven_u8str_t      - a byte string you can edit. It either owns an
 *                         allocation (create) or borrows one of yours (borrow).
 *   proven_u8str_view_t - a pointer and a length into somebody else's bytes.
 *                         It owns nothing, it is not NUL-terminated, and it is
 *                         only valid while those bytes are.
 *
 * A view is what you pass to a function that reads. A u8str is what you keep.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- an OWNED string: the allocator's memory, yours to destroy ---------- */
    /* The capacity argument is content bytes; the NUL is extra, so as_cstr is
     * always O(1) and always safe. */
    proven_result_u8str_t r = proven_u8str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 16-byte string must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u8str_t path = r.value;

    /* append is fixed-capacity: it fits or it fails, and on failure it has not
     * touched the string. It never reallocates, so it needs no allocator. */
    proven_err_t err = proven_u8str_append(&path, PROVEN_LIT("/etc/hosts"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "10 bytes fit in a 16-byte string");

    /* append_grow is the growable twin: give it the allocator the string was
     * created with and it reallocates when needed. Still failure-atomic - if the
     * allocation fails, the string is exactly as it was. */
    err = proven_u8str_append_grow(alloc, &path, PROVEN_LIT(".backup.original"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "append_grow must reallocate rather than fail");

    /* Edits in the middle. insert shifts the tail right; remove shifts it left. */
    err = proven_u8str_insert_grow(alloc, &path, 0, PROVEN_LIT("/srv"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a prefix must succeed");

    err = proven_u8str_remove(&path, proven_u8str_as_view(&path).size - 9, 9);  /* drop ".original" */
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the trailing suffix must succeed");

    /* replace_first returns PROVEN_OK when the target is absent - "nothing to do"
     * is not an error. Search first when the difference matters to you. */
    err = proven_u8str_replace_first(&path, 0, PROVEN_LIT("hosts"), PROVEN_LIT("fstab"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "replacing an existing substring must succeed");

    /* --- reading it: borrow a view, do not copy ------------------------------ */
    /* as_view is free. The view is only good until the next edit: any growing
     * call may reallocate and leave the view (and any cstr) dangling. */
    proven_u8str_view_t v = proven_u8str_as_view(&path);

    EXAMPLE_REQUIRE(proven_u8str_view_eq(v, PROVEN_LIT("/srv/etc/fstab.backup")),
                    "the edits above should have produced /srv/etc/fstab.backup");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(v, PROVEN_LIT("/srv")),
                    "the inserted prefix is at the front");

    proven_size_t dot = proven_u8str_view_find(v, 0, PROVEN_LIT(".backup"));
    EXAMPLE_REQUIRE(dot != PROVEN_INDEX_NOT_FOUND, "the suffix must be found");
```

```
    /* A slice is a view into the SAME bytes - no allocation, no copy. */
    proven_u8str_view_t stem = proven_u8str_view_slice(v, 0, dot);
    EXAMPLE_REQUIRE(proven_u8str_view_eq(stem, PROVEN_LIT("/srv/etc/fstab")),
                    "slicing at the suffix leaves the stem");

    /* as_cstr is the escape hatch to C APIs, and it is only valid because the
     * owned string keeps a NUL past its length. Do NOT do this with a view:
     * `stem.ptr` is not NUL-terminated - it just points into `path`. */
    printf("owned:  %s\n", proven_u8str_as_cstr(&path));

    /* --- a BORROWED string: your memory, no allocation at all --------------- */
    /* Same type, same operations - but the bytes are this stack buffer. `cap`
     * includes the NUL, so this holds 31 content bytes. */
    proven_byte_t line[32];
    proven_u8str_t status = proven_u8str_borrow(line, sizeof line);

    err = proven_u8str_append(&status, PROVEN_LIT("mounted "));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending into a borrowed buffer needs no allocator");
    err = proven_u8str_append(&status, stem);
    EXAMPLE_REQUIRE(proven_is_ok(err), "a view can be appended just like a literal");

    /* The growing calls exist for a borrowed string, but they refuse to
     * reallocate memory they do not own: too much data is OUT_OF_BOUNDS, and
     * `line` is left untouched. A borrowed string cannot silently escape to the
     * heap behind your back. */
    err = proven_u8str_append_grow(alloc, &status,
                                   PROVEN_LIT(" ...and a great deal more text than fits"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a borrowed string reports overflow instead of reallocating caller memory");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&status), PROVEN_LIT("mounted /srv/etc/fstab")),
                    "the failed append must have left the string unchanged");

    printf("borrowed: %s\n", proven_u8str_as_cstr(&status));

    /* reset truncates to empty and keeps the buffer, so the next frame reuses
     * the same 32 bytes with no allocation. */
    err = proven_u8str_reset(&status);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reset must succeed on a borrowed string");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&status).size == 0, "reset empties the string");

    /* --- destroy: the ownership rule, spelled out --------------------------- */
    /* destroy on the borrowed string is a no-op - `line` is not the library's to
     * free. Calling it anyway is correct and costs nothing, and it means the
     * teardown code does not have to know which kind of string it holds. */
    proven_u8str_destroy(alloc, &status);

    /* destroy on the owned string frees the allocation, and it must be given the
     * allocator the string was created with. */
    proven_u8str_destroy(alloc, &path);
    return EXAMPLE_OK();
}
```

Worked example: when the buffer is fixed and the data is not

The example above grows the string whenever it runs out of room. A great deal of real code cannot do that: a record has a fixed field width, a log line has a hard limit, a string lives in an arena where every reallocation leaves the old copy behind until the next reset. Those callers need to know which of three answers a call gives when the data does not fit.

Kind of call	What it does when the data does not fit	Calls
Atomic, fixed capacity	Refuses with `PROVEN_ERR_OUT_OF_BOUNDS` and changes nothing.	`proven_u8str_append`, `proven_u8str_insert`, `proven_u8str_replace_at`
Best-effort (truncating)	Writes what fits, returns `PROVEN_ERR_OUT_OF_BOUNDS` **and** the byte count it wrote.	`proven_u8str_append_partial`
Atomic, growable	Reallocates through the allocator; on allocation failure changes nothing.	`proven_u8str_append_grow`, `proven_u8str_replace_at_grow`, `proven_u8str_append_byte`

Two supporting calls appear here as well. `proven_u8str_reserve()` raises the capacity once, up front, so later growth does not reallocate — on the heap that saves copying, and in an arena it saves the dead storage every reallocation leaves behind. `proven_u8str_is_valid()` checks that a string handle's own fields are consistent; it is worth asserting where a string arrives from other code, not after every edit.

The program below writes a bounded log line, refuses an oversized append, truncates deliberately with the best-effort call, and edits a path in the middle with `proven_u8str_replace_at()` — first shrinking (which always fits), then growing (which does not, and is refused), then the same edit again through `proven_u8str_replace_at_grow()`. It ends by checking the file extension with `proven_u8str_view_ends_with()`.

```
/*
 * The previous example grew a string whenever it ran out of room. This one is
 * about the case where growing is not allowed - a fixed-size record, a log line
 * with a hard length limit, a buffer in an arena that must not be reallocated -
 * and about the two honest answers a call can give when the data does not fit:
 *
 *   "no, and I changed nothing"      - the atomic calls: append, insert,
 *                                      replace_at. They check the capacity
 *                                      first, so a refusal leaves the string
 *                                      exactly as it was.
 *   "some of it, and here is how much" - the best-effort call:
 *                                      append_partial. It fills what it can and
 *                                      tells you the byte count it wrote.
 *
 * Both are useful; picking the wrong one silently truncates a record or
 * silently drops one. The `_grow` variants are the third answer - "yes, I found
 * more room" - and appear at the end for contrast.
 */

#define FIELD_CAP 32u

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_u8str_t r = proven_u8str_create(alloc, FIELD_CAP);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating the field buffer must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u8str_t field = r.value;

    /* is_valid checks the handle's own structure - a pointer with a capacity and
```

```
     * a length that do not contradict each other. Worth asserting once at the
     * boundary of your code when a string arrives from somewhere else; it is not
     * a check you need after every edit, because every edit maintains it. */
    EXAMPLE_REQUIRE(proven_u8str_is_valid(&field), "a freshly created string must be structurally valid");

    /* --- reserving room up front ------------------------------------------ */

    /* reserve raises the capacity now, so later growth does not reallocate. On
     * the heap that saves copies; in an arena it saves something worse, because
     * every reallocation there leaks the old block until the next reset. Ask for
     * what you expect to need, once. */
    proven_err_t err = proven_u8str_reserve(alloc, &field, 64);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reserving 64 bytes must succeed");
    EXAMPLE_REQUIRE(field.internal.cap >= 64, "the capacity must actually be at least what was asked for");

    /* --- the atomic calls: fit, or change nothing ------------------------- */

    err = proven_u8str_append(&field, PROVEN_LIT("2026-01-01 level=info "));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the prefix fits in the reserved capacity");

    /* append_byte adds one byte, which is what separators, terminators and
     * escape characters are. It takes the allocator because it is a growing
     * call - one byte is exactly the case where a capacity check would fail on
     * a boundary you did not think about. */
    err = proven_u8str_append_byte(alloc, &field, (proven_u8)'[');
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending a single separator byte must succeed");

    err = proven_u8str_append(&field, PROVEN_LIT("disk full"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the message fits");

    err = proven_u8str_append_byte(alloc, &field, (proven_u8)']');
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing the bracket must succeed");

    /* Now ask for more than the capacity can hold. The atomic append refuses and
     * - this is the property worth relying on - the string still holds exactly
     * what it held before the call. */
    proven_size_t before = proven_u8str_as_view(&field).size;
    err = proven_u8str_append(&field, PROVEN_LIT(" and a very long trailing explanation that certainly does
not fit"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "an oversized atomic append must be refused");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&field).size == before, "and must leave the string untouched");

    /* --- the best-effort call: as much as fits, and the count ------------- */

    /* A fixed-width column in a report is the case for this one: write what
     * fits, and know how much was written so the caller can mark the value as
     * truncated instead of pretending it is complete. */
    proven_result_size_t part = proven_u8str_append_partial(&field, PROVEN_LIT(" ...more text than there is
room for"));
    EXAMPLE_REQUIRE(part.err == PROVEN_ERR_OUT_OF_BOUNDS, "a partial append that truncates still reports the
truncation");
    EXAMPLE_REQUIRE(part.value > 0, "but it wrote what it could");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&field).size == before + part.value,
                    "and the string grew by exactly the number of bytes it reports");
    printf("partial append wrote %zu byte(s) before the buffer was full\n", (size_t)part.value);

    /* --- editing in the middle without growing ---------------------------- */

    proven_result_u8str_t r2 = proven_u8str_create(alloc, FIELD_CAP);
    EXAMPLE_REQUIRE(proven_is_ok(r2.err), "creating the second buffer must succeed");
```

```
    proven_u8str_t path = r2.value;
    err = proven_u8str_append(&path, PROVEN_LIT("var/log/service.log"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the path fits");

    /* insert shifts the tail right. Fixed-capacity: it fits or it refuses. */
    err = proven_u8str_insert(&path, 0, PROVEN_LIT("/"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a leading slash must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&path), PROVEN_LIT("/var/log/service.log")),
                    "the insert lands at index 0");

    /* replace_at replaces old_len bytes at an index with data of any length, as
     * long as the result still fits. Replacing "service" (7) with "daemon" (6)
     * shrinks the string, so this cannot fail on capacity. */
    proven_size_t at = proven_u8str_view_find(proven_u8str_as_view(&path), 0, PROVEN_LIT("service"));
    EXAMPLE_REQUIRE(at != PROVEN_SIZE_MAX, "the substring must be found before it can be replaced");
    err = proven_u8str_replace_at(&path, at, 7, PROVEN_LIT("daemon"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "a shortening replacement must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&path), PROVEN_LIT("/var/log/daemon.log")),
                    "and produce the expected path");

    /* The same edit the other way round overflows a 32-byte buffer, and the
     * fixed-capacity call refuses it rather than truncating a path - which is
     * the failure that silently writes to the wrong file. */
    before = proven_u8str_as_view(&path).size;
    err = proven_u8str_replace_at(&path, at, 6, PROVEN_LIT("a-replacement-name-far-too-long-for-this-
buffer"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "a replacement that does not fit must be refused");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&path).size == before, "and must leave the path unchanged");

    /* replace_at_grow is the same edit with permission to reallocate. Use it
     * when the buffer is heap-backed and the length is genuinely unbounded;
     * prefer the fixed-capacity call when the limit is part of the format. */
    err = proven_u8str_replace_at_grow(alloc, &path, at, 6, PROVEN_LIT("a-replacement-name-far-too-long-for-
this-buffer"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the growing variant makes room instead of refusing");
    EXAMPLE_REQUIRE(proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT(".log")),
                    "the extension is still at the end after the edit");

    /* ends_with answers the question an extension check actually asks. Doing it
     * with an index computed by hand is where the off-by-one lives; doing it
     * with strcmp on a pointer requires a NUL that a view does not have. */
    EXAMPLE_REQUIRE(!proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT(".txt")),
                    "and it is not a .txt file");

    /* An empty suffix is a suffix of everything, which is the answer that keeps
     * loops over a list of suffixes from needing a special case. */
    EXAMPLE_REQUIRE(proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT("")),
                    "every string ends with the empty suffix");

    printf("log line: %s\n", proven_u8str_as_cstr(&field));
    printf("path:     %s\n", proven_u8str_as_cstr(&path));

    proven_u8str_destroy(alloc, &path);
    proven_u8str_destroy(alloc, &field);
    return EXAMPLE_OK();
}
```

Wrong — treating the best-effort call as if it were atomic:

```
(void)proven_u8str_append_partial(&line, field);   /* wrong: ignores the count */
```

The return value is the only place the truncation is reported. Discarding it turns "the record was cut short" into "the record looked fine".

Wrong — reading the byte count as a character count:

```
if (part.value == field.size) { /* all of it was written */ }
```

That comparison is correct, and it is correct in **bytes**. A UTF-8 character can be up to four of them, so a partial append can stop in the middle of one. When the tail must stay valid UTF-8, decide the cut point yourself rather than letting the capacity decide it.

Worked example: assembling a UTF-16 string for a system call

`proven_u16str_t` earns its place only at the boundary where an operating system call demands UTF-16 — the Windows wide API being the usual reason. The pattern is always the same: assemble the code units, then hand `proven_u16str_as_ptr()` to the call.

Three points decide whether this code is right:

- **The unit is a code unit, not a byte and not a character.** A capacity of 32 is 32 code units, which is 64 bytes; a character outside the Basic Multilingual Plane (BMP) — an emoji, many rarer CJK characters — occupies two of them.
- `proven_u16str_as_ptr()` **does not copy**, and the pointer it returns is good only until the next append that grows the string.
- **The result is NUL-terminated**, including after a deliberate truncation, so it is safe to pass to a system call that expects a terminator.

```
/*
 * UTF-16 exists in this library for one reason: some operating system calls
 * take it and nothing else. The Windows "wide" API is the usual case - the file
 * name you hand to CreateFileW is a NUL-terminated run of 16-bit code units,
 * not bytes.
 *
 * So the job this type does is narrow: assemble the code units, keep the count
 * right, and produce the pointer the system call wants. Everything else in your
 * program should stay UTF-8.
 *
 * The one thing to keep straight is the unit. A capacity of 32 here means 32
 * CODE UNITS, which is 64 bytes, and a character outside the Basic Multilingual
 * Plane - an emoji, most of the rarer CJK characters - costs two of them. A
 * count of code units is not a count of characters and never has been.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* The argument is a code-unit limit, not a byte limit. */
    proven_result_u16str_t r = proven_u16str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 32-code-unit string must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u16str_t name = r.value;
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == 0, "a new string is empty");

    /* PROVEN_U16_LIT builds a view from a u"..." literal and computes the unit
     * count from the literal itself, so the count cannot disagree with the text. */
    proven_err_t err = proven_u16str_append(&name, PROVEN_U16_LIT("C:\\logs\\"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the directory prefix fits in 32 code units");

    err = proven_u16str_append(&name, PROVEN_U16_LIT("service.log"));
```

```
    EXAMPLE_REQUIRE(proven_is_ok(err), "the file name fits too");
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == 8 + 11, "the length is a count of code units");

    /* Atomic, like its byte-string twin: too much data is refused and the string
     * is left exactly as it was, so a path is never half-written. */
    proven_size_t before = proven_u16str_len(&name);
    err = proven_u16str_append(&name, PROVEN_U16_LIT(".a-suffix-long-enough-to-overflow-the-capacity"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "an oversized append must be refused");
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == before, "and must not truncate the path");

    /* --- the pointer the system call wants ------------------------------- */

    /* as_ptr hands back the internal code units, NUL-terminated, without
     * copying. It is the last step before the call, and the pointer is only
     * valid until the next append: a growing append may move the storage. */
    const proven_u16 *wide = proven_u16str_as_ptr(&name);
    EXAMPLE_REQUIRE(wide != NULL, "an assembled string must yield a pointer");
    EXAMPLE_REQUIRE(wide[0] == (proven_u16)'C', "the first code unit is the drive letter");
    EXAMPLE_REQUIRE(wide[proven_u16str_len(&name)] == 0, "the sequence is NUL-terminated for the system
call");
    /* On Windows this is the whole point of the type:
     *     HANDLE h = CreateFileW((LPCWSTR)wide, ...);
     * Nothing here calls it, because this example must also run everywhere else. */

    /* --- when truncation is the correct answer --------------------------- */

    /* Some system structures have a fixed-width field - a 16-unit label, say -
     * where a name that does not fit is meant to be cut, not rejected. That is
     * what the partial append is for: it fills what it can and reports the unit
     * count it wrote, so the caller can mark the value as truncated. */
    proven_result_u16str_t r2 = proven_u16str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(r2.err), "creating the fixed-width label must succeed");
    proven_u16str_t label = r2.value;

    proven_result_size_t wrote = proven_u16str_append_partial(&label, PROVEN_U16_LIT("a-label-that-is-longer-
than-the-field"));
    EXAMPLE_REQUIRE(wrote.err == PROVEN_ERR_OUT_OF_BOUNDS, "the truncation is reported, not hidden");
    EXAMPLE_REQUIRE(wrote.value == 16, "it filled the field exactly");
    EXAMPLE_REQUIRE(proven_u16str_len(&label) == wrote.value, "and the length matches what it says it wrote");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&label)[wrote.value] == 0,
                    "a truncated string is still NUL-terminated, so it is still safe to pass on");

    printf("assembled %zu code unit(s); label truncated to %zu\n",
           (size_t)proven_u16str_len(&name), (size_t)wrote.value);

    proven_u16str_destroy(alloc, &label);
    proven_u16str_destroy(alloc, &name);
    return EXAMPLE_OK();
}
```

Wrong — sizing a UTF-16 buffer in bytes:

```
proven_result_u16str_t r = proven_u16str_create(alloc, sizeof(buf));   /* wrong */
```

The argument is a count of code units. Passing a byte count asks for twice the storage you meant, or — when the byte count came from a UTF-8 string — for a buffer that cannot hold the conversion at all.

Wrong — keeping the pointer across an append:

```
const proven_u16 *w = proven_u16str_as_ptr(&name);
proven_err_t e = proven_u16str_append_grow(alloc, &name, more);   /* may reallocate */
use_wide(w);   /* wrong: w may dangle */
```

Take the pointer immediately before the call that consumes it.

Chapter 4: Containers and Algorithms

Part III — Data structures. Prerequisite: Part II (1, 2, 3). After this chapter you can pick the right container for a job, sort and search with a guaranteed bound, hash for the right reason, and turn bytes into text and back.

This chapter covers `array.h`, `list.h`, `ring.h`, `map.h`, `algorithm.h`, `hash.h`, and `encode.h`. Every container here takes an allocator, which is why Chapter 2 comes first.

Table of contents

1. Dynamic array

The problem: the array you write by hand, every time

A growable array is the data structure every C program eventually writes, and writes slightly differently:

```
int *items = malloc(cap * sizeof(int));
/* ... later ... */
if (n == cap) {
    cap *= 2;
    items = realloc(items, cap * sizeof(int));   /* wrong on two counts */
    items[n++] = x;
}
```

Two bugs in one line, and both are classics. `cap * sizeof(int)` can **wrap** for a large `cap`, producing a small allocation for a huge count — see Chapter 1 §4. And assigning `realloc`'s result straight back to `items` **leaks the old block when it returns NULL**, because the original pointer is gone and the memory it named is still allocated.

Beyond the bugs, the hand-written version has to be rewritten for every element type, or made generic with `void *` and lose type checking.

What this library does instead

`proven_array_t` is a growable vector that keeps the element size and alignment it was created with, so it works for any type without a template and without `void *` at the call site — the `PROVEN_ARRAY_*` macros take the type and do the casting where the compiler can still check it.

- Growth uses checked arithmetic, so the overflow above is `PROVEN_ERR_OVERFLOW`, not a small allocation.
- Growth is **failure-atomic**: if it cannot grow, your existing elements are untouched and still valid. Nothing is leaked and nothing is lost.
- It **stores its allocator internally**, unlike the string types — which is why `PROVEN_ARRAY_DESTROY` takes only the array.

`proven_array_t` owns contiguous element storage, so it is also what you hand to `proven_array_sort` and the searches in §5.

Wrong — holding a pointer across a push:

```
int *first = PROVEN_ARRAY_GET(&arr, int, 0);
(void)PROVEN_ARRAY_PUSH(&arr, int, 42);   /* may reallocate */
*first = 7;                               /* wrong: `first` may point at freed memory */
```

This is the one hazard the type cannot remove. Growth moves the storage, so **any pointer or view into the array is invalidated by any operation that can grow it**. Index instead of pointing, or re-fetch the pointer after the push.

Structures

```
typedef struct {
    proven_allocator_t alloc;
    proven_byte_t *data;
    proven_size_t len;
    proven_size_t cap;
    proven_size_t elem_size;
    proven_size_t align;
} proven_array_t;

typedef struct {
    proven_err_t err;
    proven_array_t value;
} proven_result_array_t;
```

Fields:

- `alloc`: allocator used for growth and destruction.
- `data`: byte storage for elements.
- `len`: current element count.
- `cap`: capacity in elements.
- `elem_size`: size of each element.
- `align`: alignment of each element.

Functions

API	Intent	Return
`proven_array_create(alloc, init_cap, elem_size, align)`	Create a generic array.	`proven_result_array_t`.
`proven_array_is_valid(arr)`	Validate public array invariants.	`bool`.
`proven_array_reserve(arr, new_cap)`	Ensure at least `new_cap` elements.	`proven_err_t`.
`proven_array_push(arr, element)`	Append one element by copying from pointer.	`proven_err_t`.
`proven_array_pop(arr, out_element)`	Pop last element; `out_element` may be null to discard.	`proven_err_t`.
`proven_array_get_mut(arr, index)`	Get mutable element pointer.	pointer or null.
`proven_array_get(arr, index)`	Get const element pointer.	pointer or null.
`proven_array_destroy(arr)`	Free storage and clear state.	void.

Macros

Macro	Intent
`PROVEN_ARRAY_INIT(alloc, type, init_cap)`	Type-safe create using `sizeof(type)` and `alignof(type)`.
`PROVEN_ARRAY_PUSH(arr_ptr, type, value)`	Push rvalue or lvalue by temporary compound literal.
`PROVEN_ARRAY_POP(arr_ptr, type, out_ptr)`	Pop into output pointer or discard with null.
`PROVEN_ARRAY_GET(arr_ptr, type, index)`	Typed const element pointer.

`PROVEN_ARRAY_GET_MUT(arr_ptr, type, index)`	Typed mutable element pointer.
`PROVEN_ARRAY_DESTROY(arr_ptr)`	Destroy array.

Example:

```
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, int, 4);
if (!proven_is_ok(r.err)) {
    return;
}
proven_array_t nums = r.value;

(void)PROVEN_ARRAY_PUSH(&nums, int, 10);
(void)PROVEN_ARRAY_PUSH(&nums, int, 20);

/* GET points into the array's own storage, and returns NULL out of range. */
const int *first = PROVEN_ARRAY_GET(&nums, int, 0);
if (first) {
    proven_println("first={}", PROVEN_ARG(*first));
}

/* The array stores its allocator, so destroy needs nothing but the array. */
PROVEN_ARRAY_DESTROY(&nums);
```

2. Intrusive list

The problem: the list you were taught allocates a node per element

`struct node { struct node *next; void *data; };`

That is the textbook linked list, and it has two costs the textbook does not mention. Every insertion **allocates** — a thousand items means a thousand allocations that exist only to hold pointers, each one able to fail and each one needing a matching free. And `data` is a `void *`, so the list has no idea what it holds: every read back out is a cast the compiler cannot check.

What an intrusive list does instead

It turns the relationship inside out: **you own the memory, and the link lives inside it.**

```
typedef struct {
    int                id;
    proven_list_node_t  link;   /* the list's hook, inside your struct */
} task_t;
```

Now inserting is writing two pointers. It allocates nothing, so it **cannot fail** — notice that `proven_list_push_back` returns `void`, which is unusual in this library and is the point. Removing is the same. The objects can live on the stack, in an array, in an arena, anywhere; the list only rearranges pointers that are already inside them.

Getting from a link back to your object is `PROVEN_LIST_ENTRY(node, task_t, link)`, which subtracts the member's offset from the node's address. It is arithmetic the compiler checks the types of, not a cast you hope is right.

What you give up: **an object can be in only as many lists as it has link members**, and you decide that when you declare the struct. If an item needs to be in a queue and an index at the same time, it needs two links.

`proven_list_t` is an intrusive doubly-linked **circular** list — the head is a sentinel node, so insertion and removal never special-case the ends. It allocates no nodes.

Wrong — removing while walking with the plain iterator:

```
PROVEN_LIST_FOR_EACH(it, &queue) {
    if (should_drop(it)) proven_list_remove(it);   /* wrong: it->next is read after unlink */
}
```

`proven_list_remove` writes through the node's own pointers, so the loop then reads `next` from a node that has just been detached. `PROVEN_LIST_FOR_EACH_SAFE` takes a second variable and reads the next pointer *before* the body runs, which is exactly why it exists.

Worked example: a queue whose links live in the caller's structs
Compiled and run by the test suite. It builds a queue of stack-allocated tasks, walks it, removes one from inside a safe walk, and inserts in the middle — with no allocator anywhere in the program.

```
/*
 * An INTRUSIVE list puts the links inside your struct instead of allocating a
 * node to hold your data.
 *
 * The list you were taught looks like this:
 *
 *     struct node { struct node *next; void *data; };
 *
 * Every insertion allocates a node, so a list of a thousand items costs a
 * thousand allocations that exist only to hold pointers, each one a chance to
 * fail and a thing to free. Worse, `data` is a void* - the list has no idea what
 * it holds, so every read is a cast the compiler cannot check.
 *
 * Intrusive lists invert it: YOU own the memory, and the link lives in it.
 * Inserting allocates nothing and cannot fail. Removing allocates nothing and
 * cannot fail. And because the link is a member of a known type, getting back
 * from a link to the object is arithmetic the compiler does for you, not a cast.
 *
 * The trade is that an object can only be in as many lists as it has link
 * members, and that is a decision you make when you declare the struct.
 */

/* The link is a member. This task can be in exactly one list at a time. */
typedef struct {
    int                id;
    proven_list_node_t  link;
} task_t;

int main(void) {
    /* No allocator anywhere in this program: the tasks are on the stack, and the
     * list only ever rearranges pointers that live inside them. */
    task_t a = { .id = 1 };
    task_t b = { .id = 2 };
    task_t c = { .id = 3 };

    proven_list_t queue;
    proven_list_init(&queue);
    EXAMPLE_REQUIRE(proven_list_is_empty(&queue), "a freshly initialised list is empty");

    /* --- pushing cannot fail, because nothing is allocated ------------- */
    proven_list_push_back(&queue, &a.link);
    proven_list_push_back(&queue, &b.link);
    proven_list_push_back(&queue, &c.link);
    EXAMPLE_REQUIRE(!proven_list_is_empty(&queue), "three tasks are queued");

    /* --- walking: PROVEN_LIST_ENTRY gets the object back from the link -- */
    proven_list_node_t *it = NULL;
    int seen[3] = {0}, n = 0;
```

```
    PROVEN_LIST_FOR_EACH(it, &queue) {
        task_t *t = PROVEN_LIST_ENTRY(it, task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 3 && seen[0] == 1 && seen[1] == 2 && seen[2] == 3,
                    "the walk visits every task, in insertion order");

    /* --- removing while walking needs the SAFE form -------------------- */
    /* proven_list_remove writes through the node's own next/prev pointers, so a
     * plain FOR_EACH would read `it->next` from a node that has just been
     * unlinked. The _SAFE form reads the next pointer BEFORE the body runs. */
    proven_list_node_t *safe = NULL;
    PROVEN_LIST_FOR_EACH_SAFE(it, safe, &queue) {
        task_t *t = PROVEN_LIST_ENTRY(it, task_t, link);
        if (t->id == 2) proven_list_remove(it);
    }

    n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
        task_t *t = PROVEN_LIST_ENTRY(it, task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 2 && seen[0] == 1 && seen[1] == 3, "task 2 was unlinked");

    /* --- inserting in the middle is a pointer swap --------------------- */
    task_t d = { .id = 4 };
    proven_list_insert_after(&a.link, &d.link);

    n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
        task_t *t = PROVEN_LIST_ENTRY(it, task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 3 && seen[0] == 1 && seen[1] == 4 && seen[2] == 3,
                    "task 4 sits directly after task 1");

    /* There is nothing to destroy. The list never owned anything: `a`, `c` and
     * `d` are still perfectly good local variables, and their lifetime is the
     * function's, exactly as it would be without the list. */
    return EXAMPLE_OK();
}
```

Structures

```
typedef struct proven_list_node_t {
    struct proven_list_node_t *next;
    struct proven_list_node_t *prev;
} proven_list_node_t;

typedef struct {
    proven_list_node_t head;
} proven_list_t;
```

Functions and macros

API	Intent	Return
`proven_list_init(list)`	Initialize circular sentinel.	void.
`proven_list_insert_after(target, node)`	Insert `node` after `target`.	void.
`proven_list_push_back(list, node)`	Insert at tail.	void.

`proven_list_remove(node)`	Detach node and poison links to null.	void.
`proven_list_is_empty(list)`	Test empty or null list.	int truth value.
`PROVEN_CONTAINER_OF(ptr, type, member)`	Convert member pointer to parent object pointer.	pointer.
`PROVEN_LIST_FOR_EACH(iter, list)`	Iterate nodes.	loop macro.
`PROVEN_LIST_FOR_EACH_SAFE(iter, safe_next, list)`	Iterate safely while removing current node.	loop macro.
`PROVEN_LIST_ENTRY(ptr, type, member)`	Convert list node to containing object.	pointer.

Example:

```
typedef struct Item {
    int value;
    proven_list_node_t link;   /* the list lives inside the object; it allocates nothing */
} Item;

proven_list_t list;
proven_list_init(&list);

Item a = { .value = 1 };
Item b = { .value = 2 };
proven_list_push_back(&list, &a.link);
proven_list_push_back(&list, &b.link);

int total = 0;
proven_list_node_t *it = NULL;
PROVEN_LIST_FOR_EACH(it, &list) {
    /* The iterator walks nodes; ENTRY converts a node back to its owner. */
    Item *item = PROVEN_LIST_ENTRY(it, Item, link);
    total += item->value;
}
proven_println("total={}", PROVEN_ARG(total));   /* 3 */
```

3. Ring buffer

The problem: a queue that must not grow

A queue between a fast producer and a slow consumer has to answer one question: what happens when the consumer falls behind? A growable queue answers "allocate more", which turns a temporary slowdown into unbounded memory growth and, eventually, into something worse than dropped work.

The hand-written alternative is an array plus a head index plus a tail index plus modulo arithmetic — and it has one famous bug. When `head == tail`, is the buffer empty or full? Both states look identical unless you keep a separate count or deliberately waste one slot, and every generation of programmers rediscovers this.

What this library does instead

`proven_ring_t` is a fixed-capacity FIFO that keeps the count, so empty and full are distinct, and that **refuses** rather than grows:

- `proven_ring_push` on a full ring returns `PROVEN_ERR_OUT_OF_BOUNDS`. It does not overwrite the oldest entry and it does not reallocate. The caller decides — wait, drop the new item, or report backpressure — because only the caller knows which is right.
- `proven_ring_pop` on an empty ring fails rather than handing back a stale slot.

The capacity is chosen once, at creation, and that number *is* the policy: it says how far ahead the producer may get. Reach for a ring for an event queue, a log of recent items, an audio or sensor buffer — anywhere a bound is part of the design rather than a limitation.

Unlike most types in this library, `proven_ring_t` **stores its allocator internally**, which is why `PROVEN_RING_DESTROY` takes only the ring.

Wrong — treating a full ring as an error to retry immediately:

```
while (PROVEN_RING_PUSH(&ring, event_t, e) != PROVEN_OK) { }   /* wrong: spins forever */
```

Nothing in that loop consumes anything, so the ring stays full. A refusal from a bounded queue is information about the *consumer*, and the loop that ignores it is a hang.

Worked example: pushing until full, and what refusal looks like
Compiled and run by the test suite.

```
/*
 * A ring buffer is a fixed-size queue that never moves its contents and never
 * grows. You give it a capacity once; push adds at the tail, pop removes from
 * the head, and when it is full, push REFUSES.
 *
 * The C you would otherwise write is an array plus two indices plus the modulo
 * arithmetic to wrap them, and the bug is always the same: "is it full or is it
 * empty?" - both states have head == tail unless you keep a count or waste a
 * slot. This ring keeps the count, so the question does not arise.
 *
 * Use one when a producer and a consumer run at different speeds and you want a
 * hard bound on how far ahead the producer may get: an event queue, a log ring,
 * an audio buffer. The refusal on a full ring is the feature - it is
 * backpressure. A growable queue would answer a burst by eating memory until
 * something worse happens.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* Capacity of 4 events. It will never be 5. */
    proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 4-slot ring should succeed");
    proven_ring_t ring = r.value;

    /* --- push until full ------------------------------------------- */
    for (int i = 1; i <= 4; ++i) {
        proven_err_t err = PROVEN_RING_PUSH(&ring, int, i);
        EXAMPLE_REQUIRE(proven_is_ok(err), "the first four pushes fit");
    }

    /* The fifth push is refused. It does not overwrite the oldest entry, and it
     * does not grow: a full ring is a full ring, and the caller decides what to
     * do about it - wait, drop the new event, or report backpressure. */
    proven_err_t full = PROVEN_RING_PUSH(&ring, int, 5);
    EXAMPLE_REQUIRE(full == PROVEN_ERR_OUT_OF_BOUNDS,
                    "pushing into a full ring must be refused, not silently absorbed");

    /* --- pop in the order they were pushed ------------------------- */
    int out = 0;
    proven_err_t err = PROVEN_RING_POP(&ring, int, &out);
    EXAMPLE_REQUIRE(proven_is_ok(err) && out == 1, "pop returns the oldest entry first");

    /* Now there is room again, and the ring wraps around its storage without
```

```
     * moving anything. */
    err = PROVEN_RING_PUSH(&ring, int, 5);
    EXAMPLE_REQUIRE(proven_is_ok(err), "after a pop there is room for one more");

    int expected[] = { 2, 3, 4, 5 };
    for (int i = 0; i < 4; ++i) {
        err = PROVEN_RING_POP(&ring, int, &out);
        EXAMPLE_REQUIRE(proven_is_ok(err) && out == expected[i],
                        "entries come out in the order they went in");
    }

    /* --- empty behaves like full: it refuses, it does not invent data --- */
    err = PROVEN_RING_POP(&ring, int, &out);
    EXAMPLE_REQUIRE(!proven_is_ok(err), "popping an empty ring must fail rather than return junk");

    PROVEN_RING_DESTROY(&ring);
    return EXAMPLE_OK();
}
```

Structures

```
typedef struct {
    proven_allocator_t alloc;
    proven_mem_mut_t internal;
    proven_size_t head;
    proven_size_t tail;
    proven_size_t len;
    proven_size_t cap;
    proven_size_t elem_size;
    proven_size_t align;
} proven_ring_t;

typedef struct {
    proven_err_t err;
    proven_ring_t value;
} proven_result_ring_t;
```

Functions and macros

API	Intent	Return
`proven_ring_create(alloc, cap, elem_size, align)`	Create fixed-capacity ring.	`proven_result_ring_t`.
`proven_ring_is_valid(ring)`	Validate ring invariants.	`bool`.
`proven_ring_push(ring, element)`	Push one element; fails when full.	`proven_err_t`.
`proven_ring_pop(ring, out_element)`	Pop one element; null output discards.	`proven_err_t`.
`proven_ring_destroy(ring)`	Free storage.	void.
`PROVEN_RING_INIT(alloc, type, cap)`	Type-safe create.	`proven_result_ring_t`.
`PROVEN_RING_PUSH(ring_ptr, type, value)`	Type-safe push.	`proven_err_t`.
`PROVEN_RING_POP(ring_ptr, type, out_ptr)`	Type-safe pop.	`proven_err_t`.
`PROVEN_RING_DESTROY(ring_ptr)`	Destroy ring.	void.

Example:

```
proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 8);
if (!proven_is_ok(r.err)) {
    return;
}
```

```
proven_ring_t q = r.value;

proven_err_t e = PROVEN_RING_PUSH(&q, int, 7);
if (!proven_is_ok(e)) {
    /* The ring is fixed-capacity: a full ring is PROVEN_ERR_OUT_OF_BOUNDS,
     * never a silent grow. */
    PROVEN_RING_DESTROY(&q);
    return;
}

int out = 0;
e = PROVEN_RING_POP(&q, int, &out);   /* FIFO: out == 7. Empty ring is OUT_OF_BOUNDS. */
(void)e;

PROVEN_RING_DESTROY(&q);
```

4. Hash map

`proven_map_t` is an open-addressing hash map with tombstones. It supports integer keys and borrowed or owned U8 string keys. It stores its allocator internally.

Structures

```
typedef enum {
    PROVEN_KEY_TYPE_INT,          /* keys are proven_size_t integers */
    PROVEN_KEY_TYPE_U8_BORROWED,  /* keys are u8 views; caller keeps the bytes alive */
    PROVEN_KEY_TYPE_U8_OWNED      /* keys are u8 views; the map copies and frees the bytes */
} proven_key_type_t;

typedef union {
    proven_size_t id;             /* used when key_type == PROVEN_KEY_TYPE_INT */
    proven_u8str_view_t str;      /* used for the two U8 key modes */
} proven_map_key_t;

typedef struct {
    proven_allocator_t alloc;     /* allocator for the bucket array and owned keys */
    proven_mem_mut_t internal;    /* the single contiguous bucket array (ptr + size) */
    proven_size_t len;            /* live entries */
    proven_size_t used;           /* live entries + tombstones (drives the load factor) */
    proven_size_t cap;            /* number of buckets, always a power of two */
    proven_size_t elem_size;      /* size of one stored value */
    proven_size_t align;          /* alignment requested for the value */
    proven_size_t bucket_stride;  /* bytes per bucket: align_up(header + elem_size) */
    proven_size_t payload_offset; /* bytes from a bucket start to its value payload */
    proven_key_type_t key_type;   /* key mode chosen at create time */
} proven_map_t;

typedef struct {
    proven_err_t err;
    proven_map_t value;
} proven_result_map_t;
```

How it works internally

The map is a single flat array of `cap` buckets — there are no per-entry allocations for values (and none for keys in INT/BORROWED mode), so lookups stay cache-friendly. Each bucket is laid out as:

```
[ header: state + key ][ padding to the value's alignment ][ value payload ]
^ bucket start                                              ^ payload_offset
¦<------------------------ bucket_stride ----------------------------------->¦
```

The header's `state` is one of **EMPTY** (never used), **OCCUPIED** (holds a live key+value), or **TOMBSTONE** (a removed entry). `payload_offset` and `bucket_stride` are computed once at create time

from `elem_size`/`align`, so addressing bucket `i`'s value is just `internal.ptr + i*bucket_stride + payload_offset`.

- **Hashing, and why the default is the safe one.** Integer keys go through a SplitMix/Murmur-style bit-mix finaliser (so sequential ids spread across buckets). **String keys are hashed with keyed SipHash-2-4 under a per-process secret** drawn once from the OS CSPRNG — because a map that hashes *untrusted* keys with a predictable function is a denial of service waiting to happen: an attacker who controls the keys computes collisions offline, floods them all into one bucket, and turns every lookup into a linear scan. Keying the hash with a secret they cannot see is what closes that, and it is the same choice Python, Rust, and the Linux kernel made for their own tables. If your keys all come from your own program, `proven_map_create_trusted` opts into fast unkeyed FNV-1a instead; `proven_map_hash` shows you which function a given map actually uses. (On a freestanding target, which has no CSPRNG and no attacker model, string keys fall back to FNV.)
- **Probing.** Linear open addressing: the start bucket is `hash & (cap - 1)` (cheap because `cap` is a power of two), then the search walks forward one bucket at a time, wrapping around, until it finds the key (OCCUPIED with an equal key) or an EMPTY bucket (which proves the key is absent). Linear probing keeps the walked buckets contiguous in memory.
- **Removal and tombstones.** `proven_map_remove` cannot just blank a bucket — that would cut a probe chain and hide later keys. It marks the bucket TOMBSTONE instead. Tombstones are skipped by lookups but still consume a slot, which is why `used` (live + tombstones) — not `len` — drives growth.
- **Load factor and resize.** When `used >= cap * 3/4`, the map allocates a new bucket array of the next power of two and **rehashes** every OCCUPIED entry into it, dropping all tombstones in the process. Capacity only grows; it never shrinks. Reserve ahead with `proven_map_reserve` (especially with arena allocators, to avoid leaving dead arrays behind).

Key modes — choosing one

- `PROVEN_KEY_TYPE_INT`: keys are `proven_size_t`. No key storage.
- `PROVEN_KEY_TYPE_U8_BORROWED`: the bucket stores the *view* (pointer + length). **The map never copies the bytes**, so the caller must keep the exact bytes alive and unmoved for the whole time the entry exists. Cheapest for keys that already live somewhere stable (string literals, interned strings).
- `PROVEN_KEY_TYPE_U8_OWNED`: on insert (`proven_map_set_u8_owned` / `PROVEN_MAP_SET_U8_OWNED`) the map **duplicates** the key bytes into its own storage and frees them on remove/destroy, so the source buffer may be released or reused immediately after the call.

The `set_with_scratch` / alias case

If the `element` you pass to `proven_map_set` points *into the map's own bucket array* (for example, copying a value from one key to another via a pointer returned by `proven_map_get`), a rehash triggered by that same insert would move or free the bytes you are reading. `proven_map_set_with_scratch` (and the `*_WITH_SCRATCH_*` macros) capture the source bytes into a temporary buffer from the scratch allocator first, so the insert is alias-safe. Persistent storage still uses `map->alloc`; the scratch allocator is only for that transient copy.

Functions

API	Intent	Return
`proven_map_create(alloc, init_cap, key_type, elem_size, align)`	Create a map.	`proven_result_map_t`.
`proven_map_is_valid(map)`	Validate map invariants.	`bool`.
`proven_map_reserve(map, new_cap)`	Ensure capacity.	`proven_err_t`.
`proven_map_set_with_scratch(map, key, element, scratch)`	Insert/update using scratch for temporary alias-safe copies.	`proven_err_t`.

`proven_map_set(map, key, element)`	Insert/update.	`proven_err_t`.
`proven_map_set_u8_owned(map, key, element)`	Insert/update with map-owned U8 key storage.	`proven_err_t`.
`proven_map_get_mut(map, key)`	Lookup mutable value.	pointer or null.
`proven_map_get(map, key)`	Lookup const value.	pointer or null.
`proven_map_remove(map, key)`	Remove key if present.	`proven_err_t`.
`proven_map_destroy(map)`	Free map storage.	void.

Macros

Macro	Intent
`proven_map_create_with_capacity(...)`	Alias emphasizing capacity allocation.
`PROVEN_MAP_INIT_INT(alloc, type, init_cap)`	Create integer-key typed map.
`PROVEN_MAP_INIT_U8_BORROWED(alloc, type, init_cap)`	Create borrowed-string-key typed map.
`PROVEN_MAP_INIT_U8_OWNED(alloc, type, init_cap)`	Create owned-string-key typed map.
`PROVEN_MAP_SET_INT(map_ptr, int_key, type, value)`	Set integer key.
`PROVEN_MAP_SET_WITH_SCRATCH_INT(map_ptr, int_key, type, value, scratch)`	Set integer key using scratch allocator.
`PROVEN_MAP_SET_U8_BORROWED(map_ptr, u8_view, type, value)`	Set borrowed U8 key.
`PROVEN_MAP_SET_U8_OWNED(map_ptr, u8_view, type, value)`	Set owned U8 key.
`PROVEN_MAP_SET_WITH_SCRATCH_U8_BORROWED(map_ptr, u8_view, type, value, scratch)`	Set borrowed U8 key using scratch allocator.
`PROVEN_MAP_GET_INT(map_ptr, type, int_key)`	Get const value by integer key.
`PROVEN_MAP_GET_U8_BORROWED(map_ptr, type, u8_view)`	Get const value by borrowed U8 key.
`PROVEN_MAP_GET_U8_OWNED(map_ptr, type, u8_view)`	Get const value by owned U8 key.
`PROVEN_MAP_GET_MUT_INT(map_ptr, type, int_key)`	Get mutable value by integer key.
`PROVEN_MAP_GET_MUT_U8_BORROWED(map_ptr, type, u8_view)`	Get mutable value by borrowed U8 key.
`PROVEN_MAP_GET_MUT_U8_OWNED(map_ptr, type, u8_view)`	Get mutable value by owned U8 key.
`PROVEN_MAP_REMOVE_INT(map_ptr, int_key)`	Remove integer key.
`PROVEN_MAP_REMOVE_U8_BORROWED(map_ptr, u8_view)`	Remove U8 key.
`PROVEN_MAP_REMOVE_U8_OWNED(map_ptr, u8_view)`	Remove owned U8 key.
`PROVEN_MAP_DESTROY(map_ptr)`	Destroy map.

Owned key path:

`PROVEN_KEY_TYPE_U8_OWNED` stores a duplicate of the key bytes inside the map. Use it when the source buffer may move or be freed after insertion. The owned bytes are released on remove and destroy, and they move with the entry during rehash.

`PROVEN_HARDENED` and debug validation can reject some borrowed-key pointers that fall inside the map's own internal storage. That check is defensive only; it does not make borrowed keys self-owning.

Example:

```
typedef struct UserInfo {
    int level;
    double budget;
} UserInfo;
```

```
proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, UserInfo, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_map_t users = r.value;

UserInfo u = { .level = 3, .budget = 99.0 };
(void)PROVEN_MAP_SET_INT(&users, 404, UserInfo, u);

/* GET returns a pointer into the bucket array, or NULL when absent. Any insert
 * that rehashes invalidates it: look it up, use it, drop it. */
const UserInfo *found = PROVEN_MAP_GET_INT(&users, UserInfo, 404);
if (found) {
    proven_println("level={}", PROVEN_ARG(found->level));
}

PROVEN_MAP_DESTROY(&users);
```

5. Algorithms

Array algorithms operate on `proven_array_t` and a comparison callback.

Comparison function

```
typedef int (*proven_compare_fn_t)(const void *a, const void *b);
```

Return negative if `a < b`, zero if equal, positive if `a > b`.

Functions

API	Intent	Return
`proven_array_sort(arr, cmp)`	Sort array in place.	void.
`proven_array_binary_search(arr, key, cmp)`	Search sorted array.	pointer to element or null.
`proven_array_linear_search(arr, key, cmp)`	Search any array by scanning.	pointer to element or null.

`proven_array_sort` is an introsort: a Bentley-McIlroy three-way partition, an insertion-sort cutoff for small ranges, and a heapsort fallback once the recursion exceeds `2*log2(n)` levels.

Two properties are worth stating, because they are the ones that bite:

- **O(n log n) is a guarantee, not a typical case.** The heapsort fallback is what makes it one. A sort whose worst case can be reached by an adversarial ordering is a denial of service in any program that sorts data it did not author.
- **Duplicate keys are the fast case, not the slow one.** Elements equal to the pivot are collected into a run that is final and never recursed into, so all-equal input costs a single pass. This matters because low-cardinality keys
  - a status column, an enum, a bucket id - are what callers actually sort by, and they are exactly what a naive two-way partition degrades on.

The sort is not stable: equal elements may be reordered.

The comparator is an ordinary file-scope function, so the shape is:

```
static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);   /* not (x - y): that overflows */
}
```

```
proven_array_sort(&nums, cmp_int);
int key = 20;
int *hit = proven_array_binary_search(&nums, &key, cmp_int);
```

The worked example at the end of this chapter (`manual/examples/ex_04_array.c`) is the compiled-and-run version of exactly this.

6. Hashing, by use case

There is no single "hash". Which function is correct depends entirely on what you are doing with the result, and reaching for the wrong one gives you either a program that is needlessly slow or one that is quietly insecure. `proven/hash.h` offers exactly one primitive per job so the choice is made once you name the job:

You are…	Use	And crucially
hashing keys into **your own** table, **trusted** input	`proven_hash_bytes` (FNV-1a)	fast; a crypto hash here is ~50× slower for no gain
hashing keys from **untrusted** input into a table	`proven_hash_keyed` (SipHash-2-4)	FNV lets an attacker collide every key into one bucket and turn your O(1) table into O(n^2)
checking data was not **corrupted** in transit or on disk	`proven_crc32`	a checksum; interoperates with gzip/zlib/PNG
fingerprinting content: dedup, content-addressing, "same file?"	`proven_sha256`	the only one safe against a *deliberately* forged match

The one line to remember: **CRC-32 and FNV detect accident, not attack.** Do not use them to decide whether two things are "the same" when someone might benefit from fooling you — that is what `proven_sha256` is for. And a keyed hash is only safe if the key is a real secret chosen once from real randomness; a fixed key is no key at all.

Every function is byte-exact and endianness-independent: the same input gives the same output on any target, because a fingerprint that changed with the machine would be a fingerprint of the machine, not the content. All four are royalty-free algorithms (FNV, CRC-32: public domain; SipHash: CC0; SHA-256: FIPS 180-4, unpatented), implemented from their specifications and checked against each one's official known-answer vectors.

Reference

API	Intent	Return
`proven_hash_bytes(view)`	FNV-1a 64. A hash-table hash for keys you chose yourself.	`proven_u64`.
`proven_hash_keyed(view, key[16])`	SipHash-2-4 under a 16-byte secret. The same job, for keys an attacker supplies.	`proven_u64`.
`proven_crc32(view)`	One-shot CRC-32 (IEEE, reflected) — the one gzip/zlib/PNG carry.	`proven_u32`.
`proven_crc32_update(crc, view)`	The same CRC over a stream of chunks. Start from `0`; the value you hold between calls is the real CRC, so you can store it, log it, and resume.	`proven_u32`.

`proven_sha256(view, out[32])`	One-shot SHA-256.	void; writes `PROVEN_SHA256_SIZE` bytes.
`proven_sha256_init/_update/_final`	The same digest over content you cannot hold in memory at once. The digest depends only on the bytes, never on how they were chunked.	void.
`proven_sha256_to_hex(digest, out[65])`	The 64-character lowercase spelling `sha256sum` and `git` print. NUL-terminated.	void.

The one structure you hold:

```
typedef struct {
    /* Opaque. A running SHA-256: the 8-word chain value, a 64-byte block being
       filled, and the message length. Declare one, init it, update it, finalise it. */
} proven_sha256_t;

#define PROVEN_SHA256_SIZE 32   /* the digest; size your output buffer with this */
```

`proven_sha256_t` allocates nothing — it is caller-owned state, so there is nothing to destroy.

Cautions, and what goes wrong

A keyed hash with a fixed key is not a keyed hash. The security of `proven_hash_keyed` rests entirely on the key being a secret drawn once from real randomness. This is why `proven_map_create` does it for you and why you should almost never call `proven_hash_keyed` yourself for a table.

Wrong:

```
proven_byte_t key[16] = { 0 };            /* wrong: a "secret" everyone knows */
proven_u64 h = proven_hash_keyed(user_input, key);
```

```
proven_byte_t key[16];
memcpy(key, &timestamp, sizeof timestamp); /* wrong: guessable, and mostly zero */
```

Correct — draw it once, at startup, from the OS:

```
proven_byte_t key[16];
if (proven_random_bytes(key, sizeof key)) {
    proven_u64 h = proven_hash_keyed(
        proven_mem_view_from_u8(PROVEN_LIT("session-token")), key);
    (void)h;
}
```

Do not use CRC-32 or FNV to decide two things are "the same" when someone might gain by fooling you. Both are trivially forgeable: producing a second input with the same CRC-32 is schoolbook arithmetic.

Wrong — a content-addressed store an attacker can poison:

```
if (proven_crc32(incoming) == stored_crc) {
    accept_as_identical(incoming);   /* wrong: a forged collision costs seconds */
}
```

Correct — a fingerprint that must not be foolable is `proven_sha256`, compared over all 32 bytes.

A digest buffer that is not `PROVEN_SHA256_SIZE` is a buffer overflow. `proven_sha256` writes exactly 32 bytes and does not know how big your array is.

```
proven_byte_t digest[16];        /* wrong: SHA-256 is 32 bytes */
proven_sha256(data, digest);     /* writes 32 into a 16-byte buffer */
```

Compiled and run by the test suite:

```
/*
 * Hashing, by use case. The module gives you exactly one function per job, so the only
 * decision is which job you have - and getting THAT wrong is the whole danger.
 */

int main(void) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("the quick brown fox"));

    /* Job 1: hash a key into your own table, trusted input. Fast, non-cryptographic. */
    proven_u64 table_hash = proven_hash_bytes(data);
    EXAMPLE_REQUIRE(table_hash != 0, "FNV-1a produces a spread-out 64-bit value");

    /* Job 2: hash a key from UNTRUSTED input. Same purpose, but an attacker who picks the
     * input still cannot make everything collide, because they do not have the key. Pick
     * the key once at startup from real randomness; a fixed key defeats the point. */
    proven_byte_t key[16] = { 0 };   /* in real code: fill from a random source, once */
    proven_u64 safe_hash = proven_hash_keyed(data, key);
    EXAMPLE_REQUIRE(safe_hash != table_hash, "a keyed hash is a different function");

    /* Job 3: did these bytes get corrupted? A checksum, not a hash. Interoperates with
     * gzip/zlib/PNG, which all carry this exact CRC-32. */
    proven_u32 checksum = proven_crc32(data);
    /* The canonical CRC-32 sanity value, so you can see it is the real one: */
    EXAMPLE_REQUIRE(proven_crc32(proven_mem_view_from_u8(PROVEN_LIT("123456789"))) == 0xcbf43926u,
                    "CRC-32 of \"123456789\" is the shared check value");
    (void)checksum;

    /* Job 4: fingerprint content - dedup, content-addressing, "are these the same file",
     * answered safely even against someone trying to forge a match. This is the one you
     * reach for when the answer must not be foolable. */
    proven_byte_t digest[PROVEN_SHA256_SIZE];
    proven_sha256(data, digest);

    char hex[65];
    proven_sha256_to_hex(digest, hex);
    /* The same spelling sha256sum and git print, so it interoperates: */
    EXAMPLE_REQUIRE(hex[64] == '\0' && proven_cstr_len(hex) == 64,
                    "a SHA-256 fingerprint is 64 lowercase hex characters");

    /* SHA-256 streams, for content you cannot hold in memory at once - the digest depends
     * only on the bytes, never on how they were chunked. */
    proven_sha256_t ctx;
    proven_sha256_init(&ctx);
    proven_sha256_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("the quick ")));
    proven_sha256_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("brown fox")));
    proven_byte_t streamed[PROVEN_SHA256_SIZE];
    proven_sha256_final(&ctx, streamed);

    bool same = true;
    for (proven_size_t i = 0; i < PROVEN_SHA256_SIZE; ++i) {
        if (streamed[i] != digest[i]) same = false;
    }
    EXAMPLE_REQUIRE(same, "two updates of the halves equal one hash of the whole");

    return EXAMPLE_OK();
}
```

7. Bytes to text: hex and Base64

Once you can hash a thing (above) and draw a random token (`random.h`), you need to write those bytes somewhere that only holds text — a URL, an HTTP header, a log line, a JSON string. That is `encode.h`. No cryptography, no compression; the two encodings everything already agrees on, done without hidden allocation and without the two ways they are usually got wrong.

You want	Use	Alphabet
A digest or a few bytes a human reads	`proven_hex_encode`	lowercase hex, what `sha256sum` and `git` print
Bytes in a URL, a cookie, a filename	`proven_base64url_encode`	`-` `_`, **no** padding — nothing to escape, no `=` to mangle
Bytes in an HTTP header, MIME, JSON	`proven_base64_encode`	standard `+` `/`, `=`-padded

Two refusals are the point:

- **A decoder validates its whole input before writing a byte.** Text from outside the program is not guaranteed to be valid; a stray character, a bad length, bad padding, or embedded whitespace is `PROVEN_ERR_INVALID_ENCODING` with nothing committed — not a read past the end, and not a silently short result one byte into which the caller finds the corruption. `proven_base64_decode` accepts **both** alphabets and padded-or-not, because a decoder that only takes what it emits rejects half the Base64 in the world.
- **The output size is a call, not a guess** — `proven_hex_encoded_size`, `proven_base64_encoded_size`, and their decode counterparts. A buffer one byte too small is `PROVEN_ERR_OUT_OF_BOUNDS` with nothing written, never a truncated prefix.

It is pure computation — no allocation, no OS — and available freestanding.

Reference

Every call takes the input, a caller-owned output buffer and its capacity, and an optional `written_out`. None of them allocate.

API	Intent	Return
`proven_hex_encoded_size(n)`	The characters `proven_hex_encode` will write for `n` bytes: `n * 2`, no NUL.	`proven_size_t`.
`proven_hex_decoded_size(n)`	The bytes `proven_hex_decode` will write for `n` characters: `n / 2`.	`proven_size_t`.
`proven_hex_encode(data, out, cap, &w)`	Lowercase hex.	`PROVEN_OK`; `OUT_OF_BOUNDS` if `cap` is short (**nothing written**); `INVALID_ARG` for a NULL out or a `{NULL, >0}` view.
`proven_hex_decode(text, out, cap, &w)`	Decode hex; upper and lower case both accepted.	`INVALID_ENCODING` for an odd length or any non-hex byte (**nothing committed**).
`proven_base64_encoded_size(n)`	An upper bound for both Base64 forms: `4 * ceil(n/3)`.	`proven_size_t`.
`proven_base64_decoded_size(n)`	An upper bound for **padded and unpadded** text: `3 * ceil(n/4)`.	`proven_size_t`.

`proven_base64_encode(data, out, cap, &w)`	Standard alphabet (`+` `/`), `=`-padded.	as above.
`proven_base64url_encode(data, out, cap, &w)`	URL-safe alphabet (`-` `_`), **no padding**.	as above.
`proven_base64_decode(text, out, cap, &w)`	Decode. Accepts **both** alphabets, padded or not.	`INVALID_ENCODING` for a stray byte, a bad length, or bad padding.

`written_out` may be NULL. It is set to `0` on entry and on every error path, so it is never stale.

Cautions, and what goes wrong

Size the output with the size function, not by eye. The encoders refuse a short buffer rather than truncating — which means the failure you get from a hand-computed size is an error you have to handle, not a silent corruption. That is the good outcome; the point is you will still have to handle it.

Wrong — the classic off-by-one, and it now *fails* instead of overflowing:

```
proven_byte_t out[16];                      /* wrong: 12 bytes of hex needs 24 chars */
proven_hex_encode(twelve_bytes, out, sizeof out, &w);   /* OUT_OF_BOUNDS, nothing written */
```

Correct:

```
proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("twelve bytes"));
proven_byte_t out[64];
proven_size_t w = 0;
if (proven_hex_encoded_size(data.size) <= sizeof out &&
    proven_is_ok(proven_hex_encode(data, out, sizeof out, &w))) {
    /* `w` characters of lowercase hex in `out` - not NUL-terminated */
}
```

Base64URL emits no padding, and the decoder accepts that. `proven_base64_decoded_size` rounds *up* so it is a correct upper bound for unpadded text too. Do not "improve" on it with `3 * (n / 4)` — that floors away the 1-2 bytes an unpadded tail carries, and you will fail to decode this library's own URL-safe output. (It did exactly that, once, and an audit caught it.)

```
proven_size_t cap = 3 * (text_len / 4);   /* wrong: 0 for "QQ", which decodes to 1 byte */
```

Never hand-roll the decoder. The whole reason these exist is that a decode reads text from outside your program, and the two-line loop everyone writes trusts it.

Wrong — reads past the end on odd input, and accepts junk as data:

```
for (size_t i = 0; i < len; i += 2)              /* wrong: no length check */
    out[i/2] = (hexval(text[i]) << 4) | hexval(text[i+1]);   /* wrong: no validation */
```

`proven_hex_decode` validates the **whole** input before writing a single byte, so a stray character near the end cannot leave you holding a half-decoded prefix you believe is complete.

Whitespace is not skipped, on purpose. A pasted, line-wrapped Base64 blob is `INVALID_ENCODING`, not a silently different result. If you *want* to accept wrapped input, strip the whitespace yourself — deliberately, where you can see it.

Size the destination with the size functions, never by hand. Each encoding has a pair — `proven_hex_encoded_size()` / `proven_hex_decoded_size()` and `proven_base64_encoded_size()` / `proven_base64_decoded_size()` — and they differ in what they promise:

Direction	What the number is	How to use it
Encoding	**Exact** for hex and standard Base64. Base64URL omits padding, so its output	Allocate this many bytes; the call reports what it wrote.

	can be shorter — the number is still a safe upper bound.	
Decoding	An **upper bound**, because padding and the alphabet in use are not known until the text is read.	Allocate the bound, then use the reported count, not the bound, as the length.

The example below ends with that: it allocates a destination sized by `proven_base64_encoded_size()`, encodes into it, sizes the reverse buffer with `proven_base64_decoded_size()`, decodes, and round-trips the same bytes through `proven_hex_decode()` — including the uppercase input real tools produce, and the odd-length input that cannot be a whole number of bytes.

Compiled and run by the test suite:

```
/*
 * Bytes to text, by use case. The rule is the same one hashing follows: one function per job,
 * and the danger is picking the wrong job. Hex for something a human reads; Base64URL for
 * something that goes in a URL; standard Base64 for something that goes on the wire.
 */

int main(void) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("the quick brown fox"));

    /* Job 1: a digest a human will read or paste - hex, the spelling sha256sum and git use. */
    proven_byte_t hex[64];   /* proven_hex_encoded_size(19) = 38 */
    proven_size_t hn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_encode(data, hex, sizeof hex, &hn)),
                    "hex encode into a buffer sized by proven_hex_encoded_size");
    EXAMPLE_REQUIRE(hn == proven_hex_encoded_size(data.size), "two hex chars per byte");

    /* Job 2: a token that goes in a URL - Base64URL, so nothing needs percent-escaping and
     * there is no '=' padding for a parser to trip over. */
    proven_byte_t token_bytes[16] = { 0 };   /* in real code: proven_random_bytes(token_bytes, 16) */
    proven_byte_t url[32];
    proven_size_t un = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64url_encode(
                        (proven_mem_view_t){ token_bytes, sizeof token_bytes }, url, sizeof url, &un)),
                    "base64url encode a token");
    /* No '=' in a URL-safe token. */
    bool has_pad = false;
    for (proven_size_t i = 0; i < un; ++i) if (url[i] == '=') has_pad = true;
    EXAMPLE_REQUIRE(!has_pad, "the URL form emits no padding");

    /* Job 3: bytes on the wire - standard Base64, the +/= alphabet HTTP and MIME expect. */
    proven_byte_t b64[64];
    proven_size_t bn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_encode(data, b64, sizeof b64, &bn)),
                    "standard base64 encode");

    /* And it round-trips: decode gives back exactly the bytes. A decoder that accepts both
     * alphabets and padded-or-not is deliberate - real input comes in every shape. */
    proven_byte_t back[32];
    proven_size_t dn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_decode(
                        (proven_mem_view_t){ b64, bn }, back, sizeof back, &dn)),
                    "decode the base64 back");
    EXAMPLE_REQUIRE(dn == data.size && proven_memcmp(back, data.ptr, dn) == 0,
                    "what comes back is exactly what went in");
```

```
    /* The point of a validating decoder: junk is refused, not guessed. A caller who fed this
     * to a two-line loop would read past the end or get a silently short result. */
    proven_err_t bad = proven_base64_decode(
        proven_mem_view_from_u8(PROVEN_LIT("not valid base64!!")), back, sizeof back, &dn);
    EXAMPLE_REQUIRE(bad == PROVEN_ERR_INVALID_ENCODING,
                    "a stray character is INVALID_ENCODING, with nothing committed");

    /* And a buffer one byte too small is refused, never truncated. */
    proven_byte_t tiny[4];
    EXAMPLE_REQUIRE(proven_hex_encode(data, tiny, sizeof tiny, &hn) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a too-small output buffer is OUT_OF_BOUNDS, not a truncated prefix");

    /* --- sizing the destination buffer, instead of guessing ---------------- */

    /* Every encode and decode above wrote into a fixed array that was obviously
     * big enough. Real code allocates the destination, and then the size has to
     * be computed rather than remembered. That is what the four size functions
     * are for - one per direction, per encoding. */
    proven_allocator_t alloc = proven_heap_allocator();

    proven_size_t need = proven_base64_encoded_size(data.size);
    EXAMPLE_REQUIRE(need > 0, "an encoded size must be computed, not guessed");

    proven_result_mem_mut_t out = alloc.alloc_fn(alloc.ctx, need, 1);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "allocating exactly the encoded size must succeed");

    proven_size_t wrote = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_encode(data, out.value.ptr, out.value.size, &wrote)),
                    "encoding into a buffer sized by the size function must fit exactly");
    EXAMPLE_REQUIRE(wrote == need, "and use all of it: this size is exact for standard base64");

    /* The decode direction is an UPPER BOUND, not an exact count: base64 text
     * may carry padding, so the decoder reports how many bytes it really wrote.
     * Size the buffer with the bound; use the reported count afterwards. */
    proven_size_t bound = proven_base64_decoded_size(wrote);
    EXAMPLE_REQUIRE(bound >= data.size, "the decoded bound must cover the real output");

    proven_result_mem_mut_t plain = alloc.alloc_fn(alloc.ctx, bound, 1);
    EXAMPLE_REQUIRE(proven_is_ok(plain.err), "allocating the decode bound must succeed");

    proven_size_t got = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_decode(
                        (proven_mem_view_t){ out.value.ptr, wrote }, plain.value.ptr, plain.value.size,
&got)),
                    "decoding back must succeed");
    EXAMPLE_REQUIRE(got == data.size && proven_memcmp(plain.value.ptr, data.ptr, got) == 0,
                    "and reproduce the original bytes exactly");

    /* Hex has the same pair. Its decoded bound is exact when the input length is
     * even, which valid hex always is - an odd length is malformed input, and
     * the decoder says so rather than dropping the last digit. */
    proven_size_t hex_len = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_encode(data, hex, sizeof hex, &hex_len)),
                    "re-encode to hex, since the refused call above left its count unusable");

    proven_size_t hex_bound = proven_hex_decoded_size(hex_len);
    EXAMPLE_REQUIRE(hex_bound == data.size, "two hex characters decode to one byte");

    proven_byte_t from_hex[32];
    proven_size_t hex_got = 0;
```

```
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_decode(
                        (proven_mem_view_t){ hex, hex_len }, from_hex, sizeof from_hex, &hex_got)),
                    "hex decodes back to the original bytes");
    EXAMPLE_REQUIRE(hex_got == data.size && proven_memcmp(from_hex, data.ptr, hex_got) == 0,
                    "and the round trip is exact");

    /* Hex is case-insensitive on the way in, which matters because different
     * tools print different cases of the same digest. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_decode(
                        proven_mem_view_from_u8(PROVEN_LIT("DEADBEEF")), from_hex, sizeof from_hex,
&hex_got)),
                    "uppercase hex decodes too");
    EXAMPLE_REQUIRE(hex_got == 4, "and yields one byte per two characters");

    /* An odd number of characters cannot be a whole number of bytes. */
    EXAMPLE_REQUIRE(proven_hex_decode(proven_mem_view_from_u8(PROVEN_LIT("abc")),
                                      from_hex, sizeof from_hex, &hex_got) == PROVEN_ERR_INVALID_ENCODING,
                    "an odd-length hex string is malformed, not silently truncated");

    alloc.free_fn(alloc.ctx, plain.value.ptr);
    alloc.free_fn(alloc.ctx, out.value.ptr);

    (void)url; (void)un;
    return EXAMPLE_OK();
}
```

8. Examples and misuse cases

Pointers into arrays can become stale

Wrong:

```
int *p = PROVEN_ARRAY_GET_MUT(&nums, int, 0);
PROVEN_ARRAY_PUSH(&nums, int, 30);
*p = 99; /* wrong: push may have reallocated the array */
```

Correct:

```
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, int, 2);
if (!proven_is_ok(r.err)) {
    return;
}
proven_array_t nums = r.value;
(void)PROVEN_ARRAY_PUSH(&nums, int, 10);

/* Push first, then take the pointer. A pointer fetched before a push may point
 * at a block the array has already reallocated away. */
(void)PROVEN_ARRAY_PUSH(&nums, int, 30);
int *p = PROVEN_ARRAY_GET_MUT(&nums, int, 0);
if (p) {
    *p = 99;
}

PROVEN_ARRAY_DESTROY(&nums);
```

One intrusive node belongs to one list at a time

Wrong:

```
proven_list_push_back(&list_a, &item.link);
proven_list_push_back(&list_b, &item.link); /* wrong */
```

Use one embedded node per membership.

Remove while iterating with the safe macro

Correct:

```
typedef struct Item {
    int value;
    proven_list_node_t link;
} Item;

proven_list_t list;
proven_list_init(&list);
Item a = { .value = 1 };
Item b = { .value = 2 };
proven_list_push_back(&list, &a.link);
proven_list_push_back(&list, &b.link);

/* The SAFE form reads `it->next` into `next` before the body runs, so removing
 * `it` (which nulls its links) cannot strand the loop. */
proven_list_node_t *it = NULL;
proven_list_node_t *next = NULL;
PROVEN_LIST_FOR_EACH_SAFE(it, next, &list) {
    Item *item = PROVEN_LIST_ENTRY(it, Item, link);
    if (item->value % 2 == 0) {
        proven_list_remove(it);
    }
}
```

Ring push does not grow

Wrong:

```
PROVEN_RING_PUSH(&q, int, value); /* wrong if you ignore full-ring errors */
```

Correct:

```
proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 2);
if (!proven_is_ok(r.err)) {
    return;
}
proven_ring_t q = r.value;

int value = 7;
for (int i = 0; i < 3; ++i) {
    proven_err_t e = PROVEN_RING_PUSH(&q, int, value);
    if (e == PROVEN_ERR_OUT_OF_BOUNDS) {
        /* The ring is full. Drop the item, or drain one first - but decide. */
        int drop = 0;
        (void)PROVEN_RING_POP(&q, int, &drop);
        e = PROVEN_RING_PUSH(&q, int, value);
    }
    (void)e;
}

PROVEN_RING_DESTROY(&q);
```

Borrowed map keys must outlive entries

Wrong:

```
proven_u8str_t key = make_key(alloc);
PROVEN_MAP_SET_U8_BORROWED(&m, proven_u8str_as_view(&key), int, 1);
proven_u8str_destroy(alloc, &key);
/* wrong: map still points at freed key bytes */
```

Correct options:

- Use string literals or other stable storage for borrowed keys.
- Store owned key objects elsewhere and destroy them after removing map entries.
- Use `PROVEN_KEY_TYPE_U8_OWNED` and `proven_map_set_u8_owned()` when the map should own key storage.

Use scratch when inserting a value borrowed from the same map

If `element` points into map storage and insertion can rehash, use `proven_map_set_with_scratch()` or the scratch macros.

```
proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, int, 4);
if (!proven_is_ok(r.err)) {
    return;
}
proven_map_t m = r.value;
(void)PROVEN_MAP_SET_INT(&m, 1, int, 42);

/* `src` points into the map's own bucket array. Passing it straight to
 * proven_map_set would be an alias: a rehash triggered by that same insert can
 * free the bytes it is still reading from. The scratch form captures the source
 * bytes into a temporary from `scratch` first, so the insert is alias-safe. */
const int *src = PROVEN_MAP_GET_INT(&m, int, 1);
if (src) {
    (void)proven_map_set_with_scratch(&m, (proven_map_key_t){ .id = 2 }, src, scratch);
}

PROVEN_MAP_DESTROY(&m);
```

Binary search requires sorted input

Wrong:

```
int *hit = proven_array_binary_search(&nums, &key, cmp_int);
/* wrong if nums was not sorted by cmp_int */
```

Worked example: array, sort, binary search

Compiled and run by the test suite. Note the sort's behaviour on duplicate keys: they are the *fast* case, not the quadratic one.

```
/*
 * proven_array_t is a growable vector of one element type. It stores the
 * allocator you created it with, so push can grow and destroy can free without
 * you passing the allocator back in - which also means the array must be
 * destroyed with nothing but itself.
 *
 * The PROVEN_ARRAY_* macros are the type-safe face of the void* core: they
 * derive sizeof/alignof from the type and hand the element to the array by
 * pointer, so pushing an int into an array of records will not compile.
 */

/* A task queue keyed by priority. Real data is low-cardinality: three
 * priorities, many tasks. That matters for the sort - see below. */
typedef struct {
    int priority;
    int id;
} task_t;

/* The comparator is the array's ordering. The same one MUST be used for the
 * sort and for the binary search - a search under a different order is a bug
 * the compiler cannot see.
 *
 * The (x > y) - (x < y) form is deliberate: subtracting the two ints would
 * overflow for large values and hand back a nonsense sign. */
static int cmp_priority(const void *a, const void *b) {
```

```
    int x = ((const task_t *)a)->priority;
    int y = ((const task_t *)b)->priority;
    return (x > y) - (x < y);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- push / get / pop ------------------------------------------------- */
    /* init_cap is a hint, not a limit: push grows past it. Sizing it right just
     * avoids the reallocations. */
    proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, task_t, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating an array of task_t must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_array_t tasks = r.value;

    /* Deliberately duplicate-heavy, because that is what real keys look like:
     * a status column, an enum, a bucket id. */
    static const task_t seed[] = {
        { .priority = 2, .id = 10 }, { .priority = 0, .id = 11 },
        { .priority = 2, .id = 12 }, { .priority = 1, .id = 13 },
        { .priority = 2, .id = 14 }, { .priority = 0, .id = 15 },
        { .priority = 1, .id = 16 }, { .priority = 2, .id = 17 },
    };

    for (proven_size_t i = 0; i < sizeof seed / sizeof seed[0]; ++i) {
        proven_err_t err = PROVEN_ARRAY_PUSH(&tasks, task_t, seed[i]);
        EXAMPLE_REQUIRE(proven_is_ok(err), "pushing into a growable array must succeed");
        if (!proven_is_ok(err)) {
            PROVEN_ARRAY_DESTROY(&tasks);
            return 1;
        }
    }
    EXAMPLE_REQUIRE(tasks.len == 8, "eight pushes give eight elements");

    /* get returns a pointer INTO the array's storage - it is not a copy, and the
     * next push may reallocate and leave it dangling. Fetch it after the pushes,
     * use it, do not store it. */
    const task_t *front = PROVEN_ARRAY_GET(&tasks, task_t, 0);
    EXAMPLE_REQUIRE(front && front->id == 10, "element 0 is the first one pushed");

    /* Out of range is a null pointer, not a crash and not UB. */
    EXAMPLE_REQUIRE(PROVEN_ARRAY_GET(&tasks, task_t, 99) == NULL, "an out-of-range index yields NULL");

    /* pop copies the last element out (pass NULL to just discard it). */
    task_t last = {0};
    proven_err_t err = PROVEN_ARRAY_POP(&tasks, task_t, &last);
    EXAMPLE_REQUIRE(proven_is_ok(err), "popping a non-empty array must succeed");
    EXAMPLE_REQUIRE(last.id == 17 && tasks.len == 7, "pop returns the last element and shrinks the array");

    /* Put it back: the rest of the example wants all eight. */
    err = PROVEN_ARRAY_PUSH(&tasks, task_t, last);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing the popped element back must succeed");

    /* --- sort ------------------------------------------------------------- */
    /* An introsort: O(n log n) is a guarantee, not an average - the heapsort
     * fallback rules out the quadratic case an adversary could otherwise steer
     * you into. And equal keys are the FAST path here: everything equal to the
```

```
     * pivot is partitioned into a run that is never recursed into, so the
     * duplicate priorities above cost less than distinct ones would, not more.
     *
     * It is not stable: task 10 and task 12 may come out in either order. */
    proven_array_sort(&tasks, cmp_priority);

    for (proven_size_t i = 1; i < tasks.len; ++i) {
        const task_t *prev = PROVEN_ARRAY_GET(&tasks, task_t, i - 1);
        const task_t *cur  = PROVEN_ARRAY_GET(&tasks, task_t, i);
        EXAMPLE_REQUIRE(prev && cur && prev->priority <= cur->priority,
                        "after sorting, priorities must be non-decreasing");
    }

    /* --- binary search ------------------------------------------------------ */
    /* Only legal because the array is sorted by this exact comparator. The key
     * is a whole element, but only the fields the comparator reads matter. */
    task_t key = { .priority = 1, .id = 0 };
    const task_t *hit = (const task_t *)proven_array_binary_search(&tasks, &key, cmp_priority);
    EXAMPLE_REQUIRE(hit != NULL, "priority 1 is present, so the search must find it");
    EXAMPLE_REQUIRE(hit && hit->priority == 1, "the hit must be an element with the searched key");
    /* With duplicate keys it finds SOME element with that key, not the first one.
     * If you need the first, scan backwards from the hit. */

    task_t absent = { .priority = 9, .id = 0 };
    EXAMPLE_REQUIRE(proven_array_binary_search(&tasks, &absent, cmp_priority) == NULL,
                    "a key that is not there must return NULL");

    printf("tasks: %zu sorted, found priority %d (id %d)\n",
           (size_t)tasks.len, hit ? hit->priority : -1, hit ? hit->id : -1);

    /* The array owns its element storage; destroy returns it through the
     * allocator the array was created with. Elements are plain data here - if
     * they owned anything, you would have to destroy each one first. */
    PROVEN_ARRAY_DESTROY(&tasks);
    return EXAMPLE_OK();
}
```

Worked example: a map with integer keys and with owned string keys
Compiled and run by the test suite. The owned-key half proves the point that is easy to get wrong: the map copies the key, so the buffer you built it in is yours to reuse immediately.

```
/*
 * proven_map_t is a flat open-addressing hash map. The value type is fixed at
 * create time and stored inline in the bucket array - there is no per-entry
 * allocation for values, and get hands back a pointer straight into that array.
 *
 * The interesting decision is the KEY:
 *
 *   PROVEN_KEY_TYPE_INT          - the key is a proven_size_t. Nothing to own.
 *   PROVEN_KEY_TYPE_U8_BORROWED  - the bucket stores your pointer and length.
 *                                  The map never copies the bytes, so YOU must
 *                                  keep them alive and unmoved for as long as
 *                                  the entry exists. Right for string literals.
 *   PROVEN_KEY_TYPE_U8_OWNED     - the map copies the key bytes into its own
 *                                  storage and frees them again. Right for keys
 *                                  built at runtime, which is most of them.
 *
 * The second half of this example is the reason OWNED exists.
 */
```

```c
typedef struct {
    int  level;
    long score;
} player_t;

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- integer keys ------------------------------------------------------ */
    proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, player_t, 8);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating an int-keyed map must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_map_t by_id = r.value;

    proven_err_t err = PROVEN_MAP_SET_INT(&by_id, 404, player_t, ((player_t){ .level = 3, .score = 990 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting into the map must succeed");
    err = PROVEN_MAP_SET_INT(&by_id, 7, player_t, ((player_t){ .level = 1, .score = 10 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a second key must succeed");

    /* set on an existing key replaces the value; it does not add an entry. */
    err = PROVEN_MAP_SET_INT(&by_id, 7, player_t, ((player_t){ .level = 2, .score = 40 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "re-setting an existing key must succeed");
    EXAMPLE_REQUIRE(by_id.len == 2, "re-setting a key replaces its value rather than adding an entry");

    /* get returns a pointer into the bucket array, or NULL when absent. It is
     * invalidated by any insert that rehashes - look it up, use it, drop it. */
    const player_t *p = PROVEN_MAP_GET_INT(&by_id, player_t, 7);
    EXAMPLE_REQUIRE(p && p->level == 2 && p->score == 40, "get must see the replaced value");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&by_id, player_t, 999) == NULL, "a missing key yields NULL");

    err = PROVEN_MAP_REMOVE_INT(&by_id, 7);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing a present key must succeed");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&by_id, player_t, 7) == NULL, "a removed key is gone");
    EXAMPLE_REQUIRE(by_id.len == 1, "removal decrements the live entry count");

    PROVEN_MAP_DESTROY(&by_id);

    /* --- owned string keys ------------------------------------------------- */
    /* Same map, keyed by a name that we build at runtime - the case where a
     * borrowed key would be a dangling pointer waiting to happen. */
    proven_result_map_t rm = PROVEN_MAP_INIT_U8_OWNED(alloc, player_t, 8);
    EXAMPLE_REQUIRE(proven_is_ok(rm.err), "creating an owned-string-keyed map must succeed");
    if (!proven_is_ok(rm.err)) {
        return 1;
    }
    proven_map_t by_name = rm.value;

    /* A scratch buffer we intend to reuse for every key. With a BORROWED map
     * that plan is fatal: every entry would point at these same bytes. */
    proven_byte_t scratch[32];
    proven_u8str_t name = proven_u8str_borrow(scratch, sizeof scratch);

    err = proven_u8str_append(&name, PROVEN_LIT("ada"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "building the first key must succeed");

    /* set_u8_owned COPIES the key bytes into map storage. After it returns, the
     * map's key no longer has anything to do with `scratch`. */
    err = PROVEN_MAP_SET_U8_OWNED(&by_name, proven_u8str_as_view(&name), player_t,
```

```
                                ((player_t){ .level = 9, .score = 5000 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting with an owned key must succeed");

    /* So the buffer is immediately free to be reused for the next key... */
    err = proven_u8str_reset(&name);
    EXAMPLE_REQUIRE(proven_is_ok(err), "the key buffer is ours again the moment set returns");
    err = proven_u8str_append(&name, PROVEN_LIT("grace"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "overwriting the buffer with the next key must succeed");

    err = PROVEN_MAP_SET_U8_OWNED(&by_name, proven_u8str_as_view(&name), player_t,
                                ((player_t){ .level = 4, .score = 700 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting the second owned key must succeed");

    /* ...and the first entry is untouched by that overwrite. This is the whole
     * point: the map holds its own copy of "ada", not a view of a buffer that
     * now says "grace". A BORROWED map would report two entries both keyed
     * "grace" - or worse, one keyed by freed memory. */
    const player_t *ada = PROVEN_MAP_GET_U8_OWNED(&by_name, player_t, PROVEN_LIT("ada"));
    EXAMPLE_REQUIRE(ada && ada->score == 5000, "the copied key survives the caller reusing its buffer");

    const player_t *grace = PROVEN_MAP_GET_U8_OWNED(&by_name, player_t, PROVEN_LIT("grace"));
    EXAMPLE_REQUIRE(grace && grace->score == 700, "the second key is a separate entry");
    EXAMPLE_REQUIRE(by_name.len == 2, "two distinct keys means two entries");

    /* Remove frees the key copy the map made - you never free it yourself. */
    err = PROVEN_MAP_REMOVE_U8_OWNED(&by_name, PROVEN_LIT("ada"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing an owned key must succeed");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_U8_OWNED(&by_name, player_t, PROVEN_LIT("ada")) == NULL,
                    "the removed entry is gone");

    printf("map: %zu name(s) left, grace at level %d\n",
           (size_t)by_name.len, grace ? grace->level : -1);

    /* destroy frees the bucket array AND every key copy still in it ("grace"
     * here). `scratch` is ours and outlives the map; the borrowed `name` handle
     * has nothing to free. */
    PROVEN_MAP_DESTROY(&by_name);
    proven_u8str_destroy(alloc, &name);
    return EXAMPLE_OK();
}
```

Worked example: sizing containers, searching unsorted data, and streaming a checksum
The worked examples above take one container at a time. This one is the program those pieces end up inside — a small event intake — and it answers the questions that only come up once several containers are in play:

- **Stop a container reallocating while it fills.** `proven_array_reserve()` and `proven_map_reserve()` set the capacity once. On the heap that saves copying; behind an arena it saves the dead storage every reallocation leaves behind until the next reset. For a map it also matters for correctness of your pointers: a rehash invalidates every pointer a previous `get_mut` returned.
- **Check a handle you did not create.** `proven_array_is_valid()`, `proven_ring_is_valid()` and `proven_map_is_valid()` ask whether the handle's own fields are consistent — the check that catches a zero-initialised or never-created container at the boundary of your code, rather than at the first push.
- **Search data that is not sorted.** The event log is kept in arrival order, because that order is the thing being recorded, so binary search may not be used on it — on unsorted input it does not return "not found", it returns a wrong answer. `proven_array_linear_search()` is the correct call, and it returns the *first* match in order.

- **Update a value already in a map, with one lookup.** `proven_map_get_mut()` returns a pointer into the map's own storage, so a counter is incremented where it lives instead of being read, changed and written back.
- **Decide which hash the keys deserve.** `proven_map_create()` hashes string keys with keyed SipHash, so an attacker who chooses the keys cannot collide them all into one bucket. `proven_map_create_trusted()` uses fast FNV-1a and is right only when your own code chooses every key. `proven_map_hash()` returns the value the map actually places a key by, so the difference between the two is observable rather than a claim.
- **Checksum data that arrives in pieces.** `proven_crc32_update()` folds each chunk into a running value; the result equals `proven_crc32()` over the whole, whatever the chunk boundaries were.

```
/*
 * The container chapters show each structure on its own. A real program uses
 * several at once, and the questions that come up then are not about any single
 * container:
 *
 *   - How do I stop a container reallocating while it fills?  reserve.
 *   - How do I check a container somebody handed me is usable?  is_valid.
 *   - How do I search when the data is NOT sorted?  linear search - and knowing
 *     why binary search would give a wrong answer here.
 *   - How do I update a value already in a map without looking it up twice?
 *     get_mut.
 *   - Do I need the attack-resistant hash, or the fast one?  it depends on who
 *     chooses the keys, and map_hash lets you see the difference.
 *   - How do I checksum data that arrives in pieces?  crc32_update.
 *
 * The program is a small event intake: events arrive, are counted per client,
 * the most recent few are kept for a diagnostic dump, and the batch is
 * checksummed as it streams past.
 */

typedef struct {
    proven_u32 client;
    proven_u32 code;      /* an event code; 0 means "connection closed" */
} event_t;

/* Search by client id: this is the comparison for finding an event, not for
 * sorting them. The array below is in ARRIVAL order and stays that way. */
static int by_client(const void *a, const void *b) {
    const event_t *x = (const event_t *)a;
    const event_t *y = (const event_t *)b;
    return (x->client > y->client) - (x->client < y->client);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 1. reserve: decide the capacity once ----------------------------- */

    proven_result_array_t ar = PROVEN_ARRAY_INIT(alloc, event_t, 4);
    EXAMPLE_REQUIRE(proven_is_ok(ar.err), "creating the event log must succeed");
    if (!proven_is_ok(ar.err)) {
        return 1;
    }
    proven_array_t events = ar.value;

    /* We know the batch size before we start, so ask for the room once. Without
     * this the array doubles as it fills, copying its contents each time; behind
```

```
 * an arena allocator each of those copies also leaves the old block behind
 * until the arena is reset. */
proven_err_t err = proven_array_reserve(&events, 16);
EXAMPLE_REQUIRE(proven_is_ok(err), "reserving room for the whole batch must succeed");
EXAMPLE_REQUIRE(events.cap >= 16, "the capacity is now at least what was asked for");

/* is_valid asks whether the handle itself is structurally sound - a pointer,
 * a length and a capacity that agree. Assert it where a container arrives
 * from other code, or after a zero-initialised handle might have escaped;
 * it is not something to repeat after every push. */
EXAMPLE_REQUIRE(proven_array_is_valid(&events), "a created array must be structurally valid");

proven_array_t never_created = {0};
EXAMPLE_REQUIRE(!proven_array_is_valid(&never_created),
                "a zero-initialised handle is not a usable array");

static const event_t batch[] = {
    { .client = 7, .code = 200 }, { .client = 3, .code = 200 },
    { .client = 7, .code = 404 }, { .client = 9, .code = 200 },
    { .client = 3, .code = 500 }, { .client = 7, .code = 0   },
};
proven_size_t batch_len = sizeof batch / sizeof batch[0];

proven_size_t cap_before = events.cap;
for (proven_size_t i = 0; i < batch_len; ++i) {
    err = PROVEN_ARRAY_PUSH(&events, event_t, batch[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing an event must succeed");
}
EXAMPLE_REQUIRE(events.cap == cap_before, "the reserve was enough: nothing reallocated while filling");

/* --- 2. searching data that is not sorted ------------------------------ */

/* The log is in arrival order, which is the order we want to keep: it is the
 * thing being recorded. Binary search would be faster and WRONG here, since
 * it may only be used on a sorted range - on unsorted data it does not
 * return "not found", it returns nonsense. Linear search is the correct
 * tool, and O(n) over a batch this size is nothing. */
event_t key = { .client = 9, .code = 0 };
const event_t *found = (const event_t *)proven_array_linear_search(&events, &key, by_client);
EXAMPLE_REQUIRE(found != NULL, "client 9 appears in the batch");
EXAMPLE_REQUIRE(found->code == 200, "and linear search returns the FIRST match in order");

event_t absent = { .client = 42, .code = 0 };
EXAMPLE_REQUIRE(proven_array_linear_search(&events, &absent, by_client) == NULL,
                "a client that never appeared is reported as not found");

/* --- 3. a ring for the most recent events ------------------------------ */

proven_result_ring_t rr = PROVEN_RING_INIT(alloc, event_t, 4);
EXAMPLE_REQUIRE(proven_is_ok(rr.err), "creating the recent-events ring must succeed");
proven_ring_t recent = rr.value;
EXAMPLE_REQUIRE(proven_ring_is_valid(&recent), "a created ring must be structurally valid");

proven_ring_t unset = {0};
EXAMPLE_REQUIRE(!proven_ring_is_valid(&unset), "a zero-initialised ring handle is not usable");

/* This ring refuses when full rather than overwriting, so "keep the most
 * recent four" means dropping the oldest ourselves before pushing. */
for (proven_size_t i = 0; i < batch_len; ++i) {
    if (recent.len == recent.cap) {
```

```
        event_t dropped;
        err = PROVEN_RING_POP(&recent, event_t, &dropped);
        EXAMPLE_REQUIRE(proven_is_ok(err), "a full ring must yield its oldest element");
    }
    err = PROVEN_RING_PUSH(&recent, event_t, batch[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing after making room must succeed");
}
EXAMPLE_REQUIRE(recent.len == 4, "the ring holds the four most recent events");

/* --- 4. counting per client, with one lookup per update --------------- */

/* create_with_capacity is proven_map_create under a name that says why the
 * capacity argument is there: sizing it now avoids rehashing later, and a
 * rehash both copies every bucket and invalidates every pointer previously
 * returned by get_mut. */
proven_result_map_t mr = proven_map_create_with_capacity(alloc, 8, PROVEN_KEY_TYPE_INT,
                                                         sizeof(proven_u32), alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(mr.err), "creating the counter map must succeed");
proven_map_t counts = mr.value;
EXAMPLE_REQUIRE(proven_map_is_valid(&counts), "a created map must be structurally valid");

err = proven_map_reserve(&counts, 32);
EXAMPLE_REQUIRE(proven_is_ok(err), "reserving map capacity must succeed");

for (proven_size_t i = 0; i < batch_len; ++i) {
    proven_map_key_t k = { .id = batch[i].client };

    /* get_mut returns a pointer INTO the map's storage, so the counter is
     * incremented where it lives: one lookup, no copy back. The pointer is
     * good only until the next insert - which is another reason the capacity
     * was reserved above. */
    proven_u32 *seen = (proven_u32 *)proven_map_get_mut(&counts, k);
    if (seen) {
        *seen += 1;
    } else {
        proven_u32 one = 1;
        err = proven_map_set(&counts, k, &one);
        EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a new client must succeed");
    }
}

const proven_u32 *seven = PROVEN_MAP_GET_INT(&counts, proven_u32, 7);
EXAMPLE_REQUIRE(seven != NULL && *seven == 3, "client 7 sent three events");

/* A closing event (code 0) means the client is gone: drop its counter. */
for (proven_size_t i = 0; i < batch_len; ++i) {
    if (batch[i].code == 0) {
        err = proven_map_remove(&counts, (proven_map_key_t){ .id = batch[i].client });
        EXAMPLE_REQUIRE(proven_is_ok(err), "removing a present key must succeed");
    }
}
EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&counts, proven_u32, 7) == NULL, "the closed client is gone");

/* --- 5. which hash, and how to see the difference --------------------- */

/* Two string-key maps over the same keys. The default one hashes with keyed
 * SipHash, so an attacker who chooses the keys cannot force them all into
 * one bucket. The trusted one uses fast FNV-1a and is the right choice ONLY
 * when your own code chooses every key. */
proven_result_map_t untrusted = PROVEN_MAP_INIT_U8_BORROWED(alloc, proven_u32, 8);
```

```
    EXAMPLE_REQUIRE(proven_is_ok(untrusted.err), "creating the default string-key map must succeed");
    proven_map_t from_network = untrusted.value;

    proven_result_map_t tr = proven_map_create_trusted(alloc, 8, PROVEN_KEY_TYPE_U8_BORROWED,
                                                       sizeof(proven_u32), alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(tr.err), "creating the trusted-key map must succeed");
    proven_map_t internal = tr.value;

    EXAMPLE_REQUIRE(from_network.trusted_keys == false, "the default map defends against chosen keys");
    EXAMPLE_REQUIRE(internal.trusted_keys == true, "the trusted map opts out of that defence");

    /* map_hash exposes the value the map actually places a key by, so the
     * choice is observable rather than something you take on faith. */
    proven_map_key_t name = { .str = PROVEN_LIT("user-agent") };
    proven_u64 keyed = proven_map_hash(&from_network, name);
    proven_u64 fast  = proven_map_hash(&internal, name);
    EXAMPLE_REQUIRE(keyed != fast, "the same key hashes differently under the two functions");
    EXAMPLE_REQUIRE(proven_map_hash(&internal, name) == fast, "and each function is deterministic");

    /* Keys that live in memory the map does not own are BORROWED: the bytes must
     * outlive the map. These are string literals, so they do. When the key comes
     * from a buffer you are about to reuse, use an owned-key map instead
     * (PROVEN_MAP_INIT_U8_OWNED / proven_map_set_u8_owned), which copies. */
    proven_u32 hits = 1;
    err = proven_map_set(&from_network, name, &hits);
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a borrowed string key must succeed");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_U8_BORROWED(&from_network, proven_u32, PROVEN_LIT("user-agent")) != NULL,
                    "and it can be looked up by an equal view, not the same pointer");

    /* --- 6. checksumming a stream in chunks ------------------------------- */

    /* The batch is checksummed as it goes past, which is what a program reading
     * a file or a socket has to do: it never holds the whole thing. Start the
     * running value at 0, feed each chunk in, and the final value is the same
     * one a single call over the concatenation would produce. */
    proven_u32 running = 0;
    for (proven_size_t i = 0; i < batch_len; ++i) {
        proven_mem_view_t chunk = { .ptr = (const proven_byte_t *)&batch[i], .size = sizeof batch[i] };
        running = proven_crc32_update(running, chunk);
    }
    proven_mem_view_t whole = { .ptr = (const proven_byte_t *)batch, .size = sizeof batch };
    EXAMPLE_REQUIRE(running == proven_crc32(whole),
                    "chunked and whole-buffer CRC-32 must agree, whatever the chunking");

    printf("events=%zu recent=%zu crc32=%08x\n",
           (size_t)events.len, (size_t)recent.len, (unsigned)running);

    proven_map_destroy(&internal);
    proven_map_destroy(&from_network);
    proven_map_destroy(&counts);
    PROVEN_RING_DESTROY(&recent);
    PROVEN_ARRAY_DESTROY(&events);
    return EXAMPLE_OK();
}
```

Wrong — binary search on the arrival-ordered log:

```
const event_t *e = proven_array_binary_search(&events, &key, by_client);   /* wrong */
```

`proven_array_binary_search()` may only be used on a range sorted by the same comparison. On unsorted input it does not fail; it halves its way to whatever element happens to be there and reports it, or reports nothing while the element is present two slots away.

Wrong — holding a `get_mut` pointer across an insert:

```
proven_u32 *seen = proven_map_get_mut(&counts, k);
proven_err_t e = proven_map_set(&counts, other_key, &one);   /* may rehash */
*seen += 1;                                                  /* wrong: may dangle */
```

Hold the key, not the pointer, across anything that can insert.

Wrong — the fast hash on keys from outside the program:

```
proven_result_map_t m = proven_map_create_trusted(alloc, 64, PROVEN_KEY_TYPE_U8_BORROWED,
                                                  sizeof(session_t), alignof(session_t));
/* keys are HTTP header names taken from the request */   /* wrong */
```

That is exactly the case the keyed hash defends against. Use `proven_map_create()` whenever the keys are chosen by anyone other than you.

Chapter 8: Formatting and Scanning (v26.09.04a)

Part IV — Text in and out. Prerequisite: Chapter 3 §3–§4. After this chapter you can format any value, teach the formatter a type of your own, parse input with the cursor under your control, and recover from a partial parse.

This chapter is the **reference half** of the text material; Chapter 3 is the tutorial half. Chapter 3 introduces the formatter and the scanner beside the string types and gives you enough to be productive. This chapter gives the exact syntax, every parameter shape, every return value, and the places where callers usually go wrong. If you are meeting `{}` for the first time, read Chapter 3 first — this one assumes you have.

Table of contents

1. Design model

Why the type is never written twice

Both halves of this chapter exist to eliminate one thing: a place where the programmer states a type that the compiler cannot check against the value.

`printf("%d", x)` states it twice — once as `%d`, once by passing `x` — and varargs erases the second, so nothing can compare them. `scanf("%d", &x)` is worse: the format decides both how to parse *and* what to write through the pointer, so a mismatch corrupts memory rather than printing nonsense. Both are the same defect from opposite directions, and both compile silently.

Here the placeholder carries **no type at all**. `{}` marks a position; the type comes from `PROVEN_ARG(x)` on the formatting side and `PROVEN_SCAN_ARG(&x)` on the scanning side, both resolved by `_Generic` at compile time against the argument's static type. The spec after `:` controls presentation only — width, fill, alignment, precision, base — never interpretation. There is no `%d`-versus-`double` to get wrong because you never wrote a type in the string.

The second design decision is that **neither side owns a buffer**. Formatting appends into a destination you supply and refuses when it does not fit; scanning reads from a view you supply and moves a cursor you can read. Nothing here allocates unless you hand it an allocator, which is what lets the whole chapter work in a freestanding build (§13).

The formatting side and the scanning side solve opposite problems.

- Formatting takes typed values and renders text.

- Scanning takes text and writes typed values.

The project keeps both sides intentionally small:

- formatting supports a compact placeholder language, positional reuse, simple alignment, width, and hex rendering for numeric values;
- scanning supports typed destination pointers, strict placeholder counting, and literal matching with whitespace collapsing;
- neither side tries to become a full `printf` or `scanf` clone.

The practical result is that the APIs are easier to reason about than large general-purpose format engines, but the syntax is still expressive enough for common systems-code tasks.

2. Formatter data model

proven_fmt_result_t

```
typedef struct {
proven_err_t  err;
proven_size_t written;
proven_size_t required;
} proven_fmt_result_t;
```

Meaning:

- `err`: the status code for the operation.
- `written`: bytes actually written into the destination.
- `required`: total bytes needed for the full formatted output.

Use `err` first. The other fields are most useful for truncating or partially successful operations.

A successful result looks like this:

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_trunc(
        &s,
        "hello {}",
        PROVEN_ARG("world")
    );
    if (proven_is_ok(r.err)) {
        proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
    }

    proven_u8str_destroy(alloc, &s);
}
```

A truncating result can still tell you how much more space you would have needed. Here the destination is deliberately too small, so `written` stops short of `required` - and the string is still valid:

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 8);   /* on purpose: too small */
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_trunc(
        &s,
        "name={} score={}",
        PROVEN_ARG("ada"),
        PROVEN_ARG(42)
    );
    /* r.written is what fit; r.required is what the whole output needed. */
```

```
    proven_println("wrote {} of {} bytes",
                   PROVEN_ARG(r.written), PROVEN_ARG(r.required));

    proven_u8str_destroy(alloc, &s);
}
```

proven_arg_type_t

```
typedef enum {
PROVEN_ARG_NONE,
PROVEN_ARG_I32,
PROVEN_ARG_U32,
PROVEN_ARG_I64,
PROVEN_ARG_U64,
#ifndef PROVEN_FMT_NO_FLOAT
PROVEN_ARG_F64,
#endif
PROVEN_ARG_CSTR,
PROVEN_ARG_STR_VIEW,
PROVEN_ARG_DATETIME,
PROVEN_ARG_PTR,
PROVEN_ARG_FN,
} proven_arg_type_t;
```

The formatter currently recognizes these value classes:

- signed 32-bit integers
- unsigned 32-bit integers
- signed 64-bit integers
- unsigned 64-bit integers
- floating-point values, unless `PROVEN_FMT_NO_FLOAT` is defined
- trusted C strings
- borrowed U8 string views
- datetimes
- object pointers
- function pointers

proven_arg_t

```
typedef struct {
proven_arg_type_t type;
union {
proven_i32 i32;
proven_u32 u32;
proven_i64 i64;
proven_u64 u64;
double f64;
const char *cstr;
proven_u8str_view_t str_view;
proven_datetime_t datetime;
const void *ptr;
void (*fn)(void);
} value;
} proven_arg_t;
```

The union field must match the selected `type`. Do not manufacture a `proven_arg_t` by writing a mismatched union field and hoping the formatter will guess.

Wrong:

```
proven_arg_t arg = {0};
arg.type = PROVEN_ARG_I64;
arg.value.u64 = 123; /* wrong: type and union field do not match */
```

Correct:

```
proven_arg_t arg = proven_arg_i64(123);
(void)arg;   /* pass it to a formatting macro; PROVEN_ARG(arg) accepts it as-is */
```

3. Formatter constructors and selectors

Constructor summary

API	Parameters	Returns	Intent
`proven_arg_none(void)`	none	`proven_arg_t`	Internal sentinel value.
`proven_arg_i32(int v)`	signed integer	`proven_arg_t`	Render as 32-bit signed integer.
`proven_arg_u32(unsigned int v)`	unsigned integer	`proven_arg_t`	Render as 32-bit unsigned integer.
`proven_arg_i64(long long v)`	wide signed integer	`proven_arg_t`	Render as 64-bit signed integer.
`proven_arg_u64(unsigned long long v)`	wide unsigned integer	`proven_arg_t`	Render as 64-bit unsigned integer.
`proven_arg_f64(double v)`	floating-point value	`proven_arg_t`	Render as floating-point text, unless float formatting is disabled.
`proven_arg_cstr(const char *v)`	trusted live C string	`proven_arg_t`	Render a NUL-terminated C string.
`proven_arg_cstr_n(const char *v, proven_size_t max_len)`	possibly bounded C string	`proven_arg_t`	Render only up to `max_len` while searching for NUL.
`proven_arg_str_view(proven_u8str_view_t v)`	borrowed U8 view	`proven_arg_t`	Render a borrowed view without assuming NUL termination.
`proven_arg_datetime(proven_datetime_t v)`	datetime value	`proven_arg_t`	Render a datetime using the formatter's datetime rules.
`proven_arg_ptr(const void *v)`	object pointer	`proven_arg_t`	Render the pointer value.
`proven_arg_fn(void (*v)(void))`	function pointer	`proven_arg_t`	Render the raw function-pointer representation.
`proven_arg_ucstr(const unsigned char *v)`	unsigned-char string	`proven_arg_t`	Convenience wrapper around `proven_arg_cstr`.
`proven_arg_identity(proven_arg_t v)`	existing argument object	`proven_arg_t`	Pass-through helper.
`proven_arg_bool(bool v)`	boolean	`proven_arg_t`	Render `true` / `false` as words, not as `1` / `0`.
`proven_arg_char(char v)`	a character	`proven_arg_t`	Render the **character**. This is why a `char` VARIABLE renders as a character while the literal `PROVEN_ARG('Z')` still renders

			`90`: in C, `'Z'` has type `int`, and no amount of `_Generic` can tell it apart from the number 90.
`proven_arg_custom(const void *v, proven_fmt_custom_fn fn)`	any type at all	`proven_arg_t`	Render a type the library has never heard of, through a function you supply. See Formatting a user-defined type.

`PROVEN_ARG(x)`

`PROVEN_ARG(x)` is the usual entry point. It uses `_Generic` so the compiler chooses a constructor from the type of `x`.

The current mapping is:

- `_Bool`, `char`, `signed char`, `short`, `int` -> `proven_arg_i32`
- `unsigned char`, `unsigned short`, `unsigned int` -> `proven_arg_u32`
- `long`, `long long` -> `proven_arg_i64`
- `unsigned long`, `unsigned long long` -> `proven_arg_u64`
- `double`, `float` -> `proven_arg_f64`, unless `PROVEN_FMT_NO_FLOAT` is defined
- `const char *`, `char *` -> `proven_arg_cstr`
- `unsigned char *`, `const unsigned char *` -> `proven_arg_ucstr`
- `void *`, `const void *` -> `proven_arg_ptr`
- `proven_u8str_view_t` -> `proven_arg_str_view`
- `proven_datetime_t` -> `proven_arg_datetime`
- `proven_arg_t` -> `proven_arg_identity`

Important consequence:

- `PROVEN_ARG` does not select function pointers.
- Use `PROVEN_ARG_FN(f)` for function pointers.

Wrong:

```
void helper(void) {}
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(helper)); /* wrong */
```

Correct:

```
void helper(void);   /* whatever function you want to print the address of */

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(alloc, &rs.value, "{}",
                                                         PROVEN_ARG_FN(helper));
    (void)r;
    proven_u8str_destroy(alloc, &rs.value);
}
```

`PROVEN_ARG_FN(f)`

This macro exists so callers can pass function pointers without casting them through `void *`. It is a small safety wrapper around `proven_arg_fn`.

Example:

```
void helper(void);

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
```

```c
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &s,
        "callback = {}",
        PROVEN_ARG_FN(helper)
    );
    if (!PROVEN_FMT_IS_OK(r)) {
        proven_eprintln("formatting the callback failed");
    }

    proven_u8str_destroy(alloc, &s);
}
```

PROVEN_ARG_CSTR_N(v, max_len)

This macro is the bounded-string helper. Use it when the source may not be a fully trusted C string, but you still want C-string-like input handling.

It scans for NUL only up to `max_len` and then formats the bounded prefix as a view.

Good use case - `buf` came off the wire, so nothing promises it is NUL-terminated:

```c
char buf[128];                      /* filled from an untrusted source */
(void)proven_mem_copy(buf, sizeof buf, proven_mem_view_from_u8(PROVEN_LIT("payload")));

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &rs.value,
        "payload={}",
        PROVEN_ARG_CSTR_N(buf, sizeof buf)   /* looks for NUL only within 128 bytes */
    );
    (void)r;
    proven_u8str_destroy(alloc, &rs.value);
}
```

Bad use case:

```c
const char *buf = get_network_buffer();
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(buf)); /* wrong if buf is not trusted */
```

Float formatting note

If `PROVEN_FMT_NO_FLOAT` is defined, float support is removed from the generic selector and the float constructor is not available. That is a compile-time configuration choice, not a runtime toggle.

The default `{}` rendering of a `double` produces a fixed six-digit fractional form for finite values and switches to scientific notation when the magnitude is too large or too small for the compact form. Unlike earlier versions, this output is now computed by an **exact, integer-only** engine (no `double`/`long double` approximation): the digits are correctly rounded (round-half-to-even), so `{}` matches what `printf("%.6f")` / `%.6e` would print on the same value. For a shortest round-trippable form, or for a precision other than six, use the `proven_float_format_*` policy API described below.

Accuracy and limits

- The default `{}` float output uses six fractional digits, correctly rounded to nearest with ties to even (matching glibc `%.6f`/`%.6e`). It is exact at any magnitude — there is no `long double` and no precision/magnitude ceiling beyond the configurable big-integer capacity.
- For round-trip serialization use the **shortest** policy (`proven_float_format_options_shortest()`), which emits the shortest decimal string that parses back to the exact same value.

- Decimal-to-double scanning is correctly rounded to IEEE-754 binary64 with round-to-nearest, ties-to-even, matching the host `strtod` bit for bit. Values below the half-way threshold to the smallest subnormal round to signed zero with the input sign preserved.
- The parser is layered `Clinger fast path -> Eisel-Lemire -> exact big-integer fallback`; every tier is exact and the fallback is the arbiter, so the result is correctly rounded for every input. The cached power tables are generated source (`scripts/generate_float_decimal_tables.py`).
- The exact-fallback big-integer capacity is tunable with `PROVEN_FLOAT_BIGINT_LIMBS` (see `include/proven/float_config.h`) for embedded targets; the fast paths never touch the big integer.
- Validation: the formatter and parser are checked exhaustively over all 4.28 billion finite `binary32` values and over 2.56 billion random `binary64` values against the host C library, with zero mismatches. See `docs/float-correctness-and-performance.md` for algorithms, methodology, and a benchmark against glibc.

Inside the engine (conceptual)
You do not need any of this to use the API — it is here for readers who want to know why the output is trustworthy. Full detail is in `docs/float-correctness-and-performance.md`.

Parsing (decimal → binary64), three tiers, fastest first. The result is always correctly rounded; the tiers are purely a speed staircase, each one only taken when it can guarantee the exact answer:

1. *Clinger fast path.* When the value has few significant digits and a small exponent, both the significand and `10^exp` are exactly representable as `double`, so a single rounded multiply/divide is provably correct. Covers most everyday numbers.
2. *Eisel-Lemire.* A 64×128-bit fixed-point multiply by a cached power of ten, with a check that the result is far enough from a rounding boundary to be certain. If the check is inconclusive (the value sits on a halfway tie), it falls through.
3. *Exact big-integer fallback.* Builds the value as a ratio of big integers (`significand` and `5^q`/`2^q`) and compares against the candidate `double` and its neighbor exactly — this is the arbiter that makes ties and subnormals correct. A seeded ±16-ULP window keeps the search to a few comparisons. The big-integer capacity is bounded by `PROVEN_FLOAT_BIGINT_LIMBS`; the tier is the only one that allocates limbs (on the stack), and the fast paths never reach it.

Formatting (binary64/32 → decimal). Two engines, no `long double` anywhere:

- *Shortest* (`proven_float_format_options_shortest()`): a **Grisu3** fast path produces the minimal round-trippable digits for almost all inputs in ~90 ns; the rare cases where Grisu3 cannot prove minimality fall back to an exact **Dragon4** (Burger–Dybvig, round-to-even) core. The result is the unique shortest decimal that parses back to the same bits.
- *Fixed* `%f` */ scientific* `%e` (the default `{}` and the fixed options): an exact integer engine scales the value by `10^precision` with big-integer `mul_pow5`/shift, does an integer `divmod`, and rounds half-to-even — so it matches glibc at any precision and magnitude, with no `2^64`/precision ceiling. Extreme exponents do real arbitrary-precision work and are correspondingly slower (rare in practice).

Public float parsing APIs
Three entry points, sharing one correctly-rounded backend:

- `proven_scan_f64(scan)` — parse from a `proven_scan_t` cursor; restores the cursor on failure. The native, length-bounded path (no NUL terminator needed).
- `proven_parse_double_ascii(view)` — parse one locale-free ASCII token from a `proven_u8str_view_t` and report the consumed length.
- `proven_strtod(nptr, endptr)` — a `strtod`-style convenience wrapper over a C string: skips leading ASCII whitespace and reports `endptr`.

Worked example: parsing

```
#include "proven/scan.h"
#include "proven/float_parse.h"

/* (1) Native, view-based parsing through a scanner cursor. */
proven_scan_t sc = proven_scan_init(proven_u8str_view_from_cstr("3.14159e2 rest"));
proven_result_f64_t r = proven_scan_f64(&sc);
if (r.err == PROVEN_OK) {
    /* r.val == 314.159; the cursor now sits at " rest". */
    proven_println("parsed {}", PROVEN_ARG(r.val));
}

/* (2) strtod-style wrapper for C strings. endptr reports where parsing stopped. */
char *end = NULL;
double v = proven_strtod("  -0.5\t", &end);   /* v == -0.5, *end == '\t' */
(void)v;

/* A trailing exponent marker with no digits stops like strtod: "1e" parses 1,
   leaving endptr at 'e'. Inputs with hundreds of significant digits and extreme
   exponents are still rounded correctly via the exact fallback. */
```

Worked example: formatting with the policy API

`proven_float_format_f64_policy` / `_f32_policy` write directly into a caller buffer and report the number of bytes written. They never allocate.

```
#include "proven/float_format.h"

char buf[64];
proven_size_t n = 0;

/* Shortest round-trippable form: 0.1 -> "0.1" (not "0.10000000000000001"). */
(void)proven_float_format_f64_policy(buf, sizeof buf, 0.1,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(), &n);
/* buf == "0.1", n == 3 */

/* Fixed precision (correctly rounded, round-half-to-even). */
proven_float_format_options_t opt = proven_float_format_options_fixed_default();
opt.precision = 2;
(void)proven_float_format_f64_policy(buf, sizeof buf, 3.14159,
    PROVEN_FLOAT_FORMAT_POLICY_DEFAULT, opt, &n);
/* buf == "3.14" */

/* Always scientific - this is what {:e} selects. Six fractional digits by default,
   a signed two-digit-minimum exponent: exactly what printf's %e prints. */
proven_float_format_options_t sci = proven_float_format_options_scientific();
sci.precision = 2;
(void)proven_float_format_f64_policy(buf, sizeof buf, 42.0,
    PROVEN_FLOAT_FORMAT_POLICY_DEFAULT, sci, &n);
/* buf == "4.20e+01" - where fixed would give "42.00" and shortest "42" */
```

- `PROVEN_FLOAT_FORMAT_POLICY_RYU` selects shortest output; `DEFAULT`/`SIMPLE` select the exact fixed-precision path (`%f`, switching to `%e` for very large or very small magnitudes).
- Returns `PROVEN_ERR_OUT_OF_BOUNDS` if the buffer is too small (the value is never truncated silently) and `PROVEN_ERR_INVALID_ARG` for an unsupported policy.
- The generic `{}` formatter (`proven_u8str_append_fmt*`) uses the `DEFAULT` policy internally, so everyday logging needs no direct call to this API.

4. Format string grammar

The formatter accepts a deliberately small grammar.

Replacement fields

Supported forms:

- `{}`: next positional argument
- `{0}`: first user argument
- `{1}`: second user argument
- `{2}`: third user argument
- and so on

The numbering is user-facing and zero-based. The implementation stores a hidden sentinel at index 0 and maps user index `0` to internal argument slot `1`.

Escaped braces

- `{{` becomes a literal `{`
- `}}` becomes a literal `}`

The layout spec

`{:[[fill]align][sign][#][0][width][.precision][type]}`

Every part is optional, and the order is the one the rest of the world uses, so a spec copied from Python or Rust means here what it means there.

Part	Values	What it does
`align`	`< > ^`	left, right, centre. Default `>`.
`fill`	any character, before an align	the pad character. Default space.
`sign`	`+` or a space	force a sign on a non-negative number, or reserve a space for one.
`#`		alternate form: `0x`, `0X`, `0o`, `0b` prefixes. Integers only.
`0`		zero-fill. `{:08}` on 42 is `00000042`.
`width`	digits, up to 10000	minimum field width.
`.precision`	`.N`, up to 60	decimals. **Floats only.**
`type`	`x X o b d` (int), `f g e` (float)	base and case; `f` fixed, `g` shortest round-trip, `e` scientific (printf `%e`).

Two things are worth knowing because they used to be false:

- **A leading `0` is zero-fill, not the first digit of the width.** `{:08}` produced `"      42"` — space-padded, no error — until v26.07.12f. An explicit fill still wins: `{:*>08}` pads with `*`.
- **Zero-padding goes between the sign and the digits.** `{:+08}` on 42 is `+0000042`, never `0000+42`. Padding is part of the number, and a number's sign comes first.

Floats

`{}` gives six decimals, correctly rounded. `{:.3}` gives three, `{:.0}` gives none, `{:e}` forces scientific notation - mantissa, `precision` fractional digits (six by default), a signed two-digit-minimum exponent, correctly rounded, exactly as printf's `%e` - which is the form `{:f}` and `{:g}` do not reach: `{:f}` never shows an exponent, and `{:g}` uses one only when it is shorter. `{:f}` forces the fixed form, and `{:g}` gives the shortest representation that round-trips.

Until v26.07.12i **none of these existed**: every float came out with exactly six decimals, forever. The exact engine could always do all of it — the `{}` grammar simply could not reach it. The visible cost

was that a float column could not be aligned, because `12.5` rendered nine characters wide and `100.0` rendered ten:

```
proven_byte_t buf[64];
proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);
(void)proven_u8str_append_fmt(&line, "{:>9.2}", PROVEN_ARG(12.5));    /* "    12.50" */
```

A spec the argument cannot honour is an error

`{:x}` on a double, `{:.2}` on an integer, `{:f}` on an integer, `{:#}` on a string: all `PROVEN_ERR_INVALID_FORMAT`.

They used to be *ignored* — `{:x}` on a double printed `3.500000` and reported success. The caller asked for something, got something else, and was told it had worked. That is the worst available outcome, and it is worse than refusing.

`char` and `bool`

`PROVEN_ARG('Z')` renders `Z`, and a `bool` renders `true` or `false`. Both used to go through the integer path, so a character printed as `90` and there was no way to emit one at all — the ASCII column of a hex dump had to be built by hand in a separate buffer and passed as a string. An uppercase hex dump is now one loop:

```
proven_byte_t hexbuf[64];
proven_u8str_t hexline = proven_u8str_borrow(hexbuf, sizeof hexbuf);
unsigned char byte = 0xde;
(void)proven_u8str_append_fmt(&hexline, "{:02X} ", PROVEN_ARG((unsigned)byte));
(void)proven_u8str_append_fmt(&hexline, "{}", PROVEN_ARG((char)'.'));
```

5. Formatting APIs

proven_u8str_fmt_internal(...)

```
proven_fmt_result_t proven_u8str_fmt_internal(
proven_allocator_t alloc,
proven_u8str_t *str,
bool trunc,
const char *fmt,
proven_allocator_t scratch,
const proven_arg_t *args,
proven_size_t args_count
);
```

This is the internal formatting engine. User code should normally call the public macros instead.

Parameters:

- `alloc`: allocator used when the string must grow
- `str`: destination U8 string
- `trunc`: if true, allow best-effort truncation; if false, keep atomic behavior
- `fmt`: format text
- `scratch`: allocator used for temporary alias-patching when needed
- `args`: array of format arguments, including the hidden sentinel at index 0
- `args_count`: total length of `args`, including the sentinel

Return value:

- a `proven_fmt_result_t`

Important rules:

- `args_count` must match the number of placeholders plus the hidden sentinel
- extra unused arguments are an error
- missing arguments are an error

- if the engine detects aliasing between the destination string and a borrowed view argument, it may use the scratch allocator to preserve failure atomicity

`proven_u8str_append_fmt(str, fmt, ...)`
Atomic formatting into a fixed-capacity string. If the result does not fit, the function reports failure and leaves the destination unchanged.

Use this when you want all-or-nothing behavior.

`proven_u8str_append_fmt_trunc(str, fmt, ...)`
Best-effort formatting. It writes as much as fits and reports how much was written and how much was required.

Use this when partial output is acceptable.

`proven_u8str_append_fmt_grow(alloc, str, fmt, ...)`
Growable formatting. It may reallocate the destination string through the supplied allocator. On allocation failure, the old string remains valid.

Use this when you want the output to fit without manual capacity planning.

`proven_u8str_append_fmt_with_scratch(alloc, str, fmt, scratch, ...)`
Growable formatting with a separate scratch allocator. This is useful when the argument list contains string views that may alias the destination buffer and temporary patching is needed.

Use a real allocator for both `alloc` and `scratch`. Do not pass a dead arena or a one-shot temporary buffer unless its lifetime is long enough for the call.

Float format policy seam
The public float policy header provides an explicit policy layer for float formatting. It is intentionally small and keeps the exact fixed-precision formatter as the default path.

The main entry points are:

- `proven_float_format_f64_policy(...)`
- `proven_float_format_f32_policy(...)`
- `proven_float_format_options_fixed_default()`
- `proven_float_format_options_shortest()`

Policy notes:

- `PROVEN_FLOAT_FORMAT_POLICY_DEFAULT` and `PROVEN_FLOAT_FORMAT_POLICY_SIMPLE` select the exact fixed-precision output (correctly rounded, round-half-to-even).
- `PROVEN_FLOAT_FORMAT_POLICY_RYU` is the shortest-output policy branch.
- The policy API returns `PROVEN_ERR_INVALID_ARG` for unsupported enum values.
- The policy API returns `PROVEN_ERR_OUT_OF_BOUNDS` when the caller-provided buffer is too small.

Example:

```
char buf[128];
proven_size_t written = 0;
proven_err_t err = proven_float_format_f64_policy(
    buf,
    sizeof buf,
    0.1,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(),
    &written
);
if (proven_is_ok(err)) {
    /* buf holds the shortest form that parses back to exactly 0.1 */
```

```
    proven_println("{}", PROVEN_ARG_CSTR_N(buf, written));
}
```

PROVEN_FMT_IS_OK(res)
A small helper macro for checking `proven_fmt_result_t`. Use it when you want the intent to stay compact.

Example:

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 32);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &s,
        "name={} score={:0>4}",
        PROVEN_ARG("ada"),
        PROVEN_ARG(42)
    );
    if (!PROVEN_FMT_IS_OK(r)) {
        /* the string is untouched: grow-mode formatting is failure-atomic */
        proven_eprintln("formatting failed");
    }

    proven_u8str_destroy(alloc, &s);
}
```

Console-style helpers
The `sysio` layer provides print helpers that use the same formatter machinery:

- `proven_print(fmt, ...)`
- `proven_println(fmt, ...)`
- `proven_eprint(fmt, ...)`
- `proven_eprintln(fmt, ...)`

They are convenient when you want formatted output directly to stdout or stderr. They still return `proven_err_t`, so check the result when the output matters.

Example:

```
if (!proven_is_ok(proven_println("hello {}", PROVEN_ARG("world")))) {
    /* the write to stdout failed - a closed pipe, a full disk */
    proven_eprintln("stdout is not writable");
}
```

5.1. Formatting a type of your own

`PROVEN_ARG` is built on `_Generic`, and `_Generic` can only dispatch on types it was told about at compile time. It cannot be told about yours. So the formatter's argument set — integers, floats, strings, pointers, datetimes — was a **closed** one: a `rect_t`, a `uuid_t`, a `vec3_t` could not be passed to `{}` at all.

The two ways around it were both bad. Pre-format the value into a scratch string and pass *that*: an allocation and a copy for every value, in the logging path, which is the one path that must keep working when allocation is exactly what has failed. Or print the fields one at a time and give up on ever aligning the column.

`PROVEN_ARG_OF(&obj, render)` is the door:

```
proven_err_t render(proven_fmt_sink_t out, const void *obj);
```

Three things follow from the shape of that signature:

- **The renderer gets a sink, not a buffer.** It does not need to know how much room there is, and it cannot overflow anything. It emits with `proven_fmt_put`.
- **It composes.** The renderer may call the formatter again — into a stack buffer, with no allocator — and hand the result to the sink. A type whose fields are themselves user types nests naturally.
- **Width, fill and alignment work.** The formatter runs the renderer **twice**: once against a counting sink to learn how wide the output is, then once for real, with the padding applied around it. This is why `{:>10}` lines up a column of your type exactly as it lines up a column of ints — and it is why the renderer must be **deterministic** and must not mutate `obj`. If the two passes disagree, the formatter returns `PROVEN_ERR_INVALID_ARG` rather than emit a field of the wrong width into an aligned column and let you find out later.

What the formatter will **not** do is guess. `{:x}` on a rectangle, `{:.2}` on a UUID, `{:+}` on a matrix — the library has no idea what those would mean for your type, and so it refuses them with `PROVEN_ERR_INVALID_FORMAT`. Inventing a plausible answer and reporting success is how a formatter starts lying; a type letter you did not ask for is not better than an error.

Compiled and run by the test suite:

```
/*
 * Formatting a type the library has never heard of.
 *
 * `PROVEN_ARG` is built on `_Generic`, which can only dispatch on types it was told
 * about - and it cannot be told about yours. So before `PROVEN_ARG_OF` existed, a
 * `rect_t` simply could not be printed. You either pre-formatted it into a scratch
 * string and passed that (an allocation and a copy per value, in the logging path,
 * which is the one path that must not allocate), or you printed the fields one at a
 * time and gave up on ever aligning the column.
 *
 * A renderer receives a *sink*, not a buffer. That is what makes it compose: it can
 * call the formatter again, and its output is just bytes going somewhere. And because
 * the formatter measures the renderer's output before emitting it - by running it once
 * against a counting sink - width, fill and alignment work on a user type exactly as
 * they do on an int.
 */

typedef struct { int w, h; } rect_t;

static proven_err_t render_rect(proven_fmt_sink_t out, const void *obj) {
    const rect_t *r = (const rect_t *)obj;

    /* Compose: the formatter, into a stack buffer, no allocator anywhere. */
    proven_byte_t tmp[64];
    proven_u8str_t s = proven_u8str_borrow(tmp, sizeof tmp);
    proven_fmt_result_t f = proven_u8str_append_fmt(&s, "{}x{}",
                                                    PROVEN_ARG(r->w), PROVEN_ARG(r->h));
    if (!PROVEN_FMT_IS_OK(f)) return f.err;

    return proven_fmt_put(out, proven_u8str_as_view(&s));
}

int main(void) {
    rect_t a = { .w = 1920, .h = 1080 };
    rect_t b = { .w = 640,  .h = 480  };

    proven_byte_t buf[128];
    proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);

    /* Just like any other argument. */
    proven_fmt_result_t r = proven_u8str_append_fmt(&line, "mode={}", PROVEN_ARG_OF(&a, render_rect));
```

```
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "a user type should format");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&line), PROVEN_LIT("mode=1920x1080")),
                    "the renderer's bytes should be what came out");

    /* And it aligns, which is the whole reason the formatter measures it first: a
     * column of user-defined values lines up like a column of anything else. */
    (void)proven_u8str_reset(&line);
    r = proven_u8str_append_fmt(&line, "[{:>10}]\n[{:>10}]",
                                PROVEN_ARG_OF(&a, render_rect),
                                PROVEN_ARG_OF(&b, render_rect));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "two user types should format");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&line),
                                         PROVEN_LIT("[ 1920x1080]\n[   640x480]")),
                    "both rows should be right-aligned to the same width");

    /* A spec the library cannot interpret for your type is refused, not guessed at.
     * `{:x}` on a rectangle has no meaning, and answering it with something plausible
     * while reporting success is how a formatter starts lying to you. */
    (void)proven_u8str_reset(&line);
    r = proven_u8str_append_fmt(&line, "{:x}", PROVEN_ARG_OF(&a, render_rect));
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_INVALID_FORMAT,
                    "a type letter on a user type should be an error");

    return EXAMPLE_OK();
}
```

6. Console print helpers

What `proven_println` is, and what it costs

`proven_println("{}", PROVEN_ARG(x))` is the shortest way to get text out of a program, and it is the right tool for a diagnostic, a one-off tool, or a program whose output is a few lines.

It is the wrong tool for a loop, and the reason is worth stating because it is invisible at the call site: **each call is its own write syscall.** Ten thousand `proven_println` calls are ten thousand syscalls, which is roughly two orders of magnitude more expensive than the formatting itself. `printf` hides this behind a buffer that libc flushes for you; this library does not buffer behind your back, because a buffer you did not ask for is a buffer that surprises you at exit, on a crash, or when two writers interleave.

One allocation caveat. The line is formatted into a 512-byte stack buffer, so a typical line costs zero allocations. A line that does *not* fit falls back to the global heap for that call rather than being refused — refusing to print something because it is long would be worse. This is the one place in the library where a call with no allocator parameter can still allocate, and it is bounded to the over-long case. If you need the guarantee that nothing allocates, format into your own buffer and write that.

When output volume matters, take a buffered writer from `proven_sysio_stdout_buffered` and format into that — same argument rules, one syscall per flush instead of one per line. Chapter 5 covers the stream layer; this section is short because that is where the I/O API is documented.

Wrong — reaching for the scanning argument constructor when printing:

```
proven_println("{}", PROVEN_SCAN_ARG(&x));   /* wrong: that builds a scan destination */
```

The important point for formatter users is that the console helpers share the same argument rules as the string append APIs.

Common mistakes:

- using `PROVEN_SCAN_ARG` with `proven_println`

- assuming `PROVEN_LIT` is needed for every format string
- forgetting that output functions can still fail

7. Scanner data model

The scanner is a cursor over bytes you already have. It does not own the text, it does not copy it, and it does not read from anywhere - `proven_scan_t` is a view plus an offset, and that is the whole of it.

```
typedef struct {
    proven_u8str_view_t view;    /* the bytes being read; not owned */
    proven_size_t       cursor;  /* how far in we are */
} proven_scan_t;
```

Two consequences worth stating, because they are what make this different from `scanf`:

- **The scanner never allocates and never writes into your input.** A scanned word comes back as a `proven_u8str_view_t` pointing *into* the original bytes. It is valid exactly as long as those bytes are, and no longer. If it must outlive them, copy it with `proven_u8str_create_from_view()`.
- **The cursor is yours.** It is a plain field. You may save it, restore it, or step it by hand (§12 does exactly that after `proven_scan_skip_until`). Nothing in the scanner is hidden from you, so nothing has to be undone for you.

`proven_scan_init()` normalizes a malformed view (`size > 0` with a null pointer) to an empty one rather than trusting it, so a scanner built from garbage reads as end-of-input instead of dereferencing.

Each primitive returns a result struct pairing the value with the error that guards it: `proven_result_i64_t`, `proven_result_u64_t`, `proven_result_f64_t`, `proven_result_u8str_view_t`. **The value is meaningless unless the error is `PROVEN_OK`** - the scanner does not use a sentinel value to mean failure, because every sentinel is also a legitimate input.

8. Scanner primitive APIs

```
void                       proven_scan_skip_whitespace(proven_scan_t *scan);
proven_result_i64_t        proven_scan_i64(proven_scan_t *scan);
proven_result_u64_t        proven_scan_u64(proven_scan_t *scan);
proven_result_f64_t        proven_scan_f64(proven_scan_t *scan);
proven_result_u8str_view_t proven_scan_str(proven_scan_t *scan);
proven_err_t               proven_scan_skip_until(proven_scan_t *scan, proven_u8str_view_t target);
void                       proven_scan_skip_until_number(proven_scan_t *scan);
```

The value-returning ones are `[[nodiscard]]`: a scan whose result you throw away is a scan you did not need to make.

Shared behaviour

- **Leading whitespace is skipped** by every value scanner. You do not need to call `proven_scan_skip_whitespace()` first; it exists for when you want to position the cursor yourself.
- **Scanning stops at the first byte that cannot belong to the value.** `"12abc"` yields `12` and leaves the cursor on the `a`. That is not an error - the scanner answered the question you asked and left the rest for whoever asks next.
- **On failure the cursor is restored**, so a failed scan is a non-event: you can turn around and parse the same position as something else. §12 does this.

The integer scanners

Input	`proven_scan_i64`	Why
`"42"`, `"+42"`, `"-42"`	`OK` - 42, 42, -42	a sign is part of the number
`"9223372036854775808"`	`PROVEN_ERR_OVERFLOW`	one past `INT64_MAX`; it does **not** wrap
`"abc"`, `""`	`PROVEN_ERR_INVALID_ARG`	there is no number here

`"0x10"`	`OK` - **0**, cursor at 1	**decimal only**: a zero, followed by text

That last row is the one that surprises people. `proven_scan_i64` and `proven_scan_u64` read decimal. There is no hex, no octal, no base prefix. `0x10` is the integer zero, and `x10` is still in the input.

`proven_scan_u64` means unsigned: `"-1"` is `PROVEN_ERR_INVALID_ARG`, not a wrap to `18446744073709551615`. A scanner that quietly turns a negative number into a huge positive one is how a bounds check gets defeated.

The float scanner

`proven_scan_f64` routes through the same correctly-rounded decimal engine as the rest of the library: round-to-nearest, ties-to-even, no `long double` anywhere. It accepts `nan` and `inf`.

Its two boundary behaviours are **deliberately asymmetric**, and the asymmetry is the point:

- `"1e309"` gives `PROVEN_ERR_OVERFLOW`. There is no correct finite answer, so it refuses rather than handing you an infinity you did not ask for.
- `"1e-400"` gives `PROVEN_OK` and `0.0`, with the sign preserved. Underflow to zero *is* the correctly rounded answer. Reporting it as an error would mean reporting correct arithmetic as a failure.

Words and navigation

`proven_scan_str` returns the next whitespace-delimited run as a view into the input. Nothing left but whitespace is `PROVEN_ERR_INVALID_ARG`.

On a **complete view**, "the input ran out" and "the input is wrong" are the same fact - there is no more input, so a number cut off at the end really is malformed. Over a **stream** they are opposite facts, and the difference decides whether you wait or report an error. The scanner sets `proven_scan_t::needs_more` when the parse ran off the end of what it had, and the buffered scanner uses exactly that to refill and retry: a pipe that delivers `-` and then, a moment later, `12` scans as `-12`. Before, it was a malformed number. A wrong byte that is actually *present* - a letter where a digit belongs - is still an error, and no amount of further input will change that; the scanner does not wait for it.

`proven_scan_skip_until(scan, target)` moves the cursor **to** the target, not past it - you decide how much of it to consume. If the target is not there the result is `PROVEN_ERR_NOT_FOUND` **and the cursor does not move**: the scanner does not consume input it failed to navigate.

`proven_scan_skip_until_number` stops at the first digit, or at a sign immediately followed by a digit. If there is no number it runs the cursor to the end of the input - so check `scan.cursor < scan.view.size` before assuming there is something to read.

9. Scan argument model

A scan argument is a **typed pointer to your destination**, selected by the compiler:

```
PROVEN_SCAN_ARG(&x)     /* _Generic on the pointer type */
```

This is where the scanner differs most sharply from `scanf`. There is no format letter to get wrong, because there is no format letter at all. `%d` against a `long`, or `%s` against a buffer that is too small, are not mistakes available here: the destination's type *is* the specification, and a mismatch is a compile error rather than a corrupted stack.

Supported destinations: `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `double`, and `proven_u8str_view_t`.

`PROVEN_SCAN_ARG_LONG(&x)` and `PROVEN_SCAN_ARG_ULONG(&x)` exist for callers who want to be explicit at the call site; `PROVEN_SCAN_ARG` already handles `long*` and `unsigned long*`.

Narrow destinations are range-checked. Scanning `"70000"` into a `short` is `PROVEN_ERR_OVERFLOW`, not a truncated `4464`. The value is parsed at 64 bits and checked against the destination's range before anything is stored.

The `proven_scan_arg_*` constructors are public if you need to build an argument array by hand, but the macros are what callers use.

10. Structural scan grammar

Why parsing with a format string is more dangerous than printing with one
Formatting with a bad format string produces wrong output. **Parsing with one corrupts memory**, because the format decides what to write through the pointers you passed. `sscanf("%d", &c)` where `c` is a `char` writes four bytes into a one-byte object, and nothing in the call says so.

That asymmetry shapes this section. The structural scan uses the same `{}` grammar as the formatter, but the destination type comes from `PROVEN_SCAN_ARG(&x)` — the same `_Generic` dispatch, so the width written is the width of the object, and a value too large for it is `PROVEN_ERR_OVERFLOW` rather than three neighbouring bytes.

The one property to internalise before using it: **the structural scan is not transactional across placeholders.** If the third `{}` fails, the first two destinations have already been written. That is a deliberate trade — buffering every destination until the whole line parsed would need allocation, and this scanner allocates nothing — but it means a failed `proven_scan_fmt` leaves your variables in a partly-updated state. Either treat them as garbage on failure, or scan into locals and copy out only on success. §11.1 shows both patterns.

Literals in the pattern are the other half of the grammar and the part people underuse: anything that is not a placeholder must match the input exactly, so `"{}:{}"` on `"12-34"` fails at the literal rather than quietly returning one field.

The scan format string is the formatter's, read backwards:

- a placeholder consumes one argument, in order;
- anything else is a **literal that must match the input exactly**.

There are no specs inside a placeholder on the scanning side. Width, fill and alignment are formatting concerns; the scanner reads what is there.

Whitespace in the format is not special. The value scanners skip leading whitespace themselves, so a format with a space between two placeholders and one without parse `"7 8"` identically - the space in the format matches the space in the input, and had it not been there, the second scanner would have skipped it anyway.

The number of placeholders must equal the number of arguments. Too few values in the input is an error; **too many is not** (§11.1).

11. Scan formatting APIs

```
proven_scan_fmt(view, fmt, ...)           /* scan a view from the beginning */
proven_scan_fmt_cursor(&scan, fmt, ...)   /* continue from an existing cursor */
proven_err_t proven_scan_fmt_internal(...) /* what the macros expand to */
```

Use `proven_scan_fmt` for a self-contained line. Use `proven_scan_fmt_cursor` when the scan is one step in a longer walk over the same input: it advances the cursor you own, so it mixes freely with the primitives of §8.

11.1. Scan error code guide and recovery

Code	What actually happened	What to do

`PROVEN_OK`	Every placeholder was filled and every literal matched.	Publish the values.
`PROVEN_ERR_INVALID_ARG`	The input is not the shape you asked for - a placeholder had no value to read, or the input ran out.	The line does not match. Report it; do not retry the same shape.
`PROVEN_ERR_NEED_MORE`	**Buffered scanner only.** The token is cut in half by the read boundary: the rest of it has not arrived yet. You will not normally see this - `proven_sysio_scanner_scan` refills and retries for you - it is what the scanner says to itself.	Nothing. It is handled.
`PROVEN_ERR_NOT_FOUND`	A **literal** in the format did not match.	The line has a different shape than expected. Try another format, from a saved cursor.
`PROVEN_ERR_OVERFLOW`	A number was well-formed but does not fit the destination.	The input may be valid and your destination too narrow, or the input may be hostile. Those are very different situations - tell them apart before widening the type.
`PROVEN_ERR_INVALID_FORMAT`	The format string itself is malformed.	A bug in your code, not in the input.

The structural scanner is not transactional, and this is the one that bites.

When a literal fails to match, the placeholders *before* the mismatch have already been written through. The call returns `PROVEN_ERR_NOT_FOUND` and your destination holds a value anyway - `id` below is 7, from a call that failed:

```
int id = -1;
double ratio = -1.0;
proven_err_t err = proven_scan_fmt(line, "id={} XXX={}",
                                   PROVEN_SCAN_ARG(&id), PROVEN_SCAN_ARG(&ratio));
/* err == PROVEN_ERR_NOT_FOUND, and id == 7: it was written before the failure. */
```

So: **on failure, treat every destination as clobbered.** If you need all-or-nothing, scan into locals and publish them only once the call has succeeded - the worked example in §12 shows the shape. Alternatively, save `scan.cursor` before the call and restore it afterwards. The cursor is a plain field, and that is deliberate.

Trailing input is not an error. Scanning one placeholder against `"7 8"` succeeds with the value 7 and leaves `8` unconsumed. The scanner matched what you asked for and stopped; it does not police what you did not ask about. If the whole line must be consumed, check the cursor yourself:

```
if (scan.cursor != scan.view.size) { /* there is unparsed input left */ }
```

12. Examples and misuse cases

Worked example: formatting a line and scanning it back
Compiled and run by the test suite. It formats into a stack-borrowed string (atomic on overflow) and into an allocator-backed one (grows), uses a bounded argument for untrusted bytes, and then parses the line back - with the float round-tripping exactly.

```
/*
 * Formatting and scanning are the two halves of the same idea: `{}` renders typed
```

```
 * values into text, and the scanner reads text back into typed destinations. Both
 * are type-checked at the call site (_Generic picks the constructor), so there is
 * no format-string/argument mismatch to get wrong at runtime.
 *
 * The choice that matters is *where the bytes go*:
 *
 *   append_fmt       - fixed capacity, atomic. Too long? Nothing is written and
 *                      you get PROVEN_ERR_OUT_OF_BOUNDS. No allocator involved,
 *                      so it works on a stack buffer.
 *   append_fmt_grow  - allocator-backed. Grows to fit; on allocation failure the
 *                      string is left exactly as it was.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- fixed capacity: no allocator, no allocation ----------------------- */
    /* borrow wraps caller memory, so this string lives entirely on the stack. `cap`
     * includes the NUL, so 32 bytes hold 31 of content. Nothing to destroy. */
    proven_byte_t stack_buf[32];
    proven_u8str_t fixed = proven_u8str_borrow(stack_buf, sizeof stack_buf);

    proven_fmt_result_t r = proven_u8str_append_fmt(&fixed, "port={}", PROVEN_ARG(8080));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "a short line should fit in 32 bytes");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&fixed), PROVEN_LIT("port=8080")),
                    "the fixed-capacity append should have rendered the port");

    /* Atomic means atomic: an append that does not fit changes nothing. The string
     * is still valid and still holds what it held before - no truncated tail to
     * clean up. (Use append_fmt_trunc if a truncated tail is what you want.) */
    proven_fmt_result_t too_long = proven_u8str_append_fmt(
        &fixed, " and a great deal more text than will ever fit here {}", PROVEN_ARG(1));
    EXAMPLE_REQUIRE(too_long.err == PROVEN_ERR_OUT_OF_BOUNDS, "the overlong append must fail");
    EXAMPLE_REQUIRE(too_long.required > too_long.written, "it reports what it would have needed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&fixed), PROVEN_LIT("port=8080")),
                    "a failed atomic append must leave the string untouched");

    /* --- specs: fill, align, width, hex ------------------------------------ */
    proven_result_u8str_t created = proven_u8str_create(alloc, 8);   /* deliberately small */
    EXAMPLE_REQUIRE(proven_is_ok(created.err), "creating the output string should succeed");
    if (!proven_is_ok(created.err)) return 1;
    proven_u8str_t out = created.value;

    /* grow reallocates as needed, so the initial capacity is a hint, not a limit.
     * `{:0>4}` = fill '0', align right, width 4. `{:x}` = lowercase hex, no 0x. */
    r = proven_u8str_append_fmt_grow(alloc, &out, "id={:0>4} tag={:*^9} addr=0x{:x}",
                                     PROVEN_ARG(7),
                                     PROVEN_ARG(PROVEN_LIT("ok")),
                                     PROVEN_ARG(48879));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "the growing append should succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out),
                                         PROVEN_LIT("id=0007 tag=***ok**** addr=0xbeef")),
                    "fill/align/width/hex should render exactly this");
    printf("%s\n", proven_u8str_as_cstr(&out));

    /* --- untrusted text is bounded, never trusted to be NUL-terminated ------ */
    /* PROVEN_ARG on a char* means "walk it until a NUL turns up" - fine for a
     * literal, a buffer-overread for anything that came off a socket. This buffer
     * has no NUL at all; PROVEN_ARG_CSTR_N stops at the length instead, so it reads
     * only what actually exists. Use it for anything you did not create yourself. */
```

```
    const char untrusted[4] = {'a', 'b', 'c', 'd'};   /* no terminator, on purpose */
    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "reset should keep the buffer");
    r = proven_u8str_append_fmt_grow(alloc, &out, "payload={}",
                                     PROVEN_ARG_CSTR_N(untrusted, sizeof untrusted));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "the bounded append should succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("payload=abcd")),
                    "the bounded argument should render its whole 4 bytes and stop");

    /* --- format a record, then scan it back ---------------------------------- */
    proven_i64 sensor_id = 42;
    double reading = 3.14159;

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "reset should keep the buffer");
    r = proven_u8str_append_fmt_grow(alloc, &out, "{} {} {}",
                                     PROVEN_ARG(sensor_id),
                                     PROVEN_ARG(PROVEN_LIT("boiler")),
                                     PROVEN_ARG(reading));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "formatting the record should succeed");
    printf("record: %s\n", proven_u8str_as_cstr(&out));

    /* One scanner over one view. Each call advances the cursor past what it
     * consumed, so the calls compose left to right - and each one can fail
     * independently, which is the difference between a parser and a guess. */
    proven_scan_t sc = proven_scan_init(proven_u8str_as_view(&out));

    proven_result_i64_t id = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(id.err), "the first field should parse as an integer");
    EXAMPLE_REQUIRE(id.val == sensor_id, "the integer should round-trip");

    /* scan_str returns a view *into the scanned string* - it copies nothing and
     * owns nothing, so it is only valid while `out` is. */
    proven_result_u8str_view_t name = proven_scan_str(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(name.err), "the second field should parse as a word");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(name.val, PROVEN_LIT("boiler")), "the name should round-trip");

    proven_result_f64_t temp = proven_scan_f64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(temp.err), "the third field should parse as a float");

    /* Exactly equal, not approximately: the scanner is correctly rounded, so it
     * returns the nearest double to the text - and the text the formatter produced
     * (six fractional digits) names this value unambiguously. Bit-for-bit, this is
     * the same double we started with. For a value that needs more than six
     * fractional digits, format it with the shortest policy
     * (proven_float_format_options_shortest) and the same round-trip holds. */
    EXAMPLE_REQUIRE(temp.val == reading, "the float must round-trip exactly, not approximately");

    /* The input is fully consumed: nothing was silently left on the table. */
    proven_result_i64_t extra = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(!proven_is_ok(extra.err), "there should be nothing left to scan");

    proven_u8str_destroy(alloc, &out);
    return EXAMPLE_OK();
}
```

Worked example: the scanner's error codes, and recovering from them
Compiled and run by the test suite. Every code in the table above is provoked on purpose here, including the non-transactional failure - because a contract you have only read is a contract you have not learned.

```c
/*
 * The scanner's error codes, and how to recover from them.
 *
 * The scanner is not scanf. It never writes through a pointer it was not given,
 * it never guesses a width, and it tells you which of several different things
 * went wrong. That last part only helps if you know what the codes mean - so
 * this program provokes each one on purpose.
 */

static proven_u8str_view_t v(const char *s) {
    return proven_u8str_view_from_cstr(s);
}

int main(void) {
    /* --- the primitives restore the cursor when they fail ----------------- */
    /* A failed scan is a non-event: the cursor is where it was, so you can try
     * to parse the same position as something else. */
    {
        proven_scan_t sc = proven_scan_init(v("abc"));
        proven_result_i64_t n = proven_scan_i64(&sc);
        EXAMPLE_REQUIRE(n.err == PROVEN_ERR_INVALID_ARG, "'abc' is not an integer");
        EXAMPLE_REQUIRE(sc.cursor == 0, "a failed integer scan leaves the cursor alone");

        /* So the same position can be read as a word instead. */
        proven_result_u8str_view_t w = proven_scan_str(&sc);
        EXAMPLE_REQUIRE(proven_is_ok(w.err) && proven_u8str_view_eq(w.val, PROVEN_LIT("abc")),
                        "the same bytes parse fine as a word");
    }

    /* --- a number that does not fit is OVERFLOW, not a wrapped value ------ */
    {
        proven_scan_t sc = proven_scan_init(v("9223372036854775808"));   /* INT64_MAX + 1 */
        proven_result_i64_t n = proven_scan_i64(&sc);
        EXAMPLE_REQUIRE(n.err == PROVEN_ERR_OVERFLOW, "one past INT64_MAX must not wrap");
        EXAMPLE_REQUIRE(sc.cursor == 0, "the cursor is restored on overflow too");
    }

    /* --- but a float that underflows is NOT an error ---------------------- */
    /* Too large is OVERFLOW; too small is zero, with the sign kept. That
     * asymmetry is deliberate - underflow to zero is the correctly rounded
     * answer, while overflow has no correct finite answer at all. */
    {
        proven_scan_t big = proven_scan_init(v("1e309"));
        proven_result_f64_t b = proven_scan_f64(&big);
        EXAMPLE_REQUIRE(b.err == PROVEN_ERR_OVERFLOW, "1e309 does not fit a double");

        proven_scan_t tiny = proven_scan_init(v("-1e-400"));
        proven_result_f64_t t = proven_scan_f64(&tiny);
        EXAMPLE_REQUIRE(proven_is_ok(t.err), "1e-400 underflows, which is not an error");
        EXAMPLE_REQUIRE(t.val == 0.0, "it rounds to zero");
    }

    /* --- the integer scanners are decimal only ---------------------------- */
    /* "0x10" is not sixteen. It is a zero, followed by text the scanner has not
     * been asked to look at. This surprises people, so it is worth knowing. */
    {
        proven_scan_t sc = proven_scan_init(v("0x10"));
        proven_result_i64_t n = proven_scan_i64(&sc);
        EXAMPLE_REQUIRE(proven_is_ok(n.err) && n.val == 0, "0x10 scans as the integer 0");
        EXAMPLE_REQUIRE(sc.cursor == 1, "and the cursor stops before the 'x'");
```

```
}

/* --- scanning stops at the first byte that cannot belong to the value -- */
{
    proven_scan_t sc = proven_scan_init(v("12abc"));
    proven_result_i64_t n = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(n.err) && n.val == 12, "12abc yields 12");
    EXAMPLE_REQUIRE(sc.cursor == 2, "and leaves 'abc' for whoever asks next");
}

/* --- unsigned means unsigned ------------------------------------------ */
{
    proven_scan_t sc = proven_scan_init(v("-1"));
    proven_result_u64_t n = proven_scan_u64(&sc);
    EXAMPLE_REQUIRE(n.err == PROVEN_ERR_INVALID_ARG,
                    "-1 is rejected rather than wrapping to a huge unsigned value");
}

/* --- navigating to a value: skip_until -------------------------------- */
/* skip_until leaves the cursor ON the target, not past it, so you decide
 * how much of it to consume. */
{
    proven_scan_t sc = proven_scan_init(v("port=8080"));
    proven_err_t err = proven_scan_skip_until(&sc, PROVEN_LIT("="));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the '=' is there");
    EXAMPLE_REQUIRE(sc.cursor == 4, "the cursor sits on the '=' itself");

    ++sc.cursor;                                /* step over it */
    proven_result_i64_t port = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(port.err) && port.val == 8080, "the port parses");

    /* Not finding it is NOT_FOUND, and the cursor does not move - the
     * scanner does not consume the input it failed to navigate. */
    proven_scan_t sc2 = proven_scan_init(v("port=8080"));
    proven_err_t missing = proven_scan_skip_until(&sc2, PROVEN_LIT("#"));
    EXAMPLE_REQUIRE(missing == PROVEN_ERR_NOT_FOUND, "there is no '#'");
    EXAMPLE_REQUIRE(sc2.cursor == 0, "and the cursor stayed put");
}

/* --- the structural scanner ------------------------------------------- */
{
    int id = 0;
    double ratio = 0.0;
    proven_u8str_view_t name = {0};

    proven_err_t err = proven_scan_fmt(v("id=7 ratio=0.5 name=ada"),
                                       "id={} ratio={} name={}",
                                       PROVEN_SCAN_ARG(&id),
                                       PROVEN_SCAN_ARG(&ratio),
                                       PROVEN_SCAN_ARG(&name));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the line matches the shape");
    EXAMPLE_REQUIRE(id == 7 && ratio == 0.5, "the values land in the right places");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(name, PROVEN_LIT("ada")), "including the word");
}

/* --- the structural scanner is NOT transactional ---------------------- */
/*
 * This is the one that bites. When a literal fails to match, the scan
 * returns an error - but the placeholders BEFORE the mismatch have already
 * been written through. `id` is 7 even though the call failed.
```

```
     *
     * So: on failure, treat every destination as clobbered. If you need
     * all-or-nothing, scan into locals and only publish them once the call
     * succeeded, which is what the code below does.
     */
    {
        int id = -1;
        double ratio = -1.0;
        proven_err_t err = proven_scan_fmt(v("id=7 ratio=0.5"),
                                           "id={} XXX={}",       /* the literal is wrong */
                                           PROVEN_SCAN_ARG(&id),
                                           PROVEN_SCAN_ARG(&ratio));
        EXAMPLE_REQUIRE(err == PROVEN_ERR_NOT_FOUND, "the literal 'XXX=' is not in the input");
        EXAMPLE_REQUIRE(id == 7, "and yet id was already written: the scan is not atomic");

        /* The safe shape: scan into locals, publish on success. */
        int good_id = 0;
        double good_ratio = 0.0;
        int published_id = -1;
        proven_err_t ok = proven_scan_fmt(v("id=7 ratio=0.5"), "id={} ratio={}",
                                          PROVEN_SCAN_ARG(&good_id), PROVEN_SCAN_ARG(&good_ratio));
        if (proven_is_ok(ok)) published_id = good_id;
        EXAMPLE_REQUIRE(published_id == 7, "publish only what a successful scan produced");
    }

    /* --- running out of input, and having input left over ------------------ */
    {
        int a = 0, b = 0;
        proven_err_t short_input = proven_scan_fmt(v("5"), "{} {}",
                                                   PROVEN_SCAN_ARG(&a), PROVEN_SCAN_ARG(&b));
        EXAMPLE_REQUIRE(!proven_is_ok(short_input), "two placeholders, one value: that fails");

        /* Trailing input is NOT an error. The scanner matched what you asked for
         * and stopped; it does not police what you did not ask about. If the
         * whole line must be consumed, check that yourself. */
        int only = 0;
        proven_scan_t sc = proven_scan_init(v("7 8"));
        proven_err_t err = proven_scan_fmt_cursor(&sc, "{}", PROVEN_SCAN_ARG(&only));
        EXAMPLE_REQUIRE(proven_is_ok(err) && only == 7, "the first value scans");
        EXAMPLE_REQUIRE(sc.cursor < sc.view.size, "and '8' is still sitting there, unconsumed");
    }

    /* --- narrow destinations are range-checked ----------------------------- */
    {
        short small = 0;
        proven_err_t err = proven_scan_fmt(v("70000"), "{}", PROVEN_SCAN_ARG(&small));
        EXAMPLE_REQUIRE(err == PROVEN_ERR_OVERFLOW,
                        "70000 does not fit a short, and the scanner says so rather than truncating");
    }

    return EXAMPLE_OK();
}
```

Misuse: assuming `0x10` is sixteen
It is zero. The integer scanners are decimal only, and `x10` is still in the input. If you need hex, you are writing that digit loop yourself.

Misuse: treating trailing input as an error
It is not one. One placeholder against `"7 8"` succeeds. Check the cursor if you care.

Misuse: trusting destinations after a failed scan
They are clobbered. See §11.1.

Misuse: keeping a scanned word after its input is gone
`proven_scan_str` returns a **view into the input**, not a copy. When the buffer goes, so does the word. Copy it with `proven_u8str_create_from_view()` if it has to outlive the bytes it came from.

13. Freestanding and build-mode notes

Why float is the one thing that gets compiled out
Everything in this chapter is portable computation except one part, and that part is unusually expensive.

Formatting a `double` correctly — so that the shortest decimal that round-trips is what you get, on every input including subnormals — needs big-integer arithmetic and lookup tables. It is the largest single piece of code in the formatter, and most firmware never prints a `double` at all. So the freestanding profile sets `PROVEN_FMT_NO_FLOAT` and drops it, and a build for a microcontroller does not carry kilobytes of decimal-conversion tables it will never call.

This is a build-time decision rather than a run-time one on purpose: the code is *absent*, not merely unreachable, so the linker cannot be talked into keeping it. §8a of the freestanding guide covers the related knob — the big-integer capacity — for builds that do want floats on a small target.

The scanner is core: it does no I/O, allocates nothing, and touches no platform layer, so it is available in a freestanding build exactly as it is in a hosted one.

The one build-mode dependency is float. The freestanding build sets `PROVEN_FMT_NO_FLOAT` (which compiles out the float *formatter*) along with `PROVEN_NO_U16STR`. `proven_scan_f64` pulls in the decimal parsing engine (`float_parse.c` and `float_decimal.c`), which is integer-only - no `long double`, no libm, no soft-float helper calls - but it is not free in code size. On a target where that matters and you do not need to read floats, simply do not call it: nothing else in the scanner references the float path.

`proven_sysio_scanner_*` (Chapter 5) is a **different** thing: a buffered scanner over a file, hosted-only, because it does I/O. The scanner described in this chapter reads bytes you already have.

Worked example: wider numbers, loose text, and floating point
The examples above scan rigid formats into `proven_i32`. Three things come up as soon as the input is real, and this example covers them together.

The destination type is the contract. `proven_scan_arg_i64()`, `proven_scan_arg_u32()` and `proven_scan_arg_u64()` name the type the value is written into, and the scanner reports an overflow rather than wrapping. A byte count that needs more than 32 bits, and a value that must never be negative, are the two cases where the wrong destination type is a bug nobody notices until the numbers get big. The generic `PROVEN_SCAN_ARG()` macro chooses these from the pointer you hand it; the named forms are what it chooses, written out.

Not every input is a rigid format. `proven_scan_skip_whitespace()` advances past spaces, tabs and newlines, and `proven_scan_skip_until_number()` runs the cursor forward to the next digit — or to a sign immediately followed by one, so a negative number keeps its sign. Neither can fail: "there was nothing to skip" is not an error, and at the end of the input the cursor stops instead of running off.

Floating point has a locale problem, and this library does not have it. `strtod()` reads `"3,5"` as `3.5` under one locale and as `3` under another, so the same program disagrees with itself across machines. `proven_parse_double_ascii()` is locale-free by construction: a comma is never a decimal point. It reports how many bytes the number used, so a caller can carry on from there, and it reports unparsable text as an error rather than as a `0` that cannot be told from a real one.

`proven_parse_f64_ascii()` is the same function under its earlier name, kept so existing call sites still read correctly.

On the way out, `proven_arg_f64()` is how a floating-point value enters the formatter — both `float` and `double` go through it, so what a program prints does not depend on the width of the variable it was kept in. When the exact spelling matters, `proven_float_format_f32_policy()` and its `f64` sibling take the policy and options explicitly. Shortest mode asks for the fewest digits that read back as exactly this value, which is what a serialiser wants: an exact round trip without printing seventeen digits for numbers that do not need them.

```
#include <string.h>

/*
 * Reading numbers out of text, and writing them back.
 *
 * The text here is the sort a program actually meets: a log line with mixed
 * widths and signs, and a measurements file where the interesting number is
 * buried in prose. That means three jobs the earlier chapter-8 examples do not
 * cover:
 *
 *   - Scanning into types WIDER than int, and into unsigned ones, where the
 *     destination type is the whole question - a byte count that does not fit in
 *     32 bits is the classic overflow nobody notices until the file is 5 GB.
 *   - Positioning the cursor by hand - skipping whitespace, or skipping ahead to
 *     wherever the next number starts - when the input is not a rigid format.
 *   - Converting a decimal string to a double and back, exactly, without going
 *     through the C library's locale-dependent conversions.
 *
 * The float part matters more than it sounds. strtod reads "3,5" as 3.5 in one
 * locale and as 3 in another, and the same program then disagrees with itself
 * across machines. The parser used here is locale-free by construction: a comma
 * is never a decimal point, whatever the environment says.
 */

int main(void) {
    /* --- 1. scanning into the type the value needs ----------------------- */

    /* A log line: a request id that needs 64 bits, a byte count that is never
     * negative and can exceed 4 GB, a small status code, and a negative offset. */
    proven_scan_t scan = proven_scan_init(
        PROVEN_LIT("id=9007199254740993 bytes=5368709120 status=404 delta=-17"));

    proven_i64 id = 0;
    proven_u64 bytes = 0;
    proven_u32 status = 0;
    proven_i32 delta = 0;

    /* Each argument constructor names the destination type, so the scanner
     * writes the right width and reports an overflow instead of wrapping. The
     * generic PROVEN_SCAN_ARG() macro picks these for you from the pointer's
     * type; the named forms are what it picks, and what you write when you want
     * the choice visible. */
    proven_err_t err = proven_scan_fmt_cursor(&scan, "id={} bytes={} status={} delta={}",
                                      proven_scan_arg_i64(&id),
                                      proven_scan_arg_u64(&bytes),
                                      proven_scan_arg_u32(&status),
                                      proven_scan_arg_i32(&delta));
    EXAMPLE_REQUIRE(proven_is_ok(err), "scanning the log line must succeed");
    EXAMPLE_REQUIRE(id == 9007199254740993LL, "a 64-bit id survives, which a 32-bit destination could not");
```

```
EXAMPLE_REQUIRE(bytes == 5368709120ULL, "and so does a byte count larger than 4 GiB");
EXAMPLE_REQUIRE(status == 404u, "the small unsigned value reads normally");
EXAMPLE_REQUIRE(delta == -17, "and a signed destination accepts the minus sign");

/* The destination type is a contract, not a hint. A negative number has no
 * unsigned representation, and the scanner refuses rather than wrapping it
 * to something enormous. */
proven_scan_t neg = proven_scan_init(PROVEN_LIT("-17"));
proven_u32 nowhere = 12345;
err = proven_scan_fmt_cursor(&neg, "{}", proven_scan_arg_u32(&nowhere));
EXAMPLE_REQUIRE(err != PROVEN_OK, "a negative value cannot be scanned into an unsigned destination");
EXAMPLE_REQUIRE(nowhere == 12345, "and the destination is left as it was");

/* --- 2. moving the cursor when the input is not rigid ----------------- */

/* Free-form text: the numbers matter, the words between them do not. */
proven_scan_t notes = proven_scan_init(
    PROVEN_LIT("   sample A measured 42 units; sample B measured -8 units"));

/* skip_whitespace advances past spaces, tabs and newlines. It is what you
 * call between fields of a format you are driving by hand - it never fails,
 * because "there was no whitespace" is not an error. */
proven_scan_skip_whitespace(&notes);
EXAMPLE_REQUIRE(notes.cursor == 3, "the three leading spaces are consumed");

/* skip_until_number runs the cursor forward to the first digit, or to a sign
 * immediately followed by one. It is the call that turns "find the number in
 * this line" from a hand-written loop into one statement. */
proven_scan_skip_until_number(&notes);
proven_i32 first = 0;
err = proven_scan_fmt_cursor(&notes, "{}", proven_scan_arg_i32(&first));
EXAMPLE_REQUIRE(proven_is_ok(err) && first == 42, "the first number in the line is 42");

proven_scan_skip_until_number(&notes);
proven_i32 second = 0;
err = proven_scan_fmt_cursor(&notes, "{}", proven_scan_arg_i32(&second));
EXAMPLE_REQUIRE(proven_is_ok(err) && second == -8,
                "and the next one keeps its sign, because the sign is part of the number");

/* At the end there is nothing left to find, and the cursor stops rather than
 * running off: the scan is over, not broken. */
proven_scan_skip_until_number(&notes);
EXAMPLE_REQUIRE(notes.cursor == notes.view.size, "with no number left, the cursor lands at the end");

/* --- 3. decimal text to double, without a locale ---------------------- */

proven_parse_double_result_t d = proven_parse_double_ascii(PROVEN_LIT("3.14159 rest"));
EXAMPLE_REQUIRE(proven_is_ok(d.err), "parsing a decimal number must succeed");
EXAMPLE_REQUIRE(d.val > 3.14158 && d.val < 3.14160, "and produce the value the digits spell");

/* consumed says where the number ended, so the caller can carry on from
 * there - which is how you parse a list without copying it into pieces. */
EXAMPLE_REQUIRE(d.consumed == 7, "consumed reports exactly the bytes the number used");

/* A comma is not a decimal point here and never will be, whatever locale the
 * program is running under. The number ends at the comma. */
proven_parse_double_result_t comma = proven_parse_double_ascii(PROVEN_LIT("3,5"));
EXAMPLE_REQUIRE(proven_is_ok(comma.err) && comma.consumed == 1 && comma.val == 3.0,
                "a comma ends the number: the parser is locale-free by construction");
```

```
/* proven_parse_f64_ascii is the same function under its earlier name, kept
 * for code written before the current one existed. New code should use
 * proven_parse_double_ascii; both are here so an old call site still reads
 * correctly. */
proven_parse_f64_result_t same = proven_parse_f64_ascii(PROVEN_LIT("3.14159 rest"));
EXAMPLE_REQUIRE(same.val == d.val && same.consumed == d.consumed,
                "the compatibility name is the same parser");

/* Text that is not a number at all is an error, not a silent zero - which is
 * what atof() would have handed back with nothing to distinguish it from a
 * genuine "0". */
proven_parse_double_result_t junk = proven_parse_double_ascii(PROVEN_LIT("not a number"));
EXAMPLE_REQUIRE(!proven_is_ok(junk.err), "unparsable text is reported, not turned into 0");
EXAMPLE_REQUIRE(junk.consumed == 0, "and nothing was consumed");

/* --- 4. writing a float back ------------------------------------------- */

/* proven_arg_f64 is how a floating-point value enters the formatter. Both
 * float and double go through it, so the digits a program prints do not
 * depend on the width of the variable it happened to be kept in. */
proven_allocator_t alloc = proven_heap_allocator();
proven_result_u8str_t line = proven_u8str_create(alloc, 64);
EXAMPLE_REQUIRE(proven_is_ok(line.err), "creating the output string must succeed");

proven_fmt_result_t out = proven_u8str_append_fmt(&line.value, "measured {} units",
                                                  proven_arg_f64(d.val));
EXAMPLE_REQUIRE(proven_is_ok(out.err), "formatting the parsed value must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_starts_with(proven_u8str_as_view(&line.value),
                                              PROVEN_LIT("measured 3.14159")),
                "and spell the number it was given");

/* The policy form is the explicit one, for when the caller needs a
 * particular spelling rather than the default. SHORTEST asks for the fewest
 * digits that read back as exactly this value - which is the property a
 * serialiser wants, because it makes the round trip exact without printing
 * seventeen digits for numbers that do not need them. */
char shortest[64];
proven_size_t wrote = 0;
float measured = 0.1f;
proven_err_t ferr = proven_float_format_f32_policy(shortest, sizeof shortest, measured,
                                                   PROVEN_FLOAT_FORMAT_POLICY_RYU,
                                                   proven_float_format_options_shortest(),
                                                   &wrote);
EXAMPLE_REQUIRE(proven_is_ok(ferr), "formatting a float in shortest mode must succeed");
EXAMPLE_REQUIRE(wrote > 0 && shortest[wrote] == '\0', "the result is written and terminated");
EXAMPLE_REQUIRE(strcmp(shortest, "0.1") == 0,
                "0.1f prints as 0.1: the shortest text that reads back as the same float");

/* And it round-trips: the text reads back as the value it came from. That is
 * the guarantee shortest mode exists to provide. */
proven_parse_double_result_t roundtrip = proven_parse_double_ascii(
    (proven_u8str_view_t){ .ptr = (const proven_byte_t *)shortest, .size = wrote });
EXAMPLE_REQUIRE(proven_is_ok(roundtrip.err), "the shortest form parses back");
EXAMPLE_REQUIRE((float)roundtrip.val == measured, "as exactly the float it was printed from");

printf("scanned id=%lld bytes=%llu; formatted %s and %s\n",
       (long long)id, (unsigned long long)bytes, proven_u8str_as_cstr(&line.value), shortest);

proven_u8str_destroy(alloc, &line.value);
```

```
    return EXAMPLE_OK();
}
```

Wrong — scanning a large value into a 32-bit destination:

```
proven_i32 bytes = 0;
proven_err_t e = proven_scan_fmt_cursor(&scan, "bytes={}", proven_scan_arg_i32(&bytes));
/* input says 5368709120 */                                    /* wrong type */
```

The value does not fit. Choosing the destination type by what is convenient, rather than by the range the field can hold, is the same mistake as declaring a file offset `int`.

Wrong — using `atof` or `strtod` for data that crosses machines:

```
double v = strtod(text, NULL);   /* wrong: the decimal point depends on the locale */
```

And `atof` cannot report failure at all: unparsable text becomes `0`, which is indistinguishable from a genuine zero in the data.

Wrong — assuming `skip_until_number` reports "not found":

```
proven_scan_skip_until_number(&scan);
proven_i32 n = 0;
proven_err_t e = proven_scan_fmt_cursor(&scan, "{}", proven_scan_arg_i32(&n));  /* may fail */
```

It moves the cursor and returns nothing. When there is no number left the cursor lands at the end of the input, and it is the scan afterwards that tells you so — check that error rather than assuming a number was found.

Worked example: naming the argument type yourself

`PROVEN_ARG(x)` and `PROVEN_SCAN_ARG(&x)` choose an argument constructor from the type of what you hand them, and most code never needs to know which one they chose. Two situations take the choice back:

1. **The macro has no type to dispatch on that means what you mean.** A `proven_u8str_view_t`, a broken-down date, a raw address printed for a diagnostic — `proven_arg_str_view()`, `proven_arg_datetime()` and `proven_arg_ptr()` exist because those are deliberate choices, not defaults to fall into.
2. **The argument list is built at run time.** A logging helper whose parameters are already `proven_arg_t` values cannot re-wrap them — a macro that turns a value into an argument has nothing to do with something that is already one. `proven_arg_identity()` and `proven_scan_arg_identity()` are what let the same macro-driven code path accept an argument built earlier.

The example writes both sides out by name: the formatting constructors for a character, a boolean, unsigned 32- and 64-bit values, the three ways of passing text (`proven_arg_str_view()`, `proven_arg_cstr()`, `proven_arg_ucstr()`), a date and a pointer; and the scanning constructors for every plain C integer width (`short`, `int`, `long`, `long long` and their unsigned forms), a `double`, and a string view captured without copying.

The three text constructors differ in how much they trust the caller, and the order is worth remembering:

Constructor	What it is given	What can go wrong
`proven_arg_str_view`	a pointer **and** a length	nothing is scanned for a terminator, so there is none to be missing — prefer this
`proven_arg_cstr`	a NUL-terminated C string	the terminator must be there and the memory must be alive, or the formatter reads past the end

proven_arg_ucstr	the same, as `unsigned char *`	exists so a byte buffer does not need a cast that silences a real warning

```
/*
 * Every value handed to the formatter arrives as a proven_arg_t, and every value
 * the scanner writes into arrives as a proven_scan_arg_t. Most of the time you
 * never see either: PROVEN_ARG(x) and PROVEN_SCAN_ARG(&x) pick the right
 * constructor from the type of what you passed, and that is the whole point of
 * them.
 *
 * Two situations take the choice back off the macro, and both are ordinary:
 *
 *   1. The macro cannot see a type it recognises. A `proven_u8str_view_t`, a
 *      broken-down date, a raw address printed for a diagnostic - these need the
 *      constructor named, because there is no plain C type for the macro to
 *      dispatch on that means what you mean.
 *
 *   2. You are building the argument list at RUNTIME. A logging helper that
 *      takes "whatever the caller already assembled" receives proven_arg_t
 *      values, not ints and strings - and a macro that turns a value into an
 *      argument cannot be handed something that is already an argument. That is
 *      what the identity constructors are for: they let the same macro-driven
 *      code path accept an argument that was built earlier.
 *
 * So this example writes both sides out by name. The width and signedness of a
 * destination is not decoration - it is what decides whether 70000 arrives as
 * 70000 or as 4464.
 */

/* A logging helper that takes arguments the caller has already built. Because
 * its parameters are proven_arg_t, PROVEN_ARG would be the wrong tool inside it
 * - the value is already an argument, and proven_arg_identity is what says so. */
static proven_fmt_result_t log_pair(proven_u8str_t *out, const char *fmt,
                                    proven_arg_t a, proven_arg_t b) {
    return proven_u8str_append_fmt(out, fmt, proven_arg_identity(a), proven_arg_identity(b));
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_u8str_t line = proven_u8str_create(alloc, 256);
    EXAMPLE_REQUIRE(proven_is_ok(line.err), "creating the output string must succeed");
    if (!proven_is_ok(line.err)) {
        return 1;
    }
    proven_u8str_t out = line.value;

    /* --- naming the formatting argument yourself -------------------------- */

    /* A single character and a flag. Written by name here so it is visible that
     * a bool prints as true/false rather than as 1/0. */
    proven_fmt_result_t r = proven_u8str_append_fmt(&out, "flag={} mark={}",
                                                    proven_arg_bool(true),
                                                    proven_arg_char('!'));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting a bool and a char must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("flag=true mark=!")),
                    "a bool renders as a word, not as a digit");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");
```

```
    /* Unsigned widths. The constructor is the declaration of what the value is:
     * an unsigned 32-bit count and an unsigned 64-bit byte total are different
     * facts, and writing them out says which one you have. */
    r = proven_u8str_append_fmt(&out, "files={} bytes={}",
                                proven_arg_u32(1200u),
                                proven_arg_u64(5368709120ULL));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting unsigned values must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("files=1200
bytes=5368709120")),
                    "a 64-bit total is printed in full, not truncated to 32 bits");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* Three ways to give the formatter text, in order of how much they trust
     * the caller:
     *
     *   proven_arg_str_view  - a pointer AND a length. Nothing is scanned for a
     *                          terminator, so there is no terminator to be
     *                          missing. Prefer this.
     *   proven_arg_cstr      - a NUL-terminated C string. The formatter walks it
     *                          looking for the terminator, so it must be there,
     *                          and the memory must still be alive.
     *   proven_arg_ucstr     - the same, for `unsigned char *`, which is what a
     *                          byte buffer is usually typed as. It exists so the
     *                          caller does not have to write a cast that
     *                          silences a real warning.
     */
    const char *name = "report.txt";
    const unsigned char *tag = (const unsigned char *)"draft";
    proven_u8str_view_t note = PROVEN_LIT("first pass");

    r = proven_u8str_append_fmt(&out, "{} [{}] {}",
                                proven_arg_cstr(name),
                                proven_arg_ucstr(tag),
                                proven_arg_str_view(note));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting the three text forms must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out),
                                         PROVEN_LIT("report.txt [draft] first pass")),
                    "all three produce the same kind of output from different kinds of pointer");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* A date, and an address. Neither has a plain C type that means what it
     * means: a date is a struct, and an address printed for a diagnostic is a
     * deliberate act rather than something to fall into. */
    proven_datetime_t when = proven_time_breakdown(0);   /* the epoch: a fixed, checkable value */
    int local = 0;

    r = proven_u8str_append_fmt(&out, "at {} object {}",
                                proven_arg_datetime(when),
                                proven_arg_ptr(&local));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting a date and a pointer must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(proven_u8str_as_view(&out), PROVEN_LIT("at 1970-01-01")),
                    "the epoch breaks down to the first of January 1970");
    EXAMPLE_REQUIRE(proven_u8str_view_find(proven_u8str_as_view(&out), 0, PROVEN_LIT("0x")) !=
PROVEN_SIZE_MAX,
                    "and an address is rendered in hexadecimal");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");
```

```
/* Arguments built here, passed on, and formatted there. */
r = log_pair(&out, "status={} retries={}", proven_arg_cstr("ok"), proven_arg_u32(3u));
EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting pre-built arguments must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("status=ok retries=3")),
                "an argument built by the caller formats exactly as one built in place");

/* --- naming the scanning destination yourself ------------------------- */

/* On the way in, the constructor names the destination, and the destination
 * decides what "too big" means. These are the plain C types - short, int,
 * long, long long and their unsigned forms - for the very common case where
 * the variable you already have is one of them rather than a fixed-width
 * type. */
proven_scan_t scan = proven_scan_init(
    PROVEN_LIT("h=-32000 uh=65000 i=-2000000 ui=4000000000 l=-9000000 ul=9000000 "
               "ll=-9007199254740993 ull=18446744073709551615 f=2.5 word=alpha"));

short h = 0;
unsigned short uh = 0;
int i = 0;
unsigned int ui = 0;
long l = 0;
unsigned long ul = 0;
long long ll = 0;
unsigned long long ull = 0;
double f = 0.0;
proven_u8str_view_t word = {0};

proven_err_t err = proven_scan_fmt_cursor(
    &scan, "h={} uh={} i={} ui={} l={} ul={} ll={} ull={} f={} word={}",
    proven_scan_arg_short(&h),
    proven_scan_arg_ushort(&uh),
    proven_scan_arg_int(&i),
    proven_scan_arg_uint(&ui),
    proven_scan_arg_long(&l),
    proven_scan_arg_ulong(&ul),
    proven_scan_arg_llong(&ll),
    proven_scan_arg_ullong(&ull),
    proven_scan_arg_f64(&f),
    proven_scan_arg_str_view(&word));
EXAMPLE_REQUIRE(proven_is_ok(err), "scanning every plain integer width must succeed");
EXAMPLE_REQUIRE(h == -32000 && uh == 65000u, "the short forms hold their values");
EXAMPLE_REQUIRE(i == -2000000 && ui == 4000000000u, "and so do the int forms");
EXAMPLE_REQUIRE(l == -9000000L && ul == 9000000UL, "and the long forms");
EXAMPLE_REQUIRE(ll == -9007199254740993LL, "a value needing 64 bits arrives whole");
EXAMPLE_REQUIRE(ull == 18446744073709551615ULL, "including the largest unsigned 64-bit value");
EXAMPLE_REQUIRE(f > 2.4999 && f < 2.5001, "the floating-point destination reads a decimal");

/* A scanned string view points INTO the text being scanned. Nothing was
 * copied and nothing was allocated, so it is valid exactly as long as that
 * text is - copy it into a proven_u8str_t if it must outlive the scan. */
EXAMPLE_REQUIRE(proven_u8str_view_eq(word, PROVEN_LIT("alpha")), "the word is captured as a view");

/* The destination's width is a promise the scanner keeps. 70000 does not fit
 * in a short, so it is refused rather than wrapped to 4464 - which is what a
 * cast would have produced, silently, in a file nobody looks at again. */
proven_scan_t narrow = proven_scan_init(PROVEN_LIT("70000"));
short too_small = 7;
err = proven_scan_fmt_cursor(&narrow, "{}", proven_scan_arg_short(&too_small));
```

```
    EXAMPLE_REQUIRE(err != PROVEN_OK, "a value that does not fit the destination is refused");
    EXAMPLE_REQUIRE(too_small == 7, "and the destination keeps the value it had");

    /* The scanning identity constructor, for the same reason as the formatting
     * one: a helper that receives ready-made scan arguments cannot re-wrap them. */
    proven_scan_t again = proven_scan_init(PROVEN_LIT("41"));
    proven_i32 answer = 0;
    proven_scan_arg_t prebuilt = proven_scan_arg_i32(&answer);
    err = proven_scan_fmt_cursor(&again, "{}", proven_scan_arg_identity(prebuilt));
    EXAMPLE_REQUIRE(proven_is_ok(err) && answer == 41, "a pre-built scan argument works unchanged");

    printf("arguments: %s\n", proven_u8str_as_cstr(&out));

    proven_u8str_destroy(alloc, &out);
    return EXAMPLE_OK();
}
```

Wrong — wrapping an argument that is already an argument:

```
static proven_fmt_result_t log_one(proven_u8str_t *out, proven_arg_t a) {
    return proven_u8str_append_fmt(out, "{}", PROVEN_ARG(a));   /* wrong */
}
```

Use `proven_arg_identity(a)`. `PROVEN_ARG` is for values; `a` is already an argument.

Wrong — keeping a scanned string view after the text is gone:

```
proven_u8str_view_t word = {0};
{
    proven_u8str_t line = read_a_line(alloc);
    proven_scan_t s = proven_scan_init(proven_u8str_as_view(&line));
    proven_err_t e = proven_scan_fmt_cursor(&s, "word={}", proven_scan_arg_str_view(&word));
    proven_u8str_destroy(alloc, &line);     /* wrong: `word` pointed into `line` */
}
use(word);
```

A scanned view points into the text being scanned. Copy it into a `proven_u8str_t` if it has to outlive that text.

Chapter 5: Hosted Services

Part V — Talking to the operating system. Prerequisite: Part II (1, 2, 3). After this chapter you can read and write files without losing data on a crash, read input a line at a time, generate randomness that suits the job, and tell wall-clock time from elapsed time.

This chapter covers `fs.h`, `sysio.h`, `mmap.h`, `time.h`, `stream.h`, and `random.h`. Everything in it needs an operating system: this is the only part of the manual that does not apply to a freestanding build.

These APIs require hosted platform support and are excluded from the current freestanding subset.

Table of contents

1. Filesystem API

The filesystem layer wraps platform file handles, paths, directory listing, metadata, permissions, links, and locks.

Raw filesystem helpers do not sanitize untrusted paths, enforce root confinement, or defend against symlink-race TOCTOU. Callers that accept untrusted paths must validate them before using the API.

Structures and enums

```
typedef struct {
    union {
        void *ptr;
        int fd;
    } internal;
} proven_fs_handle_t;

typedef proven_fs_handle_t proven_file_t;
```

Intent: represent a platform file handle without exposing POSIX or Win32 details to higher layers.

```
typedef enum {
    PROVEN_FS_READ   = 1 << 0,
    PROVEN_FS_WRITE  = 1 << 1,
    PROVEN_FS_APPEND = 1 << 2,
    PROVEN_FS_CREATE = 1 << 3,
    PROVEN_FS_TRUNC  = 1 << 4,
    PROVEN_FS_CREATE_NEW = 1 << 5
} proven_fs_mode_t;
```

File mode flags can be combined. Examples:

- `PROVEN_FS_READ`
- `PROVEN_FS_WRITE ¦ PROVEN_FS_CREATE ¦ PROVEN_FS_TRUNC`
- `PROVEN_FS_WRITE ¦ PROVEN_FS_APPEND ¦ PROVEN_FS_CREATE`

```
typedef enum {
    PROVEN_FS_TYPE_FILE,
    PROVEN_FS_TYPE_DIR,
    PROVEN_FS_TYPE_OTHER
} proven_fs_type_t;

typedef struct {
    proven_u8str_t name;
    proven_fs_type_t type;
    proven_size_t size;
} proven_fs_entry_t;
```

Directory entries own `name`. Use `proven_fs_list_destroy()` for arrays returned by `proven_fs_list()`.

```
typedef struct {
    proven_err_t err;
    proven_file_t value;
} proven_result_file_t;

typedef enum {
    PROVEN_FS_PERM_OWNER_R = 1 << 8,
    PROVEN_FS_PERM_OWNER_W = 1 << 7,
    PROVEN_FS_PERM_OWNER_X = 1 << 6,
    PROVEN_FS_PERM_GROUP_R = 1 << 5,
    PROVEN_FS_PERM_GROUP_W = 1 << 4,
    PROVEN_FS_PERM_GROUP_X = 1 << 3,
    PROVEN_FS_PERM_OTHER_R = 1 << 2,
    PROVEN_FS_PERM_OTHER_W = 1 << 1,
    PROVEN_FS_PERM_OTHER_X = 1 << 0,
    PROVEN_FS_PERM_DEFAULT = PROVEN_FS_PERM_OWNER_R ¦ PROVEN_FS_PERM_OWNER_W ¦
                             PROVEN_FS_PERM_GROUP_R ¦ PROVEN_FS_PERM_OTHER_R
} proven_fs_perms_t;

typedef enum {
    PROVEN_FS_LOCK_SHARED,
    PROVEN_FS_LOCK_EXCLUSIVE,
    PROVEN_FS_LOCK_UNLOCK
} proven_fs_lock_type_t;

typedef struct {
    proven_size_t size;          /* size in bytes (0 for directories on some hosts) */
    proven_fs_type_t type;       /* PROVEN_FS_TYPE_FILE / _DIR - there is no symlink type */
    proven_fs_perms_t perms;     /* the nine permission bits only; the file type is in `type` */
    proven_i64 created_at;       /* always 0: no birth time is queried - see below */
    proven_i64 modified_at;      /* last-modification time, seconds since the Unix epoch */
    unsigned long long dev;      /* device id (POSIX st_dev; 0 where unavailable) */
    unsigned long long ino;      /* inode number (POSIX st_ino; 0 where unavailable) */
    unsigned long long uid;      /* owner id  (POSIX st_uid; 0 on Windows — no uid/gid) */
    unsigned long long gid;      /* group id  (POSIX st_gid; 0 on Windows) */
} proven_fs_stat_t;
```

`uid`/`gid` (added in v26.06.22a) hold the POSIX owner and group ids; both are `0` on Windows, which has no `uid`/`gid` concept. Resolve them to names with the host's `getpwuid`/`getgrgid` when displaying an `ls -l`-style owner/group column.

One field is narrower than it looks, and one has a sharp edge:

- `created_at` is **always 0**. Plain `stat()` has no portable birth time, and the PAL does not ask for one. Only `modified_at` carries a real timestamp.
- `type` is `FILE` for a regular file, `DIR` for a directory, and **`PROVEN_FS_TYPE_OTHER`** for everything else — a FIFO, a socket, a device, a dangling symlink. (Until v26.07.13a all of those stat'd as `FILE`, which told a caller it could open them and read bytes out of them. A dangling symlink cannot be opened at all.)

`type` **follows symlinks**, here and in the directory walk, which is what makes the two agree: a symlink to a regular file is `FILE`, and a symlink to a directory is `DIR`. The edge that follows is real — **a recursive walker can loop**, because a symlink pointing at an ancestor is a cycle and its type says `DIR`. Carry a depth limit, or remember `(dev, ino)` pairs and refuse to descend into one you have seen.

```
proven_fs_stat_t st;
if (proven_is_ok(proven_fs_stat(scratch, PROVEN_LIT("/etc/hosts"), &st))) {
    proven_println("size={} uid={} gid={}",
        PROVEN_ARG(st.size), PROVEN_ARG(st.uid), PROVEN_ARG(st.gid));
}
```

Macro

Macro	Intent
`PROVEN_FS_PATH_SEP`	Preferred library-level separator character, currently `'/'`.

File functions

API	Intent	Return
`proven_fs_open(scratch, path, mode)`	Open a file path. `scratch` is used for path conversion.	`proven_result_file_t`.
`proven_fs_close(file)`	Close file handle.	**Returns an error.** On a file you wrote to, the close *is* part of the write: NFS, CIFS and quota-enforcing filesystems report a failed write-back here and nowhere else. On a file you only read, `(void)`-ing it is fine.
`proven_fs_read(file, dest)`	Single read attempt into mutable slice.	`proven_result_size_t`.
`proven_fs_write(file, src)`	Single write attempt from byte view.	`proven_result_size_t`.
`proven_fs_write_all(file, src)`	Retry until all bytes are written or an error occurs.	`proven_err_t`.
`proven_fs_size(file)`	Query open file size.	`proven_result_size_t`.
`proven_fs_rename(scratch, src, dest)`	Rename or move path.	`proven_err_t`.
`proven_fs_remove(scratch, path)`	Remove file.	`proven_err_t`.
`proven_fs_copy(temp_alloc, src, dest)`	Copy file using temporary buffer allocation.	`proven_err_t`.
`proven_fs_mkdir(scratch, path)`	Create directory.	`proven_err_t`.
`proven_fs_rmdir(scratch, path)`	Remove empty directory.	`proven_err_t`.
`proven_fs_list(alloc, path)`	List directory into `proven_array_t` of `proven_fs_entry_t`.	`proven_result_array_t`.
`proven_fs_list_destroy(alloc, list)`	Destroy directory listing and entry names.	void.
`proven_fs_chmod(scratch, path, perms)`	Set permissions.	`proven_err_t`.

`proven_fs_lock(file, type, wait)`	Acquire/release file lock.	`proven_err_t`.
`proven_fs_stat(scratch, path, out_stat)`	Fill metadata.	`proven_err_t`.

`proven_fs_stat()` reports only the nine permission bits in `perms`, so a stat's `perms` can be handed straight back to `proven_fs_chmod()`. It used to carry the raw POSIX `st_mode`, whose file-type bits `chmod` rejects - which made that round-trip, the obvious use of the field, fail with `PROVEN_ERR_INVALID_ARG` for every real file. Read the file type from `type`.

Position, and the difference between atomic and durable

A handle that cannot seek says so. A pipe, a FIFO or a terminal returns `PROVEN_ERR_UNSUPPORTED` from `proven_fs_seek`, not `PROVEN_ERR_IO`. Not being seekable is a property of the thing, not a failure of the call, and code that adapts to it — the scanner does — has to be able to tell them apart.

`pread` **and** `pwrite` **do not move the position.** That is what they are for: two readers sharing one handle cannot race on a cursor that neither of them moves.

Atomic and durable are different promises, and conflating them is how data gets lost.

- `proven_fs_write_file_atomic` guarantees that a *reader* never sees a half-written file. It says nothing about a power cut: the kernel may still be holding your bytes, and the rename may reach the disk before the data it points at.
- `proven_fs_write_file_durable` closes that window, in the only order that works: fsync the temp file, **then** rename, **then** fsync the directory. Syncing the file but not the directory leaves a crash window in which the bytes are safe and the name that points at them is not — which is exactly the corruption an atomic write exists to prevent.

The durable form waits for the storage device twice. Use it when losing the write would be worse than the wait, and the atomic form when it would not.

There used to be a `proven_sysio_flush` here that was none of this: it claimed to flush a buffer that did not exist. It is **deleted**. Pushing a buffered writer's bytes to the OS is `proven_writer_flush`; pushing the OS's bytes to the disk is `proven_fs_sync`. Those are two different operations, and one word could not honestly mean both.

Important behavior:

- `proven_fs_read()` and `proven_fs_write()` are single-operation APIs and may process fewer bytes than requested.
- **A read at end-of-file returns** `PROVEN_ERR_EOF`**, not a zero-byte success.** A loop written the obvious way - `if (r.value == 0) break;` - never takes that branch, and treats the end of the file as an I/O failure instead. Check for `PROVEN_ERR_EOF` explicitly; the worked example at the end of this chapter shows the shape.
- Use `proven_fs_write_all()` when all bytes must be written.
- Zero-size read/write requests should succeed with zero bytes processed without requiring a non-null buffer.
- `proven_fs_is_absolute()` recognizes POSIX absolute paths, Windows drive-root paths, UNC paths, and extended Windows path forms.
- `proven_fs_read_all()` reads to EOF; it does not read to a pre-measured size. The file's reported size only seeds the initial capacity, so a regular file is still read in one allocation and one pass. This matters because `proven_fs_size()` reports 0 for anything that is not a regular file: a FIFO, a character device, or a `/proc` entry has no size that can be known up front, and reading to EOF is the only way to get their contents. It also means a file that grows while it is being read is not silently truncated. `value.size` is always the actual byte count, and an empty source yields `{ .ptr = NULL, .size = 0 }` with `PROVEN_OK`.

- `proven_fs_read_all()` and `proven_fs_read_all_u8str()` need an allocator with a `realloc_fn` if the source outgrows its reported size; a non-growing allocator returns `PROVEN_ERR_UNSUPPORTED` in that case.
- `proven_fs_read_all_u8str()` is the whole-file read most callers want: the result is NUL-terminated, so `proven_u8str_as_view()` and `proven_u8str_as_cstr()` work on it with no second copy. The terminator slot is reserved up front, so it costs no extra allocation. Contents are not validated as UTF-8. Release it with `proven_u8str_destroy()`.
- `proven_fs_write_file()` is not atomic: a reader can observe a partially written file, and a failure mid-write leaves the file truncated. `proven_fs_write_file_atomic()` writes a sibling temp file and renames it over the target, so a concurrent reader sees either the entire old file or the entire new one. It is atomic with respect to readers, not durable across power loss: the rename may reach the disk before the data. When you need durability, ask for it explicitly with `proven_fs_write_file_durable` (or `proven_fs_sync`), described above.

Example:

```
proven_result_file_t of = proven_fs_open(
    alloc,
    PROVEN_LIT("out.txt"),
    PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC
);
if (proven_is_ok(of.err)) {
    proven_err_t e = proven_fs_write_all(
        of.value,
        proven_mem_view_from_u8(PROVEN_LIT("hello\n"))
    );
    /* The close is part of the write. Close on the failure path too - the handle is
     * ours either way - but do not throw the answer away: on a network filesystem or
     * over quota, close() is the only place the failure appears. */
    proven_err_t ce = proven_fs_close(of.value);
    if (proven_is_ok(e)) e = ce;
    if (!proven_is_ok(e)) {
        proven_eprintln("writing out.txt failed");
    }
}
```

2. System I/O and environment

Standard streams

```
proven_file_t proven_sysio_stdin(void);
proven_file_t proven_sysio_stdout(void);
proven_file_t proven_sysio_stderr(void);
```

Purpose: expose the standard streams as `proven_file_t` handles. They are also writers and readers — see The standard streams below, which is what lets you read stdin a line at a time, buffer stdout, and format straight into either.

proven_sysio_scanner_t

```
typedef struct {
    proven_file_t file;
    proven_allocator_t alloc;
    proven_u8 *buffer;
    proven_size_t capacity;
    proven_size_t cursor;
    proven_size_t length;
    bool eof;
} proven_sysio_scanner_t;
```

Purpose: buffered scanner for bounded stream input, safe for pipes and stdin as well as seekable files. When a token reaches the end of the currently loaded fragment before EOF, the scanner

refills the buffer and retries; only a token that cannot fit inside the whole buffer even after a refill is rejected, with `PROVEN_ERR_OUT_OF_BOUNDS`, instead of being accepted truncated.

Sysio functions and macros

API	Intent	Return
`proven_sysio_scanner_init(scanner, file, alloc, buffer_capacity)`	Allocate scanner buffer and bind stream.	`proven_err_t`.
`proven_sysio_scanner_deinit(scanner)`	Free scanner buffer.	void.
`proven_sysio_scanner_scan_impl(scanner, fmt, args, args_count)`	Internal scanner engine.	`proven_err_t`.
`proven_sysio_scanner_scan(scanner, fmt, ...)`	Type-safe buffered scan macro.	`proven_err_t`.
`proven_sysio_print_impl(handle, fmt, args, args_count)`	Internal printing engine.	`proven_err_t`.
`proven_sysio_scan_chunk_impl(handle, fmt, args, args_count)`	One-chunk scan engine.	`proven_err_t`.
`proven_print(fmt, ...)`	Print to stdout.	`proven_err_t`.
`proven_println(fmt, ...)`	Print to stdout with newline.	`proven_err_t`.
`proven_eprint(fmt, ...)`	Print to stderr.	`proven_err_t`.
`proven_eprintln(fmt, ...)`	Print to stderr with newline.	`proven_err_t`.
`proven_scan_fmt_from_file(file, fmt, ...)`	Scan from one fixed-size chunk of file.	`proven_err_t`.
`proven_scan_fmt_from_stdin(fmt, ...)`	Scan one fixed-size chunk from stdin.	`proven_err_t`.
`proven_env_get(alloc, key)`	Read environment variable into owned U8 string.	`proven_result_u8str_t`.

`proven_sysio_scan_chunk_impl()` is intended for seekable file inputs. It reads at most one fixed-size chunk. If the handle cannot be rewound, it returns `PROVEN_ERR_UNSUPPORTED` before reading. If the chunk fills before a complete token is available, it returns `PROVEN_ERR_OUT_OF_BOUNDS` and rewinds the file cursor to the start of the chunk. It also refuses string-view destinations with `PROVEN_ERR_UNSUPPORTED` before reading: such a view would borrow the helper's local chunk buffer and dangle on return. Use the caller-owned `proven_sysio_scanner_t` for repeated buffered scanning or borrowed string results. A string view returned by that scanner remains valid only until the next scan call or scanner deinitialization, because either operation may refill, compact, or free the shared buffer.

Example:

```
proven_println("answer={}", PROVEN_ARG(42));
proven_eprintln("warning: {}", PROVEN_ARG(PROVEN_LIT("low memory")));
```

Environment example. `proven_env_get` hands back an owned string, so it has to be destroyed:

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("PATH"));
if (proven_is_ok(env.err)) {
    proven_u8str_view_t path = proven_u8str_as_view(&env.value);
    proven_println("PATH is {} bytes", PROVEN_ARG(path.size));
    proven_u8str_destroy(alloc, &env.value);
}
```

3. Memory mapping

The problem: reading a file you do not want to copy

`proven_fs_read_all_u8str` reads a file into memory you own. For a configuration file that is exactly right. For a 4 GB database, a memory-mapped index, or a file two processes need to see at once, it is wrong in three ways: it needs 4 GB of RAM, it copies every byte whether or not you read them, and the copy is yours alone.

A memory mapping asks the operating system to make the file's contents *appear* at an address instead. Nothing is copied up front. The pages you touch are read from disk on demand; the ones you never touch are never read. Two processes mapping the same file with `PROVEN_MMAP_SHARED` see one set of pages, so a write in one is visible in the other.

What this costs, and when not to use it

This is the section of the manual where the trade is sharpest, because the failure modes do not look like errors:

- **A read can fault.** With a file in memory, an I/O error is a return value. With a mapping, touching a page whose disk read fails delivers a **signal** (`SIGBUS`), not an error code. There is no `if` you can write around it.
- **Truncation is a landmine.** If another process shortens the file after you map it, touching a page past the new end is also `SIGBUS`. Mapping a file that anything else may truncate is unsafe in a way no API can fix for you.
- **It is not free.** Setting up a mapping is a syscall and page-table work; each first touch is a page fault. For a small file, `proven_fs_read_all_u8str` is simply faster.
- `proven_mmap_sync` **is the durability point.** Writes to a shared mapping reach the file eventually; if you need them on disk *now*, ask.

Use a mapping for large files you read sparsely, for read-only data shared between processes, and for random access into a big file. Use an ordinary read for everything else.

The mapping is caller-owned state: `proven_mmap_as_view` hands you a view **into the mapping**, so that view is dead the moment `proven_mmap_destroy` runs.

Wrong — using the view after destroying the mapping:

```
proven_u8str_view_t data = proven_mmap_as_view(m);
proven_err_t e = proven_mmap_destroy(&m);
(void)e;
parse(data);                /* wrong: those addresses are no longer mapped - SIGSEGV */
```

Wrong — treating a mapping like a buffer you can grow:

```
/* wrong: a mapping is a window onto a file of a fixed size at map time.
   Appending means changing the file and mapping it again. */
```

Structures and enums

```
typedef enum {
    PROVEN_MMAP_READ  = 0x01,
    PROVEN_MMAP_WRITE = 0x02,
    PROVEN_MMAP_EXEC  = 0x04
} proven_mmap_prot_t;

typedef enum {
    PROVEN_MMAP_PRIVATE = 0x01,
    PROVEN_MMAP_SHARED  = 0x02
} proven_mmap_flags_t;

typedef struct {
    void *ptr;
```

```
    proven_size_t size;
    proven_fs_handle_t file;
    void *internal_handle;
} proven_mmap_t;

typedef struct {
    proven_err_t err;
    proven_mmap_t value;
} proven_result_mmap_t;
```

Functions

API	Intent	Return
`proven_mmap_create(file, offset, size, prot, flags)`	Map a file region. `size == 0` maps until EOF. Offset must match the platform mapping granularity.	`proven_result_mmap_t`.
`proven_mmap_destroy(mmap)`	Unmap region and clear state.	`proven_err_t`.
`proven_mmap_sync(mmap)`	Flush shared writable changes to storage.	`proven_err_t`.
`proven_mmap_as_view(mmap)`	Borrow mapped memory as U8 view.	`proven_u8str_view_t`.

Example:

```
proven_result_file_t f = proven_fs_open(alloc, PROVEN_LIT("data.bin"), PROVEN_FS_READ);
if (proven_is_ok(f.err)) {
    proven_result_mmap_t mr = proven_mmap_create(
        f.value,
        0,
        0,
        PROVEN_MMAP_READ,
        PROVEN_MMAP_PRIVATE
    );
    if (proven_is_ok(mr.err)) {
        /* The view borrows the mapping: it dies when the mapping is destroyed. */
        proven_u8str_view_t bytes = proven_mmap_as_view(mr.value);
        proven_println("mapped {} bytes", PROVEN_ARG(bytes.size));
        (void)proven_mmap_destroy(&mr.value);
    }
    (void)proven_fs_close(f.value);
}
```

4. Time API

The problem: two different questions, one word
"Time" means two things that look alike and behave nothing alike.

A **wall clock** answers *what time is it?* — the thing a user reads. It is allowed to jump: NTP corrects it, daylight saving shifts it, an administrator sets it. Measure a duration with it and you can get a negative answer, which is a bug that appears on leap-second days and on nobody's laptop during testing.

A **monotonic** clock answers *how long since?* It only moves forward and has no relationship to any calendar. It is what you time an operation with.

libc blurs the distinction. `time()` gives whole seconds of wall clock — too coarse to measure anything. `clock()` measures **CPU** time, so a program that waits on a socket appears to take no time at all. Neither name says which question it answers, and both are routinely used for the other one.

What this library does instead

`proven_time_now()` returns **nanoseconds since the Unix epoch** as a signed 64-bit value. One number that you can subtract to get a duration, or break down to get a calendar date — at a resolution fine enough to time real work.

`proven_time_breakdown()` turns that number into `proven_datetime_t`, whose fields are the ones a human uses: `month` is **1-12**, not libc's 0-11, and `year` is the actual year, not years since

1. Those two off-by-N conventions in `struct tm` have caused enough bugs to be worth diverging

from deliberately.

Formatting uses the same `{}` placeholders as Chapter 3: the name picks the field and the spec pads it, so `{month:0>2}` is zero-filled to width two. Month and weekday names come from a `proven_time_locale_t`, so rendering another language is passing a different locale rather than setting a global.

Wrong — timing with a wall clock and trusting the sign:

```
proven_time_t t0 = proven_time_now();
do_work();
proven_time_t t1 = proven_time_now();
proven_u64 ns = (proven_u64)(t1 - t0);   /* wrong: an NTP step back makes this enormous */
```

The subtraction is fine; the cast is not. Keep the difference signed, and treat a negative elapsed time as "the clock moved", not as a duration.

Wrong — assuming sleep is precise:

```
proven_time_sleep(15);
/* wrong to assume exactly 15ms have passed: sleep guarantees AT LEAST that long,
   and the scheduler decides when you actually run again. */
```

Worked example: a duration, a date, and a formatted timestamp

Compiled and run by the test suite. Note what it does *not* assert — an upper bound on the sleep — because that would be a test that fails on a busy machine.

```
/*
 * Time comes in two flavours that look identical and are not, and picking the
 * wrong one is the classic timing bug.
 *
 *   - A WALL CLOCK answers "what time is it?". It is what a user wants to see,
 *     and it is allowed to jump: NTP corrects it, daylight saving shifts it, an
 *     administrator sets it. Measuring a duration with it can produce a negative
 *     elapsed time, and did, famously, on leap-second days.
 *
 *   - A MONOTONIC clock answers "how long since?". It only moves forward, at a
 *     steady rate, and has no relationship to any calendar. It is what you time
 *     an operation with.
 *
 * libc blurs this. time() is wall clock in whole seconds - useless for
 * measurement. clock() measures CPU time, not elapsed time, so a program that
 * sleeps looks instantaneous. Neither name tells you which of the two questions
 * it is answering.
 *
 * proven_time_now() is nanoseconds since the Unix epoch: one number that both
 * formats as a date and subtracts as a duration, at a resolution fine enough to
 * time real work.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();
```

```
    /* --- as a duration ------------------------------------------------- */
    proven_time_t start = proven_time_now();
    proven_time_sleep(15);                  /* milliseconds */
    proven_time_t end = proven_time_now();

    proven_i64 elapsed_ns = end - start;
    EXAMPLE_REQUIRE(elapsed_ns > 0, "time must move forward across a sleep");
    /* Sleep guarantees AT LEAST the requested time, never at most: the scheduler
     * decides when you actually run again. Asserting an upper bound here would
     * be a test that fails on a busy machine, which is why this one does not. */
    EXAMPLE_REQUIRE(elapsed_ns >= 10 * 1000 * 1000,
                    "sleeping 15ms must take at least ~10ms of wall time");

    /* --- as a date ----------------------------------------------------- */
    proven_datetime_t dt = proven_time_breakdown(start);
    EXAMPLE_REQUIRE(dt.year >= 2020 && dt.year < 3000, "the epoch breakdown gives a plausible year");
    EXAMPLE_REQUIRE(dt.month >= 1 && dt.month <= 12, "month is 1-12, not 0-11 as in libc's tm");
    EXAMPLE_REQUIRE(dt.day >= 1 && dt.day <= 31, "day is 1-31");
    EXAMPLE_REQUIRE(dt.hour <= 23 && dt.min <= 59 && dt.sec <= 60, "sec allows 60 for leap seconds");
    EXAMPLE_REQUIRE(dt.weekday <= 6, "weekday is 0-6 with 0 = Sunday");

    /* proven_time_now_datetime() is the two calls above in one, for when you
     * only want the calendar form. */
    proven_datetime_t now = proven_time_now_datetime();
    EXAMPLE_REQUIRE(now.year == dt.year, "both routes read the same clock");

    /* --- formatting a timestamp ---------------------------------------- */
    proven_result_u8str_t s = proven_u8str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "a 64-byte string is enough for a timestamp");

    /* The locale supplies month and weekday names; proven_time_locale_en is the
     * built-in English one. Pass your own to render other languages. */
    proven_err_t err = proven_time_u8_fmt(alloc, &s.value, dt, &proven_time_locale_en,
                                          "{year}-{month:0>2}-{day:0>2} {hour:0>2}:{min:0>2}:{sec:0>2}");
    EXAMPLE_REQUIRE(proven_is_ok(err), "formatting a datetime should succeed");

    proven_u8str_view_t out = proven_u8str_as_view(&s.value);
    EXAMPLE_REQUIRE(out.size == 19, "year-month-day hour:min:sec is exactly 19 characters");
    EXAMPLE_REQUIRE(out.ptr[4] == '-' && out.ptr[7] == '-' && out.ptr[13] == ':',
                    "the separators land where the pattern put them");

    proven_println("formatted: {}", PROVEN_ARG(out));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

Types

```
typedef proven_i64 proven_time_t;
```

Nanoseconds since UNIX epoch.

```
typedef struct {
    proven_i32 year;
    proven_u8 month;
    proven_u8 day;
    proven_u8 hour;
    proven_u8 min;
    proven_u8 sec;
    proven_u32 ms;
```

```
    proven_u8 weekday;
} proven_datetime_t;
```

Field ranges:

- `month`: 1 to 12.
- `day`: 1 to 31.
- `hour`: 0 to 23.
- `min`: 0 to 59.
- `sec`: 0 to 60.
- `ms`: 0 to 999.
- `weekday`: 0 to 6, Sunday is 0.

```
typedef struct {
    const proven_u8str_view_t *month_names;
    const proven_u8str_view_t *month_short_names;
    const proven_u8str_view_t *weekday_names;
    const proven_u8str_view_t *weekday_short_names;
} proven_time_locale_t;
```

`proven_time_locale_en` is the default English locale.

Functions

API	Intent	Return
`proven_time_u8_fmt(alloc, str, dt, locale, fmt)`	Append formatted datetime to U8 string.	`proven_err_t`.
`proven_time_u16_fmt(alloc, str, dt, locale, fmt)`	Append formatted datetime to U16 string unless U16 is disabled.	`proven_err_t`.
`proven_time_now()`	Current timestamp in nanoseconds.	`proven_time_t`.
`proven_time_breakdown(time_ns)`	Convert epoch nanoseconds to broken-down UTC time.	`proven_datetime_t`.
`proven_time_now_datetime()`	Current local broken-down time.	`proven_datetime_t`.
`proven_time_sleep(ms)`	Sleep for milliseconds.	void.

Datetime format keys:

- {year}, {month}, {day}, {hour}, {min}, {sec}, {ms}, {wday_num}
- {Month}, {mon} using locale
- {Weekday}, {wday} using locale

Example:

```
proven_result_u8str_t r = proven_u8str_create(alloc, 32);
if (proven_is_ok(r.err)) {
    proven_u8str_t s = r.value;

    proven_datetime_t now = proven_time_now_datetime();
    proven_err_t e = proven_time_u8_fmt(
        alloc,
        &s,
        now,
        &proven_time_locale_en,
        "{year}-{month:0>2}-{day:0>2} {hour:0>2}:{min:0>2}:{sec:0>2}"
    );
    if (proven_is_ok(e)) {
        proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
    }
```

```
    proven_u8str_destroy(alloc, &s);
}
```

5. Examples and misuse cases

Single writes can be partial

Wrong:

```
proven_result_size_t w = proven_fs_write(file, data);
/* wrong: one write may be partial, and w.value is never looked at */
```

Correct:

```
proven_result_file_t f = proven_fs_open(alloc, PROVEN_LIT("out.txt"),
                                        PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
if (proven_is_ok(f.err)) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("payload"));
    proven_err_t e = proven_fs_write_all(f.value, data);   /* loops until done */
    proven_err_t ce = proven_fs_close(f.value);            /* and the close can fail too */
    if (proven_is_ok(e)) e = ce;
    (void)e;
}
```

Directory listings need special destruction

Wrong:

```
proven_result_array_t r = proven_fs_list(alloc, PROVEN_LIT("."));
PROVEN_ARRAY_DESTROY(&r.value); /* wrong: entry names leak */
```

Correct:

```
proven_result_array_t r = proven_fs_list(alloc, PROVEN_LIT("."));
if (proven_is_ok(r.err)) {
    proven_fs_list_destroy(alloc, &r.value);
}
```

Do not use a mapping after destroy

Wrong:

```
proven_mmap_destroy(&map);
use_bytes(map.ptr, map.size); /* wrong: mapping has been released */
```

Use buffered sysio scanning for streams

For repeated reading from stdin or pipes, prefer:

```
proven_sysio_scanner_t scanner = {0};
proven_err_t e = proven_sysio_scanner_init(&scanner, proven_sysio_stdin(), alloc, 4096);
if (proven_is_ok(e)) {
    int value = 0;
    e = proven_sysio_scanner_scan(&scanner, "{}", PROVEN_SCAN_ARG(&value));
    proven_sysio_scanner_deinit(&scanner);
}
```

Environment values are owned strings

Wrong:

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("PATH"));
use_path(proven_u8str_as_view(&env.value)); /* wrong if env.err was not checked, and it leaks */
```

Correct:

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("HOME"));
if (proven_is_ok(env.err)) {
    proven_u8str_view_t home = proven_u8str_as_view(&env.value);
    proven_println("HOME={}", PROVEN_ARG(home));
```

```
    proven_u8str_destroy(alloc, &env.value);
}
```

Worked example: reading and writing whole files

Compiled and run by the test suite. This is the whole-file API most callers actually want, including the atomic rewrite that preserves the target's permissions.

```
/*
 * The whole-file API: one call in, one call out. It exists because the
 * open/read-loop/close dance is where most file-handling bugs live - a forgotten
 * close, a partial read treated as EOF, a truncated file left behind by a failed
 * write. If you are reading or writing a file in its entirety, this is the API.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* A relative path in the current directory: the example must not depend on a
     * writable /tmp, and it removes what it creates before returning. */
    proven_u8str_view_t path = PROVEN_LIT("proven_example_wholefile.tmp");
    proven_u8str_view_t text = PROVEN_LIT("first line\nsecond line\n");

    /* --- write it in one call ------------------------------------------- */
    /* Not atomic: a concurrent reader can see this file half-written. Fine here,
     * because nobody else is looking at it yet. */
    proven_err_t err = proven_fs_write_file(alloc, path, proven_mem_view_from_u8(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the whole file should succeed");
    if (!proven_is_ok(err)) return 1;

    /* --- read it back as raw bytes -------------------------------------- */
    /* proven_fs_read_all reads to EOF rather than to a pre-measured size, so it
     * also works on a pipe or a /proc entry, whose size cannot be known up front. */
    proven_result_mem_mut_t raw = proven_fs_read_all(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(raw.err), "reading the whole file should succeed");
    if (proven_is_ok(raw.err)) {
        EXAMPLE_REQUIRE(raw.value.size == text.size, "read_all should return every byte written");
        /* The block is plain allocator memory - hand it back to the allocator that
         * produced it. There is no proven_fs_read_all_destroy. */
        alloc.free_fn(alloc.ctx, raw.value.ptr);
    }

    /* --- read it back as a string --------------------------------------- */
    /* This is the one most callers want: the result is NUL-terminated, so it can
     * be handed to a view, to as_cstr, or to the scanner with no second copy. The
     * terminator slot is reserved up front, so it costs no extra allocation. */
    proven_result_u8str_t s = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "reading the whole file as a string should succeed");
    if (!proven_is_ok(s.err)) {
        (void)proven_fs_remove(alloc, path);
        return 1;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), text),
                    "the file's contents should come back unchanged");
    printf("read back %zu bytes: %s", (size_t)proven_u8str_as_view(&s.value).size,
           proven_u8str_as_cstr(&s.value));
    proven_u8str_destroy(alloc, &s.value);

    /* --- stat, and the perms round-trip --------------------------------- */
    proven_fs_stat_t st = {0};
    err = proven_fs_stat(alloc, path, &st);
```

```
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat on a file we just wrote should succeed");
    EXAMPLE_REQUIRE(st.type == PROVEN_FS_TYPE_FILE, "a regular file should stat as a FILE");
    EXAMPLE_REQUIRE(st.size == text.size, "stat should report the size we wrote");

    /* `perms` carries the nine permission bits and nothing else, so it can be fed
     * straight back to chmod. That is the whole point of the field: read a file's
     * mode, and later restore it. (It used to carry the raw POSIX st_mode, whose
     * file-type bits chmod rejects - so this obvious round-trip failed.) */
    err = proven_fs_chmod(alloc, path, st.perms);
    EXAMPLE_REQUIRE(proven_is_ok(err), "a stat's perms must be accepted back by chmod");

    /* Now make the file owner-only, so the next check has something to prove. */
    proven_fs_perms_t private_perms = PROVEN_FS_PERM_OWNER_R | PROVEN_FS_PERM_OWNER_W;
    err = proven_fs_chmod(alloc, path, private_perms);
    EXAMPLE_REQUIRE(proven_is_ok(err), "restricting the file to its owner should succeed");

    /* --- rewrite it atomically ------------------------------------------- */
    /* A sibling temp file plus a rename: a concurrent reader sees either the whole
     * old file or the whole new one, never a half-written mix. Atomic for readers,
     * not durable across power loss. When it must be, proven_fs_write_file_durable asks. */
    proven_u8str_view_t text2 = PROVEN_LIT("replacement\n");
    err = proven_fs_write_file_atomic(alloc, path, proven_mem_view_from_u8(text2));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the atomic rewrite should succeed");

    proven_fs_stat_t st2 = {0};
    err = proven_fs_stat(alloc, path, &st2);
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat after the atomic rewrite should succeed");
    EXAMPLE_REQUIRE(st2.size == text2.size, "the file should now hold the replacement text");
    /* The rename writes a *new* inode over the old name, so the permissions would
     * be lost unless they were copied across. They are: rewriting a 0600 file does
     * not republish it as 0644. */
    EXAMPLE_REQUIRE(st2.perms == private_perms,
                    "the atomic rewrite must preserve the target's permissions");

    /* --- clean up -------------------------------------------------------- */
    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the temp file should succeed");

    return EXAMPLE_OK();
}
```

Worked example: open, read, write, close

Compiled and run by the test suite. Note the read loop: a read at end-of-file returns `PROVEN_ERR_EOF`, not a zero-byte success, so a loop that only checks for zero bytes never terminates the way its author expected.

```
/*
 * The open/read/write/close path, for when the whole-file API (ex_05_fs_wholefile)
 * is not enough: you are streaming, or you want to own the buffer.
 *
 * The one thing to get right here: a single read or write moves *up to* the
 * requested number of bytes, not exactly that many. Treating one short read as
 * end-of-file is the classic way to silently lose the tail of a file.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_u8str_view_t path = PROVEN_LIT("proven_example_stream.tmp");
    proven_u8str_view_t text = PROVEN_LIT("streamed bytes, read back in chunks\n");
```

```
/* --- write ------------------------------------------------------------- */
/* CREATE makes the file if it is absent; TRUNC empties it if it is not. The
 * allocator is only used to convert the path for the platform call. */
proven_result_file_t out = proven_fs_open(alloc, path,
                                          PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
EXAMPLE_REQUIRE(proven_is_ok(out.err), "opening the file for writing should succeed");
if (!proven_is_ok(out.err)) return 1;

/* write_all loops for us. proven_fs_write does one attempt and may write less,
 * which is almost never what a caller means. */
proven_err_t err = proven_fs_write_all(out.value, proven_mem_view_from_u8(text));

/* The close is part of the write, and on a network or quota-enforced filesystem it is
 * the ONLY place the failure appears: the bytes were buffered, write() said yes, and
 * close() is where the disk finally says no. Close on the failure path too - the
 * handle is ours either way - but do not throw the answer away. */
proven_err_t cerr = proven_fs_close(out.value);
if (proven_is_ok(err)) err = cerr;

EXAMPLE_REQUIRE(proven_is_ok(err), "writing the whole buffer should succeed");
if (!proven_is_ok(err)) {
    (void)proven_fs_remove(alloc, path);
    return 1;
}

/* --- read -------------------------------------------------------------- */
proven_result_file_t in = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(in.err), "opening the file for reading should succeed");
if (!proven_is_ok(in.err)) {
    (void)proven_fs_remove(alloc, path);
    return 1;
}

/* size is a hint for sizing the buffer, not a promise about how many bytes any
 * one read will hand over - and it is 0 for anything that is not a regular
 * file (a pipe, a device, a /proc entry). The loop below does not rely on it. */
proven_result_size_t sz = proven_fs_size(in.value);
EXAMPLE_REQUIRE(proven_is_ok(sz.err), "querying the size of an open file should succeed");
EXAMPLE_REQUIRE(sz.value == text.size, "the file should be as long as what we wrote");

proven_byte_t buf[128];
proven_size_t total = 0;

/* The partial-read loop. Each pass asks for whatever is left of the buffer and
 * advances by however much actually arrived: a short read is normal, not the
 * end of the file. The end of the file is a distinct status - PROVEN_ERR_EOF
 * with zero bytes - so the loop terminates on that, and on nothing else. The
 * loop also stops if the source outgrows the buffer; noticing that is the
 * caller's business (here it cannot happen, but a growing file could). */
for (;;) {
    if (total == sizeof buf) break;   /* buffer full: caller decides what to do */

    proven_mem_mut_t dest = { .ptr = buf + total, .size = sizeof buf - total };
    proven_result_size_t r = proven_fs_read(in.value, dest);
    if (r.err == PROVEN_ERR_EOF) break;
    if (!proven_is_ok(r.err)) {
        (void)proven_fs_close(in.value);
        (void)proven_fs_remove(alloc, path);
        EXAMPLE_REQUIRE(false, "reading from the open file should not fail");
```

```
            return 1;
        }
        total += r.value;
    }

    (void)proven_fs_close(in.value);

    EXAMPLE_REQUIRE(total == text.size, "the loop should have read every byte in the file");
    proven_u8str_view_t got = { .ptr = buf, .size = total };
    EXAMPLE_REQUIRE(proven_u8str_view_eq(got, text), "the bytes should come back unchanged");

    printf("read %zu bytes in chunks: %.*s", (size_t)total, (int)total, (const char *)buf);

    /* --- clean up ------------------------------------------------------- */
    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the temp file should succeed");

    return EXAMPLE_OK();
}
```

Reading a directory one entry at a time

`proven_fs_list` reads the **whole** directory before you see any of it, and it allocates a string for every name. Measured on 50,000 entries: **189 ms, +4.2 MB resident, 50,008 allocations**, with nothing visible until the last entry was read. That is fine for a config directory and useless for a mail spool.

`proven_fs_dir_open` / `_next` / `_close` walks the same directory one entry at a time and **allocates nothing per entry**.

API	Intent	Return
`proven_fs_dir_open(scratch, path)`	Open a streaming iterator. `scratch` is for the path conversion only.	`proven_result_dir_t`.
`proven_fs_dir_next(&dir, &entry)`	The next entry. `PROVEN_ERR_EOF` when there are no more.	`proven_err_t`.
`proven_fs_dir_close(&dir)`	Release the iterator.	void.

```
typedef struct {
    proven_u8str_view_t name;   /* BORROWED: points into the iterator's own storage,
                                   valid only until the next proven_fs_dir_next */
    proven_fs_type_t    type;   /* FILE / DIR / OTHER - and it follows symlinks */
} proven_fs_dir_entry_t;
```

Use it like this:

```
proven_result_dir_t d = proven_fs_dir_open(alloc, PROVEN_LIT("."));
if (proven_is_ok(d.err)) {
    proven_fs_dir_t dir = d.value;
    proven_fs_dir_entry_t entry;
    for (;;) {
        proven_err_t e = proven_fs_dir_next(&dir, &entry);
        if (e == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(e)) break;              /* a real error: report it */
        proven_println("{}", PROVEN_ARG(entry.name));
    }
    proven_fs_dir_close(&dir);
}
```

The name is borrowed, and it dies at the next call. This is what makes the whole thing cost no allocations — and a dangling pointer the moment you keep it.

Wrong:

```
proven_u8str_view_t names[100];
int n = 0;
while (proven_is_ok(proven_fs_dir_next(&dir, &entry)))
    names[n++] = entry.name;    /* wrong: every entry aliases the same storage, and the
                                   next _next overwrites it. All 100 end up equal - to
                                   whatever the last entry happened to be. */
```

Correct: copy the bytes (`proven_u8str_create_from_view`) for the ones you need to keep — or use `proven_fs_list`, which does exactly that for every entry and charges you for it.

`PROVEN_ERR_EOF` **is the end; anything else is a failure.** A loop that stops on "not OK" treats a permission error as a complete listing.

```
while (proven_is_ok(proven_fs_dir_next(&dir, &entry))) { ... }
/* wrong: an I/O error ends the loop exactly like the end of the directory does,
   and you cannot tell whether you saw everything. */
```

Walking a tree

`proven_fs_dir_*` walks ONE directory. `proven_fs_walk` walks a tree — and it is worth saying exactly what it refuses to do, because those refusals are the feature:

It cannot **loop**	It never descends *through* a symlink.
It cannot **escape**	Same rule: a link out of the tree is reported, not followed.
It cannot **lie**	A directory it cannot read comes back as an *error* naming that directory, and the walk goes on. A tree walker that silently skips an unreadable subtree is how a backup misses files and reports success.
It cannot **bloat**	One open handle per *level* of the current path, plus one reused path buffer. Memory is a function of depth, not of how many files there are.

A symlinked directory is still **reported** — it exists, `type` is `DIR`, `is_symlink` is true — it is simply not entered. Hiding it would be its own kind of lie. If you *want* to follow it, you have the path: open a second walk on it, and you own the cycle question.

The structure you receive

```
typedef struct {
    proven_u8str_view_t path;   /* the whole path, from the root you passed in.
                                   BORROWED: it points into the walk's one reused
                                   buffer and is valid only until the next call. */
    proven_u8str_view_t name;   /* the last component of `path`. Same lifetime. */
    proven_fs_type_t    type;   /* FILE / DIR / OTHER - and it FOLLOWS symlinks,
                                   exactly as proven_fs_stat does. */
    proven_size_t       size;   /* bytes, for a regular file; 0 otherwise. */
    proven_size_t       depth;  /* 0 for an entry directly inside the root. */
    bool                is_symlink;  /* reached through a symlink. `type` describes
                                        the TARGET; the walk does not enter it. */
} proven_fs_walk_entry_t;
```

`entry.path` and `entry.name` are borrowed from the walk's one reused buffer and are valid **until the next call**. Copy them if you need them to outlive the step; that is the price of a walk of a million entries costing one allocation instead of a million.

Wrong — the same trap the directory iterator has, for the same reason:

```
proven_u8str_view_t found[100];
int n = 0;
while (proven_is_ok(proven_fs_walk_next(&walk, &entry)))
```

```
    found[n++] = entry.path;   /* wrong: every entry aliases ONE buffer. When the loop
                                  ends, all 100 point at whatever the last path was. */
```

Two limits, both of which say so rather than going quiet:

- `max_depth` — how far to descend. A directory *at* the limit is still reported (it is an entry); it is not entered.
- `PROVEN_FS_WALK_DEPTH_LIMIT` (256) — how deep the walk's own stack goes, ever. A directory past it comes back as `PROVEN_ERR_OUT_OF_BOUNDS`, naming the directory.

Compiled and run by the test suite:

```
/*
 * Walking a tree.
 *
 * The three things a recursive walker gets wrong, and what this one does instead:
 *
 *   It loops.       A symlink pointing at an ancestor is a cycle. This walk never descends
 *                   THROUGH a symlink - the symlinked directory is still reported, it is
 *                   simply not entered - so a cycle is impossible, and so is walking out of
 *                   the tree you asked about and into the rest of the filesystem.
 *
 *   It lies.        A directory it cannot read gets skipped, and the walk reports success.
 *                   That is how a backup misses a subtree. Here the error comes back from
 *                   proven_fs_walk_next, with the entry naming the directory, and the walk
 *                   carries on from the next sibling. You decide what to do about it.
 *
 *   It bloats.      Reading a whole directory into memory before yielding anything makes a
 *                   walk of a big tree cost a big allocation. This one holds one open handle
 *                   per LEVEL of the current path and one reused path buffer - so its memory
 *                   is a function of depth, not of how many files there are.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* A small tree to walk: a file, a directory, a file inside it. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_mkdir(alloc, PROVEN_LIT("ex_walk"))), "mkdir");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_mkdir(alloc, PROVEN_LIT("ex_walk/inner"))), "mkdir inner");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_file(alloc, PROVEN_LIT("ex_walk/top.txt"),
        proven_mem_view_from_u8(PROVEN_LIT("top")))), "write top.txt");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_file(alloc, PROVEN_LIT("ex_walk/inner/deep.txt"),
        proven_mem_view_from_u8(PROVEN_LIT("deep")))), "write deep.txt");

    proven_result_walk_t walk = proven_fs_walk_open(alloc, PROVEN_LIT("ex_walk"),
                                                    PROVEN_FS_WALK_UNLIMITED);
    EXAMPLE_REQUIRE(proven_is_ok(walk.err), "the walk should open");

    proven_size_t files = 0;
    proven_size_t dirs = 0;
    proven_size_t unreadable = 0;
    proven_size_t total_bytes = 0;

    for (;;) {
        proven_fs_walk_entry_t entry = {0};
        proven_err_t err = proven_fs_walk_next(&walk.value, &entry);

        if (err == PROVEN_ERR_EOF) break;

        if (!proven_is_ok(err)) {
            /* A directory that could not be read, or one deeper than the walk's stack. It is
```

```
             * REPORTED, not skipped - `entry.path` says which one - and the walk goes on. A
             * tool that copies a tree should fail here; one that reports on a tree should
             * count it and say so. What it must not do is pretend it did not happen. */
            unreadable++;
            continue;
        }

        /* `entry.path` and `entry.name` are borrowed: they point into the walk's one reused
         * buffer and are valid until the next call. Copy them if you need them longer. */
        if (entry.type == PROVEN_FS_TYPE_DIR) {
            dirs++;
        } else if (entry.type == PROVEN_FS_TYPE_FILE) {
            files++;
            total_bytes += entry.size;
        }
    }

    proven_fs_walk_close(&walk.value);

    EXAMPLE_REQUIRE(files == 2, "two files: top.txt and inner/deep.txt");
    EXAMPLE_REQUIRE(dirs == 1, "one directory: inner");
    EXAMPLE_REQUIRE(unreadable == 0, "and nothing in this tree is unreadable");
    EXAMPLE_REQUIRE(total_bytes == 7, "three bytes plus four");

    /* Depth-limited: max_depth 0 reports what is directly inside the root and descends
     * nowhere. The directory at the limit is still an entry, so it is still reported. */
    walk = proven_fs_walk_open(alloc, PROVEN_LIT("ex_walk"), 0);
    EXAMPLE_REQUIRE(proven_is_ok(walk.err), "the shallow walk should open");

    proven_size_t shallow = 0;
    for (;;) {
        proven_fs_walk_entry_t entry = {0};
        proven_err_t err = proven_fs_walk_next(&walk.value, &entry);
        if (err == PROVEN_ERR_EOF) break;
        if (proven_is_ok(err)) shallow++;
    }
    proven_fs_walk_close(&walk.value);

    EXAMPLE_REQUIRE(shallow == 2, "top.txt and inner - but nothing inside inner");

    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/inner/deep.txt"));
    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/top.txt"));
    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/inner"));
    (void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk"));

    return EXAMPLE_OK();
}
```

Streams: writers and readers

The formatter's only sink used to be a `proven_u8str_t`. A file was a `proven_file_t`. The in-memory scanner read a view; the file scanner read something else again. **Four types, four function families, no common interface** — so you could not write one `serialize(sink, value)` that worked over both memory and a file, you could not format into a file at all, and there was no way to read a file line by line.

A **writer** is a byte sink. A **reader** is a byte source. Both are small vtables passed by value, exactly like `proven_allocator_t`, and for the same reason: the caller decides where the bytes go, and nothing is hidden.

API	Intent
`proven_writer_from_file(&file)`	Unbuffered sink over an open file.
`proven_writer_from_u8str(&state, &str, alloc)`	Appends to an owned string.
`proven_writer_from_buffer(&state)`	Fixed caller memory. Allocates nothing, ever.
`proven_writer_buffered(&state, inner, buf)`	Accumulates small writes; **you** supply the buffer.
`proven_fprint(w, fmt, ...)` / `proven_fprintln`	Format straight into a writer. No allocation.
`proven_reader_from_file(&file)` / `_from_view(&state, view)`	Byte sources.
`proven_reader_buffered(&state, inner, buf)`	Buffered source; required for line reading.
`proven_reader_read_line(&state)`	One line, without the newline.
`proven_writer_is_valid(w)` / `proven_reader_is_valid(r)`	Did the constructor succeed? A zeroed handle is invalid, and every constructor returns one on bad arguments — so this is the check, not a NULL test.
`proven_fwrite_fmt(w, scratch, fmt, ...)`	`proven_fprint` with a scratch buffer **you** size. `proven_fprint` uses a 512-byte stack buffer and returns `OUT_OF_BOUNDS` for a longer line; this is how you format one.
`proven_fmt_to_writer_impl(w, scratch, fmt, args, n)`	The function the two macros above expand to. Call it directly only if you are building your own variadic wrapper; the macros exist so you do not have to count arguments.

Four rules worth stating plainly, because each of them is a way this could have been designed badly:

- **Buffering uses memory you supply.** `proven_writer_buffered` takes a `proven_mem_mut_t`, the way `proven_arena_create` does. There is no hidden global buffer, which means there is also no destructor to flush it for you — **you must flush before the buffer goes out of scope.** In exchange, your logging path never allocates, and a program logging its way out of an out-of-memory condition can still log.
- **A full sink refuses; it does not truncate.** A fixed buffer that fills up returns `PROVEN_ERR_OUT_OF_BOUNDS` and records `overflowed`. A sink that silently drops the end of your data is worse than one that says it cannot take it.
- **A line too long for the reader's buffer is an error**, not a truncated line. A truncated line handed back as if it were whole is a corruption the caller has no way to detect. The buffer is yours; size it for the input you expect.
- **A partial write is a fact, not a failure to be papered over.** A `write_fn` returns a `proven_result_size_t`: how many bytes the sink took, *and* what went wrong. A pipe, a socket, or a filling disk really does accept 4096 of your 6000 bytes and then fail, and a trait that says "consume it all or fail" simply makes such a sink impossible to write correctly. `proven_writer_write` still means all-or-nothing (it loops); `proven_writer_write_partial` is there when you need to see how far you got. The buffered writer keeps only the tail the sink did **not** take — the first version kept the whole buffer and re-sent it, so a failing sink received the accepted prefix twice.
- **A writer that has failed stays failed.** Once a writer has lost bytes — a buffered writer whose sink died, a fixed buffer that overflowed — the stream it was producing has a hole in it that the

receiver cannot see, so every later write and flush returns the error. A shorter chunk that *would* fit is refused too: writing it would put it after the hole, and the result would look like complete output. There is no `clear()`: if you have a recovery story it involves a new writer over a new sink, not pretending this one is fine. (Before this, `flush` answered `PROVEN_OK` after a failed write — the buffer was empty, so there was nothing left to fail on — and "write, write, write, check the flush", which is how almost everyone uses a buffered writer, reported success on a full disk.)

A reader's rule is the mirror image: **a read that fails is an error, never an end of file.** `proven_reader_read` returns `PROVEN_ERR_IO`, not a clean zero-byte EOF, when the source breaks — because a file cut short by a disk error and a file that simply ended are the same thing to a caller who cannot tell them apart, and only one of them is safe to act on.

What it costs, measured over 10,000 lines to stdout:

	`write()` syscalls	`malloc()`
`proven_println`	10,000	**0** (was 10,000)
buffered writer, 8 KiB of caller memory	**24**	**0**

The `malloc()` column is for lines that fit the 512-byte stack buffer, which is the ordinary case. A line longer than that falls back to the heap for that one call rather than being refused.

`proven_println` is deliberately still one syscall per line: buffering it would need hidden global state. A caller who wants the 24 builds a buffered writer and says so.

Worked example: one serializer, three destinations, and reading it back
Compiled and run by the test suite. Note that `render_row` does not know where its bytes are going — that is the entire point.

```
/*
 * Writers and readers: one interface for "where bytes go" and one for "where bytes
 * come from".
 *
 * The point is that the code below - render_row - does not know and does not care
 * whether it is writing to a string, to a fixed buffer, or to a file. That was
 * impossible before: the formatter's only sink was a proven_u8str_t.
 */

/* One serializer. It takes a sink, not a destination. */
static proven_err_t render_row(proven_writer_t w, int id, const char *name) {
    proven_fmt_result_t r = proven_fprintln(w, "{:>4} | {}", PROVEN_ARG(id), PROVEN_ARG(name));
    return r.err;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- the same code, into a growing string ----------------------------- */
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "string create");

    proven_writer_u8str_t s_state;
    proven_writer_t to_string = proven_writer_from_u8str(&s_state, &s.value, alloc);
    EXAMPLE_REQUIRE(proven_is_ok(render_row(to_string, 7, "ada")), "render into a string");

    printf("into a string:\n%s", proven_u8str_as_cstr(&s.value));
    proven_u8str_destroy(alloc, &s.value);

    /* --- the same code, into memory you own: zero allocations ------------- */
```

```c
proven_byte_t fixed[64];
proven_writer_buf_t b_state = { .buf = { .ptr = fixed, .size = sizeof fixed } };
proven_writer_t to_buffer = proven_writer_from_buffer(&b_state);
EXAMPLE_REQUIRE(proven_is_ok(render_row(to_buffer, 8, "grace")), "render into a buffer");
EXAMPLE_REQUIRE(b_state.len > 0, "the buffer received the row");

/* A full buffer REFUSES; it does not truncate. A sink that silently drops the
 * end of your data is worse than one that says it cannot take it. */
proven_byte_t tiny[4];
proven_writer_buf_t t_state = { .buf = { .ptr = tiny, .size = sizeof tiny } };
proven_writer_t to_tiny = proven_writer_from_buffer(&t_state);
EXAMPLE_REQUIRE(proven_writer_write_str(to_tiny, PROVEN_LIT("far too long")) == PROVEN_ERR_OUT_OF_BOUNDS,
                "a full buffer refuses rather than truncating");
EXAMPLE_REQUIRE(t_state.overflowed, "and it records that it did");

/* --- the same code, into a file, buffered ------------------------------ */
/*
 * The buffer is memory YOU supply, exactly like an arena's. This library has no
 * hidden global state, so it cannot flush for you at exit - which is why you must
 * flush before the buffer goes out of scope. In exchange, your logging path never
 * allocates: ten thousand lines here cost 0 mallocs and a couple of dozen write
 * syscalls, where ten thousand proven_println calls cost 10,000 syscalls.
 */
proven_u8str_view_t path = PROVEN_LIT("example_stream_rows.txt");
proven_result_file_t f = proven_fs_open(alloc, path,
    (proven_fs_mode_t)(PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC));
EXAMPLE_REQUIRE(proven_is_ok(f.err), "open the output file");
proven_file_t file = f.value;

proven_byte_t out_buf[4096];
proven_writer_buffered_t w_state;
proven_writer_t to_file = proven_writer_buffered(&w_state,
    proven_writer_from_file(&file),
    (proven_mem_mut_t){ .ptr = out_buf, .size = sizeof out_buf });

for (int i = 0; i < 3; ++i) {
    EXAMPLE_REQUIRE(proven_is_ok(render_row(to_file, i, "row")), "render into the file");
}
EXAMPLE_REQUIRE(proven_is_ok(proven_writer_flush(to_file)),
                "flush: nothing is written until you say so");
/* And the close, which is the last thing that can tell you the write did not land. */
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(file)), "closing the written file");

/* --- reading it back, a line at a time --------------------------------- */
/* Reading a file line by line was simply not possible before: the only route was
 * loading the entire file into memory and splitting it by hand. */
proven_result_file_t rf = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(rf.err), "reopen for reading");
proven_file_t rfile = rf.value;

proven_byte_t in_buf[128];
proven_reader_buffered_t r_state;
(void)proven_reader_buffered(&r_state, proven_reader_from_file(&rfile),
                             (proven_mem_mut_t){ .ptr = in_buf, .size = sizeof in_buf });

int lines = 0;
for (;;) {
    proven_result_u8str_view_t line = proven_reader_read_line(&r_state);
    if (line.err == PROVEN_ERR_EOF) break;
    EXAMPLE_REQUIRE(proven_is_ok(line.err), "read a line");
```

```
        /* The view points INTO the reader's buffer, and is valid only until the next
         * call. Copy it if it has to outlive that. */
        printf("line %d: %.*s\n", lines, (int)line.val.size, (const char *)line.val.ptr);
        ++lines;
    }
    EXAMPLE_REQUIRE(lines == 3, "three rows in, three lines out");

    (void)proven_fs_close(rfile);
    (void)proven_fs_remove(alloc, path);

    return EXAMPLE_OK();
}
```

The structures you hold

The two **handles** are passed by value and are cheap:

```
typedef struct {
    void *ctx;
    proven_result_size_t (*write_fn)(void *ctx, proven_mem_view_t chunk);
    proven_err_t (*flush_fn)(void *ctx);   /* may be NULL: this sink holds nothing back */
} proven_writer_t;

typedef struct {
    void *ctx;
    proven_result_size_t (*read_fn)(void *ctx, proven_mem_mut_t dest);
} proven_reader_t;
```

`write_fn` reports **how much went out even when it then fails**, and that is not a nicety: a write to a pipe or a full disk really does put some bytes out and then fail. A buffered writer built on the tidier "all or nothing" lie kept its whole buffer on failure and re-sent it on the next flush — a 6000-byte payload arrived as 10,096 bytes with the first 4096 **duplicated**. Losing data is bad; silently doubling it is worse, because the receiver cannot tell.

Everything else is caller-owned state — `proven_writer_buf_t`, `proven_writer_u8str_t`, `proven_writer_buffered_t`, `proven_reader_view_t`, `proven_reader_buffered_t`. They allocate nothing, they have no destroy, and **they must not be copied or moved** while a handle points into them.

Cautions, and what goes wrong

A buffered writer that is never flushed is output that never happened. There is no hidden state, so there is no destructor to flush it for you — and nothing here registers an `atexit` handler, because a library that owns your process is a library you cannot reason about.

Wrong:

```
proven_writer_buffered_t st;
proven_writer_t w = proven_writer_buffered(&st, inner, buf);
(void)proven_fprintln(w, "the important line");
return;                       /* wrong: the buffer dies with the frame, and so does the line */
```

Correct — flush before the buffer or the inner sink goes away:

```
proven_byte_t buf[256];
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out,
    (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });
(void)proven_fprintln(w, "the important line");
(void)proven_writer_flush(w);          /* now it has happened */
```

The line a reader hands you points into its buffer. It is valid only until the next call. That is what makes reading a million lines cost one buffer instead of a million allocations — and it is a dangling pointer the moment you keep it.

Wrong:

```
proven_u8str_view_t lines[100];
for (int i = 0; i < 100; ++i) {
    proven_result_u8str_view_t ln = proven_reader_read_line(&st);
    lines[i] = ln.val;        /* wrong: every entry aliases the SAME buffer, and the
                                 next read_line overwrites what the last one returned */
}
```

Correct: copy the bytes you need to keep (into a `proven_u8str_t`, an arena, wherever), before you call again.

A flush is not a durability barrier. `proven_writer_flush` pushes a buffered writer's bytes to the thing behind it; getting a *file's* bytes onto the disk is `proven_fs_sync`. They are different operations, and one word could not honestly mean both — which is why the old `proven_sysio_flush`, which claimed to be both and was neither, is gone.

A line longer than the buffer is refused, not truncated. `PROVEN_ERR_OUT_OF_BOUNDS` — and the reader then stays wedged on that line: there is no resync, because a line you cannot hold is not a line you can skip past without deciding what to do with the bytes. Size the buffer for the input you expect. (A line that *exactly fills* the buffer is fine: the newline does not have to fit too, and neither does a final line with no newline at all.)

The standard streams

`stream.h` has writers, readers, buffered writers and a line reader. `sysio.h` has stdin, stdout and stderr. Until they were introduced to each other, two things were simply not possible — and one call was a lie.

Reading stdin a line at a time had no route. The most common thing a program does with stdin, and the choices were the token scanner or reading the whole of a stream that may never end. The bridge fixes that: a standard handle is parked in caller-owned storage, so the line reader has something stable to point at.

```
proven_byte_t buf[4096];
proven_sysio_lines_t lines;
if (proven_is_ok(proven_sysio_stdin_lines(&lines, (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf }))) {
    for (;;) {
        proven_result_u8str_view_t line = proven_sysio_read_line(&lines);
        if (line.err == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(line.err)) break;   /* OUT_OF_BOUNDS: a line longer than `buf` */
        /* `line.val` points INTO `buf` and is valid only until the next call. */
        proven_println("{}", PROVEN_ARG(line.val));
    }
}
```

It inherits the line reader's properties, which is the point of not writing a second one: the view costs no allocation, `"\r\n"` is handled, a final line with no trailing newline is still returned, and a line longer than your buffer is `PROVEN_ERR_OUT_OF_BOUNDS` — never a silently truncated line.

The formatter could not be aimed at a standard stream. `proven_fprintln` takes a writer; stdout was not one. Now it is, and it can be a *buffered* one — so a thousand small lines cost one syscall instead of a thousand.

`proven_sysio_stdout_writer(&st)`	An unbuffered writer over stdout. Every write is a write syscall.
`proven_sysio_stderr_writer(&st)`	The same for stderr — which is what you want for an error: it is out before the next line of code runs.

`proven_sysio_stdin_reader(&st)`	A reader over stdin.
`proven_sysio_stdout_buffered(&out, buf)`	stdout behind a buffered writer over a buffer you own.
`proven_sysio_file_buffered(&out, file, buf)`	The same over any open file.

And `flush` means something now. `proven_sysio_flush` used to claim to flush a buffer that did not exist: a no-op on POSIX, a *disk sync* on Windows. It is **deleted**. Pushing a buffered writer's bytes to the OS is `proven_writer_flush`; pushing the OS's bytes to the disk is `proven_fs_sync`. They are different operations and now say so.

> **You must flush a buffered writer.** Nothing reaches the terminal until the buffer fills or you flush it, and nothing in this library registers an `atexit` handler to do it behind your back — a library that owns your process is a library you cannot reason about. Buffered output that is never flushed is output that never happened. The direct calls (`proven_print`, `proven_println`, `proven_eprint`) remain unbuffered for exactly this reason: what they write is on its way out before they return.

The structures you hold

```
typedef struct { proven_file_t file; } proven_sysio_std_t;
    /* Storage for a standard handle, so a writer or reader has something stable to
       point at. proven_writer_from_file takes a proven_file_t * and the file must
       outlive the writer - so it cannot be a temporary. This is that storage. */

typedef struct {
    proven_sysio_std_t       std;
    proven_writer_buffered_t buffered;
} proven_sysio_out_t;        /* a buffered writer over a standard stream or a file */

typedef struct {
    proven_sysio_std_t       std;
    proven_reader_buffered_t buffered;
} proven_sysio_lines_t;      /* a line reader over a standard stream or a file */
```

All three are caller-owned state.

Cautions, and what goes wrong

These state structs contain a pointer to themselves. The writer you get back addresses `&st->std.file` *inside* the struct you passed. Copy the struct, or return it by value, and the writer still points at the original — which may be a dead frame.

Wrong — and an audit reproduced exactly this as a heap-use-after-free:

```
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);

proven_sysio_out_t copy = out;      /* wrong: `w` still points into `out` */
/* ... `out` goes out of scope ... */
(void)proven_writer_write_str(w, PROVEN_LIT("boom"));   /* writes through dead storage */
```

The one exception is `proven_sysio_lines_t`: `proven_sysio_read_line` re-binds it on every call, so a line reader **may** be moved. That is a deliberate courtesy, because it takes its state by pointer — the shape that says "relocatable" — and the library should not lay a trap in the shape of a promise.

A zero-initialised `proven_file_t` is not an invalid handle — on POSIX it is fd 0, which is stdin. The library cannot tell a handle you forgot to fill in from one that legitimately refers to fd 0.

```
proven_sysio_out_t out;
proven_file_t f = {0};                                  /* wrong: this is stdin */
proven_writer_t w = proven_sysio_file_buffered(&out, f, buf);   /* writes to fd 0 */
```

Buffered stdout and unbuffered stderr do not interleave in the order you wrote them. Anything you buffer sits in your buffer while stderr goes straight out. Flush before you print an error that is supposed to appear after your output.

Randomness, by use case

There is no single "random". There are two jobs that look identical and are not:

Your job	Use	Why
A key, a token, a nonce — anything an attacker must not guess.	`proven_random_bytes`, or a `proven_chacha_rng_t` seeded from it.	Only a cryptographic source is unguessable.
The same, on a target with no OS.	`proven_chacha_rng_t`, seeded from the board's own entropy.	ChaCha20 is pure arithmetic; it needs no OS. It is only as unguessable as its seed.
A simulation, a test, a game, a sample.	`proven_xoshiro256ss_t`.	Fast, and **reproducible**: the same seed replays the same run, which is what makes a failing test debuggable.
A number in a range, a shuffle, a float in [0,1).	`proven_rng_below`, `proven_rng_range`, `proven_rng_f64`, `proven_rng_shuffle` — over any source.	`% n` is biased, and everyone writes it anyway. These are not.

The two requirements are in direct opposition. Reproducible means predictable, and predictable is exactly what a token must not be: a few outputs of `proven_xoshiro256ss_t` reveal its entire state and therefore every number it will ever produce. That is a feature for a simulation you need to replay and a catastrophe for a session token — so the two carry names that cannot be confused, and the choice is visible at the call site rather than buried in how something was seeded.

The trait is infallible. `proven_rng_t` is a source of random bytes, and drawing from a valid one cannot fail. That is not a simplification; it is where the failure went. Asking an operating system for entropy *can* fail, so that failure is confined to exactly one place — seeding — which you check once, at startup. Every draw downstream is total.

`proven_random_bytes(buf, len)`	The OS CSPRNG. Returns `false` on failure; do not use `buf` then. `len == 0` is a successful no-op.
`proven_random_u64()`	One strong word from the OS, or `0` on failure.
`proven_chacha_rng_seed_from_entropy(&g)`	Seed the cryptographic generator from the entropy source. **This is the call that can fail.**
`proven_random_set_source(fn, ctx)`	Install the entropy source. The OS is already installed on a hosted target; a bare-metal target installs its board's TRNG.
`proven_chacha_rng_seed(&g, seed32)`	Seed it from 32 bytes you supply — a hardware entropy source on a board. Never the clock.
`proven_xoshiro256ss_seed(&g, seed)`	Seed the reproducible generator. Even seed 0 is fine: it is expanded through SplitMix64.

Where entropy comes from

Everything above is pure arithmetic — the generators, the helpers, and `proven_random_bytes` itself. What differs by platform is the **entropy source** behind it, because that is the one thing a program cannot compute for itself.

- **Hosted:** the OS CSPRNG is installed for you — `getrandom` on Linux, `getentropy` on the BSDs and macOS, `BCryptGenRandom` on Windows, and `/dev/urandom` where none of those exist. You call nothing.
- **Bare metal:** there is no source until you install one. A board *has* real entropy — an on-chip TRNG, a ring oscillator, an ADC's noise floor — and the library cannot know where.

```
/* On a board: hand the library its hardware entropy, once, at startup.
 * (A listing, not a fragment: it defines a function, and `hardware_rng_read` is
 * whatever your SoC calls its entropy register.) */
static bool board_trng(void *ctx, void *buf, proven_size_t len) {
    (void)ctx;
    /* read the SoC's entropy register into buf; return false if it is not ready */
    return hardware_rng_read(buf, len);
}

proven_random_set_source(board_trng, NULL);

/* From here everything above works unchanged - including the one call that turns a few
 * hundred bytes of hardware entropy into an endless cryptographic stream. */
proven_chacha_rng_t g;
if (!proven_chacha_rng_seed_from_entropy(&g)) {
    /* the TRNG was not ready. The generator is INERT - it yields zeros and an invalid
     * trait - so ignoring this does not get you plausible-looking bytes. */
}
```

With no source installed, `proven_random_bytes` returns **false**. It does not fall back to a clock-seeded PRNG, because that looks like success and is a security hole nothing reports — a refusal is a fact a caller can act on.

There is deliberately **no built-in `RDRAND` / `RNDR` backend.** On a hosted target the OS already mixes the CPU's instruction into its own pool, so calling it directly buys nothing and costs you that mixing; and a raw hardware instruction used as the *sole* source is exactly the arrangement people have argued about for a decade. If you want it, it is four lines behind this hook — and then the choice is visibly yours.

The structures you hold

All three are caller-owned state: they allocate nothing, there is nothing to destroy, and **copying one clones its sequence**.

```
typedef struct { const proven_rng_vtable_t *vt; void *ctx; } proven_rng_t;
    /* The trait: two pointers, held by value. `ctx` points at one of the generators
       below, which must outlive it. Drawing from a VALID one cannot fail - that is
       the whole design: the failure lives in seeding, not in drawing. */

typedef struct { proven_u64 s[4]; } proven_xoshiro256ss_t;
    /* 256 bits of state. Reproducible, and NOT secret-grade. */

typedef struct {
    proven_u32    state[16];   /* the ChaCha state: constants, key, counter, nonce */
    proven_byte_t block[64];   /* the keystream block currently being handed out */
    proven_size_t used;        /* how much of it is spent */
    proven_u32    seeded;      /* set only by seeding. A zero-initialised struct is
                                  the shape of "never seeded", and must stay inert. */
} proven_chacha_rng_t;
```

Reference

API	Intent	Return

`proven_random_bytes(buf, len)`	Fill from the entropy source (the OS by default). **The one call that can fail.**	`bool`. On `false`, `buf` is unspecified and must not be used. `len == 0` succeeds.
`proven_random_u64()`	One strong word from the same source.	`proven_u64`, or `0` on failure — which is also a valid draw, so use `proven_random_bytes` when you must tell them apart.
`proven_random_set_source(fn, ctx)`	Install the entropy source. Not needed on a hosted target; this is how a board hands over its TRNG.	void.
`proven_xoshiro256ss_seed(&g, seed)`	Seed the reproducible generator. Any seed is fine — even 0; it is expanded through SplitMix64.	void.
`proven_xoshiro256ss_next(&g)`	The next word. The hot path: call it directly, not through the trait.	`proven_u64`.
`proven_xoshiro256ss_rng(&g)`	View it as a `proven_rng_t`, for the helpers.	`proven_rng_t`.
`proven_chacha_rng_seed(&g, seed32)`	Seed the cryptographic generator from 32 bytes of **real entropy** you supply.	void.
`proven_chacha_rng_seed_from_entropy(&g)`	Seed it from the installed source. **Check this.**	`bool`. On `false` the generator is left INERT — it yields zeros and an invalid trait.
`proven_chacha_rng_next/_fill`	Draw. Cannot fail once seeded.	`proven_u64` / void.
`proven_chacha_rng(&g)`	View it as a `proven_rng_t`.	`proven_rng_t` — **invalid** if the generator was never successfully seeded.
`proven_rng_u64(rng)` / `proven_rng_fill(rng, buf, len)`	Draw through the trait, from whichever generator.	`proven_u64` / void. `0` / no-op for an invalid source.
`proven_rng_below(rng, bound)`	Uniform in `[0, bound)`, **unbiased**.	`proven_u64`; `0` when `bound == 0`.
`proven_rng_range(rng, lo, hi)`	Uniform in `[lo, hi]`, inclusive. The full `INT64_MIN..INT64_MAX` span does not overflow.	`proven_i64`; `lo` if `hi < lo`.
`proven_rng_f64(rng)`	Uniform in `[0, 1)`. 53 bits; never returns `1.0`.	`double`.
`proven_rng_shuffle(rng, base, count, elem_size)`	An unbiased Fisher-Yates permutation, in place.	void.

Cautions, and what goes wrong

Never generate a secret with `proven_xoshiro256ss_t`. It is fast *because* it is predictable: a handful of its outputs reveal its entire 256-bit state, and from the state every number it will ever produce. The two generators carry names that cannot be confused for exactly this reason.

Wrong — a session token an attacker can compute after watching a few:

```
proven_xoshiro256ss_t g;
proven_xoshiro256ss_seed(&g, 12345);
proven_u64 session_token = proven_xoshiro256ss_next(&g);   /* wrong: predictable */
```

Never seed the cryptographic generator from the clock, a counter, or a serial number. ChaCha20 is exactly as unguessable as its seed. A clock-derived seed produces a stream that looks perfectly random and is not — which is worse than an obvious failure, because nothing reports it.

```
proven_byte_t seed[32] = { 0 };
memcpy(seed, &now_ns, sizeof now_ns);      /* wrong: ~20 bits of real entropy, and guessable */
proven_chacha_rng_seed(&g, seed);
```

Check the seeding. It is the only thing here that can fail, which is precisely why ignoring it is tempting. If you do, the generator is inert and hands you zeros — a visibly dead value, by design, rather than a plausible one.

Wrong:

```
proven_chacha_rng_t g;
proven_chacha_rng_seed_from_entropy(&g);   /* wrong: the bool was the point */
proven_chacha_rng_fill(&g, key, 32);       /* key is now 32 zero bytes */
```

Correct:

```
proven_chacha_rng_t g;
if (!proven_chacha_rng_seed_from_entropy(&g)) {
    /* No entropy. There is nothing safe to do here except refuse to continue. */
} else {
    proven_byte_t key[32];
    proven_chacha_rng_fill(&g, key, sizeof key);   /* cannot fail: it is seeded */
}
```

`% n` is biased, and everyone writes it anyway. Unless `n` divides 2^64 the low values come up more often — invisible in a spot check, real in a shuffle or a sample.

```
proven_u64 die = proven_rng_u64(rng) % 6 + 1;   /* wrong: 1 and 2 are slightly likelier */
```

Correct: `proven_rng_below(rng, 6) + 1`.

Do not copy a seeded generator unless you mean to clone its stream. Two "independent" generators copied from one produce identical output — which is a feature for replaying a simulation and a catastrophe for issuing two tokens.

Compiled and run by the test suite:

```
/*
 * Randomness, by use case. There is no single "random": there are two jobs that look
 * identical and are not, and picking the wrong one is the whole danger.
 *
 *   A key, a token, a nonce - anything an attacker must not guess - needs a CRYPTOGRAPHIC
 *   source. A simulation, a test, a game needs a REPRODUCIBLE one, because a failing run you
 *   cannot replay is a failing run you cannot debug. The two requirements are in direct
 *   opposition: reproducible means predictable, and predictable is exactly what a token must
 *   not be. So the library gives them different names, and the choice is visible here at the
 *   call site rather than buried in how something was seeded.
 */

int main(void) {
    /* ---- Job 1: a secret. The OS CSPRNG - and the one place randomness can fail. ---- */
    proven_byte_t key[32];
    EXAMPLE_REQUIRE(proven_random_bytes(key, sizeof key),
                    "the OS must give us strong bytes on a hosted platform");

    /* ---- Job 2: lots of cryptographic bytes, or any at all on a board with no OS.
```

```
     * ChaCha20 is pure arithmetic: seed it once from real entropy and it needs nothing from
     * the operating system afterwards - no syscall per draw, and it works on bare metal.
     * Seeding is the ONLY step that can fail, so it is the only one you have to check. ---- */
    proven_chacha_rng_t crypto;
    EXAMPLE_REQUIRE(proven_chacha_rng_seed_from_entropy(&crypto), "seed the CSPRNG from the OS, once");

    proven_byte_t token[16];
    proven_chacha_rng_fill(&crypto, token, sizeof token);   /* cannot fail: it is seeded */

    /* ---- Job 3: a REPRODUCIBLE run. xoshiro256** is fast and replays exactly from its seed,
     * which is what makes a failing simulation debuggable. It is NOT secret-grade: a few of
     * its outputs reveal its whole state. Never hand it a token to generate. ---- */
    proven_xoshiro256ss_t sim;
    proven_xoshiro256ss_seed(&sim, 12345);

    proven_xoshiro256ss_t replay;
    proven_xoshiro256ss_seed(&replay, 12345);
    EXAMPLE_REQUIRE(proven_xoshiro256ss_next(&sim) == proven_xoshiro256ss_next(&replay),
                    "the same seed replays the same run - that is the whole point");

    /* ---- The helpers work over ANY source, through the proven_rng_t trait. ---- */
    proven_rng_t rng = proven_xoshiro256ss_rng(&sim);

    /* A number in a range. `rng_u64() % 6` is what everyone writes, and it is BIASED unless
     * the bound divides 2^64 - the low values come up more often. This one is not. */
    for (int i = 0; i < 100; ++i) {
        proven_u64 die = proven_rng_below(rng, 6) + 1;
        EXAMPLE_REQUIRE(die >= 1 && die <= 6, "a die roll is 1..6, uniformly");
    }

    proven_i64 temperature = proven_rng_range(rng, -40, 85);
    EXAMPLE_REQUIRE(temperature >= -40 && temperature <= 85, "an inclusive range, both ends");

    double p = proven_rng_f64(rng);
    EXAMPLE_REQUIRE(p >= 0.0 && p < 1.0, "a double in [0, 1) - never 1.0");

    /* An unbiased shuffle: Fisher-Yates over the unbiased index above. The `% n` version of
     * this loop measurably favours some orderings. */
    int deck[10];
    for (int i = 0; i < 10; ++i) deck[i] = i;
    proven_rng_shuffle(rng, deck, 10, sizeof deck[0]);

    int sum = 0;
    for (int i = 0; i < 10; ++i) sum += deck[i];
    EXAMPLE_REQUIRE(sum == 45, "a shuffle is a permutation: every card is still there, once");

    /* The cryptographic generator satisfies the same trait, so the same helpers work over it
     * when the choice must be unguessable rather than merely uniform. */
    proven_rng_t secure = proven_chacha_rng(&crypto);
    proven_u64 unguessable_index = proven_rng_below(secure, 1000);
    EXAMPLE_REQUIRE(unguessable_index < 1000, "the helpers do not care which source they draw from");

    (void)token;
    (void)key;
    return EXAMPLE_OK();
}
```

Worked example: replacing a file so a power cut cannot corrupt it
Writing over a file in place has a failure mode that testing never shows and production eventually does: the machine loses power halfway through, and the file that remains is neither the old one nor the new one. The fix is a four-step recipe, and every step of it is load-bearing:

1. Write the new contents to a **temporary file** beside the real one.
2. `proven_fs_sync()` — the new file's bytes are now on the storage device, not merely in the operating system's cache.
3. `proven_fs_rename()` — the name flips to the new file in one indivisible step. A reader sees the whole old file or the whole new one, never a mixture.
4. `proven_fs_sync_dir()` — the rename itself is now on the device.

Skip step 2 and the rename can publish a file whose contents never arrived. Skip step 4 and the contents are safe under a name that is not.

The same example covers the record-level calls that go with it:

Call	What it is for
`proven_fs_pread` / `proven_fs_pwrite`	Read or write at an absolute offset **without moving the file position** — which is what makes one handle safe to share between threads.
`proven_fs_seek` / `proven_fs_tell`	Move the position, and ask where it is. Seeking from the end with a negative offset finds the last record without knowing the length.
`proven_fs_truncate`	Set the length directly. One call, and the filesystem adjusts a number; the alternative is copying the part you keep.
`proven_fs_lock`	An **advisory** lock: it excludes other processes that also ask for one, and does not affect a program that never asks.
`proven_fs_copy`	Duplicate the bytes into a second, independent file.
`proven_fs_link`	A second **name** for the same file (a hard link). No original: the data lives until the last name goes. Same filesystem only.
`proven_fs_symlink`	A small file holding a path (a symbolic link). May cross filesystems, and may point at nothing.
`proven_fs_is_absolute`	Does this path start from the root? The rule differs per platform, which is why it is a call.
`proven_fs_rmdir`	Remove an **empty** directory. A non-empty one is refused, so a recursive delete stays an explicit decision.

```
/*
 * Updating a file so that a power cut cannot leave it half-written.
 *
 * The recipe is old and every part of it is load-bearing:
 *
 *   1. write the new contents to a TEMPORARY file beside the real one,
 *   2. proven_fs_sync   - the new file's bytes are now on the device,
 *   3. proven_fs_rename - the name flips to the new file in one step; a reader
 *                         sees either the whole old file or the whole new one,
 *   4. proven_fs_sync_dir - the rename itself is now on the device.
 *
 * Skip step 2 and the rename can publish a file whose contents never arrived.
 * Skip step 4 and the contents are safe under a name that may not be. Neither
 * failure shows up in testing; both show up in production, once.
 *
```

```
 * The same program also shows the record-level calls - seek/tell, pread/pwrite,
 * truncate - and the advisory lock that stops two copies of the program doing
 * all this at the same time.
 */

typedef struct {
    proven_u32 id;
    proven_u32 score;
} record_t;

static proven_err_t write_records(proven_file_t f, const record_t *recs, proven_size_t n) {
    proven_mem_view_t view = { .ptr = (const proven_byte_t *)recs, .size = n * sizeof recs[0] };
    return proven_fs_write_all(f, view);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_u8str_view_t live = PROVEN_LIT("proven_example_durable.dat");
    proven_u8str_view_t temp = PROVEN_LIT("proven_example_durable.dat.new");
    proven_u8str_view_t here = PROVEN_LIT(".");

    /* is_absolute answers a question worth asking before you join paths or
     * resolve one against a base directory: does this path already start from
     * the root? The rule differs per platform - a leading '/' here, a drive
     * letter or a UNC prefix on Windows - which is exactly why it is a call and
     * not a comparison against '/'. */
    EXAMPLE_REQUIRE(!proven_fs_is_absolute(live), "the working paths in this example are relative");
    EXAMPLE_REQUIRE(proven_fs_is_absolute(PROVEN_LIT("/etc/hosts")), "a leading slash is absolute on POSIX");

    static const record_t initial[] = {
        { .id = 1, .score = 10 }, { .id = 2, .score = 20 },
        { .id = 3, .score = 30 }, { .id = 4, .score = 40 },
    };

    /* --- an ordinary first write ------------------------------------------ */

    proven_result_file_t f = proven_fs_open(alloc, live, PROVEN_FS_WRITE | PROVEN_FS_CREATE |
PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the data file must succeed");
    if (!proven_is_ok(f.err)) return 1;

    proven_err_t err = write_records(f.value, initial, 4);
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the initial records must succeed");
    err = proven_fs_close(f.value);
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing after a write must succeed");

    /* --- an advisory lock, so two copies do not interleave ---------------- */

    proven_result_file_t rw = proven_fs_open(alloc, live, PROVEN_FS_READ | PROVEN_FS_WRITE);
    EXAMPLE_REQUIRE(proven_is_ok(rw.err), "reopening for update must succeed");
    if (!proven_is_ok(rw.err)) return 1;

    /* An EXCLUSIVE lock keeps every other process that also asks for one out.
     * "Advisory" means exactly that: it stops cooperating programs, and a
     * program that never asks for the lock is not affected. `wait = false`
     * returns immediately rather than blocking - the right choice when you have
     * something else to do, and the only safe choice when the other holder
     * might be waiting on you. */
    err = proven_fs_lock(rw.value, PROVEN_FS_LOCK_EXCLUSIVE, false);
```

```
EXAMPLE_REQUIRE(proven_is_ok(err), "taking the exclusive lock must succeed when nobody holds it");

/* --- reading and writing one record, by offset ------------------------ */

/* pread reads at an absolute offset and does NOT move the file position.
 * That is what makes it safe to use from two threads sharing one handle:
 * there is no shared cursor for them to race on. */
record_t third = {0};
proven_mem_mut_t into = { .ptr = (proven_byte_t *)&third, .size = sizeof third };
proven_result_size_t got = proven_fs_pread(rw.value, into, 2 * sizeof(record_t));
EXAMPLE_REQUIRE(proven_is_ok(got.err) && got.value == sizeof third, "reading record 2 must succeed");
EXAMPLE_REQUIRE(third.id == 3 && third.score == 30, "and yield the record that was written there");

/* tell reports the position; after a pread it has not moved. */
proven_result_u64_t pos = proven_fs_tell(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(pos.err) && pos.val == 0, "pread must not move the file position");

/* pwrite updates that record in place, again without touching the cursor. */
third.score = 99;
proven_mem_view_t out_view = { .ptr = (const proven_byte_t *)&third, .size = sizeof third };
proven_result_size_t put = proven_fs_pwrite(rw.value, out_view, 2 * sizeof(record_t));
EXAMPLE_REQUIRE(proven_is_ok(put.err) && put.value == sizeof third, "writing record 2 back must succeed");

/* seek is the cursor-moving alternative, and it returns the position it
 * arrived at. Seeking from the END with a negative offset is how you find
 * the last record without knowing the file length first. */
proven_result_u64_t last = proven_fs_seek(rw.value, -(proven_i64)sizeof(record_t), PROVEN_FS_SEEK_END);
EXAMPLE_REQUIRE(proven_is_ok(last.err), "seeking to the last record must succeed");
EXAMPLE_REQUIRE(last.val == 3 * sizeof(record_t), "which is three records in");

pos = proven_fs_tell(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(pos.err) && pos.val == last.val, "tell agrees with the seek result");

/* truncate sets the length directly. Dropping the last record is one call
 * and O(1); the old way - read everything, write back the part you keep -
 * was an O(n) copy for an operation the filesystem does by adjusting a
 * number. */
err = proven_fs_truncate(rw.value, 3 * sizeof(record_t));
EXAMPLE_REQUIRE(proven_is_ok(err), "truncating to three records must succeed");
proven_result_size_t size = proven_fs_size(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(size.err) && size.value == 3 * sizeof(record_t),
                "the file is now exactly three records long");

/* Release the lock explicitly. Closing the handle would also drop it, but
 * saying so keeps the critical section visible in the code. */
err = proven_fs_lock(rw.value, PROVEN_FS_LOCK_UNLOCK, false);
EXAMPLE_REQUIRE(proven_is_ok(err), "releasing the lock must succeed");
err = proven_fs_close(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the update handle must succeed");

/* --- the durable replace ---------------------------------------------- */

static const record_t replacement[] = {
    { .id = 1, .score = 11 }, { .id = 2, .score = 22 },
};

/* 1. Write the new contents beside the old file. CREATE_NEW refuses if the
 *    temporary name already exists, which is how a leftover from a crashed
 *    run is noticed instead of silently reused. */
proven_result_file_t tmp = proven_fs_open(alloc, temp,
```

```
                                        PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
EXAMPLE_REQUIRE(proven_is_ok(tmp.err), "creating the temporary file must succeed");
if (!proven_is_ok(tmp.err)) return 1;

err = write_records(tmp.value, replacement, 2);
EXAMPLE_REQUIRE(proven_is_ok(err), "writing the new contents must succeed");

/* 2. Push those bytes all the way to the storage device. This is expensive
 *    and meant to be: you are buying the guarantee that the data exists
 *    after a power cut, and the price is a real trip to the device. */
err = proven_fs_sync(tmp.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "syncing the new file's data must succeed");

err = proven_fs_close(tmp.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the temporary file must succeed");

/* 3. Flip the name. A rename within one directory is atomic: any reader
 *    sees the old file or the new one, never a partial write. */
err = proven_fs_rename(alloc, temp, live);
EXAMPLE_REQUIRE(proven_is_ok(err), "renaming the temporary file over the live one must succeed");

/* 4. Make the rename itself durable. Until the directory reaches the device,
 *    the new contents are safe under a name that might not be. */
err = proven_fs_sync_dir(alloc, here);
EXAMPLE_REQUIRE(proven_is_ok(err), "syncing the directory must succeed");

proven_result_file_t check = proven_fs_open(alloc, live, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(check.err), "the live path must now open");
size = proven_fs_size(check.value);
EXAMPLE_REQUIRE(proven_is_ok(size.err) && size.value == 2 * sizeof(record_t),
                "and hold exactly the replacement records");
err = proven_fs_close(check.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the verification handle must succeed");

/* --- copies, and the two kinds of link ------------------------------- */

/* copy duplicates the bytes: two independent files from here on. The
 * allocator is for the temporary buffer the copy moves data through. */
proven_u8str_view_t backup = PROVEN_LIT("proven_example_durable.bak");
err = proven_fs_copy(alloc, live, backup);
EXAMPLE_REQUIRE(proven_is_ok(err), "copying the file must succeed");

/* A HARD link is a second name for the same file. There is no original: the
 * data lives until the last name is removed. Both names must be on the same
 * filesystem, because a name and its data cannot span two. */
proven_u8str_view_t hard = PROVEN_LIT("proven_example_durable.hard");
err = proven_fs_link(alloc, live, hard);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a hard link must succeed");

proven_fs_stat_t st_live = {0}, st_hard = {0};
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, live, &st_live)), "stat of the live name");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, hard, &st_hard)), "stat of the hard link");
EXAMPLE_REQUIRE(st_live.ino == st_hard.ino && st_live.dev == st_hard.dev,
                "both names refer to the same file, which is what a hard link means");

/* A SYMBOLIC link is a small file holding a path. It may point at something
 * on another filesystem, and it may point at nothing at all - following it
 * then fails, which a hard link can never do. */
proven_u8str_view_t soft = PROVEN_LIT("proven_example_durable.link");
err = proven_fs_symlink(alloc, live, soft);
```

```
    EXAMPLE_REQUIRE(proven_is_ok(err), "creating a symbolic link must succeed");

    proven_fs_stat_t st_soft = {0};
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, soft, &st_soft)),
                    "stat follows the symbolic link to its target");
    EXAMPLE_REQUIRE(st_soft.size == st_live.size, "so it reports the target's size");

    /* --- directories, and cleaning up ---------------------------------- */

    proven_u8str_view_t dir = PROVEN_LIT("proven_example_durable_dir");
    err = proven_fs_mkdir(alloc, dir);
    EXAMPLE_REQUIRE(proven_is_ok(err), "creating a directory must succeed");

    /* rmdir removes an EMPTY directory only. That refusal is a feature: a
     * recursive delete is a decision the caller should have to make explicitly,
     * not something a stray path argument can trigger. */
    proven_u8str_view_t inside = PROVEN_LIT("proven_example_durable_dir/file.txt");
    proven_result_file_t child = proven_fs_open(alloc, inside, PROVEN_FS_WRITE | PROVEN_FS_CREATE);
    EXAMPLE_REQUIRE(proven_is_ok(child.err), "creating a file inside it must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(child.value)), "closing it must succeed");

    err = proven_fs_rmdir(alloc, dir);
    EXAMPLE_REQUIRE(err != PROVEN_OK, "removing a non-empty directory must be refused");

    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_remove(alloc, inside)), "removing the file must succeed");
    err = proven_fs_rmdir(alloc, dir);
    EXAMPLE_REQUIRE(proven_is_ok(err), "and then the empty directory can be removed");

    printf("durable replace complete: %zu byte(s) live\n", (size_t)size.value);

    (void)proven_fs_remove(alloc, soft);
    (void)proven_fs_remove(alloc, hard);
    (void)proven_fs_remove(alloc, backup);
    (void)proven_fs_remove(alloc, live);
    return EXAMPLE_OK();
}
```

Wrong — the in-place rewrite the recipe exists to replace:

```
proven_result_file_t f = proven_fs_open(alloc, live, PROVEN_FS_WRITE | PROVEN_FS_TRUNC);
proven_err_t e = proven_fs_write_all(f.value, new_contents);   /* wrong */
```

Between the truncate and the last byte of the write, the file on disk is incomplete. A crash there does not lose the update; it loses the data that was already there.

Wrong — renaming without syncing first:

```
proven_err_t e = proven_fs_close(tmp.value);        /* no proven_fs_sync */
e = proven_fs_rename(alloc, temp, live);            /* wrong */
```

Closing a file does not put its bytes on the device. The rename can be durable while the contents it points at are not.

Wrong — assuming the lock stops everyone:

```
proven_err_t e = proven_fs_lock(f.value, PROVEN_FS_LOCK_EXCLUSIVE, true);
/* another program writes the file without ever calling proven_fs_lock */
```

An advisory lock is a convention between programs that all use it. It is not permission control.

Worked example: readers, writers, and the standard streams

A **writer** is "somewhere bytes go" and a **reader** is "somewhere bytes come from". Each is two pointers: a small table of functions, and the state those functions work on. The whole value of the

arrangement is that code written against a writer does not know whether the bytes end up in a file, in a string, on the terminal, or in a test buffer — and code written against a reader can be tested against a string in memory instead of a file on disk.

The example puts the pieces together:

- `proven_reader_from_view()` makes a reader over bytes you already have, which is how a parser gets tested without touching the filesystem. `proven_reader_read()` fills up to the buffer size and reports what it got; a short read is normal, and end of input is `PROVEN_ERR_EOF`, not a zero-byte success.
- `proven_sysio_stdout_writer()` and `proven_sysio_stderr_writer()` are the unbuffered standard streams, and `proven_sysio_stdin_reader()` the standard input. `proven_writer_write()` means "all of it, or an error"; `proven_writer_write_partial()` moves what it can and reports the count, for callers doing their own retry or back-pressure.
- `proven_reader_is_valid()` and `proven_writer_is_valid()` catch a handle that was never built, at the boundary rather than at the first read or write.
- `proven_sysio_file_buffered()` wraps an open file in a buffered writer over a buffer **you** supply, so the memory cost of buffering is a number you chose. It must be flushed: nothing here flushes on your behalf at exit.
- `proven_sysio_lines_open()` reads that file back one line at a time, through the same kind of caller-supplied buffer. Size it for the longest line you expect — a longer one is `PROVEN_ERR_OUT_OF_BOUNDS`, never a silently cut line.
- `proven_scan_fmt_from_file()` (which calls `proven_sysio_scan_chunk_impl()`) pulls typed values straight out of a file handle when the input has a known shape rather than being free text.

```
/*
 * A writer is "somewhere bytes go" and a reader is "somewhere bytes come from",
 * each of them two pointers: a small table of functions, and the state those
 * functions work on. That is all. The value of the arrangement is that code
 * written against a writer does not know or care whether the bytes end up in a
 * file, in a string, on the terminal, or in a test buffer - and code written
 * against a reader can be tested against a string in memory instead of a file
 * on disk.
 *
 * This program shows both, and the standard-stream and file plumbing that hangs
 * off them:
 *
 *   - a reader over a view (an in-memory string), which is how you test a
 *     parser without touching the filesystem,
 *   - the unbuffered stdout and stderr writers, and the difference between
 *     "write all of this" and "write what you can",
 *   - a buffered writer over a file, which turns many small writes into few
 *     large ones,
 *   - a line reader over that same file,
 *   - reading formatted values straight out of a file handle.
 *
 * A note that costs people an afternoon: the state structs below must stay
 * where they are declared. A writer holds a pointer INTO its state struct, so
 * copying the struct leaves the copy inert and the original addressed.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 1. a reader over bytes you already have ------------------------- */

    /* The parser below does not know this is a string. It would work the same
     * over a file, a pipe or a socket - which is exactly why the test can use
```

```
     * the cheap one. */
    proven_reader_view_t src_state;
    proven_reader_t src = proven_reader_from_view(&src_state, PROVEN_LIT("id=41\nid=42\n"));

    /* is_valid asks whether the reader was actually built - a zero-initialised
     * handle is not a reader, and this is the check that says so at the
     * boundary instead of at the first read. */
    EXAMPLE_REQUIRE(proven_reader_is_valid(src), "a reader made from a view must be usable");
    proven_reader_t never_made = {0};
    EXAMPLE_REQUIRE(!proven_reader_is_valid(never_made), "a zero-initialised reader handle is not");

    /* read fills up to dest.size bytes and reports how many it actually got.
     * A short read is normal - it is not end of file, and treating it as one is
     * the classic way to lose the tail of an input. End of file has its own
     * code, PROVEN_ERR_EOF. */
    proven_byte_t chunk[5];
    proven_mem_mut_t into = { .ptr = chunk, .size = sizeof chunk };
    proven_result_size_t got = proven_reader_read(src, into);
    EXAMPLE_REQUIRE(proven_is_ok(got.err), "the first read must succeed");
    EXAMPLE_REQUIRE(got.value == 5, "and fill the buffer from the view");

    proven_size_t total = got.value;
    for (;;) {
        got = proven_reader_read(src, into);
        if (got.err == PROVEN_ERR_EOF) {
            break;
        }
        EXAMPLE_REQUIRE(proven_is_ok(got.err), "reads before end of file must succeed");
        total += got.value;
    }
    EXAMPLE_REQUIRE(total == 12, "the loop must consume the whole view, however it was chunked");

    /* --- 2. the standard streams ------------------------------------------ */

    /* The state struct holds the handle the writer points at, so it has to
     * outlive the writer. Declaring the two next to each other is the habit
     * that keeps that true. */
    proven_sysio_std_t out_state;
    proven_writer_t out = proven_sysio_stdout_writer(&out_state);
    EXAMPLE_REQUIRE(proven_writer_is_valid(out), "the stdout writer must be usable");

    /* write means "all of it, or an error". It loops internally, because a
     * single system-level write may move fewer bytes than asked. */
    proven_err_t err = proven_writer_write(out, proven_mem_view_from_u8(PROVEN_LIT("stream example:
start\n")));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing a whole line to stdout must succeed");

    /* write_partial is the honest low-level twin: it moves what it can and
     * reports the count. Use it when you are managing your own retry or
     * back-pressure; use write when you just want the bytes out. */
    proven_result_size_t part = proven_writer_write_partial(out, proven_mem_view_from_u8(PROVEN_LIT("partial
write\n")));
    EXAMPLE_REQUIRE(proven_is_ok(part.err), "a partial write to a terminal or pipe must succeed");
    EXAMPLE_REQUIRE(part.value > 0, "and report how many bytes it moved");

    /* stderr is unbuffered on purpose: a diagnostic is out before the next line
     * of code runs, which is what you need when the next line is the one that
     * crashes. */
    proven_sysio_std_t err_state;
    proven_writer_t diag = proven_sysio_stderr_writer(&err_state);
```

```
    EXAMPLE_REQUIRE(proven_writer_is_valid(diag), "the stderr writer must be usable");
    err = proven_writer_write(diag, proven_mem_view_from_u8(PROVEN_LIT("stream example: diagnostics go
here\n")));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing to stderr must succeed");

    /* A reader over stdin is built the same way. This example does not read
     * from it - a test run has no one typing - but building it shows the shape,
     * and it is the handle a filter program would loop over. */
    proven_sysio_std_t in_state;
    proven_reader_t stdin_reader = proven_sysio_stdin_reader(&in_state);
    EXAMPLE_REQUIRE(proven_reader_is_valid(stdin_reader), "the stdin reader must be usable");

    /* --- 3. buffered output to a file ----------------------------------- */

    proven_u8str_view_t path = PROVEN_LIT("proven_example_streams.txt");
    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_WRITE ¦ PROVEN_FS_CREATE ¦
PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the output file must succeed");
    if (!proven_is_ok(f.err)) return 1;

    /* The buffer is yours: the library does not allocate one behind your back,
     * so the memory cost of buffering is a number you chose and can see. Sixty
     * lines through a 256-byte buffer is a handful of writes instead of sixty. */
    proven_byte_t outbuf[256];
    proven_sysio_out_t file_out;
    proven_writer_t w = proven_sysio_file_buffered(&file_out, f.value,
                                                   (proven_mem_mut_t){ .ptr = outbuf, .size = sizeof
outbuf });
    EXAMPLE_REQUIRE(proven_writer_is_valid(w), "the buffered file writer must be usable");

    for (int i = 0; i < 20; ++i) {
        proven_fmt_result_t line_out = proven_fprintln(w, "reading {} = {}",
                                                       proven_arg_i32(i), proven_arg_i32(i * i));
        EXAMPLE_REQUIRE(proven_is_ok(line_out.err), "writing a formatted line must succeed");
    }

    /* Buffered output that is never flushed is output that never happened.
     * Nothing here flushes at exit on your behalf. */
    err = proven_writer_flush(w);
    EXAMPLE_REQUIRE(proven_is_ok(err), "the flush is what actually writes the file");
    err = proven_fs_close(f.value);
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing the file must succeed");

    /* --- 4. reading it back a line at a time ---------------------------- */

    proven_result_file_t rf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(rf.err), "reopening for reading must succeed");
    if (!proven_is_ok(rf.err)) return 1;

    /* Size the buffer for the longest LINE you expect. A longer line is
     * reported as OUT_OF_BOUNDS - never silently cut in half, which is the
     * failure that turns one bad record into two plausible ones. */
    proven_byte_t linebuf[128];
    proven_sysio_lines_t lines;
    err = proven_sysio_lines_open(&lines, rf.value, (proven_mem_mut_t){ .ptr = linebuf, .size = sizeof
linebuf });
    EXAMPLE_REQUIRE(proven_is_ok(err), "opening a line reader over the file must succeed");

    proven_size_t count = 0;
    for (;;) {
```

```
        proven_result_u8str_view_t line = proven_sysio_read_line(&lines);
        if (line.err == PROVEN_ERR_EOF) {
            break;
        }
        EXAMPLE_REQUIRE(proven_is_ok(line.err), "reading a line must succeed until end of file");
        /* The view points into linebuf and is good only until the next call.
         * That is what makes a million lines cost one buffer instead of a
         * million allocations - and why you copy a line you want to keep. */
        if (count == 0) {
            EXAMPLE_REQUIRE(proven_u8str_view_eq(line.val, PROVEN_LIT("reading 0 = 0")),
                            "the first line reads back exactly as it was written");
        }
        ++count;
    }
    EXAMPLE_REQUIRE(count == 20, "every line written must be read back");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(rf.value)), "closing the read handle must succeed");

    /* --- 5. reading values, not lines, from a file ------------------------ */

    /* When the input is a known shape rather than free text, scanning it
     * directly saves writing the split-and-convert loop by hand. This reads one
     * chunk from the file handle and pulls the two numbers out of it. */
    proven_result_file_t sf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(sf.err), "opening the file for scanning must succeed");

    proven_i32 index = -1, square = -1;
    err = proven_scan_fmt_from_file(sf.value, "reading {} = {}",
                                    proven_scan_arg_i32(&index), proven_scan_arg_i32(&square));
    EXAMPLE_REQUIRE(proven_is_ok(err), "scanning the first record must succeed");
    EXAMPLE_REQUIRE(index == 0 && square == 0, "and produce the values that were written");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(sf.value)), "closing the scan handle must succeed");

    printf("streams: %zu byte(s) read from the view, %zu line(s) round-tripped\n",
           (size_t)total, (size_t)count);

    (void)proven_fs_remove(alloc, path);
    return EXAMPLE_OK();
}
```

Wrong — copying a state struct after making a writer from it:

```
proven_sysio_out_t state;
proven_writer_t w = proven_sysio_file_buffered(&state, file, buf);
proven_sysio_out_t moved = state;    /* wrong: w still points into `state` */
```

The writer holds a pointer **into** the struct. Leave the state where you declared it, for as long as the writer lives.

Wrong — forgetting the flush:

```
proven_fmt_result_t r = proven_fprintln(w, "done");
return 0;                            /* wrong: the buffer never reached the file */
```

Worked example: mapping a file into memory, and making the change durable
A memory-mapped file is a file the processor reads as memory: the operating system arranges for its pages to appear at an address, and a read happens without a read call. It suits random access into a large file, especially one several processes share; it does not suit streaming, where a buffered reader is simpler and does not tie the file's size to your address space.

The distinction that decides whether your writes survive:

Mapping	Writes go	`proven_mmap_sync()`
`PROVEN_MMAP_SHARED`	to the file	pushes them to the storage device
`PROVEN_MMAP_PRIVATE`	to a private copy (copy-on-write) — this process only	returns `PROVEN_ERR_UNSUPPORTED`, because there is nothing to write back

That refusal is deliberate. A caller who believed the mapping was shared finds out at the sync, rather than when the data turns out to be missing.

The example ends with the calendar formatter, because the record it writes carries a date: `proven_time_u8_fmt()` for the UTF-8 form, and `proven_time_u16_fmt()` for the UTF-16 form — the latter being right in exactly one situation, handing text to a system call that takes wide strings.

```
#include <string.h>

/*
 * A memory-mapped file is a file the processor reads as memory: the operating
 * system arranges for the pages to appear at an address, and reads happen
 * without a read call. It is the right tool for one shape of problem - random
 * access into a large file that several processes look at - and the wrong tool
 * for streaming, where a buffered reader is simpler and does not tie a file's
 * size to your address space.
 *
 * The part that is easy to get wrong is durability, and it is the reason this
 * example exists:
 *
 *   PROVEN_MMAP_SHARED  - writes go to the file. proven_mmap_sync pushes them
 *                         to the storage device.
 *   PROVEN_MMAP_PRIVATE - writes are copy-on-write: they exist in this process
 *                         and nowhere else. There is nothing to write back, and
 *                         asking to sync one says so instead of quietly doing
 *                         nothing.
 *
 * This example also shows the calendar formatter alongside it, because the
 * record it writes carries a timestamp - and a timestamp written for a Windows
 * API is the one place the UTF-16 formatter is the right call.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();
    proven_u8str_view_t path = PROVEN_LIT("proven_example_mmap.dat");

    /* A mapping cannot extend a file, so the file has to be the size you intend
     * to map before you map it. */
    static const char initial[] = "record 0: pending    \n";
    proven_result_file_t create = proven_fs_open(alloc, path,
                                                 PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(create.err), "creating the backing file must succeed");
    if (!proven_is_ok(create.err)) return 1;

    proven_mem_view_t seed = { .ptr = (const proven_byte_t *)initial, .size = sizeof initial - 1 };
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_all(create.value, seed)), "writing the record must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(create.value)), "closing it must succeed");

    /* --- a shared mapping: writes reach the file -------------------------- */

    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_READ | PROVEN_FS_WRITE);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "opening the file for mapping must succeed");
    if (!proven_is_ok(f.err)) return 1;
```

```
    /* size 0 means "to the end of the file". */
    proven_result_mmap_t m = proven_mmap_create(f.value, 0, 0,
                                                PROVEN_MMAP_READ | PROVEN_MMAP_WRITE,
                                                PROVEN_MMAP_SHARED);
    EXAMPLE_REQUIRE(proven_is_ok(m.err), "mapping the file must succeed");
    if (!proven_is_ok(m.err)) {
        (void)proven_fs_close(f.value);
        return 1;
    }
    proven_mmap_t map = m.value;

    /* Reading is just reading memory - no call, no copy. as_view hands back the
     * whole mapping as a byte view, so the ordinary view helpers apply. */
    proven_u8str_view_t contents = proven_mmap_as_view(map);
    EXAMPLE_REQUIRE(contents.size == sizeof initial - 1, "the mapping covers the whole file");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(contents, PROVEN_LIT("record 0:")),
                    "and shows the bytes that were written");

    /* Writing is writing memory. The status field is a fixed width on purpose:
     * a mapping cannot make the file longer, so an in-place edit has to fit the
     * space that is already there. */
    proven_size_t at = proven_u8str_view_find(contents, 0, PROVEN_LIT("pending"));
    EXAMPLE_REQUIRE(at != PROVEN_SIZE_MAX, "the status field must be found");
    memcpy((proven_byte_t *)map.ptr + at, "done   ", 7);

    /* sync is the durability step: without it the change is in the page cache,
     * where it survives this program exiting but not the machine losing power. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_sync(&map)), "syncing a shared mapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_destroy(&map)), "unmapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(f.value)), "closing the mapped file must succeed");

    /* The edit is in the file, not merely in this process's memory. */
    proven_result_u8str_t back = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(back.err), "reading the file back must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_find(proven_u8str_as_view(&back.value), 0, PROVEN_LIT("done")) !=
PROVEN_SIZE_MAX,
                    "the mapped write reached the file");
    proven_u8str_destroy(alloc, &back.value);

    /* --- a private mapping: writes go nowhere ------------------------------ */

    proven_result_file_t pf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(pf.err), "reopening for a private mapping must succeed");

    proven_result_mmap_t pm = proven_mmap_create(pf.value, 0, 0, PROVEN_MMAP_READ, PROVEN_MMAP_PRIVATE);
    EXAMPLE_REQUIRE(proven_is_ok(pm.err), "a private read mapping must succeed");
    proven_mmap_t priv = pm.value;

    /* Asking to sync a private mapping is refused rather than accepted and
     * ignored. The refusal is the useful behaviour: a caller who believed the
     * mapping was shared finds out here, not when the data is missing. */
    EXAMPLE_REQUIRE(proven_mmap_sync(&priv) == PROVEN_ERR_UNSUPPORTED,
                    "a private mapping has nothing to write back, and says so");

    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_destroy(&priv)), "unmapping the private mapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(pf.value)), "closing it must succeed");

    /* --- the timestamp, in both string types ------------------------------- */
```

```
    proven_datetime_t now = proven_time_now_datetime();

    proven_result_u8str_t stamp = proven_u8str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(stamp.err), "creating the timestamp string must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_time_u8_fmt(alloc, &stamp.value, now, &proven_time_locale_en,
                                                    "{year}-{month:0>2}-{day:0>2}")),
                    "formatting the date as UTF-8 must succeed");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&stamp.value).size == 10, "a YYYY-MM-DD date is ten characters");

    /* The UTF-16 form of the same call, for the one place it is the right one:
     * handing the text straight to a system call that takes wide strings. */
    proven_result_u16str_t wide = proven_u16str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(wide.err), "creating the wide timestamp string must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_time_u16_fmt(alloc, &wide.value, now, &proven_time_locale_en,
                                                     "{year}-{month:0>2}-{day:0>2}")),
                    "formatting the same date as UTF-16 must succeed");
    EXAMPLE_REQUIRE(proven_u16str_len(&wide.value) == 10, "ten code units, one per character here");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&wide.value)[4] == (proven_u16)'-',
                    "and the same layout as the UTF-8 form");

    printf("mapped record updated; stamped %s\n", proven_u8str_as_cstr(&stamp.value));

    proven_u16str_destroy(alloc, &wide.value);
    proven_u8str_destroy(alloc, &stamp.value);
    (void)proven_fs_remove(alloc, path);
    return EXAMPLE_OK();
}
```

Wrong — expecting a mapping to extend the file:

```
proven_result_mmap_t m = proven_mmap_create(f.value, 0, 1 << 20, ...);  /* file is 22 bytes */
memcpy((char *)m.value.ptr + 4096, data, n);                            /* wrong */
```

Set the file's length first — `proven_fs_truncate()` does it in one call — and map what exists.

Worked example: where randomness comes from

The example above picks the right generator. This one is the layer underneath: the source of the bytes, and how to write code that does not care which source it got.

- `proven_rng_t` **is the interface.** A function that takes one works with the operating system's generator, with ChaCha20, with xoshiro, and with a fake you wrote for a test, unchanged. `proven_rng_is_valid()` says whether a source was actually built; drawing from one that was not returns 0 rather than inventing a number, so check once at the boundary instead of trusting every draw. `proven_rng_u64()` draws one 64-bit word and `proven_rng_fill()` fills a whole buffer in one call.
- **A fixed seed makes a test reproducible.** `proven_chacha_rng_seed()` takes the seed bytes directly, so `proven_chacha_rng_next()` walks a known sequence — which is what turns "fails once a week" into a failure you can replay. In production the seed must come from real entropy, because anyone who learns it knows every byte that follows.
- `proven_random_u64()` draws a single strong word straight from the entropy source. Right for a one-off — a hash key at start-up, an identifier — and wrong in a loop, where each call costs a trip to the operating system.
- `proven_random_set_source()` installs the entropy source itself. A hosted program already has the operating system's and should leave it alone; a bare-metal program has none, and this is where its hardware source goes.

```
#include <string.h>

/*
```

```
 * The other example shows which generator to pick. This one is about the layer
 * underneath: where the randomness comes FROM, and how to write code that does
 * not care.
 *
 *   proven_rng_t is a source of random bytes as a pair of pointers - a small
 *   table of functions and the generator state they work on. Code that takes a
 *   proven_rng_t works with the OS generator, with ChaCha20, with xoshiro, and
 *   with a fake you wrote for a test, without a line of change.
 *
 *   proven_random_set_source is the layer below THAT: where the raw entropy a
 *   generator is seeded from comes from. A hosted program already has one - the
 *   operating system's - and should leave it alone. A bare-metal program has
 *   none, and this is the hook where its hardware source is installed.
 *
 * The fixed-seed part matters more than it looks: a cryptographic generator
 * seeded from a KNOWN seed produces a known sequence, which is what makes a
 * test that involves randomness reproducible instead of "fails once a week".
 */

/* A source of "entropy" that is not random at all: it counts. Nothing like this
 * belongs in a real program - see the counter-example in the chapter - but it
 * is exactly the right shape for showing how the hook works, and for a test
 * that must produce the same bytes every run. */
static bool counting_entropy(void *ctx, void *buf, proven_size_t len) {
    proven_u8 *next = (proven_u8 *)ctx;
    proven_u8 *out = (proven_u8 *)buf;
    for (proven_size_t i = 0; i < len; ++i) {
        out[i] = (*next)++;
    }
    return true;
}

/* A function written against the trait. It never learns which generator it got. */
static proven_u64 roll_total(proven_rng_t rng, int rolls) {
    proven_u64 sum = 0;
    for (int i = 0; i < rolls; ++i) {
        sum += proven_rng_below(rng, 6) + 1;
    }
    return sum;
}

int main(void) {
    /* --- 1. a source you can check before you use it ---------------------- */

    proven_rng_t nothing = {0};
    EXAMPLE_REQUIRE(!proven_rng_is_valid(nothing), "a zero-initialised source is not a generator");

    /* Drawing from an invalid source does not crash and does not invent a
     * number: it returns 0. That is a defined, boring answer - but a stream of
     * zeros is not randomness, so check the source once when you receive it
     * rather than trusting every draw. */
    EXAMPLE_REQUIRE(proven_rng_u64(nothing) == 0, "an invalid source yields 0, not a fabricated value");

    /* --- 2. a cryptographic generator from a KNOWN seed ------------------- */

    /* proven_chacha_rng_seed takes the seed bytes directly, so the sequence is
     * reproducible. That is what you want in a test and never in production:
     * anyone who learns the seed knows every byte the generator will produce. */
    proven_byte_t seed[PROVEN_CHACHA_SEED_SIZE];
    memset(seed, 0xA5, sizeof seed);
```

```c
proven_chacha_rng_t a, b;
proven_chacha_rng_seed(&a, seed);
proven_chacha_rng_seed(&b, seed);

/* next returns one 64-bit word at a time. Two generators given the same
 * seed walk the same sequence - which is the property the test relies on. */
proven_u64 first = proven_chacha_rng_next(&a);
EXAMPLE_REQUIRE(first == proven_chacha_rng_next(&b), "the same seed replays the same sequence");
EXAMPLE_REQUIRE(proven_chacha_rng_next(&a) == proven_chacha_rng_next(&b), "and keeps replaying it");

/* --- 3. using it through the trait ------------------------------------ */

proven_rng_t rng = proven_chacha_rng(&a);
EXAMPLE_REQUIRE(proven_rng_is_valid(rng), "a seeded generator makes a valid source");

proven_u64 word = proven_rng_u64(rng);
(void)word;   /* any 64-bit value is a legal answer; there is nothing to assert about it */

/* fill is the bulk form: one call for a whole buffer, rather than a loop
 * over 64-bit words that has to deal with the remainder itself. */
proven_byte_t nonce[12] = {0};
proven_rng_fill(rng, nonce, sizeof nonce);

bool all_zero = true;
for (proven_size_t i = 0; i < sizeof nonce; ++i) {
    if (nonce[i] != 0) all_zero = false;
}
EXAMPLE_REQUIRE(!all_zero, "filling from a seeded generator must produce something");

/* The same function, driven by two different generators. This is the only
 * reason the trait exists. */
proven_chacha_rng_t c;
proven_chacha_rng_seed(&c, seed);
proven_u64 crypto_total = roll_total(proven_chacha_rng(&c), 50);

proven_xoshiro256ss_t fast;
proven_xoshiro256ss_seed(&fast, 7);
proven_u64 fast_total = roll_total(proven_xoshiro256ss_rng(&fast), 50);

EXAMPLE_REQUIRE(crypto_total >= 50 && crypto_total <= 300, "50 dice must total between 50 and 300");
EXAMPLE_REQUIRE(fast_total >= 50 && fast_total <= 300, "whichever generator produced them");

/* --- 4. one strong word, without holding a generator ------------------ */

/* proven_random_u64 draws straight from the entropy source. Convenient for
 * a one-off - a table's hash key at startup, a request id - and the wrong
 * tool for a loop, because each call costs a trip to the operating system.
 * For bulk output, seed a generator once and draw from that. */
proven_u64 one_off = proven_random_u64();
proven_u64 another = proven_random_u64();
EXAMPLE_REQUIRE(one_off != another || one_off != 0,
                "two draws from the OS source are essentially never the same value");

/* --- 5. installing an entropy source ---------------------------------- */

/* On a hosted target the operating system's source is already installed and
 * you should leave it there. This hook exists for the bare-metal case,
 * where the library cannot know that the board's entropy lives in a
 * particular hardware register. Here it is installed with a deliberately
```

```
     * fake source, purely to show the mechanism and to prove the switch took
     * effect - a real one must be genuine hardware entropy. */
    proven_u8 counter = 0;
    proven_random_set_source(counting_entropy, &counter);

    proven_byte_t drawn[4] = {0};
    EXAMPLE_REQUIRE(proven_random_bytes(drawn, sizeof drawn), "the installed source must answer");
    EXAMPLE_REQUIRE(drawn[0] == 0 && drawn[1] == 1 && drawn[2] == 2 && drawn[3] == 3,
                    "and it is the source we installed that answered");

    /* Put the platform default back. Leaving a test source installed is how a
     * program ends up generating predictable keys in production. */
    proven_random_set_source(NULL, NULL);

    proven_byte_t real[8] = {0};
    EXAMPLE_REQUIRE(proven_random_bytes(real, sizeof real), "the OS source is back and working");

    printf("random: first word %llu, dice totals %llu and %llu\n",
           (unsigned long long)first, (unsigned long long)crypto_total, (unsigned long long)fast_total);

    return EXAMPLE_OK();
}
```

Wrong — installing something that merely looks random:

```
static bool clock_entropy(void *ctx, void *buf, proven_size_t len) {
    proven_time_t t = proven_time_now();       /* wrong: predictable */
    memcpy(buf, &t, len < sizeof t ? len : sizeof t);
    return true;
}
proven_random_set_source(clock_entropy, NULL);
```

A clock, a serial number, an uninitialised buffer or a pseudo-random generator all produce something that passes a glance and is guessable. If a board has no real entropy, install nothing: a refusal is a fact the caller can act on, and silent predictability is not.

Wrong — leaving a test source installed:

```
proven_random_set_source(counting_entropy, &counter);
/* ... the rest of the program, now generating predictable keys ... */
```

Restore the platform default with `proven_random_set_source(NULL, NULL)` as soon as the test is over.

Chapter 6: Execution, Aliases, PAL, Freestanding, and Cross Builds

Part VI — Going further. Prerequisites: Parts II–V. After this chapter you can run work on more than one thread without the memory model surprising you, write a state machine that reads like a loop, and build for a target that has no operating system.

This chapter covers `coro.h`, `job.h`, `alias_xcv.h`, thread-safety and pointer provenance, PAL contracts, freestanding mode, and platform build notes. This is the hardest material in the manual, and it is last on purpose — nothing in Parts I–V depends on it.

Table of contents

1. Stackless coroutines

The coroutine macros implement stackless state machines using `__LINE__` labels inside a switch. A coroutine function should return `proven_i32`: `0` means yielded or waiting, `1` means done.

Structure

```
typedef struct {
proven_i32 state;
} proven_coro_t;
```

Macros

Macro	Intent
`PROVEN_CORO_INIT(c)`	Set state to zero before first use.
`PROVEN_CORO_BEGIN(c)`	Begin coroutine switch block.
`PROVEN_CORO_YIELD(c)`	Save resume point and return 0.
`PROVEN_CORO_AWAIT(c, cond)`	Return 0 until `cond` is true, then continue.
`PROVEN_CORO_END(c)`	Mark done and return 1.
`PROVEN_CORO_IS_DONE(c)`	Check whether state is -1.

How it expands (and the one rule that matters)

These macros are the classic Protothreads/Duff's-device trick. `BEGIN` opens a `switch (state)`, each `YIELD`/`AWAIT` records `__LINE__` as the resume label and `return`s, and `END` closes the switch. Conceptually:

```
proven_i32 f(Ctx *c) {
    switch (c->coro.state) {       /* PROVEN_CORO_BEGIN */
    case 0:
        /* ... code before the first yield ... */
        c->coro.state = 42; return 0; case 42:;   /* PROVEN_CORO_YIELD on line 42 */
        /* ... code after the yield ... */
    }
    c->coro.state = -1; return 1;  /* PROVEN_CORO_END */
}
```

The function **returns** at each yield and is **re-entered from the top**, jumping straight to the resume `case`. The consequence — the single rule to remember:

Local (stack) variables do NOT survive a yield. Anything whose value must persist across a `YIELD`/`AWAIT` must live in the coroutine's own struct (or be `static`), because the locals are re-created on every call.

Other constraints that follow from the expansion: two coroutine macros must not share a source line (they'd collide on `__LINE__`), and a `YIELD`/`AWAIT` must not sit inside a `switch` of your own (it would land in the wrong `case`). The coroutine has no stack, so it cannot yield from a helper function it calls — only from its own body.

Counter-example — a local across a yield

```
static proven_i32 bad(Ctx *c) {
    PROVEN_CORO_BEGIN(&c->coro);
    int i = 0;                      /* WRONG: a local */
    while (i < 3) {
        i += 1;
        PROVEN_CORO_YIELD(&c->coro); /* on re-entry `i` is re-initialized to 0 */
    }
    PROVEN_CORO_END(&c->coro);      /* loop never ends: infinite yields */
}
/* RIGHT: put `i` in Ctx (like `value` in the Counter example below). */
```

Example (a sketch: a coroutine is a whole function, so it cannot be shown as a statement fragment - the compiled version of this shape is the worked example in section 8, `manual/examples/ex_06_coro.c`):

```
typedef struct Counter {
proven_coro_t coro;
int value;
} Counter;

static proven_i32 counter_next(Counter *c) {
PROVEN_CORO_BEGIN(&c->coro);
c->value = 0;
while (c->value < 3) {
c->value += 1;
PROVEN_CORO_YIELD(&c->coro);
}
PROVEN_CORO_END(&c->coro);
}

Counter c = {0};
PROVEN_CORO_INIT(&c.coro);
while (!counter_next(&c)) {
use_value(c.value);
}
```

2. Job system

The job system owns worker threads and a bounded MPMC-style queue. Producers submit `routine(arg)` work items. Workers execute queued jobs until the system is closed and drained.

Structures

```
typedef struct {
void (*routine)(void *arg);
void *arg;
} proven_job_t;

typedef struct proven_job_sys proven_job_sys_t;
```

`proven_job_sys_t` is opaque.

Functions

API	Intent	Return
`proven_job_system_init(alloc, num_workers, max_queue_capacity, out_sys)`	Allocate and start a job system. Queue capacity must be a power of two.	`proven_err_t`.
`proven_job_system_close(sys)`	Stop accepting new jobs.	void.
`proven_job_system_destroy(sys)`	Close if needed, drain queue, join workers, free resources.	void.
`proven_job_submit(sys, routine, arg)`	Submit one job. Thread-safe with other submitters.	true if queued, false if full or closed.
`proven_job_execute_one(sys)`	Let the calling thread execute one available job.	true if a job ran.

Important constraints:

- `proven_job_system_close()` may race with submitters. A submission admitted before close either commits or returns `false` before close returns; later submissions are rejected.
- `proven_job_system_destroy()` must not race with producer threads still calling `proven_job_submit()`.
- To use close as the stop signal, call close, join every producer, and only then destroy.
- Queue sequence counters assume they do not wrap beyond signed pointer-difference range during one job-system lifetime.

Concurrency model
The queue is a fixed-size ring buffer of `max_queue_capacity` slots (must be a power of two, so a slot index is `seq & (capacity - 1)`). Producers and consumers do not lock the whole queue; they coordinate with **atomic sequence counters**:

- A producer (`proven_job_submit`) atomically claims the next enqueue position. If claiming would overrun the slowest un-consumed slot, the queue is full and submit returns `false` rather than waiting for capacity or overwriting pending work. Posting the worker wake may briefly enter platform synchronization.
- A consumer (a worker thread, or your own thread via `proven_job_execute_one`) atomically claims the next dequeue position, reads the `routine`/`arg`, marks the slot reusable, and runs the routine outside the claim.

An idle worker blocks on a platform counting semaphore instead of repeatedly polling and yielding. A successful submit publishes its slot before posting one permit. A permit remains available when no worker is waiting, so a wake cannot be lost between an empty-queue observation and the wait. `close` first stops admission and waits for already-admitted submissions to finish, then posts one final permit per worker. If an external `proven_job_execute_one` consumes a job before a worker consumes its permit, the retained permit is merely stale: the worker performs an empty recheck and parks again, or exits after close.

Because submit and execute are MPMC-safe, *many* threads may submit and *many* may consume at once. What the library does **not** do for you: it does not synchronize the *data your jobs touch*. Two jobs that write the same variable still need their own locking/atomics — the queue only orders the handoff, not the work. For the thread-safety of the allocators those jobs use, the pointer-provenance hazards of sharing allocations, and the lock-free toolbox (CAS, ABA, tagged pointers, hazard pointers, epoch-based reclamation), see Chapter 2 §7 "Allocator thread-safety & provenance".

Lifecycle state machine (drive it in this order):

```
init  --->  running  --(close)-->  closed  --(destroy: drain + join)-->  freed
```

- `init` reserves `num_workers` OS threads and the slot buffer up front.
- `close` flips a flag so every later `submit` returns `false`; already-queued and in-flight submissions settle before close returns, and every accepted job still runs.
- `destroy` closes if needed, then **blocks the calling thread until the queue is empty and every worker has joined**, then frees everything.

Memory visibility. A worker joining inside `destroy` is a synchronization point: every memory effect of every job is visible to the thread that called `destroy` *after* `destroy` *returns*. So the safe pattern for collecting results is "submit all → close → destroy → read results". Reading a job's output from another thread *before* that join needs your own synchronization.

Counter-examples

```
/* WRONG: capacity must be a power of two. */
proven_job_system_init(alloc, 4, 100, &sys);   /* 100 is not 2^n */

/* WRONG: reading a result before the join makes it visible. */
int total = 0;
(void)proven_job_submit(sys, accumulate, &total);
proven_println("total = {}", PROVEN_ARG(total));  /* data race: job may still be running */
/* RIGHT: */
proven_job_system_close(sys);
proven_job_system_destroy(sys);                   /* joins workers -> writes now visible */
proven_println("total = {}", PROVEN_ARG(total));

/* WRONG: ignoring the submit result drops work silently when the ring is full. */
proven_job_submit(sys, work, arg);   /* [[nodiscard]]: a false return means NOT queued */
```

Example (a sketch, because the job routine has to be a function of its own; the compiled version, with the atomics a shared counter really needs, is the worked example in section 8, `manual/examples/ex_06_job.c`):

```
static void increment(void *arg) {
int *p = arg;
*p += 1;
}

proven_job_sys_t *sys = NULL;
proven_err_t e = proven_job_system_init(alloc, 2, 64, &sys);
if (!proven_is_ok(e)) return e;

int value = 0;
if (!proven_job_submit(sys, increment, &value)) {
proven_job_system_close(sys);
proven_job_system_destroy(sys);
return PROVEN_ERR_BUSY;
}

proven_job_system_close(sys);
proven_job_system_destroy(sys);
```

The lifecycle itself, with no routine in sight, is ordinary statement code:

```
proven_job_sys_t *sys = NULL;
proven_err_t e = proven_job_system_init(alloc, 2, 64, &sys);  /* capacity: power of two */
if (proven_is_ok(e)) {
    /* ... submit work here, from this thread or from producers you own ... */
    proven_job_system_close(sys);     /* every later submit now returns false */
    proven_job_system_destroy(sys);   /* drains the queue, joins the workers */
}
```

3. Threads, allocators, and pointer provenance

This section moved here from Chapter 2. It was the hardest material in the manual sitting in one of the first chapters a reader opens, and nothing in Chapters 1-5 depends on it. If you are sharing `proven` objects between threads, or building a lock-free structure on top of one, this is the section you need; if you are not, you can skip it entirely and nothing later will suffer.

The `proven_allocator_t` trait is just a `ctx` plus three function pointers; it contains **no synchronization of its own**. Whether concurrent use is safe depends entirely on the concrete allocator behind the trait, and on how you share the *objects* the allocator produces. This section states the guarantees, then explains the deeper pointer-provenance hazards and the lock-free concepts you would need if you build concurrent structures on top.

3.1 What is and isn't thread-safe

Allocator	Concurrent use of one instance	Why
`proven_heap_allocator()`	**Safe** for concurrent `alloc`/`realloc`/`free`	The trait is stateless (`ctx == NULL`); it forwards to the platform `aligned_alloc`/`posix_memalign`/`_aligned_malloc`+`free`, which are thread-safe since C11.
`proven_arena_t`	**Not safe**	`proven_arena_alloc_*` does a non-atomic read-modify-write of `arena->offset`.
`proven_pool_t`	**Not safe**	`proven_pool_*` pop/push the free-list (`bin`, `bin_len`) non-atomically.

Two rules follow:

1. **"Allocator safe" is not "object safe."** Even with the heap allocator, the containers built on it (`proven_u8str_t`, `proven_array_t`, `proven_map_t`, ...) add no internal locks. A `proven_u8str_t` that one thread appends to while another reads it is a data race regardless of how thread-safe the allocator is. Shared mutable objects always need *your* synchronization.
2. **Arena and pool must not be shared.** Concurrent `proven_arena_alloc` can tear `offset` and hand the *same bytes* to two threads or drop an allocation entirely; concurrent pool pop/push can hand out the same slot twice or underflow `bin_len`. A data race is undefined behavior on its own — see §7.3 for why the consequences are worse than "a wrong number."

3.2 Pointer provenance in one paragraph

In C's abstract machine every pointer carries **provenance**: the identity of the storage instance it was derived from. Two rules matter here: (a) using a pointer outside the lifetime or bounds of its provenance object is undefined; and (b) the optimizer is allowed to assume that **pointers with different provenance do not alias**, and to reorder or elide memory accesses on that basis. Allocation, reallocation, and freeing all create and destroy provenance — which is exactly why they interact badly with unsynchronized sharing.

3.3 Where allocation + provenance bite under threads

- `realloc` **always relocates here.** The platform `realloc` is *allocate-new* + *copy* + *free-old* (an aligned block cannot be resized in place portably), so a successful grow **always** returns a new object with new provenance and ends the old one. Any retained copy of the old pointer — a cached element pointer, a borrowed `proven_mem_view_t`/`proven_u8str_view_t` — is now dangling. In a single thread you avoid this by not holding a view across a grow; under threads another thread can grow a shared container at *any* instant, leaving your view pointing into freed storage (rule (a): UB, plus a data race).

- **Address reuse / ABA.** `free` then `alloc` (especially the pool's free-list recycling) can return the *same address* for a *different* object with *fresh* provenance. A thread still holding the old pointer assumes the old provenance; by rule (b) the compiler may treat the two as non-aliasing even though the bytes coincide, and without a happens-before edge the new object's writes need not be visible. Same address, different provenance — still UB.
- `uintptr_t` **round-trips + torn reads.** The arena converts `backing.ptr` to an integer for alignment math. It is careful to derive the *result* pointer by offsetting the original `backing.ptr` (preserving its provenance) rather than fabricating a pointer from the integer — the provenance-correct technique. But if `offset` is read torn under a race, the computed pointer can land *outside* the backing object, i.e. an access with no valid provenance for that location. The race produces not just a wrong offset but a pointer with no right to point there.
- **Data race × provenance reasoning = miscompilation, not just wrong values.** Because the compiler applies single-threaded, provenance-based non-aliasing reasoning within each thread, a racy allocator can let the optimizer "prove" non-aliasing that does not hold at runtime — yielding torn pointers, double allocations the compiler believes cannot overlap, or dropped stores. The failure mode is structural, not a flaky value.

3.4 The lock-free toolbox (concepts, if you build concurrency on top)

`proven` does **not** implement any lock-free allocator or safe-memory-reclamation scheme. If you build concurrent data structures over these allocators, you supply the following yourself (`<stdatomic.h>` is available — the job system in Chapter 6 uses it for its queue indices). These are the standard pieces and how they relate to the provenance hazards above.

- **CAS (compare-and-swap).** The atomic primitive behind almost all lock-free code: `atomic_compare_exchange_strong(&p, &expected, desired)` atomically sets `p = desired` only if `p == expected`, otherwise reports the current value. It lets one thread publish a change only if no other thread changed the word first.

```
text /* Treiber-stack style push (sketch): node_t and n are illustrative, so this is a listing rather than a compiled block. */ _Atomic(node_t *) head; node_t *old = atomic_load(&head); do { n->next = old; } while (! atomic_compare_exchange_weak(&head, &old, n));
```

- **The ABA problem.** CAS compares *values*, not *history*. A lock-free pop reads `head == A`, plans to install `A->next`, then CASes. If, in between, other threads pop `A`, pop its successor, free them, and push `A` back (its address reused), the CAS still sees `head == A` and *succeeds* — installing a pointer to freed memory. The value matched (A→B→A) but the world changed. The pool's free-list is a textbook ABA candidate if naïvely made lock-free.
- **Tagged pointers / version counters.** Pack a monotonically increasing tag next to the pointer and CAS them together (a double-width CAS, or by stealing the low alignment bits / high bits). Every successful update bumps the tag, so an A→B→A sequence comes back with a *different* tag and the CAS fails — ABA detected. Costs/limits: bit-stealing needs guaranteed alignment and shrinks the usable address range; a full-width tag needs hardware double-word CAS (e.g. `cmpxchg16b`); the tag can in principle wrap. **Crucially, a tag fixes ABA *detection* on the shared word — it does not restore provenance.** The reused address is still a new object; the tag only stops you from *acting* on the stale view.
- **Hazard pointers (safe memory reclamation).** Each thread owns a few single-writer/multi-reader "hazard" slots. Before dereferencing a shared pointer it publishes that pointer into a slot and re-validates; a thread that wants to free an object first scans every hazard slot and **defers** the free (a retire list) if anyone is protecting it. This bounds memory and prevents use-after-free and reclamation-ABA, at the cost of a store + fence per protected access and a fixed number of simultaneously protected pointers per thread.
- **Epoch-based reclamation (EBR).** A global epoch counter; a thread "pins" the current epoch while touching shared structures, and retired memory is tagged with the epoch it was retired in.

Memory retired in epoch *e* is freed only once every thread has been observed past *e* (a grace period of a couple of epochs). Cheaper per operation than hazard pointers (just a pinned flag), but a thread that stalls while pinned blocks all reclamation — unbounded memory growth. Variants: quiescent-state (QSBR) and interval-based reclamation.

- **How these tie back to provenance.** Reclamation schemes (hazard pointers, EBR) exist precisely to keep a freed object's *storage alive* — its provenance valid — until no thread can still reference it. CAS + a tag keep the shared *word* consistent but say nothing about the lifetime of what it points to. That is why a correct lock-free stack typically needs **both**: a tag (for ABA on the head word) *and* an SMR scheme (for safe reclamation of popped nodes). A `proven` pool or arena gives you neither, so concurrency must be layered above them, not assumed.

3.5 Safe patterns with `proven`

- **Per-thread arena/pool.** Give each thread its own `proven_arena_t` / `proven_pool_t`. No sharing means no race and no cross-thread provenance — the simplest correct design, and usually the fastest.
- **Heap for cross-thread alloc/free, but synchronize the objects.** It is fine for thread A to allocate and thread B to free via `proven_heap_allocator()`; it is *not* fine for both to mutate the same `proven_array_t`/`proven_u8str_t` without a lock.
- **Hand off ownership with a happens-before edge.** Transfer a pointer to another thread through a mutex, atomic with release/acquire ordering, or a queue (like the job system) so that only one thread observes it at a time. This keeps each allocation's provenance confined to a single thread and makes the producer's writes visible to the consumer.
- **Do not pass borrowed views across threads** unless the owner is guaranteed not to grow/move/free for the whole duration of the borrow — and remember a grow here always relocates.
- **If you must share an arena/pool, wrap it** in your own mutex (or build a real lock-free allocator with the tools in §7.4); the built-in ones assume a single owner at a time.

4. Alias layer

Why a second set of names exists

`proven_u8str_view_slice` is 26 characters. In a file that calls twenty such functions, the prefix is a third of the line, and the part that varies — the part you actually read — is squeezed into what is left.

Prefixes are not decoration in C. The language has one global namespace for functions, so a library that exports `slice` or `create` will eventually collide with something else the program links. The prefix is what makes a C library safe to combine with others, and it is why every serious C library has one.

`alias_xcv.h` is the compromise: a shorter `xcv_` spelling for every public symbol, in a header you opt into. Include it and `xcv_u8str_view_slice` works; do not, and the short names do not exist to collide with anything.

The canonical names remain the source of truth — for the ABI, for this manual, and for the tests. The alias layer is a spelling, not an API: nothing is exported under the short name that is not exported under the long one, and a completeness gate (`tests/test_docs_alias_completeness.c`) fails the build if a public function is ever added without its alias, because an alias layer with holes is worse than none — a caller who adopts it discovers the gaps one compile error at a time.

Wrong — expecting the aliases without including the header:

```
#include "proven.h"
xcv_u8str_view_t v;   /* wrong: proven.h does not pull in the alias layer */
```

Wrong — mixing the two spellings for the same idea in one file:

```
proven_result_u8str_t s = xcv_u8str_create(alloc, 32);   /* wrong: pick one and stay with it */
```

Both compile. Neither is a good idea: a reader now has to know both vocabularies to follow one function.

Include it after the canonical headers:

```
#include "proven.h"
#include "proven/alias_xcv.h"

xcv_allocator_t heap = xcv_heap_allocator();
(void)heap;
xcv_println("{}", XCV_ARG(123));
```

Alias design rules:

- Aliases are preprocessor conveniences, not a separate ABI.
- The canonical API remains the source of truth.
- Treat `alias_xcv.h` as a template if a project wants its own shorter local prefix.
- Keep alias tests synchronized when public API names are added or removed.

Alias macro families:

- Formatting: `XCV_ARG`, `XCV_ARG_CSTR`, `XCV_ARG_CSTR_N`, `XCV_FMT_IS_OK`.
- Containers: `XCV_ARRAY_*`, `XCV_RING_*`, `XCV_MAP_*`, `XCV_LIST_*`.
- Errors and arithmetic: `XCV_ERR_*`, `XCV_IS_OK`, `XCV_CKD_*`.
- Coroutines: `XCV_CORO_*`.
- Filesystem and sysio: `xcv_fs_*`, `xcv_print*`, `xcv_scan*` aliases.
- Types and allocators: `xcv_*_t` type aliases and allocator aliases.

For an exhaustive source-grounded alias table, see Chapter 7: Alias Index.

5. PAL contract

Why the syscalls are quarantined
Every library that touches an operating system has to decide where the OS-specific code goes. The usual answer is "wherever it is needed", with `#ifdef _WIN32` sprinkled through the source. That works, and it has two costs that compound: nobody can tell what the library actually requires from the OS, and porting means auditing every file.

This library puts all of it in one directory. **`platform/` is the only place that makes a syscall.** Everything in `src/proven/` is portable C that calls through the PAL, which means:

- **The requirement list is a file listing.** What this library needs from an operating system is exactly the set of `proven_sys_*` functions — nothing hidden, nothing discovered late.
- **Porting is bounded.** A new target reimplements `platform/`. Nothing in `src/proven/` changes, and the tests that exercise the portable half keep passing.
- **Freestanding is the same mechanism, not a special case.** Build without the hosted PAL files and the portable core is what remains. That is why the freestanding guide is a list of which files to leave out rather than a separate port.

The PAL is **internal**. `proven_sys_*` functions are not the public API and their signatures may change; the public wrappers in `fs.h`, `sysio.h`, `time.h` and `random.h` are what you call. The symbols gate knows this — it exempts the PAL from the "must be documented" requirement precisely because it is not a surface you are meant to program against.

Wrong — calling the PAL directly from application code:

```
proven_sys_fs_open(path, flags);   /* wrong: internal. Use proven_fs_open. */
```

You lose the error translation, the argument validation, and any guarantee that the call keeps working next release.

PAL areas:

- `proven_sys_mem`: heap allocation backend.
- `proven_sys_fs`: files, directories, paths, links, permissions, locks.
- `proven_sys_time`: clock, sleep, time breakdown.
- `proven_sys_env`: environment variables.
- `proven_sys_thread`: threads and synchronization for the job system.
- `proven_sys_io`: standard stream I/O.
- `proven_sys_math`: math helpers where needed.

Public application code should prefer high-level APIs:

- `proven_fs_*` for filesystem operations.
- `proven_sysio_*` and print/scan macros for console I/O.
- `proven_time_*` for time.
- `proven_job_*` for threaded jobs.

PAL internal exception: thread lifecycle code may need internal platform allocation to keep opaque OS metadata alive across entry points. That exception stays inside PAL code and should not leak into core container APIs.

6. Freestanding subset

`PROVEN_FREESTANDING` builds a reduced subset for OS-free or libc-minimal targets. The current freestanding build also defines:

```
-DPROVEN_FREESTANDING -DPROVEN_FMT_NO_FLOAT -DPROVEN_NO_U16STR -ffreestanding
```

Available modules in the current freestanding configuration:

- `types`
- `error`
- `memory`
- `align`
- `allocator`
- `arena`
- `pool`
- `buffer`
- `array`
- `list`
- `ring`
- `map`
- `algorithm`
- `u8str`
- `coro`
- `scan`
- `fmt` without floating-point formatting
- `panic`

Excluded or stubbed modules:

- `heap`: compiled, but `proven_heap_allocator()` returns an invalid zero allocator.
- `u16str`: excluded by `PROVEN_NO_U16STR` in the current freestanding build.
- `time`: partial. `src/proven/time.c` is compiled, so `proven_time_breakdown()` and the datetime formatters are available, but the clock backend (`platform/proven_sys_time.c`) is not - so `proven_time_now()`, `proven_time_now_datetime()`, and `proven_time_sleep()` have no implementation to link against.
- `fs`, `sysio`, `mmap`, `job`: hosted/PAL services excluded from the current freestanding subset.

See `manual-freestanding.md` for the exact source list and command examples.

7. Cross compilation

Why the build compiles for targets it cannot run
A library that claims to be portable is making a claim that decays silently. Code that assumed 64-bit pointers, or that `char` is signed, or that unaligned loads are fine, compiles perfectly on the machine it was written on and breaks on an ARM board six months later — and the commit that broke it looked harmless.

`./nob cross` compiles the library for every target in a matrix without running anything. The configured MinGW lanes additionally link a smoke executable. That sounds weak and is not: **most portability failures are compile-time failures.** A type that is not the width you assumed, a missing intrinsic, an alignment requirement the target actually enforces, a header that does not exist there — all of them fail the compiler, on this machine, in seconds, in the commit that introduced them.

What a compile-only check cannot catch is behaviour: endianness bugs, alignment faults that only happen at run time, and anything timing-dependent. Those need real hardware or an emulator, and the matrix does not pretend otherwise. It is the cheap half of portability testing, run on every build, rather than the expensive half run never.

Cross builds are also where the freestanding profile is exercised for real targets — Cortex-M and RISC-V among them — so the "no operating system" claim is checked by a compiler rather than by a paragraph in a guide.

Wrong — treating a green cross matrix as proof the library runs on that target:

```
/* wrong: ./nob cross compiles and links objects. It does not execute anything.
   Endianness, alignment faults and real timing still need the hardware. */
```

The build driver command is:

```
./nob cross -build-root build-out/proven_c_lib
```

Current target categories:

- Native GCC hosted.
- Native Clang hosted when available.
- Linux AArch64 hosted compile-only.
- Linux ARM hard-float hosted compile-only.
- Linux i686 hosted compile-only through `i686-linux-gnu-gcc` or `gcc -m32`.
- Windows x86_64 and i686 WinAPI compile-and-link smoke checks through MinGW.
- ARM Cortex-M freestanding compile-only.
- RISC-V ELF freestanding compile-only.

Rules:

- Missing optional compilers are skipped.
- Real compile errors fail the command.
- Cross builds are compile-only except for the configured MinGW compile-and-link smoke; none is runtime verification.
- Runtime behavior still needs a target runner, emulator, device, or OS environment.

8. Examples and misuse cases

Coroutine macros must not share one source line
Wrong:

```
PROVEN_CORO_YIELD(&c->coro); PROVEN_CORO_YIELD(&c->coro);
/* wrong: both use the same __LINE__ value */
```

Correct:

```
PROVEN_CORO_YIELD(&c->coro);
PROVEN_CORO_YIELD(&c->coro);
```

Coroutine local variables are ordinary locals

A stackless coroutine returns to the caller. Any local variable whose value must survive across yields should live in the coroutine state object, not as an automatic local inside the coroutine function.

Wrong:

```
static proven_i32 next(Task *t) {
int temporary = 0;
PROVEN_CORO_BEGIN(&t->coro);
temporary = 42;
PROVEN_CORO_YIELD(&t->coro);
use_int(temporary); /* wrong assumption: ordinary local lifetime/state is not persistent */
PROVEN_CORO_END(&t->coro);
}
```

Correct:

```
typedef struct Task {
proven_coro_t coro;
int temporary;
} Task;
```

Job destroy must not race with submitters

Wrong:

```
/* thread A */
proven_job_system_destroy(sys);

/* thread B at the same time */
proven_job_submit(sys, work, arg); /* wrong: external synchronization missing */
```

Correct:

```
proven_job_system_close(sys); /* may race with submit and makes it return false */
join_producer_threads();      /* no producer can access sys after this point */
proven_job_system_destroy(sys);
```

Job argument lifetime

Wrong:

```
void submit_bad(proven_job_sys_t *sys) {
int value = 10;
proven_job_submit(sys, work, &value);
} /* wrong: value may be dead before work runs */
```

Correct:

```
JobData *data = allocate_job_data();
proven_job_submit(sys, work_and_free_data, data);
```

Alias layer should not hide canonical docs

Wrong:

```
/* Document only xcv_map_t and forget proven_map_t. */
```

Correct: document canonical `proven_` API first, then mention aliases as optional local spelling.

Worked example: the bounded job system

Compiled and run by the test suite. Note the ordering the contract requires: submit, then close, then destroy. `proven_job_system_destroy` must not race with `proven_job_submit`.

```
#include <stdatomic.h>

/*
 * The job system: worker threads plus a bounded atomic MPMC queue. Idle workers
 * park on platform synchronization. It orders the *handoff* of work - it does
 * not synchronize the data the work touches. That is why the counter below is
 * an atomic and not a plain int: two jobs incrementing the same variable is a
 * data race unless the caller says otherwise.
 *
 * The lifecycle is a straight line, and it is not optional:
 *
 *     init -> submit... -> close -> destroy
 *
 * destroy must not race with submit. Nothing in the library enforces that; the
 * caller has to stop its producers first. Here there is only one producer - this
 * thread - so "stop the producers" means "finish the submit loop before closing".
 */

#define JOB_COUNT 64

static void increment(void *arg) {
    atomic_int *counter = arg;
    /* relaxed is enough: we only need the total to be right, not to order anything
     * against it. The join inside destroy is what publishes the result to us. */
    atomic_fetch_add_explicit(counter, 1, memory_order_relaxed);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_job_sys_t *sys = NULL;
    /* Queue capacity must be a power of two - the ring maps a sequence number to a
     * slot with a mask. Sized above JOB_COUNT so a submit cannot find the ring
     * full even if every worker is still starting up. */
    proven_err_t err = proven_job_system_init(alloc, 4, 128, &sys);
    EXAMPLE_REQUIRE(proven_is_ok(err), "starting a job system with 4 workers should succeed");
    if (!proven_is_ok(err)) return 1;

    /* Lives until after destroy: a job's arg must outlive the job, and jobs run
     * until destroy has drained the queue. */
    atomic_int counter = 0;

    proven_size_t submitted = 0;
    for (proven_size_t i = 0; i < JOB_COUNT; ++i) {
        /* submit returns false when the ring is full or the system is closed. It
         * never waits for queue capacity and never drops work silently; its wake
         * path may briefly enter platform synchronization. Ignoring the answer is
         * how you lose jobs, which is why it is [[nodiscard]]. */
        if (!proven_job_submit(sys, increment, &counter)) {
            /* A real caller would back off and retry, or run the job inline with
             * proven_job_execute_one. Here a full ring means the sizing above is
             * wrong, so say so rather than paper over it. */
            EXAMPLE_REQUIRE(false, "the queue was sized to hold every job");
            break;
        }
        ++submitted;
```

```
    }

    /* This thread is the only producer, and it is done submitting - so it is safe
     * to close. close makes every later submit fail; jobs already queued still run. */
    proven_job_system_close(sys);

    /* destroy blocks until the queue is empty and every worker has been joined.
     * That join is the synchronization point: after destroy returns, every memory
     * effect of every job is visible to this thread. Reading `counter` before this
     * line would be reading a value the workers are still writing. */
    proven_job_system_destroy(sys);

    int ran = atomic_load(&counter);
    EXAMPLE_REQUIRE(submitted == JOB_COUNT, "every job should have been accepted");
    EXAMPLE_REQUIRE(ran == (int)submitted, "every submitted job should have run exactly once");

    printf("submitted %zu jobs, %d ran\n", (size_t)submitted, ran);

    return EXAMPLE_OK();
}
```

Worked example: a stackless coroutine

Compiled and run by the test suite. The rule that catches everyone: locals do not survive a yield, so every piece of state the coroutine needs across a suspension has to live in its struct.

```
/*
 * A stackless coroutine is a switch statement in disguise: BEGIN opens a
 * switch on the saved state, each YIELD records __LINE__ as a resume label and
 * *returns*, and the next call re-enters the function from the top and jumps
 * straight back to that label.
 *
 * Everything that follows comes from that one fact:
 *
 *   - Locals do NOT survive a yield. The function returned; its stack frame is
 *     gone. Anything that must persist lives in the coroutine's own struct - which
 *     is what `value` and `remaining` are doing below.
 *   - Two coroutine macros must not share a source line (they would collide on
 *     __LINE__).
 *   - It cannot yield from a helper it calls: there is no stack to suspend.
 *
 * The payoff is that a suspended coroutine costs exactly its struct - four bytes
 * of state plus whatever you put next to it - and no thread, no stack, no context
 * switch.
 */

typedef struct {
    proven_coro_t coro;
    /* The generator's state. These would be `int i` locals in a normal loop; here
     * they have to be fields, or they would be reset to their initial values on
     * every resume and the loop would never end. */
    int value;
    int remaining;
} squares_t;

/* A coroutine returns proven_i32: 0 = suspended (call me again), 1 = done. */
static proven_i32 squares_next(squares_t *g) {
    PROVEN_CORO_BEGIN(&g->coro);

    g->remaining = 5;
    g->value = 1;
```

```
    while (g->remaining > 0) {
        g->value = g->value * g->value;
        PROVEN_CORO_YIELD(&g->coro);      /* the caller reads g->value here */
        g->value = g->value + 1;          /* resumes exactly on this line */
        g->remaining -= 1;
    }

    PROVEN_CORO_END(&g->coro);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* The coroutine owns no memory, so there is nothing to destroy - but the values
     * it produces have to go somewhere, and that string does have an owner. */
    proven_result_u8str_t out = proven_u8str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "creating the output string should succeed");
    if (!proven_is_ok(out.err)) return 1;

    squares_t gen = {0};
    PROVEN_CORO_INIT(&gen.coro);   /* unconditional, exactly once, before the first call */

    int produced = 0;
    int last = 0;

    /* Drive it to completion. squares_next returns 1 on the call that runs off the
     * end of the body - that call produces no value, so the loop body only runs
     * while it returned 0. */
    while (!squares_next(&gen)) {
        proven_fmt_result_t r = proven_u8str_append_fmt_grow(alloc, &out.value, "{} ",
                                                             PROVEN_ARG(gen.value));
        EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "appending a generated value should succeed");
        last = gen.value;
        ++produced;
    }

    /* Done is sticky: the state is -1 and stays there. Calling it again would just
     * return 1 without re-running the body. */
    EXAMPLE_REQUIRE(PROVEN_CORO_IS_DONE(&gen.coro), "the generator should have finished");
    EXAMPLE_REQUIRE(squares_next(&gen) == 1, "a finished coroutine stays finished");

    /* 1, then (1+1)^2 = 4, then (4+1)^2 = 25, then 676, then 458329. */
    EXAMPLE_REQUIRE(produced == 5, "the generator yields once per iteration");
    EXAMPLE_REQUIRE(last == 458329, "the state carried across every yield");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out.value),
                                         PROVEN_LIT("1 4 25 676 458329 ")),
                    "the generated sequence should be exactly this");

    printf("squares: %s\n", proven_u8str_as_cstr(&out.value));

    proven_u8str_destroy(alloc, &out.value);
    return EXAMPLE_OK();
}
```

Proven Freestanding Mode (v26.09.04a)

Part VI — Going further. Prerequisites: Parts II–V, and Chapter 6. After this guide you can build the library for a target with no operating system and no libc, and you will know exactly which modules survive that and which do not.

This guide describes the current `PROVEN_FREESTANDING` configuration as implemented by `nob.c` and the public headers.

A note on the shape of this guide. Unlike the chapters, roughly half of it is procedure — flag lists, file listings, and two compile commands. Those sections are deliberately short: a compile command explained at length is a compile command nobody reads. The sections that carry a *decision* — why this mode exists (§0), what "excluded" really means (§3), what your panic handler has to do (§5), what you lose by dropping float (§8), why hosted calls fail at link time (§9), why the lifetime rules matter more here (§10), and how the claims in this guide are verified (§11) — are written out in full.

0. Why this mode exists, and why the whole library is shaped by it

Freestanding mode is for firmware, kernels, bootloaders, hypervisors, and anywhere else that has no operating system underneath it — no `malloc`, no `open`, no `printf`, often no `libc` at all.

The C standard has a name for this. A **hosted** implementation gives you the whole standard library and starts your program at `main`. A **freestanding** implementation guarantees only a handful of headers — `<stddef.h>`, `<stdint.h>`, `<limits.h>` and a few others — and is what a compiler targeting bare metal gives you. `-ffreestanding` tells the compiler to assume exactly that.

Most C libraries cannot be used here at all, and the reason is rarely a big one. It is a `malloc` buried three calls down, or a `printf` in an error path, or a global that is initialised by something that never runs. One hidden dependency on the host is enough to make a library unusable on a microcontroller.

This is the constraint that produced most of this library's design decisions, and it is worth seeing that connection, because it explains choices that otherwise look like taste:

Design decision	Read the earlier chapters as	And the reason is
The allocator is a parameter (Ch 2)	"so you can swap the strategy"	There is no `malloc` here. An arena over a `static` array is the only allocator, and code that took one as a parameter already works.
Errors are returned values (Ch 1)	"so you cannot ignore them"	There is no `errno` and nothing to unwind to. A return value is the only mechanism that exists.
Panics go through a hook (Ch 1 §6)	"so you can override it"	There is no `abort()` and no `stderr`. The default traps because that is all it can portably do.
Syscalls live only in `platform/`	"separation of concerns"	It is the only directory that has to be replaced for a new target. Everything in `src/proven/` is portable by construction.

Views instead of C strings (Ch 3)	"so lengths cannot get lost"	`strlen` is libc. A view brings its length with it and needs nothing.

In other words: freestanding support is not a feature bolted onto the side. The rest of the manual has been describing a library built to survive here, and this guide is where you find out what survives and what does not.

What you give up
Everything that needs the operating system, and nothing else:

- **No heap.** `proven_heap_allocator` does not exist. You supply memory — a `static` array is usual — and put an arena or a pool over it.
- **No filesystem, no standard streams, no clock, no OS randomness.** `fs.h`, `sysio.h`, `mmap.h`, `time.h` and the OS entropy source are all hosted services. `proven_chacha_rng_t` still works if you seed it yourself, which is what `proven_random_set_source` is for.
- **No float formatting by default.** `PROVEN_FMT_NO_FLOAT` drops it, because the float path is the largest piece of code in the formatter and most firmware never prints a `double`. §8 says how to turn it back on.
- **No UTF-16 strings.** `PROVEN_NO_U16STR` is on; there is no Windows API here to talk to.

What you keep is the whole portable core: errors, views, checked arithmetic, alignment, arenas, pools, buffers, strings, arrays, maps, lists, rings, sorting, searching, hashing, encoding, coroutines, and integer formatting and scanning.

What still bites you
The lifetime rules do not relax because the target got smaller — if anything they matter more, because there is no operating system to notice a mistake and no allocator that will refuse to reuse memory you still hold. §10 is short and is the section people skip.

1. Build profile

The build driver uses these freestanding flags:

```
-std=c23
-ffreestanding
-DPROVEN_FREESTANDING
-DPROVEN_FMT_NO_FLOAT
-DPROVEN_NO_U16STR
```

The driver still probes `-std=c23` first and falls back to `-std=c2x` when the compiler uses the transitional spelling. The freestanding source contract stays C23-first even though the build wrapper accepts either flag spelling.

The hosted `./nob freestanding` command also links its local freestanding test executables statically on the build host. Cross freestanding targets in `./nob cross` use compile-only object checks.

Run the project freestanding check:

```
cc nob.c -o nob
./nob freestanding -build-root build-out/proven_c_lib
```

Run the cross-build matrix:

```
./nob cross -build-root build-out/proven_c_lib
```

2. Available source files

In the current freestanding build, `nob.c` compiles the portable source files except hosted-only modules.

Included core modules:

```
src/proven/memory.c
src/proven/arena.c
src/proven/pool.c
src/proven/buffer.c
src/proven/heap.c
src/proven/u8str.c
src/proven/array.c
src/proven/ring.c
src/proven/map.c
src/proven/algorithm.c
src/proven/hash.c
src/proven/encode.c
src/proven/random.c
src/proven/float_decimal.c
src/proven/float_parse.c
src/proven/float_format.c
src/proven/time.c
src/proven/fmt.c
src/proven/scan.c
src/proven/panic.c
platform/proven_sys_math.c
```

Excluded hosted modules:

```
src/proven/u16str.c
src/proven/fs.c
src/proven/stream.c
src/proven/sysio.c
src/proven/mmap.c
src/proven/job.c
platform/proven_sys_fs.c
platform/proven_sys_thread.c
platform/proven_sys_io.c
platform/proven_sys_env.c
platform/proven_sys_time.c
platform/proven_sys_random.c
platform/proven_sys_mem.c
```

Do not add excluded hosted modules to a bare-metal build unless you also provide a real platform backend for the target.

3. Module availability

How to read this table, and what "excluded" means
A module is excluded here for exactly one reason: it needs something only an operating system can provide. It is not that the code is untested on small targets or that somebody has not got round to it — `fs.h` needs a filesystem, `sysio.h` needs standard streams, `mmap.h` needs virtual memory, `job.h` needs threads. There is nothing to port.

The interesting rows are the two that are neither available nor excluded:

- `heap.h` **is a stub.** `proven_heap_allocator()` still exists and still compiles, and it returns an allocator whose function pointers are null — one that `proven_alloc_is_valid` reports as invalid. It does not silently allocate from somewhere else and it does not fail to link. That choice matters: code written against the allocator trait keeps compiling for a bare-metal target, and the place it breaks is the one line that asked for a heap, at run time, with a check you can write. §4 shows what to pass instead.

- `time.h` **is limited.** The datetime formatting is pure arithmetic over a number you supply, so it compiles. What is missing is the number: reading a clock is a syscall, so `proven_time_now` has no backend here. Format timestamps you got from your own hardware timer.

Everything marked Available is the portable core, and "available" means the same tests that run on a hosted build run against it — the freestanding profile is built and checked by `./nob freestanding` on every release, not asserted in this table.

Module	Status in current freestanding profile	Notes
`types.h`, `error.h`, `align.h`, `memory.h`	Available	Requires fixed-width integer and `uintptr_t` support.
`allocator.h`	Available	Trait only. Caller supplies backing allocators.
`arena.h`	Available	Primary allocator for static memory regions.
`pool.h`	Available	Uses a caller-provided base allocator, often an arena.
`buffer.h`	Available	Fixed-capacity byte buffer.
`u8str.h`	Available	U8 strings and views.
`u16str.h`	Excluded	Current profile defines `PROVEN_NO_U16STR`.
`array.h`, `list.h`, `ring.h`, `map.h`	Available	No hidden OS dependency.
`algorithm.h`	Available	Sort/search helpers for arrays.
`hash.h`	Available	FNV-1a, SipHash-2-4, CRC-32, SHA-256 — byte-exact, no OS dependency.
`encode.h`	Available	Hex and Base64 — pure computation, no OS.
`fmt.h`	Available without float	Current profile defines `PROVEN_FMT_NO_FLOAT`.
`scan.h`	Available	Scanner for memory views.
`float_parse.h`	Available	`proven_strtod`, `proven_parse_double_ascii` and `proven_parse_f64_ascii` all compile here: the decimal-to-binary64 engine is integer-only and needs no libc. The one difference is that the freestanding build does not set `errno` on overflow or underflow — the returned `proven_err_t` carries that instead, which is the value to check on a target with no `errno` at all. This is separate from `fmt.h`'s float **formatting**, which the profile does compile out.

`float_format.h`	Available without `fmt.h` integration	The binary64 formatter is integer-only and compiles, but the current profile defines `PROVEN_FMT_NO_FLOAT`, so `{}` will not render a float. Call the `float_format.h` entry points directly if you need digits on a target where you have decided the code size is worth it.
`time.h`	Limited	Core datetime formatting can compile; real PAL time is excluded.
`heap.h`	Stub	`proven_heap_allocator()` returns an invalid allocator.
`fs.h`, `stream.h`, `mmap.h`, `sysio.h`, `job.h`	Excluded	Require hosted PAL services.
`random.h`	Available	The generators and helpers are pure arithmetic. `proven_random_bytes` works here too, but only once you install an entropy source with `proven_random_set_source` — a board's TRNG, ring oscillator, or ADC noise floor. With none installed it returns **false** rather than falling back to a clock-seeded PRNG, which would look like success and be a security hole nothing reports. `proven_chacha_rng_seed_from_entropy` then turns the board's entropy into an endless cryptographic stream.
`coro.h`	Available	Macro-only stackless coroutine support.
`panic.h`	Available	Override for target-specific trap/reset behavior.

4. Minimal static arena setup

There is no heap, so the memory comes from a static block and an arena is laid over it. The arena does not own that block: `alloc` bumps an offset, `free` is a no-op, and `reset` rewinds the whole thing at once.

This is the pattern the whole library was shaped to allow, and it is worth seeing why it works. Every function that can allocate takes a `proven_allocator_t` as a parameter, so code written for a hosted build against `proven_heap_allocator()` runs here unchanged once you hand it an arena instead.

Nothing in `u8str.h`, `array.h` or `map.h` knows or cares that the memory came from a `static` array in `.bss` rather than from `malloc`.

Three decisions to make when you size that block, none of which the library can make for you:

- **How big.** The arena cannot grow, so its size is a hard limit you are choosing at compile time. Too small and allocations start returning `PROVEN_ERR_NOMEM`; too large and you have spent RAM the rest of the firmware needed. This is the trade you are being asked to make explicitly rather than discovering it as heap fragmentation months later.
- **When to reset.** An arena's whole advantage is freeing everything at once. The natural points are a loop iteration, a received packet, a command — anywhere a batch of work has a clear end. Everything allocated during that batch dies at the reset, so **nothing may outlive it**.
- **Alignment of the backing array.** `alignas(max_align_t)` on the declaration, as below. Without it the array may start at an address that cannot hold a `double`, and the arena has nothing to fix that with.

Wrong — a view built in the arena that outlives the reset:

```
proven_u8str_view_t label = build_label(arena_alloc);   /* lives in the arena */
proven_arena_reset(&arena);
send(label);   /* wrong: those bytes are free space now, and the next allocation takes them */
```

On a hosted build this often survives long enough to look fine in testing. Here the next allocation gets the same bytes immediately, so it does not.

```
#include "proven/types.h"
#include "proven/memory.h"
#include "proven/arena.h"
#include "proven/array.h"
#include "proven/panic.h"

/* static storage: no OS, no heap, known at link time */
static alignas(PROVEN_MAX_ALIGN) proven_byte_t storage[4096];

proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t arena_alloc = proven_arena_as_allocator(&arena);

proven_result_array_t r = PROVEN_ARRAY_INIT(arena_alloc, proven_i32, 16);
if (proven_is_ok(r.err)) {
    proven_array_t values = r.value;
    proven_err_t e = PROVEN_ARRAY_PUSH(&values, proven_i32, 42);
    (void)e;
    PROVEN_ARRAY_DESTROY(&values); /* arena free is a no-op */
}

proven_arena_destroy(&arena);      /* also a no-op: the caller owns `storage` */
```

Common mistake:

```
proven_allocator_t heap = proven_heap_allocator();
heap.alloc_fn(heap.ctx, 64, 8); /* wrong: heap is invalid in PROVEN_FREESTANDING */
```

Correct:

```
proven_allocator_t heap = proven_heap_allocator();
if (!proven_alloc_is_valid(heap)) {
    /* use an arena, pool, or target-provided allocator instead */
    proven_panic("no heap on this target");
}
```

5. Panic handler override

Why installing one is not optional here
On a hosted system a panic that traps is survivable: the process dies, the operating system cleans up, and something restarts it. On bare metal there is nothing underneath. A trap halts the core, and whatever the device was doing — holding a motor at speed, keeping a radio link, driving a heater — it is still doing when the CPU stops.

So the default handler is a placeholder for the one you must write, and what yours does is a product decision rather than a programming one. The usual shapes:

- **Put the hardware in a safe state, then halt.** Motors off, outputs to a known level, then spin or wait for a debugger. Correct for anything with a physical actuator.
- **Record and reset.** Write a reason code to a register or a reserved RAM area that survives reset, then trigger the watchdog. The next boot reports why the last one ended.
- **Halt loudly.** Blink an LED in a recognisable pattern. On a board with no console this is the entire diagnostic channel, and it is worth more than it sounds.

The rule from Chapter 1 §6 applies with more force here: **the handler must not return.** If it does, `proven_arena_alloc_or_panic` proceeds with a block that was never allocated, and the failure moves from "the device stopped" to "the device is writing through a null pointer".

Wrong — a handler that logs and returns:

```
static void my_panic(const char *msg) {
    uart_write(msg);   /* wrong: this returns, and the caller uses a block it never got */
}
```

`proven_arena_alloc_or_panic()` and related panic paths call `proven_panic()`, which dispatches to the handler installed with `proven_set_panic_handler()`. The default handler traps.

The handler is a whole function, so this is a listing rather than a fragment:

```
#include "proven/panic.h"

static void my_panic(const char *msg) {
(void)msg;
/* optional: write msg to UART or crash log */
for (;;) {
/* or reset the MCU */
}
}

/* install once during startup; pass NULL to restore the default */
proven_set_panic_handler(my_panic);
```

A production panic handler should not return. If it returns, the allocation result that triggered the panic is not guaranteed to be valid.

6. Example Cortex-M compile command

```
arm-none-eabi-gcc \
-std=c23 \
-mcpu=cortex-m4 -mthumb \
-ffreestanding -nostdlib \
-DPROVEN_FREESTANDING \
-DPROVEN_FMT_NO_FLOAT \
-DPROVEN_NO_U16STR \
-Iinclude -Iplatform \
-c src/proven/memory.c -o memory.o
```

A real firmware build should compile the included freestanding modules listed above, compile your target startup code, and link with your linker script.

7. Example RISC-V ELF compile command

```
riscv64-elf-gcc \
-std=c23 \
-ffreestanding -nostdlib \
-DPROVEN_FREESTANDING \
-DPROVEN_FMT_NO_FLOAT \
-DPROVEN_NO_U16STR \
-Iinclude -Iplatform \
-c src/proven/arena.c -o arena.o
```

If your toolchain uses the `riscv64-unknown-elf-gcc` name, use that compiler instead. The project cross matrix checks both names when available.

8. Formatting in freestanding mode

What is left, and why that is usually enough
The formatter is portable computation — it builds bytes in a destination you supply — so almost all of it survives here. What is gone is one placeholder type: `double`.

That sounds like a big loss and rarely is. Correct float formatting means emitting the shortest decimal that reads back as the same value, on every input including subnormals, and doing that requires big-integer arithmetic and lookup tables. It is the largest single piece of code in the formatter. Most firmware formats integers, string views, characters and pointers — a sensor reading is a scaled integer, a status line is text — so the profile drops the float path by default and the binary is smaller for it.

What remains: `{}` for integers of every width, views, characters, booleans and pointers; the whole spec grammar (width, fill, alignment, base, sign); `PROVEN_ARG_OF` for your own types; and the scanner in full, including float *parsing*, which does not carry the same weight.

If you do need floats on a small target, the switch is `PROVEN_FMT_NO_FLOAT` and §8a covers the capacity knob that goes with it. Measure the size change before deciding — on a part with 32 KB of flash it is not a rounding error.

Note the other consequence: **there is no `proven_println` here.** Formatting appends into a `proven_u8str_t` or a buffer, and getting those bytes to a UART or an RTT channel is your platform code, because a freestanding build has no standard output to assume.

Float formatting is disabled by `PROVEN_FMT_NO_FLOAT`. Integer and string-view formatting remain available.

Correct (`alloc` here is an arena allocator, as in section 4 - there is no heap):

```
proven_result_u8str_t r = proven_u8str_create(alloc, 32);
if (proven_is_ok(r.err)) {
    proven_fmt_result_t f = proven_u8str_append_fmt_grow(alloc, &r.value,
                                                         "value={}", PROVEN_ARG(123));
    (void)f;
    proven_u8str_destroy(alloc, &r.value);
}
```

Wrong:

```
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(3.14));
/* wrong in the current freestanding profile: float args are excluded, so
   PROVEN_ARG has no _Generic association for double and this fails to compile */
```

8a. Tuning the float big-integer capacity

The exact decimal-to-binary64 fallback and the big-integer division helper use a fixed-size big integer whose capacity is set by `PROVEN_FLOAT_BIGINT_LIMBS` (default 160, in `proven/float_config.h`). This is the dominant factor in their stack usage: each big integer is `8 * PROVEN_FLOAT_BIGINT_LIMBS` bytes, and a division uses several plus 32-bit scratch.

Targets with tight stacks can lower it, for example:

```
-DPROVEN_FLOAT_BIGINT_LIMBS=48
```

The kept-significand cap (`PROVEN_FLOAT_MAX_SIGNIFICAND_DIGITS`) is derived from the capacity, so a smaller value never overflows: the parser still rounds correctly for inputs up to the derived number of significant digits and stays within one ULP for longer inputs. The Clinger and Eisel-Lemire fast paths do not use the big integer, so typical short inputs are unaffected. Keep the default (160) when exact rounding of pathological long decimals matters, since a binary64 rounding boundary can need up to 767 significant digits.

9. Avoid hosted APIs

Why this is a link error rather than a run-time surprise

The hosted modules are not compiled into a freestanding build at all. Calling one is therefore a **link error** — an undefined symbol, at build time, naming the function you should not have used.

That is the design working. The alternative, which many embedded libraries choose, is to provide stubs that return an error at run time; then a call to `proven_fs_open` on a microcontroller compiles, links, ships, and fails in the field on a path nobody exercised. Here it cannot get out of the build.

The practical consequence for your own code: if a module is shared between a hosted tool and firmware, the hosted-only calls need to be behind `#ifndef PROVEN_FREESTANDING`, and the link error tells you exactly which ones you missed.

Do not call these in the current freestanding profile:

```
proven_fs_open(...);
proven_println(...);
proven_env_get(...);
proven_mmap_create(...);
proven_job_system_init(...);
```

They require hosted PAL files that are intentionally excluded.

10. Lifetime rules still apply

The section people skip, and why it costs more here

Freestanding does not relax any of the ownership rules in Chapter 0 §5. If anything it sharpens them, for three reasons that all point the same way:

- **There is no allocator between you and the memory.** A dangling pointer into a heap block often survives for a while because the allocator has not handed that block out again. An arena hands the same bytes out on the very next allocation after a reset, so a stale view is overwritten immediately and deterministically.
- **There is nothing to catch you.** No MMU trap on a small target, no operating system to kill the process, no sanitizer in the field. A use-after-reset does not crash — it reads someone else's data and carries on, and the symptom appears somewhere unrelated.
- **The consequences are physical.** The wrong byte here can be a motor command or a radio packet.

The rules are the same ones you already know: a view is valid only while its owner is, an owned object is destroyed exactly once with the allocator that made it, and caller-owned state structs must not be copied. What changes is the margin for error, which is zero.

Freestanding does not relax container rules.

Wrong:

```
int *p = PROVEN_ARRAY_GET_MUT(&arr, int, 0);
PROVEN_ARRAY_PUSH(&arr, int, 9);
*p = 1; /* wrong: push may have moved the array */
```

Wrong:

```
proven_u8str_view_t key = make_stack_view();
PROVEN_MAP_SET_U8_BORROWED(&map, key, int, 1);
return; /* wrong if map survives after key bytes go out of scope */
```

11. Verification

Why this guide is checked rather than believed
Everything above is a claim about what compiles and links without an operating system, and claims like that rot quietly. One `#include <stdio.h>` added to a portable source file in an unrelated commit and the freestanding profile is broken — on a host build nothing would notice, because `stdio.h` is right there.

So the profile is built on every release rather than described. `./nob freestanding` compiles the portable core with `-ffreestanding` and the profile's defines, links the checks below **statically** on the build host, and runs them. `./nob cross` then compiles the same profile for real embedded targets — Cortex-M and RISC-V among them — as a compile-only matrix.

The two checks are chosen for what they would catch:

- The **heap stub** check proves `proven_heap_allocator()` still exists and returns something `proven_alloc_is_valid` rejects. If somebody made it fail to link instead, trait-based code would stop compiling for bare metal — the failure §3 explains this design avoids.
- The **compile check** builds a representative program against the profile, which is what catches a hosted header sneaking into a portable file.

Neither runs on the actual hardware, and this guide does not claim otherwise: alignment faults, endianness and timing need a board. What is verified is the part that can be, on every build, rather than the whole thing on no build.

The project freestanding command builds and runs these local checks on the build host:

```
tests/test_portability_freestanding_heap_stub
tests/test_portability_compile_freestanding
tests/test_portability_compile_nofloat
tests/test_portability_compile_nou16str
tests/test_portability_freestanding
```

The cross command performs compile-only checks for available embedded compilers:

```
freestanding-arm-cortex-m4        arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb
freestanding-riscv64-elf          riscv64-elf-gcc
freestanding-riscv64-unknown-elf  riscv64-unknown-elf-gcc
```

Missing toolchains are skipped. Real compile failures fail the command. Runtime behavior still needs validation on the target or emulator.

Appendix A: `alias_xcv.h` Alias Index (Chapter 7)

This is a lookup table, not reading material. It is the one file in the manual with no prose to work through: 500-odd rows mapping every `xcv_` short name to its canonical `proven_` name. Skip it on a first read and come back when you meet an `xcv_` spelling you do not recognise, or when you are deciding whether to adopt the short names in your own code.

The file keeps its `manual-07` number because that number is a stable identifier used by links and by the build; its position in the reading order is "appendix", which is what this title says.

This appendix is generated from `include/proven/alias_xcv.h`. The alias layer is optional. The canonical `proven_` and `PROVEN_` names remain the source of truth for ABI, documentation, and tests.

Usage rule

Include aliases after the normal headers:

```
#include "proven.h"
#include "proven/alias_xcv.h"
```

Do not mix alias documentation with canonical API documentation. Use this index only as a spelling map.

Alias table

501 aliases: 375 lowercase `xcv_` function names and 126 uppercase `XCV_` macro names. The table is generated from `include/proven/alias_xcv.h`; `tests/test_docs_alias_completeness` fails the build if a public function has no alias. That gate compares the headers with each other, not with this appendix — the table had fallen 61 rows behind the header before it was regenerated, so regenerate it from the header rather than editing rows by hand.

There is deliberately no line-number column. It was wrong after every alias that got inserted above it, which is worse than having no column at all.

Alias macro	Expands to
XCV_ARG	PROVEN_ARG
XCV_ARG_CSTR	proven_arg_cstr
XCV_ARG_DATETIME	proven_arg_datetime
XCV_ARG_F64	proven_arg_f64
XCV_ARG_FN	PROVEN_ARG_FN(f)
XCV_ARG_I32	proven_arg_i32
XCV_ARG_I64	proven_arg_i64
XCV_ARG_NONE	proven_arg_none
XCV_ARG_PTR	proven_arg_ptr
XCV_ARG_STR_VIEW	proven_arg_str_view
XCV_ARG_U32	proven_arg_u32
XCV_ARG_U64	proven_arg_u64
XCV_ARRAY_DESTROY	PROVEN_ARRAY_DESTROY
XCV_ARRAY_GET	PROVEN_ARRAY_GET
XCV_ARRAY_GET_MUT	PROVEN_ARRAY_GET_MUT
XCV_ARRAY_INIT	PROVEN_ARRAY_INIT
XCV_ARRAY_IS_VALID	proven_array_is_valid
XCV_ARRAY_POP	PROVEN_ARRAY_POP
XCV_ARRAY_PUSH	PROVEN_ARRAY_PUSH

XCV_CKD_ADD	PROVEN_CKD_ADD
XCV_CKD_MUL	PROVEN_CKD_MUL
XCV_CKD_SUB	PROVEN_CKD_SUB
XCV_CONTAINER_OF	PROVEN_CONTAINER_OF
XCV_CORO_AWAIT	PROVEN_CORO_AWAIT
XCV_CORO_BEGIN	PROVEN_CORO_BEGIN
XCV_CORO_END	PROVEN_CORO_END
XCV_CORO_INIT	PROVEN_CORO_INIT
XCV_CORO_IS_DONE	PROVEN_CORO_IS_DONE
XCV_CORO_YIELD	PROVEN_CORO_YIELD
XCV_DEFAULT_ALIGNMENT	PROVEN_DEFAULT_ALIGNMENT
XCV_ERR_AGAIN	PROVEN_ERR_AGAIN
XCV_ERR_BUSY	PROVEN_ERR_BUSY
XCV_ERR_EOF	PROVEN_ERR_EOF
XCV_ERR_INVALID_ARG	PROVEN_ERR_INVALID_ARG
XCV_ERR_INVALID_ENCODING	PROVEN_ERR_INVALID_ENCODING
XCV_ERR_INVALID_STATE	PROVEN_ERR_INVALID_STATE
XCV_ERR_IO	PROVEN_ERR_IO
XCV_ERR_NOMEM	PROVEN_ERR_NOMEM
XCV_ERR_NOT_FOUND	PROVEN_ERR_NOT_FOUND
XCV_ERR_NEED_MORE	PROVEN_ERR_NEED_MORE
XCV_ERR_OUT_OF_BOUNDS	PROVEN_ERR_OUT_OF_BOUNDS
XCV_ERR_OVERFLOW	PROVEN_ERR_OVERFLOW
XCV_ERR_PERMISSION	PROVEN_ERR_PERMISSION
XCV_ERR_UNSUPPORTED	PROVEN_ERR_UNSUPPORTED
XCV_FS_APPEND	PROVEN_FS_APPEND
XCV_FS_CREATE	PROVEN_FS_CREATE
XCV_FS_LOCK_EXCLUSIVE	PROVEN_FS_LOCK_EXCLUSIVE
XCV_FS_LOCK_SHARED	PROVEN_FS_LOCK_SHARED
XCV_FS_LOCK_UNLOCK	PROVEN_FS_LOCK_UNLOCK
XCV_FS_PATH_SEP	PROVEN_FS_PATH_SEP
XCV_FS_PERM_DEFAULT	PROVEN_FS_PERM_DEFAULT
XCV_FS_PERM_GROUP_R	PROVEN_FS_PERM_GROUP_R
XCV_FS_PERM_GROUP_W	PROVEN_FS_PERM_GROUP_W
XCV_FS_PERM_GROUP_X	PROVEN_FS_PERM_GROUP_X
XCV_FS_PERM_OTHER_R	PROVEN_FS_PERM_OTHER_R
XCV_FS_PERM_OTHER_W	PROVEN_FS_PERM_OTHER_W
XCV_FS_PERM_OTHER_X	PROVEN_FS_PERM_OTHER_X
XCV_FS_PERM_OWNER_R	PROVEN_FS_PERM_OWNER_R
XCV_FS_PERM_OWNER_W	PROVEN_FS_PERM_OWNER_W
XCV_FS_PERM_OWNER_X	PROVEN_FS_PERM_OWNER_X
XCV_FS_READ	PROVEN_FS_READ
XCV_FS_TRUNC	PROVEN_FS_TRUNC
XCV_FS_TYPE_DIR	PROVEN_FS_TYPE_DIR
XCV_FS_TYPE_FILE	PROVEN_FS_TYPE_FILE

XCV_FS_TYPE_OTHER	PROVEN_FS_TYPE_OTHER
XCV_FS_WRITE	PROVEN_FS_WRITE
XCV_INDEX_NOT_FOUND	PROVEN_INDEX_NOT_FOUND
XCV_IS_OK	PROVEN_IS_OK
XCV_KEY_TYPE_INT	PROVEN_KEY_TYPE_INT
XCV_KEY_TYPE_U8_BORROWED	PROVEN_KEY_TYPE_U8_BORROWED
XCV_KEY_TYPE_U8_OWNED	PROVEN_KEY_TYPE_U8_OWNED
XCV_LIST_ENTRY	PROVEN_LIST_ENTRY
XCV_LIST_FOR_EACH	PROVEN_LIST_FOR_EACH
XCV_LIST_FOR_EACH_SAFE	PROVEN_LIST_FOR_EACH_SAFE
XCV_LIT	PROVEN_LIT
XCV_MAP_DESTROY	PROVEN_MAP_DESTROY
XCV_MAP_GET_INT	PROVEN_MAP_GET_INT
XCV_MAP_GET_MUT_INT	PROVEN_MAP_GET_MUT_INT
XCV_MAP_GET_MUT_U8_BORROWED	PROVEN_MAP_GET_MUT_U8_BORROWED
XCV_MAP_GET_U8_BORROWED	PROVEN_MAP_GET_U8_BORROWED
XCV_MAP_GET_U8_OWNED	PROVEN_MAP_GET_U8_OWNED
XCV_MAP_INIT_INT	PROVEN_MAP_INIT_INT
XCV_MAP_INIT_U8_BORROWED	PROVEN_MAP_INIT_U8_BORROWED
XCV_MAP_INIT_U8_OWNED	PROVEN_MAP_INIT_U8_OWNED
XCV_MAP_IS_VALID	proven_map_is_valid
XCV_MAP_REMOVE_INT	PROVEN_MAP_REMOVE_INT
XCV_MAP_REMOVE_U8_BORROWED	PROVEN_MAP_REMOVE_U8_BORROWED
XCV_MAP_REMOVE_U8_OWNED	PROVEN_MAP_REMOVE_U8_OWNED
XCV_MAP_SET_INT	PROVEN_MAP_SET_INT
XCV_MAP_SET_WITH_SCRATCH_INT	PROVEN_MAP_SET_WITH_SCRATCH_INT
XCV_MAP_SET_U8_BORROWED	PROVEN_MAP_SET_U8_BORROWED
XCV_MAP_SET_U8_OWNED	PROVEN_MAP_SET_U8_OWNED
XCV_MAP_SET_WITH_SCRATCH_U8_BORROWED	PROVEN_MAP_SET_WITH_SCRATCH_U8_BORROWED
XCV_MAX_ALIGN	PROVEN_MAX_ALIGN
XCV_MMAP_EXEC	PROVEN_MMAP_EXEC
XCV_MMAP_PRIVATE	PROVEN_MMAP_PRIVATE
XCV_MMAP_READ	PROVEN_MMAP_READ
XCV_MMAP_SHARED	PROVEN_MMAP_SHARED
XCV_MMAP_WRITE	PROVEN_MMAP_WRITE
XCV_OK	PROVEN_OK
XCV_RING_DESTROY	PROVEN_RING_DESTROY
XCV_RING_INIT	PROVEN_RING_INIT
XCV_RING_IS_VALID	proven_ring_is_valid
XCV_RING_POP	PROVEN_RING_POP
XCV_RING_PUSH	PROVEN_RING_PUSH
XCV_SCAN_ARG	PROVEN_SCAN_ARG(x)
XCV_SCAN_ARG_LONG	PROVEN_SCAN_ARG_LONG(ptr)
XCV_SCAN_ARG_TYPE_F64	PROVEN_SCAN_ARG_TYPE_F64
XCV_SCAN_ARG_TYPE_I32	PROVEN_SCAN_ARG_TYPE_I32

XCV_SCAN_ARG_TYPE_I64	PROVEN_SCAN_ARG_TYPE_I64
XCV_SCAN_ARG_TYPE_INT	PROVEN_SCAN_ARG_TYPE_INT
XCV_SCAN_ARG_TYPE_LLONG	PROVEN_SCAN_ARG_TYPE_LLONG
XCV_SCAN_ARG_TYPE_LONG	PROVEN_SCAN_ARG_TYPE_LONG
XCV_SCAN_ARG_TYPE_NONE	PROVEN_SCAN_ARG_TYPE_NONE
XCV_SCAN_ARG_TYPE_SHORT	PROVEN_SCAN_ARG_TYPE_SHORT
XCV_SCAN_ARG_TYPE_STR_VIEW	PROVEN_SCAN_ARG_TYPE_STR_VIEW
XCV_SCAN_ARG_TYPE_U32	PROVEN_SCAN_ARG_TYPE_U32
XCV_SCAN_ARG_TYPE_U64	PROVEN_SCAN_ARG_TYPE_U64
XCV_SCAN_ARG_TYPE_UINT	PROVEN_SCAN_ARG_TYPE_UINT
XCV_SCAN_ARG_TYPE_ULLONG	PROVEN_SCAN_ARG_TYPE_ULLONG
XCV_SCAN_ARG_TYPE_ULONG	PROVEN_SCAN_ARG_TYPE_ULONG
XCV_SCAN_ARG_TYPE_USHORT	PROVEN_SCAN_ARG_TYPE_USHORT
XCV_SCAN_ARG_ULONG	PROVEN_SCAN_ARG_ULONG(ptr)
XCV_U16_LIT	PROVEN_U16_LIT
XCV_VERSION_NUM	PROVEN_VERSION_NUM
XCV_VERSION_STRING	PROVEN_VERSION_STRING
xcv_alloc_fn_t	proven_alloc_fn_t
xcv_alloc_is_valid	proven_alloc_is_valid
xcv_allocator_t	proven_allocator_t
xcv_arena_alloc	proven_arena_alloc
xcv_arena_alloc_aligned	proven_arena_alloc_aligned
xcv_arena_alloc_aligned_or_panic	proven_arena_alloc_aligned_or_panic
xcv_arena_alloc_or_panic	proven_arena_alloc_or_panic
xcv_arena_alloc_trait	proven_arena_alloc_trait
xcv_arena_as_allocator	proven_arena_as_allocator
xcv_arena_create	proven_arena_create
xcv_arena_destroy	proven_arena_destroy
xcv_arena_free_trait	proven_arena_free_trait
xcv_arena_realloc_aligned	proven_arena_realloc_aligned
xcv_arena_realloc_trait	proven_arena_realloc_trait
xcv_arena_reset	proven_arena_reset
xcv_arena_t	proven_arena_t
xcv_arg_bool	proven_arg_bool
xcv_arg_char	proven_arg_char
xcv_arg_cstr	proven_arg_cstr
xcv_arg_cstr_n	proven_arg_cstr_n
xcv_arg_custom	proven_arg_custom
xcv_arg_datetime	proven_arg_datetime
xcv_arg_f64	proven_arg_f64
xcv_arg_fn	proven_arg_fn
xcv_arg_i32	proven_arg_i32
xcv_arg_i64	proven_arg_i64
xcv_arg_identity	proven_arg_identity
xcv_arg_none	proven_arg_none

xcv_arg_ptr	proven_arg_ptr
xcv_arg_str_view	proven_arg_str_view
xcv_arg_t	proven_arg_t
xcv_arg_type_t	proven_arg_type_t
xcv_arg_u32	proven_arg_u32
xcv_arg_u64	proven_arg_u64
xcv_arg_ucstr	proven_arg_ucstr
xcv_array_binary_search	proven_array_binary_search
xcv_array_create	proven_array_create
xcv_array_destroy	proven_array_destroy
xcv_array_get	proven_array_get
xcv_array_get_mut	proven_array_get_mut
xcv_array_is_valid	proven_array_is_valid
xcv_array_linear_search	proven_array_linear_search
xcv_array_pop	proven_array_pop
xcv_array_push	proven_array_push
xcv_array_reserve	proven_array_reserve
xcv_array_sort	proven_array_sort
xcv_array_t	proven_array_t
xcv_buf_append	proven_buf_append
xcv_buf_create	proven_buf_create
xcv_buf_destroy	proven_buf_destroy
xcv_buf_t	proven_buf_t
xcv_byte_t	proven_byte_t
xcv_c_lib	proven_c_lib
xcv_compare_fn_t	proven_compare_fn_t
xcv_coro_t	proven_coro_t
xcv_crc32	proven_crc32
xcv_crc32_update	proven_crc32_update
xcv_cstr_len	proven_cstr_len
xcv_datetime_t	proven_datetime_t
xcv_env_get	proven_env_get
xcv_eprint	proven_eprint
xcv_eprintln	proven_eprintln
xcv_err_t	proven_err_t
xcv_file_t	proven_file_t
xcv_float_format_f32_policy	proven_float_format_f32_policy
xcv_float_format_f64_policy	proven_float_format_f64_policy
xcv_float_format_options_fixed_default	proven_float_format_options_fixed_default
xcv_float_format_options_scientific	proven_float_format_options_scientific
xcv_float_format_options_shortest	proven_float_format_options_shortest
xcv_fmt_put	proven_fmt_put
xcv_fmt_to_writer_impl	proven_fmt_to_writer_impl
xcv_fprint	proven_fprint
xcv_free_fn_t	proven_free_fn_t

xcv_fs_chmod	proven_fs_chmod
xcv_fs_close	proven_fs_close
xcv_fs_copy	proven_fs_copy
xcv_fs_dir_close	proven_fs_dir_close
xcv_fs_dir_next	proven_fs_dir_next
xcv_fs_dir_open	proven_fs_dir_open
xcv_fs_entry_t	proven_fs_entry_t
xcv_fs_handle_t	proven_fs_handle_t
xcv_fs_is_absolute	proven_fs_is_absolute
xcv_fs_link	proven_fs_link
xcv_fs_list	proven_fs_list
xcv_fs_list_destroy	proven_fs_list_destroy
xcv_fs_lock	proven_fs_lock
xcv_fs_lock_type_t	proven_fs_lock_type_t
xcv_fs_mkdir	proven_fs_mkdir
xcv_fs_mode_t	proven_fs_mode_t
xcv_fs_open	proven_fs_open
xcv_fs_perms_t	proven_fs_perms_t
xcv_fs_pread	proven_fs_pread
xcv_fs_pwrite	proven_fs_pwrite
xcv_fs_read	proven_fs_read
xcv_fs_read_all	proven_fs_read_all
xcv_fs_read_all_u8str	proven_fs_read_all_u8str
xcv_fs_remove	proven_fs_remove
xcv_fs_rename	proven_fs_rename
xcv_fs_rmdir	proven_fs_rmdir
xcv_fs_seek	proven_fs_seek
xcv_fs_size	proven_fs_size
xcv_fs_stat	proven_fs_stat
xcv_fs_stat_t	proven_fs_stat_t
xcv_fs_symlink	proven_fs_symlink
xcv_fs_sync	proven_fs_sync
xcv_fs_sync_dir	proven_fs_sync_dir
xcv_fs_tell	proven_fs_tell
xcv_fs_truncate	proven_fs_truncate
xcv_fs_type_t	proven_fs_type_t
xcv_fs_walk_close	proven_fs_walk_close
xcv_fs_walk_next	proven_fs_walk_next
xcv_fs_walk_open	proven_fs_walk_open
xcv_fs_write	proven_fs_write
xcv_fs_write_all	proven_fs_write_all
xcv_fs_write_file	proven_fs_write_file
xcv_fs_write_file_atomic	proven_fs_write_file_atomic
xcv_fs_write_file_durable	proven_fs_write_file_durable
xcv_hash_bytes	proven_hash_bytes

xcv_hash_keyed	proven_hash_keyed
xcv_base64_decode	proven_base64_decode
xcv_base64_decoded_size	proven_base64_decoded_size
xcv_base64_encode	proven_base64_encode
xcv_base64_encoded_size	proven_base64_encoded_size
xcv_base64url_encode	proven_base64url_encode
xcv_hex_decode	proven_hex_decode
xcv_hex_decoded_size	proven_hex_decoded_size
xcv_hex_encode	proven_hex_encode
xcv_hex_encoded_size	proven_hex_encoded_size
xcv_heap_allocator	proven_heap_allocator
xcv_i16	proven_i16
xcv_i32	proven_i32
xcv_i64	proven_i64
xcv_i8	proven_i8
xcv_intptr_t	proven_intptr_t
xcv_is_ok	proven_is_ok
xcv_is_pow2	proven_is_pow2
xcv_job_execute_one	proven_job_execute_one
xcv_job_submit	proven_job_submit
xcv_job_sys	proven_job_sys
xcv_job_sys_t	proven_job_sys_t
xcv_job_system_close	proven_job_system_close
xcv_job_system_destroy	proven_job_system_destroy
xcv_job_system_init	proven_job_system_init
xcv_job_t	proven_job_t
xcv_key_type_t	proven_key_type_t
xcv_list_init	proven_list_init
xcv_list_insert_after	proven_list_insert_after
xcv_list_is_empty	proven_list_is_empty
xcv_list_node_t	proven_list_node_t
xcv_list_push_back	proven_list_push_back
xcv_list_remove	proven_list_remove
xcv_list_t	proven_list_t
xcv_map_create	proven_map_create
xcv_map_create_trusted	proven_map_create_trusted
xcv_map_create_with_capacity	proven_map_create_with_capacity
xcv_map_destroy	proven_map_destroy
xcv_map_get	proven_map_get
xcv_map_get_mut	proven_map_get_mut
xcv_map_hash	proven_map_hash
xcv_map_is_valid	proven_map_is_valid
xcv_map_key_t	proven_map_key_t
xcv_map_remove	proven_map_remove
xcv_map_reserve	proven_map_reserve

xcv_map_set	proven_map_set
xcv_map_set_u8_owned	proven_map_set_u8_owned
xcv_map_set_with_scratch	proven_map_set_with_scratch
xcv_map_t	proven_map_t
xcv_mem_align_up	proven_mem_align_up
xcv_mem_copy	proven_mem_copy
xcv_mem_move	proven_mem_move
xcv_mem_mut_from_owned	proven_mem_mut_from_owned
xcv_mem_mut_slice_checked	proven_mem_mut_slice_checked
xcv_mem_mut_slice_unchecked	proven_mem_mut_slice_unchecked
xcv_mem_mut_t	proven_mem_mut_t
xcv_mem_t	proven_mem_t
xcv_mem_view_from_owned	proven_mem_view_from_owned
xcv_mem_view_from_u8	proven_mem_view_from_u8
xcv_mem_view_slice_checked	proven_mem_view_slice_checked
xcv_mem_view_slice_unchecked	proven_mem_view_slice_unchecked
xcv_mem_view_t	proven_mem_view_t
xcv_memcmp	proven_memcmp
xcv_mmap_as_view	proven_mmap_as_view
xcv_mmap_create	proven_mmap_create
xcv_mmap_destroy	proven_mmap_destroy
xcv_mmap_flags_t	proven_mmap_flags_t
xcv_mmap_prot_t	proven_mmap_prot_t
xcv_mmap_sync	proven_mmap_sync
xcv_mmap_t	proven_mmap_t
xcv_panic	proven_panic
xcv_parse_double_ascii	proven_parse_double_ascii
xcv_parse_f64_ascii	proven_parse_f64_ascii
xcv_pool_as_allocator	proven_pool_as_allocator
xcv_pool_destroy	proven_pool_destroy
xcv_pool_init	proven_pool_init
xcv_pool_t	proven_pool_t
xcv_print	proven_print
xcv_println	proven_println
xcv_ptrdiff_t	proven_ptrdiff_t
xcv_chacha_rng	proven_chacha_rng
xcv_chacha_rng_fill	proven_chacha_rng_fill
xcv_chacha_rng_next	proven_chacha_rng_next
xcv_chacha_rng_seed	proven_chacha_rng_seed
xcv_chacha_rng_seed_from_entropy	proven_chacha_rng_seed_from_entropy
xcv_entropy_fn	proven_entropy_fn
xcv_random_bytes	proven_random_bytes
xcv_random_set_source	proven_random_set_source
xcv_random_u64	proven_random_u64
xcv_rng_below	proven_rng_below

xcv_rng_f64	proven_rng_f64
xcv_rng_fill	proven_rng_fill
xcv_rng_is_valid	proven_rng_is_valid
xcv_rng_range	proven_rng_range
xcv_rng_shuffle	proven_rng_shuffle
xcv_rng_u64	proven_rng_u64
xcv_xoshiro256ss_next	proven_xoshiro256ss_next
xcv_xoshiro256ss_rng	proven_xoshiro256ss_rng
xcv_xoshiro256ss_seed	proven_xoshiro256ss_seed
xcv_range_contains_ptr	proven_range_contains_ptr
xcv_reader_buffered	proven_reader_buffered
xcv_reader_from_file	proven_reader_from_file
xcv_reader_from_view	proven_reader_from_view
xcv_reader_is_valid	proven_reader_is_valid
xcv_reader_read	proven_reader_read
xcv_reader_read_line	proven_reader_read_line
xcv_realloc_fn_t	proven_realloc_fn_t
xcv_result_array_t	proven_result_array_t
xcv_result_buf_t	proven_result_buf_t
xcv_result_cstr_t	proven_result_cstr_t
xcv_result_f64_t	proven_result_f64_t
xcv_result_file_t	proven_result_file_t
xcv_result_i64_t	proven_result_i64_t
xcv_result_map_t	proven_result_map_t
xcv_result_mem_mut_t	proven_result_mem_mut_t
xcv_result_mem_view_t	proven_result_mem_view_t
xcv_result_mmap_t	proven_result_mmap_t
xcv_result_ring_t	proven_result_ring_t
xcv_result_size_t	proven_result_size_t
xcv_result_u16str_t	proven_result_u16str_t
xcv_result_u64_t	proven_result_u64_t
xcv_result_u8str_t	proven_result_u8str_t
xcv_result_u8str_view_t	proven_result_u8str_view_t
xcv_ring_create	proven_ring_create
xcv_ring_destroy	proven_ring_destroy
xcv_ring_is_valid	proven_ring_is_valid
xcv_ring_pop	proven_ring_pop
xcv_ring_push	proven_ring_push
xcv_ring_t	proven_ring_t
xcv_scan_arg_f64	proven_scan_arg_f64
xcv_scan_arg_i32	proven_scan_arg_i32
xcv_scan_arg_i64	proven_scan_arg_i64
xcv_scan_arg_identity	proven_scan_arg_identity
xcv_scan_arg_int	proven_scan_arg_int
xcv_scan_arg_llong	proven_scan_arg_llong

xcv_scan_arg_long	proven_scan_arg_long
xcv_scan_arg_none	proven_scan_arg_none
xcv_scan_arg_short	proven_scan_arg_short
xcv_scan_arg_str_view	proven_scan_arg_str_view
xcv_scan_arg_t	proven_scan_arg_t
xcv_scan_arg_type_t	proven_scan_arg_type_t
xcv_scan_arg_u32	proven_scan_arg_u32
xcv_scan_arg_u64	proven_scan_arg_u64
xcv_scan_arg_uint	proven_scan_arg_uint
xcv_scan_arg_ullong	proven_scan_arg_ullong
xcv_scan_arg_ulong	proven_scan_arg_ulong
xcv_scan_arg_ushort	proven_scan_arg_ushort
xcv_scan_f64	proven_scan_f64
xcv_scan_fmt	proven_scan_fmt
xcv_scan_fmt_cursor	proven_scan_fmt_cursor
xcv_scan_fmt_from_file	proven_scan_fmt_from_file
xcv_scan_fmt_from_stdin	proven_scan_fmt_from_stdin
xcv_scan_fmt_internal	proven_scan_fmt_internal
xcv_scan_fmt_internal_view	proven_scan_fmt_internal_view
xcv_scan_i64	proven_scan_i64
xcv_scan_init	proven_scan_init
xcv_scan_skip_until	proven_scan_skip_until
xcv_scan_skip_until_number	proven_scan_skip_until_number
xcv_scan_skip_whitespace	proven_scan_skip_whitespace
xcv_scan_str	proven_scan_str
xcv_scan_t	proven_scan_t
xcv_scan_u64	proven_scan_u64
xcv_set_panic_handler	proven_set_panic_handler
xcv_sha256	proven_sha256
xcv_sha256_final	proven_sha256_final
xcv_sha256_init	proven_sha256_init
xcv_sha256_to_hex	proven_sha256_to_hex
xcv_sha256_update	proven_sha256_update
xcv_size_t	proven_size_t
xcv_strtod	proven_strtod
xcv_sysio_file_buffered	proven_sysio_file_buffered
xcv_sysio_lines_open	proven_sysio_lines_open
xcv_sysio_lines_t	proven_sysio_lines_t
xcv_sysio_out_t	proven_sysio_out_t
xcv_sysio_read_line	proven_sysio_read_line
xcv_sysio_std_t	proven_sysio_std_t
xcv_sysio_stderr_writer	proven_sysio_stderr_writer
xcv_sysio_stdin_lines	proven_sysio_stdin_lines
xcv_sysio_stdin_reader	proven_sysio_stdin_reader
xcv_sysio_stdout_buffered	proven_sysio_stdout_buffered

xcv_sysio_stdout_writer	proven_sysio_stdout_writer
xcv_sysio_print_impl	proven_sysio_print_impl
xcv_sysio_scan_chunk_impl	proven_sysio_scan_chunk_impl
xcv_sysio_scanner_deinit	proven_sysio_scanner_deinit
xcv_sysio_scanner_init	proven_sysio_scanner_init
xcv_sysio_scanner_scan_impl	proven_sysio_scanner_scan_impl
xcv_sysio_stderr	proven_sysio_stderr
xcv_sysio_stdin	proven_sysio_stdin
xcv_sysio_stdout	proven_sysio_stdout
xcv_time_breakdown	proven_time_breakdown
xcv_time_locale_en	proven_time_locale_en
xcv_time_locale_t	proven_time_locale_t
xcv_time_now	proven_time_now
xcv_time_now_datetime	proven_time_now_datetime
xcv_time_sleep	proven_time_sleep
xcv_time_t	proven_time_t
xcv_time_u16_fmt	proven_time_u16_fmt
xcv_time_u8_fmt	proven_time_u8_fmt
xcv_u16	proven_u16
xcv_u16str_append	proven_u16str_append
xcv_u16str_append_grow	proven_u16str_append_grow
xcv_u16str_append_partial	proven_u16str_append_partial
xcv_u16str_as_ptr	proven_u16str_as_ptr
xcv_u16str_create	proven_u16str_create
xcv_u16str_create_from_view	proven_u16str_create_from_view
xcv_u16str_destroy	proven_u16str_destroy
xcv_u16str_len	proven_u16str_len
xcv_u16str_t	proven_u16str_t
xcv_u16str_view_t	proven_u16str_view_t
xcv_u32	proven_u32
xcv_u64	proven_u64
xcv_u8	proven_u8
xcv_u8str_append	proven_u8str_append
xcv_u8str_append_byte	proven_u8str_append_byte
xcv_u8str_append_fmt	proven_u8str_append_fmt
xcv_u8str_append_fmt_grow	proven_u8str_append_fmt_grow
xcv_u8str_append_fmt_trunc	proven_u8str_append_fmt_trunc
xcv_u8str_append_fmt_with_scratch	proven_u8str_append_fmt_with_scratch
xcv_u8str_append_grow	proven_u8str_append_grow
xcv_u8str_append_partial	proven_u8str_append_partial
xcv_u8str_as_cstr	proven_u8str_as_cstr
xcv_u8str_as_view	proven_u8str_as_view
xcv_u8str_borrow	proven_u8str_borrow
xcv_u8str_create	proven_u8str_create
xcv_u8str_create_from_view	proven_u8str_create_from_view

xcv_u8str_destroy	proven_u8str_destroy
xcv_u8str_fmt_internal	proven_u8str_fmt_internal
xcv_u8str_insert	proven_u8str_insert
xcv_u8str_insert_grow	proven_u8str_insert_grow
xcv_u8str_is_valid	proven_u8str_is_valid
xcv_u8str_mut_t	proven_u8str_mut_t
xcv_u8str_remove	proven_u8str_remove
xcv_u8str_replace_at	proven_u8str_replace_at
xcv_u8str_replace_at_grow	proven_u8str_replace_at_grow
xcv_u8str_replace_first	proven_u8str_replace_first
xcv_u8str_reserve	proven_u8str_reserve
xcv_u8str_reset	proven_u8str_reset
xcv_u8str_t	proven_u8str_t
xcv_u8str_view_ends_with	proven_u8str_view_ends_with
xcv_u8str_view_eq	proven_u8str_view_eq
xcv_u8str_view_find	proven_u8str_view_find
xcv_u8str_view_from_cstr	proven_u8str_view_from_cstr
xcv_u8str_view_slice	proven_u8str_view_slice
xcv_u8str_view_starts_with	proven_u8str_view_starts_with
xcv_u8str_view_t	proven_u8str_view_t
xcv_u8str_view_to_cstr	proven_u8str_view_to_cstr
xcv_uintptr_align_up	proven_uintptr_align_up
xcv_uintptr_t	proven_uintptr_t
xcv_writer_buffered	proven_writer_buffered
xcv_writer_flush	proven_writer_flush
xcv_writer_from_buffer	proven_writer_from_buffer
xcv_writer_from_file	proven_writer_from_file
xcv_writer_from_u8str	proven_writer_from_u8str
xcv_writer_is_valid	proven_writer_is_valid
xcv_writer_write	proven_writer_write
xcv_writer_write_partial	proven_writer_write_partial
xcv_writer_write_str	proven_writer_write_str